Ecole Nationale Supérieure des Mines de Rabat

Département Mines
Spécialité : Aménagement et Exploitation du Sol et sous-sol (AESSS)

Projet Data Science / Traitement d'Images

Estimation de la granulométrie d'un minerai
à partir de micrographies : segmentation et métrologie automatisées

Réalisé par :

Oumayma Oughris
Errichi Raouan
Cheiguer Elycheikh

Sous la supervision de :

Pr. Ibtissam Ben Miloud

Année universitaire : **2025–2026**

Table des matières

Résumé de l'étude **1**

Introduction **2**

Source et description des données **3**

Pipeline de traitement et étapes principales **4**
Étape 1 : Initialisation du projet et structuration des données 4
Étape 2 : Calibration spatiale et préparation des entrées 4
Étape 3 : Segmentation classique et extraction de contours (seuillage et gradient) 4
Étape 4 : Mesure planimétrique par méthode Line Intercept 4
Étape 5 : Segmentation par HED et production de priors 4
Étape 6 : Apprentissage supervisé U-Net (fusion image + priors) et évaluation 5
Étape 7 : Labellisation, extraction de variables morphométriques et restitution (visualisation + interface) 5

Variables de l'étude **5**

Description détaillée des variables **6**
Variables de segmentation et objets d'analyse 6
Variables morphométriques et mesures géométriques 7
Calibration, conversion en unités physiques et indicateurs de performance 7

Méthode Line Intercept avec seuil manuel **14**

Méthode Gradient / Canny(version v1) **26**

Méthode Gradient / Canny (version v2 avec masques filtrés **39**

Initialisation & Line Intercept avec HEs **46**

Planimétrie HED & mesures géométriques des grains **76**

Segmentation modulaire des grains et étiquetage automatique **102**

Conclusion **314**

Résumé de l'étude

Ce travail vise à mettre en place une chaîne **complète et reproductible** d'estimation de la granulométrie à partir de micrographies, en combinant des approches classiques de traitement d'images et des méthodes d'apprentissage profond. Le pipeline démarre par le **chargement** des images, leur **prétraitement** et la **calibration** de l'échelle (conversion pixel → microns) afin de rendre les mesures interprétables physiquement.

Plusieurs stratégies de **segmentation des joints de grains** sont ensuite explorées et comparées : seuillage et opérations morphologiques, segmentation par **gradient** (Canny + post-traitement), et segmentation basée sur les cartes de contours **HED** (Holistically-Nested Edge Detection) via OpenCV/DNN. En parallèle, un modèle **U-Net** de segmentation est entraîné sur un jeu supervisé constitué de **grains artificiels** (type Voronoï) et de **grains réels annotés**, avec une inférence robuste pouvant exploiter plusieurs entrées (image + priors).

À partir des masques binaires, le pipeline construit des **cartes de labels** (connected components), puis extrait, grain par grain, des **caractéristiques géométriques** (aire, diamètre équivalent, axes majeur/mineur, circularité, rapport d'axes, centroïdes) qui sont ensuite converties en **unités physiques**. La qualité de segmentation est évaluée quantitativement (Dice, IoU, précision, rappel, F1-score, etc.) et les résultats sont rendus exploitables via des visualisations et une **interface interactive Gradio** (overlay + identifiants + tableau de mesures).

Introduction

La granulométrie et la morphologie des grains constituent des informations essentielles pour la caractérisation des matériaux et l'aide au contrôle de procédés : elles permettent de quantifier la taille moyenne, la dispersion des tailles et certains indicateurs de forme, à partir d'observations micrographiques. Cependant, l'estimation manuelle (ou semi-manuelle) des grains à partir d'images reste souvent **longue**, **difficile à reproduire** et sensible à l'opérateur, notamment lorsque le volume d'images augmente.

Dans ce contexte, le traitement d'images et l'intelligence artificielle offrent une voie d'automatisation pertinente. Le présent notebook met en œuvre un cadre complet allant : (i) du prétraitement des micrographies et de la mise à l'échelle, (ii) à la segmentation par méthodes classiques (seuillage, gradient/Canny, HED), (iii) puis à une segmentation avancée via un U-Net supervisé (grains artificiels + grains réels), avant de terminer par (iv) l'extraction automatique de mesures géométriques, leur conversion en unités physiques, et la construction de sorties directement exploitables (tableaux de caractéristiques, distributions granulométriques, overlays annotés).

Source et description des données

Les sources de données utilisées dans ce projet sont accessibles publiquement via Kaggle :

- **Micrographies réelles (316L ExONE, 500×)** : https://www.kaggle.com/datasets/peterwarren/exone-stainless-steel-316l-grains-500x
- **Grains artificiels (Voronoï)** : https://www.kaggle.com/datasets/peterwarren/voronoi-artificial-grains-gen

Les données exploitées dans ce projet proviennent d'un jeu d'images de joints de grains d'un acier inoxydable 316L obtenu par fabrication additive de type *Binder Jetting*. Le matériau est un acier inoxydable 316L imprimé sur une machine ExONE. Les micrographies ont été acquises au microscope optique à un grossissement de 500×. À l'origine, 21 micrographies de taille 1600$×1200$pixelsonttcollectes, puissubdivisesentuilesde400×$300 pixels afin d'augmenter le nombre d'exemples disponibles pour l'analyse et l'apprentissage. Une partie des masques de joints de grains a été obtenue par des méthodes classiques de seuillage, tandis qu'un autre sous-ensemble a été segmenté manuellement (par exemple dans un logiciel de type Paint) pour servir de vérité terrain supervisée.

Un deuxième niveau de données repose sur des grains artificiels générés par une méthode de cellules de Voronoï. Ces images synthétiques, accompagnées de leurs masques exacts, permettent de compléter le jeu d'apprentissage et de mieux contrôler la distribution de tailles et de formes des grains.

Dans la structure de fichiers du projet (sous la racine `BASE_DIR`, montée depuis Google Drive), les principaux dossiers sont :

- `Grains/` : micrographies de référence de la microstructure réelle (tuiles 400×300).
- `HED_raw/` et `HED_filtered/` : cartes de contours brutes et filtrées produites par le modèle HED.
- `FilteredGradV2/` : images issues de filtrages de gradient (version 2).
- `Segmented/` : résultats de segmentation obtenus par diverses méthodes (seuillage, gradient, HED, U-Net).
- `weights HED/` : fichiers `deploy.prototxt` et `hed_pretrained_bsds.caffemodel` nécessaires à l'initialisation du réseau HED.
- `GRAIN DATA SET/` : jeu de données supervisé pour l'U-Net, organisé en :
  - `AG/` et `AGMask/` : images de grains artificiels (Voronoï) et leurs masques annotés.
  - `RG/` et `RGMask/` : images de grains réels (316L imprimé) et masques de vérité terrain.

Un facteur d'échelle de 2,25 pixels par micron est utilisé pour la conversion des mesures en unités physiques via un paramètre de calibration noté `P2M`. Ce facteur permet de transformer l'aire (en pixels) en μm^2 et les longueurs (diamètre équivalent, axes) en microns. Le jeu de données réel et artificiel est documenté dans la littérature et sur des plateformes publiques (par exemple la thèse accessible via https://stars.library.ucf.edu/etd2020/1693/).

Pipeline de traitement et étapes principales

Étape 1 : Initialisation du projet et structuration des données

Le notebook met en place l'environnement de travail (organisation des répertoires et accès aux données) et charge les bibliothèques nécessaires au traitement d'images, à l'analyse de données et à l'apprentissage automatique. Les chemins de travail sont centralisés (par exemple via `DRIVE_ROOT` et `BASE_DIR`) et déclinés en sous-dossiers dédiés aux images sources, aux sorties intermédiaires (cartes de bords, masques) et aux artefacts de modèles. L'objectif est d'assurer une séparation claire entre *entrées* (images) et *sorties* (masques/mesures/modèles).

Étape 2 : Calibration spatiale et préparation des entrées

Une calibration d'échelle est définie afin de convertir les mesures en pixels vers des unités physiques. Le repère d'échelle utilisé est `225` pixels pour `100` microns, conduisant à `scale = 225/100 = 2.25` (pixels par micron) et `P2M = 1/scale` (microns par pixel). Cette calibration est ensuite utilisée par les fonctions de conversion et de mesure. Les images sont préparées selon les besoins des méthodes (niveaux de gris, cohérence dimensionnelle, et mise au format requis pour l'apprentissage supervisé lorsque nécessaire).

Étape 3 : Segmentation classique et extraction de contours (seuillage et gradient)

Deux familles de segmentation « classiques » sont appliquées pour produire des masques/contours exploitables :

- **Seuillage manuel** : la fonction `manual_threshold_and_blur` produit un masque binaire à partir d'un seuillage et d'un lissage (médian) visant à stabiliser les contours des grains.
- **Gradient (Canny)** : la fonction `segment_gradient` applique une détection de bords (Canny) puis un seuillage sur la sortie afin d'obtenir une image binaire des contours.

La sortie principale de cette étape est un masque/contour binaire par image, utilisé ensuite pour le comptage planimétrique et/ou la construction de *priors*.

Étape 4 : Mesure planimétrique par méthode *Line Intercept*

Les masques/contours issus des méthodes classiques alimentent une mesure planimétrique basée sur des interceptions « grille × contours ». Une grille régulière est construite avec `N_LINES_GRID = 20` (lignes horizontales et verticales), puis les intersections sont comptabilisées via des différences discrètes (`np.diff`) sur l'image d'intersection. Les résultats sont structurés sous forme de matrices `Total_Grains_X` et `Total_Grains_Y`, ainsi que de synthèses par moyennes (`AVG_GrainX`, `AVG_GrainY`).

Étape 5 : Segmentation par HED et production de *priors*

Le pipeline met en œuvre une détection de bords HED (Holistically-Nested Edge Detection) à partir de poids pré-entraînés (fichiers `deploy.prototxt` et `hed_pretrained_bsds.caffemodel`) et d'une couche personnalisée `CropLayer` pour l'alignement dimensionnel. Les cartes de bords HED sont générées puis transformées en masques binaires, de manière à fournir une alternative de segmentation et à construire un *prior* `hed_prior` exploitable par le modèle U-Net.

Étape 6 : Apprentissage supervisé U-Net (fusion image + priors) et évaluation

Un modèle U-Net est entraîné sur un jeu supervisé construit à partir d'images et de masques, avec mise au format `256×256` et un canal (`IMG_HEIGHT=256`, `IMG_WIDTH=256`, `IMG_CH_IMG=1`). Le notebook prévoit plusieurs configurations (entrée simple, entrée image + HED, ou entrée image + `grad_prior` + `hed_prior`), et compile notamment le modèle `U_Net_GrainSeg_GradHED` avec :

- une perte `bce_dice_loss`,
- des métriques incluant `dice_coef` et `MeanIoUThreshold` (nommée `iou_score`),
- ainsi que des métriques de classification binaire (`bin_acc`, `precision`, `recall`).

Étape 7 : Labellisation, extraction de variables morphométriques et restitution (visualisation + interface)

À partir d'un masque binaire (issu des méthodes classiques, HED, ou U-Net), les grains sont isolés par labellisation (`label_components` basée sur `cv2.connectedComponentsWithStats`), avec possibilité de filtrage par aire (`filter_components_by_area`). Les variables morphométriques sont ensuite extraites via :

- `compute_grain_features_from_labels` (s'appuyant sur `regionprops_table`),
- et, lorsque nécessaire, une procédure de repli `fallback_features_from_cc`.

En complément, une mesure géométrique « contour-based » est fournie par `measure_grains_planimetric`, qui calcule (après filtrage des petites particules) des mesures en unités physiques via `p2m`, notamment l'aire (µm^2), le périmètre (µm), la circularité ((C = 4 A / P^2)) et des indicateurs géométriques basés sur la boîte englobante (*aspect ratio*) et des distances de type Feret. La conversion vers des unités physiques est gérée par `add_physical_units` sur la base de `P2M`. Enfin, le notebook propose une restitution visuelle (superposition/annotation des grains, par ex. via `draw_grain_ids_on_image`) et une interface Gradio (`segment_and_measure`) permettant de choisir une méthode (`"U-Net"`, `"HED"`, `"Gradient"`, `"Threshold"`) et d'obtenir à la fois l'overlay et un tableau de mesures.

Variables de l'étude

Les variables retenues dans le cadre de l'analyse se structurent autour de : (i) l'identification des objets segmentés (image/grain), (ii) les mesures morphométriques par grain, (iii) la calibration et la conversion en unités physiques, et (iv) les indicateurs de qualité/quantification associés aux segmentations (U-Net et méthodes classiques) et à la planimétrie.

Variable	Description	Rôle dans l'étude
`image_id`	Identifiant d'image (associé au fichier source).	Regroupement des grains et mesures par image.
`grain_id`	Identifiant d'un grain (label de composante).	Indexation des objets segmentés au sein d'une image.
`area_px`	Aire du grain en pixels.	Variable centrale pour distributions de taille (pixels).
`perimeter_px`	Périmètre du grain en pixels.	Base pour métriques de forme (ex. circularité).
`equiv_diameter_px`	Diamètre équivalent en pixels.	Résumé scalaire de la taille du grain.
`centroid_row`, `centroid_col`	Centroïde dans l'image.	Localisation (overlay/numérotation).
`major_axis_px`, `minor_axis_px`	Axes majeur et mineur en pixels.	Caractérisation de l'allongement.
`axis_ratio`	Rapport axe majeur / axe mineur.	Indicateur de forme.
`circularity`	Mesure dérivée de l'aire et du périmètre.	Quantification de l'écart à une forme circulaire.
`P2M`	Facteur pixels → microns (issu de la calibration).	Conversion pixels → unités physiques.
`area_um2`	Aire en μm^2 après conversion.	Analyse granulométrique en unités interprétables.
`equiv_diameter_um`	Diamètre équivalent en microns.	Comparaisons en unités physiques.
`major_axis_um`, `minor_axis_um`	Axes majeur/mineur en microns.	Morphologie en unités physiques.
`dice_score`, `iou_score`	Dice/IoU entre masque prédit et référence.	Évaluation de la qualité de segmentation.
`Total_Grains_X`, `Total_Grains_Y`	Comptes d'interceptions (horizontal/vertical).	Indicateurs planimétriques (*line intercept*).
`AVG_GrainX`, `AVG_GrainY`	Moyennes des interceptions.	Synthèse planimétrique par orientation.

Description détaillée des variables

Variables de segmentation et objets d'analyse

Le pipeline manipule des images et des masques binaires issus de plusieurs méthodes (`"Threshold"`, `"Gradient"`, `"HED"`, `"U-Net"`). Le masque constitue la sortie structurante : il sert à isoler les grains via labellisation (`label_components`), à alimenter l'extraction de variables morphométriques (`compute_grain_features_from_labels`) et à produire des indicateurs planimétriques par intersections. Une extraction minimale peut également être produite via `fallback_features_from_cc` selon les cas.

Variables morphométriques et mesures géométriques

Les variables morphométriques décrivent la taille, la forme et la localisation des grains. Elles incluent des mesures de taille (par ex. `area_px`, `equiv_diameter_px`), de forme (par ex. `major_axis_px`, `minor_axis_px`, `axis_ratio`, `circularity`) et de position (centroïdes). En parallèle, `measure_grains_planimetric` extrait des mesures géométriques à partir des contours (aire, périmètre, circularité) et des indicateurs géométriques basés sur la boîte englobante (*aspect ratio*) et des distances de type Feret.

Calibration, conversion en unités physiques et indicateurs de performance

La conversion en unités physiques repose sur la calibration `scale = 2.25` (pixels par micron) et `P2M = 1/scale` (microns par pixel). Les variables physiques (par ex. `area_um2` et des longueurs en microns) sont générées via `add_physical_units`. La qualité de la segmentation est évaluée à l'aide de métriques de recouvrement, notamment `dice_score` et `iou_score`. En complément, la quantification planimétrique par *line intercept* produit `Total_Grains_X`/`Total_Grains_Y` et leurs moyennes (`AVG_GrainX`/`AVG_GrainY`) afin de relier la segmentation à un indicateur basé sur les intersections.

[ ]:
```python
# 1) Monter Google Drive
from google.colab import drive
drive.mount("/content/drive", force_remount=True)

# 2) standards
import os
from pathlib import Path
import cv2
import numpy as np
from matplotlib import pyplot as plt
import scipy.stats as stats
import pandas as pd

# 3) Racine de ton Drive dans Colab
DRIVE_ROOT = Path("/content/drive/MyDrive")

#  Adapter "cheigher" au nom EXACT du dossier projet dans ton Drive
BASE_DIR = DRIVE_ROOT / "Data"

print(f"[INFO] BASE_DIR = {BASE_DIR}")
if not BASE_DIR.is_dir():
    raise FileNotFoundError(
        f"Le dossier {BASE_DIR} n'existe pas.\n"
        "  Vérifie le nom du dossier dans ton Google Drive et adapte BASE_DIR."
    )

# === Sous-dossiers principaux du projet (toutes les sources d'images) ===
GRAINS_DIR        = BASE_DIR / "Grains"
HED_RAW_DIR       = BASE_DIR / "HED_raw"
```

```
HED_FILTERED_DIR  = BASE_DIR / "HED_filtered"
SEGMENTED_DIR     = BASE_DIR / "Segmented"
FILTERED_GRAD_DIR = BASE_DIR / "FilteredGradV2"
WEIGHTS_HED_DIR   = BASE_DIR / "weights HED"

# === Dossier du jeu de données supervisé (U-Net / ML) ===
GRAIN_DS_DIR   = BASE_DIR / "GRAIN DATA SET"
AG_DIR         = GRAIN_DS_DIR / "AG"
AGMASK_DIR     = GRAIN_DS_DIR / "AGMask"
RG_DIR         = GRAIN_DS_DIR / "RG"
RGMASK_DIR     = GRAIN_DS_DIR / "RGMask"
THRESH_PRE_DIR = GRAIN_DS_DIR / "THRESH_PRE"
HED_PRE_DIR    = GRAIN_DS_DIR / "HED_PRE"
GRAD_PRE_DIR   = GRAIN_DS_DIR / "GRAD_PRE"

# === Alias rétro-compatibles avec ton notebook existant ===
IMG_DIR = GRAINS_DIR
# === Petit helper pour vérifier les dossiers + nb de fichiers ===
def check_dir(name, path, max_files=5, exts=None):
    """
    Affiche si un dossier existe, le nombre de fichiers et quelques exemples.
    exts : liste d'extensions pour filtrer (['.png', '.jpg', ...]) ou None pour␣
 ↪tout.
    """
    exists = path.is_dir()
    status = "OK" if exists else " "
    print(f"{name:15s} -> {status} | {path}")
    if exists:
        if exts is None:
            files = sorted([p.name for p in path.iterdir() if p.is_file()])
        else:
            exts_lower = tuple(e.lower() for e in exts)
            files = sorted([
                p.name for p in path.iterdir()
                if p.is_file() and p.suffix.lower() in exts_lower
            ])
        print(f"  {len(files)} fichiers trouvés.")
        print(f"  Exemples : {files[:max_files]}")
    print()

# Vérification rapide de tous les dossiers utiles (avec filtre d'extensions␣
 ↪pour les images)
for name, path in [
    ("BASE_DIR",    BASE_DIR),
    ("GRAINS_DIR",  GRAINS_DIR),
    ("HED_RAW_DIR", HED_RAW_DIR),
    ("HED_FILTRÉ",  HED_FILTERED_DIR),
```

```
    ("SEGMENTED",   SEGMENTED_DIR),
    ("FGRADV2_DIR", FILTERED_GRAD_DIR),
    ("WEIGHTS_HED", WEIGHTS_HED_DIR),
    ("GRAIN_DS",    GRAIN_DS_DIR),
    ("AG_DIR",      AG_DIR),
    ("AGMASK_DIR",  AGMASK_DIR),
    ("RG_DIR",      RG_DIR),
    ("RGMASK_DIR",  RGMASK_DIR),
    ("THRESH_PRE",  THRESH_PRE_DIR),
    ("HED_PRE",     HED_PRE_DIR),
    ("GRAD_PRE",    GRAD_PRE_DIR),
]:
    # pour les dossiers d'images : filtrer png / jpg / jpeg
    if "DIR" in name or name in {"BASE_DIR", "GRAIN_DS", "WEIGHTS_HED"}:
        exts = [".png", ".jpg", ".jpeg"] if name not in {"BASE_DIR",␣
 ↪"GRAIN_DS", "WEIGHTS_HED"} else None
        check_dir(name, path, exts=exts)

# === Paramètres physiques globaux (calibration pixels  microns) ===
pixels = 225
microns = 100
scale = pixels / microns    # pixels par micron
M2P   = scale               # microns -> pixels
P2M   = 1 / scale           # pixels -> microns
print(f"[INFO] Échelle : {scale:.3f} pixels / micron (P2M = {P2M:.4f})")
```

```
Mounted at /content/drive
Requirement already satisfied: opencv-python in /usr/local/lib/python3.12/dist-
packages (4.12.0.88)
Requirement already satisfied: numpy<2.3.0,>=2 in
/usr/local/lib/python3.12/dist-packages (from opencv-python) (2.0.2)
[INFO] BASE_DIR = /content/drive/MyDrive/Data
BASE_DIR        -> OK | /content/drive/MyDrive/Data
  0 fichiers trouvés.
  Exemples : []

GRAINS_DIR      -> OK | /content/drive/MyDrive/Data/Grains
  336 fichiers trouvés.
  Exemples : ['A_01_01.png', 'A_01_02.png', 'A_01_03.png', 'A_01_04.png',
'A_02_01.png']

HED_RAW_DIR     ->   | /content/drive/MyDrive/Data/HED_raw

FGRADV2_DIR     ->   | /content/drive/MyDrive/Data/FilteredGradV2

WEIGHTS_HED     -> OK | /content/drive/MyDrive/Data/weights HED
  2 fichiers trouvés.
  Exemples : ['deploy.prototxt', 'hed_pretrained_bsds.caffemodel']
```

```
GRAIN_DS        -> OK | /content/drive/MyDrive/Data/GRAIN DATA SET
  1 fichiers trouvés.
  Exemples : ['.DS_Store']

AG_DIR          -> OK | /content/drive/MyDrive/Data/GRAIN DATA SET/AG
  800 fichiers trouvés.
  Exemples : ['AG_NOISE1.png', 'AG_NOISE10.png', 'AG_NOISE100.png',
'AG_NOISE101.png', 'AG_NOISE102.png']

AGMASK_DIR      -> OK | /content/drive/MyDrive/Data/GRAIN DATA SET/AGMask
  800 fichiers trouvés.
  Exemples : ['AG1.png', 'AG10.png', 'AG100.png', 'AG101.png', 'AG102.png']

RG_DIR          -> OK | /content/drive/MyDrive/Data/GRAIN DATA SET/RG
  480 fichiers trouvés.
  Exemples : ['RG10_1_1.jpg', 'RG10_1_2.jpg', 'RG10_1_3.jpg', 'RG10_1_4.jpg',
'RG10_2_1.jpg']

RGMASK_DIR      -> OK | /content/drive/MyDrive/Data/GRAIN DATA SET/RGMask
  480 fichiers trouvés.
  Exemples : ['RGMask10_1_1.jpg', 'RGMask10_1_2.jpg', 'RGMask10_1_3.jpg',
'RGMask10_1_4.jpg', 'RGMask10_2_1.jpg']

[INFO] Échelle : 2.250 pixels / micron (P2M = 0.4444)
```

Le code commence par monter le Google Drive dans l'environnement d'exécution afin de travailler directement sur les fichiers stockés dans l'espace personnel de l'utilisateur. Le dossier racine du projet est ensuite défini via `BASE_DIR`, et une vérification immédiate garantit que ce dossier existe bien. En cas de problème (nom incorrect, mauvais emplacement), une erreur explicite est déclenchée pour obliger à corriger la configuration avant de poursuivre.
Les différents sous-dossiers du projet sont ensuite déclarés : dossier des micrographies originales, dossiers des résultats HED bruts et filtrés, dossiers des masques segmentés et des versions filtrées Gradient, dossier contenant les poids du modèle HED, ainsi que les dossiers du jeu de données supervisé (images et masques pour les séries AG et RG, plus les variantes prétraitées). Une fonction utilitaire `check_dir` inspecte chaque dossier, indique s'il existe, compte le nombre de fichiers et affiche quelques exemples de noms, ce qui permet de vérifier rapidement que les données sont présentes et correctement organisées.
Enfin, le code fixe la calibration physique en s'appuyant sur une barre d'échelle connue (225 pixels pour 100 microns). À partir de ces valeurs, il calcule le nombre de pixels par micron et son inverse (microns par pixel), puis affiche ces facteurs de conversion. Cette calibration est utilisée dans tout le pipeline pour transformer les longueurs mesurées en pixels (longueurs d'interception, diamètres, dimensions des grains) en grandeurs physiques exprimées en microns, ce qui rend les résultats directement interprétables dans un contexte d'ingénierie des matériaux.

[ ]:
```
#  Bloc corrigé : construction de la liste complète d'images + image test

# Liste triée de toutes les images de grains (PNG)
```

```
image_files = sorted([p.name for p in IMG_DIR.glob("*.png")])
print(f"[INFO] Nombre total d'images de grains trouvées dans IMG_DIR :␣
 ↪{len(image_files)}")
print("Exemples de fichiers :", image_files[:10])

if len(image_files) == 0:
    raise RuntimeError(f"Aucune image .png trouvée dans {IMG_DIR}")

# On garde aussi la liste des chemins complets pour un usage futur sur TOUT le␣
 ↪dataset
grain_image_paths = [IMG_DIR / fname for fname in image_files]

print(f"[INFO] Exemple de chemins complets (3 premiers) :")
for p in grain_image_paths[:3]:
    print("   -", p)

#  Définition d'une fonction réutilisable de seuillage + flous médians
def manual_threshold_and_blur(gray_img, thresh_value=135):
    """
    Applique le seuillage manuel + la chaîne de flous médians
    utilisée dans le notebook d'origine.

    gray_img : image en niveaux de gris (uint8)
    thresh_value : seuil binaire (0-255)
    """
    # Seuil manuel
    _, thresh = cv2.threshold(gray_img, thresh_value, 255, cv2.THRESH_BINARY)

    # Série de flous médians
    blur1 = cv2.medianBlur(thresh, 5)
    blur2 = cv2.medianBlur(blur1, 5)
    blur3 = cv2.medianBlur(blur2, 7)
    blur4 = cv2.medianBlur(blur3, 7)
    return blur4

#  On utilise la première image seulement comme "image de référence"
#     pour vérifier les dimensions et calculer les longueurs physiques.
test_fname = image_files[0]
test_path = IMG_DIR / test_fname
print("\n[INFO] Image test de référence :", test_path)

img = cv2.imread(str(test_path), cv2.IMREAD_GRAYSCALE)
if img is None:
    raise FileNotFoundError(f"Impossible de lire l'image : {test_path}")

print("Image test shape :", img.shape, "| dtype :", img.dtype)
```

```python
# Application du pipeline manuel sur l'image de référence
Blur_Med_4_img = manual_threshold_and_blur(img)

# Dimensions réelles de cette image
numrows, numcols = Blur_Med_4_img.shape
print("Taille utilisée pour les calculs (référence) :", numrows, "x", numcols)

# Longueurs physiques des lignes d'interception (en microns)
row_length = numcols * P2M   # longueur physique d'une ligne horizontale
col_length = numrows * P2M   # longueur physique d'une ligne verticale
print("row_length (microns) :", row_length)
print("col_length (microns) :", col_length)

#  On mémorise ces infos pour la suite du notebook (interceptions, surfaces,␣
 ↪etc.)
IMG_SHAPE_REF = (numrows, numcols)
ROW_LENGTH_MICRONS = row_length
COL_LENGTH_MICRONS = col_length
print("\n[INFO] Paramètres de référence enregistrés :")
print("   IMG_SHAPE_REF        =", IMG_SHAPE_REF)
print("   ROW_LENGTH_MICRONS   =", ROW_LENGTH_MICRONS)
print("   COL_LENGTH_MICRONS   =", COL_LENGTH_MICRONS)
```

```
[INFO] Nombre total d'images de grains trouvées dans IMG_DIR : 336
Exemples de fichiers : ['A_01_01.png', 'A_01_02.png', 'A_01_03.png',
'A_01_04.png', 'A_02_01.png', 'A_02_02.png', 'A_02_03.png', 'A_02_04.png',
'A_03_01.png', 'A_03_02.png']
[INFO] Exemple de chemins complets (3 premiers) :
   - /content/drive/MyDrive/Data/Grains/A_01_01.png
   - /content/drive/MyDrive/Data/Grains/A_01_02.png
   - /content/drive/MyDrive/Data/Grains/A_01_03.png

[INFO] Image test de référence : /content/drive/MyDrive/Data/Grains/A_01_01.png
Image test shape : (300, 400) | dtype : uint8
Taille utilisée pour les calculs (référence) : 300 x 400
row_length (microns) : 177.77777777777777
col_length (microns) : 133.33333333333331

[INFO] Paramètres de référence enregistrés :
   IMG_SHAPE_REF        = (300, 400)
   ROW_LENGTH_MICRONS   = 177.77777777777777
   COL_LENGTH_MICRONS   = 133.33333333333331
```

Ce bloc commence par construire la **liste complète des micrographies** à partir du dossier `IMG_DIR`. Les noms de fichiers sont triés, le nombre total d'images est affiché, et quelques exemples sont listés pour vérifier visuellement que le dataset est correctement chargé. Une protection est ajoutée : si aucune image `.png` n'est trouvée, une erreur explicite est levée afin de signaler immédiatement un problème de configuration des données.

Les chemins complets vers chaque image sont ensuite stockés dans `grain_image_paths`. Cette liste servira plus tard pour parcourir de manière systématique l'ensemble du jeu de données, sans avoir à reconstruire les chemins à chaque traitement.

Le bloc définit également une fonction réutilisable `manual_threshold_and_blur`. Cette fonction applique d'abord un **seuillage binaire manuel** sur une image en niveaux de gris, puis enchaîne plusieurs **flous médians** successifs. L'objectif est de reproduire une même chaîne de prétraitement pour toutes les images, en éliminant le bruit tout en conservant les contours des grains.

Une image de référence est ensuite choisie (la première de la liste). Elle est chargée en niveaux de gris, et ses dimensions sont affichées pour documenter la taille caractéristique d'une micrographie (par exemple 300 × 400 pixels). La fonction de prétraitement est appliquée à cette image de référence, puis les dimensions finales de l'image binaire/floutée sont utilisées pour calculer les **longueurs physiques** des lignes d'interception :
- `row_length` correspond à la longueur d'une ligne horizontale en microns ;
- `col_length` correspond à la longueur d'une ligne verticale en microns.

Enfin, ces informations de référence sont mémorisées dans des variables globales (`IMG_SHAPE_REF`, `ROW_LENGTH_MICRONS`, `COL_LENGTH_MICRONS`). Elles serviront dans tout le reste du pipeline pour convertir les comptages d'interceptions en tailles de grains exprimées en microns, en garantissant que les calculs reposent toujours sur la même géométrie d'image de référence.

[ ]:
```python
#  Étapes 3 & 4 - Différences + visualisation (image de référence)

# On utilise l'image prétraitée de référence déjà calculée : Blur_Med_4_img
# (issue du bloc précédent avec manual_threshold_and_blur)

diff_x = np.diff(Blur_Med_4_img, axis=1)  # différences horizontales
diff_y = np.diff(Blur_Med_4_img, axis=0)  # différences verticales

print("[INFO] diff_x shape :", diff_x.shape)
print("[INFO] diff_y shape :", diff_y.shape)
```

```
[INFO] diff_x shape : (300, 399)
[INFO] diff_y shape : (299, 400)
```

Ce bloc calcule deux cartes de différences à partir de l'image de référence prétraitée (`Blur_Med_4_img`) : `diff_x` pour les variations horizontales et `diff_y` pour les variations verticales. L'utilisation de `np.diff` met en évidence les transitions entre le fond et les grains, c'est-à-dire les changements brusques de niveau de gris entre pixels voisins. Les dimensions affichées confirment simplement que le calcul des différences réduit d'une unité la taille dans la direction considérée. Ces cartes de différences serviront ensuite de base pour le **comptage des interceptions** le long des lignes horizontales et verticales La figure affiche l'image de référence après le prétraitement par seuillage binaire et flous médians. Les grains apparaissent en zones blanches compactes, bien séparées du fond noir. Cette visualisation confirme que le prétraitement supprime une grande partie du bruit tout en conservant correctement les contours des grains, ce qui prépare l'image pour les calculs ultérieurs d'interceptions et de mesures géométriques..

Méthode Line Intercept avec seuil manuel

Cette section met en place la **méthode des interceptions linéaires** sur l'image de référence prétraitée. L'idée est de compter combien de fois les contours des grains coupent des lignes horizontales et verticales, afin de relier ces comptages à une taille moyenne de grain. On commence ici par un bloc d'expérimentation sur une seule image pour valider la logique du comptage avant de la généraliser à tout le jeu de données.

[ ]:
```
#  Étape 5 - Comptage d'interceptions sur l'image de référence (PRACTICE)

value_intercept = 255  # valeur de transition à compter

Total_Grains_X_practice = np.zeros(numrows, dtype=np.float32)
Total_Grains_Y_practice = np.zeros(numcols, dtype=np.float32)

# Comptage horizontal
for x in range(numrows):
    count = 0
    for element in diff_x[x, :]:
        if element == value_intercept:
            count += 1
    Total_Grains_X_practice[x] = count

# Comptage vertical
for y in range(numcols - 1):
    count = 0
    for element in diff_y[:, y]:
        if element == value_intercept:
            count += 1
    Total_Grains_Y_practice[y] = count

print("[PRACTICE] Total_Grains_X (extrait 20 premières lignes) :")
print(Total_Grains_X_practice[:20])
print("[PRACTICE] Total_Grains_Y (extrait 20 premières colonnes) :")
print(Total_Grains_Y_practice[:20])
print("Shape X (practice) :", Total_Grains_X_practice.shape,
      "| Shape Y (practice) :", Total_Grains_Y_practice.shape)
```

```
[PRACTICE] Total_Grains_X (extrait 20 premières lignes) :
[4. 4. 4. 4. 4. 4. 5. 5. 5. 5. 5. 5. 5. 5. 5. 5. 5. 5. 5. 5.]
[PRACTICE] Total_Grains_Y (extrait 20 premières colonnes) :
[2. 2. 2. 2. 2. 2. 2. 3. 3. 4. 4. 4. 5. 5. 6. 7. 6. 6. 6. 6.]
Shape X (practice) : (300,) | Shape Y (practice) : (400,)
```

Ce bloc calcule, pour chaque ligne horizontale et chaque colonne verticale de l'image de référence, le **nombre d'interceptions** entre les grains et les lignes d'exploration. À partir des cartes de différences `diff_x` (horizontale) et `diff_y` (verticale), le code compte le nombre de transitions correspondant à la valeur choisie (`value_intercept = 255`) et stocke ces comptages dans deux vecteurs :

- `Total_Grains_X_practice` pour les lignes horizontales ;
- `Total_Grains_Y_practice` pour les colonnes verticales.

Les impressions montrent, pour les 20 premières lignes et colonnes, des valeurs typiquement comprises entre 2 et 7 interceptions, ainsi que les dimensions complètes des vecteurs (`300` lignes, `400` colonnes). Ce bloc sert donc de **validation pédagogique** : il confirme que la méthode Line Intercept repère bien les coupures des grains sur l'image de référence avant de l'étendre à l'ensemble des micrographies.

[ ]:
```
#  Étapes 6 à 11 - Méthode des intercepts sur TOUTES les images de Grains

# 1 Sélection du nombre d'images utilisées
#    Ici on prend toutes les images disponibles, mais tu peux
#    fixer max_images_line_intercept à une valeur plus petite si nécessaire.
max_images_line_intercept = len(image_files)  # utiliser toutes les images
n = min(max_images_line_intercept, len(image_files))

print(f"[INFO] Nombre total d'images disponibles : {len(image_files)}")
print(f"[INFO] Nombre d'images utilisées pour la méthode des intercepts : {n}")

# 2 Initialisation des matrices 2D : une colonne par image
Total_Grains_X = np.zeros((numrows, n), dtype=np.float32)
Total_Grains_Y = np.zeros((numcols, n), dtype=np.float32)

print("Dimensions Total_Grains_X :", Total_Grains_X.shape)
print("Dimensions Total_Grains_Y :", Total_Grains_Y.shape)

value_intercept = 255      # valeur de transition à compter
N_LINES_GRID    = 20       # nombre de lignes horizontales / verticales pour le␣
 ↪maillage

# 3 Boucle sur toutes les images sélectionnées
for t in range(n):

    fname = image_files[t]
    img_path = IMG_DIR / fname
    print(f"[{t+1}/{n}] Traitement image :", fname)

    img_gray = cv2.imread(str(img_path), cv2.IMREAD_GRAYSCALE)
    if img_gray is None:
        raise FileNotFoundError(f"Impossible de lire l'image : {img_path}")

    # Vérification de la taille : on impose la même taille que l'image de␣
 ↪référence
    if img_gray.shape != IMG_SHAPE_REF:
        raise ValueError(
            f"Image {fname} de taille {img_gray.shape}, "
            f"différente de la taille de référence {IMG_SHAPE_REF}"
```

```
        )

    #  Pipeline de prétraitement identique à l'étape de référence
    Blur_Med_4_img_t = manual_threshold_and_blur(img_gray, thresh_value=135)

    #  Maillage : 20 lignes horizontales + 20 verticales
    black = np.zeros_like(Blur_Med_4_img_t)

    # Lignes horizontales
    for i in range(N_LINES_GRID):
        y = int(i * img_gray.shape[0] / N_LINES_GRID)
        cv2.line(black, (0, y), (img_gray.shape[1], y), (255, 255, 255), 1)

    # Lignes verticales
    for i in range(N_LINES_GRID):
        x = int(i * img_gray.shape[1] / N_LINES_GRID)
        cv2.line(black, (x, 0), (x, img_gray.shape[0]), (255, 255, 255), 1)

    #  Intersection lignes × image prétraitée
    result = cv2.bitwise_and(black, Blur_Med_4_img_t)

    diff_x_t = np.diff(result, axis=1)
    diff_y_t = np.diff(result, axis=0)

    #  Comptage des interceptions sur cette image
    for x in range(numrows):
        count = 0
        for element in diff_x_t[x, :]:
            if element == value_intercept:
                count += 1
        Total_Grains_X[x, t] = count

    for y in range(numcols - 1):
        count = 0
        for element in diff_y_t[:, y]:
            if element == value_intercept:
                count += 1
        Total_Grains_Y[y, t] = count

# 4  Visualisation du dernier maillage (contrôle visuel)
plt.imshow(black, cmap='gray')
plt.title("Maillage (lignes d'intercepts) sur la dernière image traitée")
plt.axis('off')
plt.show()

# 5  Remplacement des zéros pour éviter les divisions par 0
Total_Grains_X[Total_Grains_X == 0] = 1
```

```
Total_Grains_Y[Total_Grains_Y == 0] = 1

# 6 Tailles moyennes des grains (méthode des intercepts)
AVG_GrainX_ManThresh = ROW_LENGTH_MICRONS / Total_Grains_X
AVG_GrainY_ManThresh = COL_LENGTH_MICRONS / Total_Grains_Y

print("AVG_GrainX_ManThresh shape :", AVG_GrainX_ManThresh.shape)
print("AVG_GrainY_ManThresh shape :", AVG_GrainY_ManThresh.shape)

# 7 Sauvegarde des CSV dans ton dossier de projet sur Drive
out_TGX = BASE_DIR / "Total_Grains_X_ManThresh.csv"
out_TGY = BASE_DIR / "Total_Grains_Y_ManThresh.csv"
out_AX  = BASE_DIR / "AVG_GrainX_ManThresh.csv"
out_AY  = BASE_DIR / "AVG_GrainY_ManThresh.csv"

np.savetxt(str(out_TGX), Total_Grains_X, delimiter=",")
np.savetxt(str(out_TGY), Total_Grains_Y, delimiter=",")
np.savetxt(str(out_AX),  AVG_GrainX_ManThresh, delimiter=",")
np.savetxt(str(out_AY),  AVG_GrainY_ManThresh, delimiter=",")

print("[SAVE] CSV sauvegardés dans :")
print("   ", out_TGX)
print("   ", out_TGY)
print("   ", out_AX)
print("   ", out_AY)

# 8 Statistiques globales (moyenne & écart-type)
meanX = np.mean(AVG_GrainX_ManThresh)
meanY = np.mean(AVG_GrainY_ManThresh)
STDX  = np.std(AVG_GrainX_ManThresh)
STDY  = np.std(AVG_GrainY_ManThresh)

print("\n[STATS GLOBALES - Méthode des intercepts / seuil manuel]")
print("meanX (µm) :", meanX)
print("meanY (µm) :", meanY)
print("STDX  (µm) :", STDX)
print("STDY  (µm) :", STDY)
```

```
[INFO] Nombre total d'images disponibles : 336
[INFO] Nombre d'images utilisées pour la méthode des intercepts : 336
Dimensions Total_Grains_X : (300, 336)
Dimensions Total_Grains_Y : (400, 336)
[1/336] Traitement image : A_01_01.png
[2/336] Traitement image : A_01_02.png
[3/336] Traitement image : A_01_03.png
[4/336] Traitement image : A_01_04.png
[5/336] Traitement image : A_02_01.png
[6/336] Traitement image : A_02_02.png
```

```
[7/336] Traitement image : A_02_03.png
[8/336] Traitement image : A_02_04.png
[9/336] Traitement image : A_03_01.png
[10/336] Traitement image : A_03_02.png
[11/336] Traitement image : A_03_03.png
[12/336] Traitement image : A_03_04.png
[13/336] Traitement image : A_04_01.png
[14/336] Traitement image : A_04_02.png
[15/336] Traitement image : A_04_03.png
[16/336] Traitement image : A_04_04.png
[17/336] Traitement image : B_01_01.png
[18/336] Traitement image : B_01_02.png
[19/336] Traitement image : B_01_03.png
[20/336] Traitement image : B_01_04.png
[21/336] Traitement image : B_02_01.png
[22/336] Traitement image : B_02_02.png
[23/336] Traitement image : B_02_03.png
[24/336] Traitement image : B_02_04.png
[25/336] Traitement image : B_03_01.png
[26/336] Traitement image : B_03_02.png
[27/336] Traitement image : B_03_03.png
[28/336] Traitement image : B_03_04.png
[29/336] Traitement image : B_04_01.png
[30/336] Traitement image : B_04_02.png
[31/336] Traitement image : B_04_03.png
[32/336] Traitement image : B_04_04.png
[33/336] Traitement image : C_01_01.png
[34/336] Traitement image : C_01_02.png
[35/336] Traitement image : C_01_03.png
[36/336] Traitement image : C_01_04.png
[37/336] Traitement image : C_02_01.png
[38/336] Traitement image : C_02_02.png
[39/336] Traitement image : C_02_03.png
[40/336] Traitement image : C_02_04.png
[41/336] Traitement image : C_03_01.png
[42/336] Traitement image : C_03_02.png
[43/336] Traitement image : C_03_03.png
[44/336] Traitement image : C_03_04.png
[45/336] Traitement image : C_04_01.png
[46/336] Traitement image : C_04_02.png
[47/336] Traitement image : C_04_03.png
[48/336] Traitement image : C_04_04.png
[49/336] Traitement image : D_01_01.png
[50/336] Traitement image : D_01_02.png
[51/336] Traitement image : D_01_03.png
[52/336] Traitement image : D_01_04.png
[53/336] Traitement image : D_02_01.png
[54/336] Traitement image : D_02_02.png
```

```
[55/336] Traitement image : D_02_03.png
[56/336] Traitement image : D_02_04.png
[57/336] Traitement image : D_03_01.png
[58/336] Traitement image : D_03_02.png
[59/336] Traitement image : D_03_03.png
[60/336] Traitement image : D_03_04.png
[61/336] Traitement image : D_04_01.png
[62/336] Traitement image : D_04_02.png
[63/336] Traitement image : D_04_03.png
[64/336] Traitement image : D_04_04.png
[65/336] Traitement image : E_01_01.png
[66/336] Traitement image : E_01_02.png
[67/336] Traitement image : E_01_03.png
[68/336] Traitement image : E_01_04.png
[69/336] Traitement image : E_02_01.png
[70/336] Traitement image : E_02_02.png
[71/336] Traitement image : E_02_03.png
[72/336] Traitement image : E_02_04.png
[73/336] Traitement image : E_03_01.png
[74/336] Traitement image : E_03_02.png
[75/336] Traitement image : E_03_03.png
[76/336] Traitement image : E_03_04.png
[77/336] Traitement image : E_04_01.png
[78/336] Traitement image : E_04_02.png
[79/336] Traitement image : E_04_03.png
[80/336] Traitement image : E_04_04.png
[81/336] Traitement image : F_01_01.png
[82/336] Traitement image : F_01_02.png
[83/336] Traitement image : F_01_03.png
[84/336] Traitement image : F_01_04.png
[85/336] Traitement image : F_02_01.png
[86/336] Traitement image : F_02_02.png
[87/336] Traitement image : F_02_03.png
[88/336] Traitement image : F_02_04.png
[89/336] Traitement image : F_03_01.png
[90/336] Traitement image : F_03_02.png
[91/336] Traitement image : F_03_03.png
[92/336] Traitement image : F_03_04.png
[93/336] Traitement image : F_04_01.png
[94/336] Traitement image : F_04_02.png
[95/336] Traitement image : F_04_03.png
[96/336] Traitement image : F_04_04.png
[97/336] Traitement image : G_01_01.png
[98/336] Traitement image : G_01_02.png
[99/336] Traitement image : G_01_03.png
[100/336] Traitement image : G_01_04.png
[101/336] Traitement image : G_02_01.png
[102/336] Traitement image : G_02_02.png
```

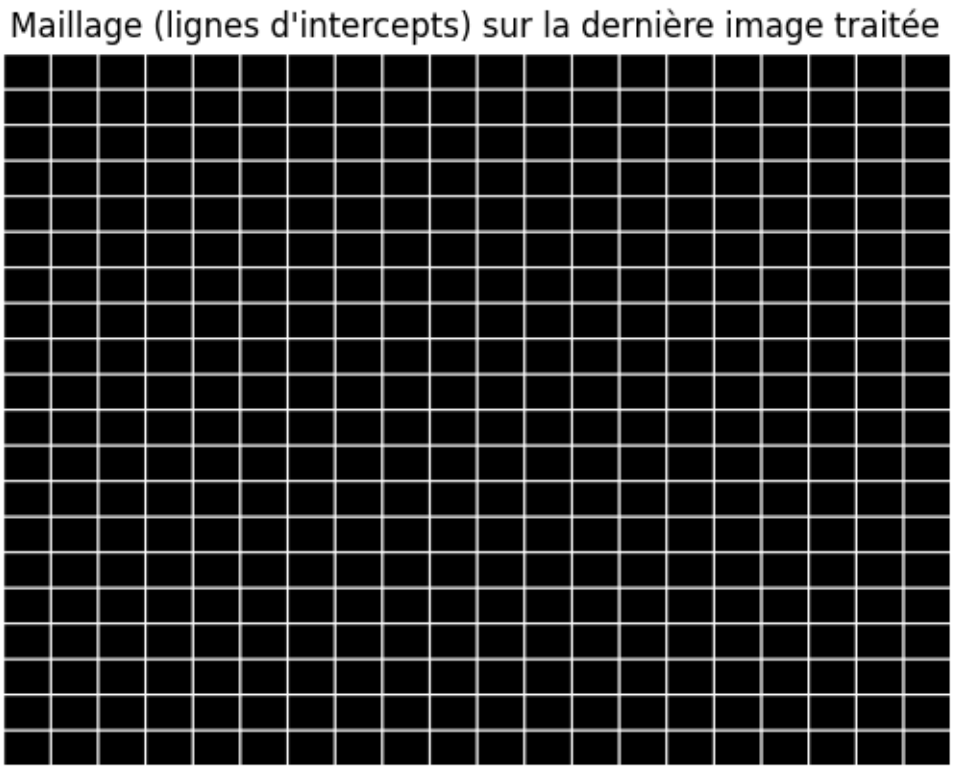
```
[103/336] Traitement image : G_02_03.png
[104/336] Traitement image : G_02_04.png
[105/336] Traitement image : G_03_01.png
[106/336] Traitement image : G_03_02.png
[107/336] Traitement image : G_03_03.png
[108/336] Traitement image : G_03_04.png
[109/336] Traitement image : G_04_01.png
[110/336] Traitement image : G_04_02.png
[111/336] Traitement image : G_04_03.png
[112/336] Traitement image : G_04_04.png
[113/336] Traitement image : H_01_01.png
[114/336] Traitement image : H_01_02.png
[115/336] Traitement image : H_01_03.png
[116/336] Traitement image : H_01_04.png
[117/336] Traitement image : H_02_01.png
[118/336] Traitement image : H_02_02.png
[119/336] Traitement image : H_02_03.png
[120/336] Traitement image : H_02_04.png
[121/336] Traitement image : H_03_01.png
[122/336] Traitement image : H_03_02.png
[123/336] Traitement image : H_03_03.png
[124/336] Traitement image : H_03_04.png
[125/336] Traitement image : H_04_01.png
[126/336] Traitement image : H_04_02.png
[127/336] Traitement image : H_04_03.png
[128/336] Traitement image : H_04_04.png
[129/336] Traitement image : I_01_01.png
[130/336] Traitement image : I_01_02.png
[131/336] Traitement image : I_01_03.png
[132/336] Traitement image : I_01_04.png
[133/336] Traitement image : I_02_01.png
[134/336] Traitement image : I_02_02.png
[135/336] Traitement image : I_02_03.png
[136/336] Traitement image : I_02_04.png
[137/336] Traitement image : I_03_01.png
[138/336] Traitement image : I_03_02.png
[139/336] Traitement image : I_03_03.png
[140/336] Traitement image : I_03_04.png
[141/336] Traitement image : I_04_01.png
[142/336] Traitement image : I_04_02.png
[143/336] Traitement image : I_04_03.png
[144/336] Traitement image : I_04_04.png
[145/336] Traitement image : J_01_01.png
[146/336] Traitement image : J_01_02.png
[147/336] Traitement image : J_01_03.png
[148/336] Traitement image : J_01_04.png
[149/336] Traitement image : J_02_01.png
[150/336] Traitement image : J_02_02.png
```

```
[151/336] Traitement image : J_02_03.png
[152/336] Traitement image : J_02_04.png
[153/336] Traitement image : J_03_01.png
[154/336] Traitement image : J_03_02.png
[155/336] Traitement image : J_03_03.png
[156/336] Traitement image : J_03_04.png
[157/336] Traitement image : J_04_01.png
[158/336] Traitement image : J_04_02.png
[159/336] Traitement image : J_04_03.png
[160/336] Traitement image : J_04_04.png
[161/336] Traitement image : K_01_01.png
[162/336] Traitement image : K_01_02.png
[163/336] Traitement image : K_01_03.png
[164/336] Traitement image : K_01_04.png
[165/336] Traitement image : K_02_01.png
[166/336] Traitement image : K_02_02.png
[167/336] Traitement image : K_02_03.png
[168/336] Traitement image : K_02_04.png
[169/336] Traitement image : K_03_01.png
[170/336] Traitement image : K_03_02.png
[171/336] Traitement image : K_03_03.png
[172/336] Traitement image : K_03_04.png
[173/336] Traitement image : K_04_01.png
[174/336] Traitement image : K_04_02.png
[175/336] Traitement image : K_04_03.png
[176/336] Traitement image : K_04_04.png
[177/336] Traitement image : L_01_01.png
[178/336] Traitement image : L_01_02.png
[179/336] Traitement image : L_01_03.png
[180/336] Traitement image : L_01_04.png
[181/336] Traitement image : L_02_01.png
[182/336] Traitement image : L_02_02.png
[183/336] Traitement image : L_02_03.png
[184/336] Traitement image : L_02_04.png
[185/336] Traitement image : L_03_01.png
[186/336] Traitement image : L_03_02.png
[187/336] Traitement image : L_03_03.png
[188/336] Traitement image : L_03_04.png
[189/336] Traitement image : L_04_01.png
[190/336] Traitement image : L_04_02.png
[191/336] Traitement image : L_04_03.png
[192/336] Traitement image : L_04_04.png
[193/336] Traitement image : M_01_01.png
[194/336] Traitement image : M_01_02.png
[195/336] Traitement image : M_01_03.png
[196/336] Traitement image : M_01_04.png
[197/336] Traitement image : M_02_01.png
[198/336] Traitement image : M_02_02.png
```

```
[199/336] Traitement image : M_02_03.png
[200/336] Traitement image : M_02_04.png
[201/336] Traitement image : M_03_01.png
[202/336] Traitement image : M_03_02.png
[203/336] Traitement image : M_03_03.png
[204/336] Traitement image : M_03_04.png
[205/336] Traitement image : M_04_01.png
[206/336] Traitement image : M_04_02.png
[207/336] Traitement image : M_04_03.png
[208/336] Traitement image : M_04_04.png
[209/336] Traitement image : N_01_01.png
[210/336] Traitement image : N_01_02.png
[211/336] Traitement image : N_01_03.png
[212/336] Traitement image : N_01_04.png
[213/336] Traitement image : N_02_01.png
[214/336] Traitement image : N_02_02.png
[215/336] Traitement image : N_02_03.png
[216/336] Traitement image : N_02_04.png
[217/336] Traitement image : N_03_01.png
[218/336] Traitement image : N_03_02.png
[219/336] Traitement image : N_03_03.png
[220/336] Traitement image : N_03_04.png
[221/336] Traitement image : N_04_01.png
[222/336] Traitement image : N_04_02.png
[223/336] Traitement image : N_04_03.png
[224/336] Traitement image : N_04_04.png
[225/336] Traitement image : O_01_01.png
[226/336] Traitement image : O_01_02.png
[227/336] Traitement image : O_01_03.png
[228/336] Traitement image : O_01_04.png
[229/336] Traitement image : O_02_01.png
[230/336] Traitement image : O_02_02.png
[231/336] Traitement image : O_02_03.png
[232/336] Traitement image : O_02_04.png
[233/336] Traitement image : O_03_01.png
[234/336] Traitement image : O_03_02.png
[235/336] Traitement image : O_03_03.png
[236/336] Traitement image : O_03_04.png
[237/336] Traitement image : O_04_01.png
[238/336] Traitement image : O_04_02.png
[239/336] Traitement image : O_04_03.png
[240/336] Traitement image : O_04_04.png
[241/336] Traitement image : P_01_01.png
[242/336] Traitement image : P_01_02.png
[243/336] Traitement image : P_01_03.png
[244/336] Traitement image : P_01_04.png
[245/336] Traitement image : P_02_01.png
[246/336] Traitement image : P_02_02.png
```

```
[247/336] Traitement image : P_02_03.png
[248/336] Traitement image : P_02_04.png
[249/336] Traitement image : P_03_01.png
[250/336] Traitement image : P_03_02.png
[251/336] Traitement image : P_03_03.png
[252/336] Traitement image : P_03_04.png
[253/336] Traitement image : P_04_01.png
[254/336] Traitement image : P_04_02.png
[255/336] Traitement image : P_04_03.png
[256/336] Traitement image : P_04_04.png
[257/336] Traitement image : Q_01_01.png
[258/336] Traitement image : Q_01_02.png
[259/336] Traitement image : Q_01_03.png
[260/336] Traitement image : Q_01_04.png
[261/336] Traitement image : Q_02_01.png
[262/336] Traitement image : Q_02_02.png
[263/336] Traitement image : Q_02_03.png
[264/336] Traitement image : Q_02_04.png
[265/336] Traitement image : Q_03_01.png
[266/336] Traitement image : Q_03_02.png
[267/336] Traitement image : Q_03_03.png
[268/336] Traitement image : Q_03_04.png
[269/336] Traitement image : Q_04_01.png
[270/336] Traitement image : Q_04_02.png
[271/336] Traitement image : Q_04_03.png
[272/336] Traitement image : Q_04_04.png
[273/336] Traitement image : R_01_01.png
[274/336] Traitement image : R_01_02.png
[275/336] Traitement image : R_01_03.png
[276/336] Traitement image : R_01_04.png
[277/336] Traitement image : R_02_01.png
[278/336] Traitement image : R_02_02.png
[279/336] Traitement image : R_02_03.png
[280/336] Traitement image : R_02_04.png
[281/336] Traitement image : R_03_01.png
[282/336] Traitement image : R_03_02.png
[283/336] Traitement image : R_03_03.png
[284/336] Traitement image : R_03_04.png
[285/336] Traitement image : R_04_01.png
[286/336] Traitement image : R_04_02.png
[287/336] Traitement image : R_04_03.png
[288/336] Traitement image : R_04_04.png
[289/336] Traitement image : S_01_01.png
[290/336] Traitement image : S_01_02.png
[291/336] Traitement image : S_01_03.png
[292/336] Traitement image : S_01_04.png
[293/336] Traitement image : S_02_01.png
[294/336] Traitement image : S_02_02.png
```

```
[295/336] Traitement image : S_02_03.png
[296/336] Traitement image : S_02_04.png
[297/336] Traitement image : S_03_01.png
[298/336] Traitement image : S_03_02.png
[299/336] Traitement image : S_03_03.png
[300/336] Traitement image : S_03_04.png
[301/336] Traitement image : S_04_01.png
[302/336] Traitement image : S_04_02.png
[303/336] Traitement image : S_04_03.png
[304/336] Traitement image : S_04_04.png
[305/336] Traitement image : T_01_01.png
[306/336] Traitement image : T_01_02.png
[307/336] Traitement image : T_01_03.png
[308/336] Traitement image : T_01_04.png
[309/336] Traitement image : T_02_01.png
[310/336] Traitement image : T_02_02.png
[311/336] Traitement image : T_02_03.png
[312/336] Traitement image : T_02_04.png
[313/336] Traitement image : T_03_01.png
[314/336] Traitement image : T_03_02.png
[315/336] Traitement image : T_03_03.png
[316/336] Traitement image : T_03_04.png
[317/336] Traitement image : T_04_01.png
[318/336] Traitement image : T_04_02.png
[319/336] Traitement image : T_04_03.png
[320/336] Traitement image : T_04_04.png
[321/336] Traitement image : U_01_01.png
[322/336] Traitement image : U_01_02.png
[323/336] Traitement image : U_01_03.png
[324/336] Traitement image : U_01_04.png
[325/336] Traitement image : U_02_01.png
[326/336] Traitement image : U_02_02.png
[327/336] Traitement image : U_02_03.png
[328/336] Traitement image : U_02_04.png
[329/336] Traitement image : U_03_01.png
[330/336] Traitement image : U_03_02.png
[331/336] Traitement image : U_03_03.png
[332/336] Traitement image : U_03_04.png
[333/336] Traitement image : U_04_01.png
[334/336] Traitement image : U_04_02.png
[335/336] Traitement image : U_04_03.png
[336/336] Traitement image : U_04_04.png
```




```
AVG_GrainX_ManThresh shape : (300, 336)
AVG_GrainY_ManThresh shape : (400, 336)
[SAVE] CSV sauvegardés dans :
    /content/drive/MyDrive/Data/Total_Grains_X_ManThresh.csv
    /content/drive/MyDrive/Data/Total_Grains_Y_ManThresh.csv
    /content/drive/MyDrive/Data/AVG_GrainX_ManThresh.csv
    /content/drive/MyDrive/Data/AVG_GrainY_ManThresh.csv

[STATS GLOBALES - Méthode des intercepts / seuil manuel]
meanX (µm) : 18.469652
meanY (µm) : 15.702522
STDX  (µm) : 9.128215
STDY  (µm) : 14.4650955
```

Ce bloc applique la méthode des interceptions linéaires à l'ensemble des 336 micrographies de grains : pour chaque image, il vérifie la taille, applique le même prétraitement que sur l'image de référence (seuil manuel puis flous médians), superpose un maillage régulier de 20 lignes horizontales et 20 lignes verticales, calcule les cartes de différences sur l'intersection maillage × image prétraitée, puis compte le nombre d'interceptions par ligne et par colonne, ce qui remplit les matrices 2D `Total_Grains_X` (300 × 336) et `Total_Grains_Y` (400 × 336) ; après remplacement des zéros pour éviter les divisions par zéro, les tailles moyennes de grains sont obtenues par direction en divisant les longueurs physiques de référence par les comptages (`AVG_GrainX_ManThresh` et `AVG_GrainY_ManThresh`), ces matrices ainsi que les comptages bruts sont sauvegardés dans quatre

fichiers CSV dans le dossier du projet, et les statistiques globales montrent des tailles moyennes d'environ 18,47 µm (horizontalement) et 15,70 µm (verticalement), avec des écarts-types respectifs d'environ 9,13 µm et 14,47 µm, fournissant ainsi une première estimation quantitative de la taille des grains par la méthode des intercepts avec seuil manuel sur tout le jeu de données.

Méthode Gradient / Canny(version v1)

Dans cette section, la méthode des interceptions linéaires est réappliquée en remplaçant le seuillage manuel par une détection de contours basée sur le gradient (Canny). L'objectif est d'obtenir une estimation de la taille des grains plus robuste aux variations d'éclairage et de contraste, en s'appuyant sur les frontières détectées automatiquement plutôt que sur un seuil fixe.

[ ]:
```
#  Étape 12 - Initialisation de la méthode Gradient (Canny)
#    VERSION COLAB + DRIVE + TOUTES LES IMAGES

VALUE_INTERCEPT = 255  # valeur à compter (binaire)
MAX_IMAGES_GRAD = len(image_files)  # on dispose de toutes les images de Grains
n_grad = min(MAX_IMAGES_GRAD, len(image_files))  # par défaut = toutes

print(f"[GRAD] Nombre total d'images disponibles : {len(image_files)}")
print(f"[GRAD] Nombre d'images utilisées pour la méthode Gradient : {n_grad}")

# Matrices de stockage pour la méthode Gradient (Canny)
Total_Grains_X_grad = np.zeros((numrows, n_grad), dtype=np.float32)
Total_Grains_Y_grad = np.zeros((numcols, n_grad), dtype=np.float32)

print("Dimensions Total_Grains_X_grad :", Total_Grains_X_grad.shape)
print("Dimensions Total_Grains_Y_grad :", Total_Grains_Y_grad.shape)
print("Nombre d'images (n_grad) :", n_grad)
```

```
[GRAD] Nombre total d'images disponibles : 336
[GRAD] Nombre d'images utilisées pour la méthode Gradient : 336
Dimensions Total_Grains_X_grad : (300, 336)
Dimensions Total_Grains_Y_grad : (400, 336)
Nombre d'images (n_grad) : 336
```

Ce bloc prépare la version Gradient de la méthode en fixant la valeur binaire à compter (`VALUE_INTERCEPT = 255`), en choisissant le nombre d'images à traiter (ici les 336 micrographies disponibles) et en initialisant deux matrices de stockage pour les comptages d'interceptions : `Total_Grains_X_grad` de dimension 300 × 336 pour les lignes horizontales et `Total_Grains_Y_grad` de dimension 400 × 336 pour les lignes verticales. Les messages affichés confirment que toutes les images du jeu de données seront utilisées et que les structures de données nécessaires au remplissage des comptages Gradient sont correctement dimensionnées.

[ ]:
```
#  Étape 13 - Test Canny/Gradient sur une image (contrôle visuel)

test_grad_fname = image_files[0]
test_grad_path = IMG_DIR / test_grad_fname
```

```python
print("[GRAD] Image test (Gradient) :", test_grad_path)

img_grad_test = cv2.imread(str(test_grad_path), cv2.IMREAD_GRAYSCALE)
if img_grad_test is None:
    raise FileNotFoundError(f"Impossible de lire l'image : {test_grad_path}")

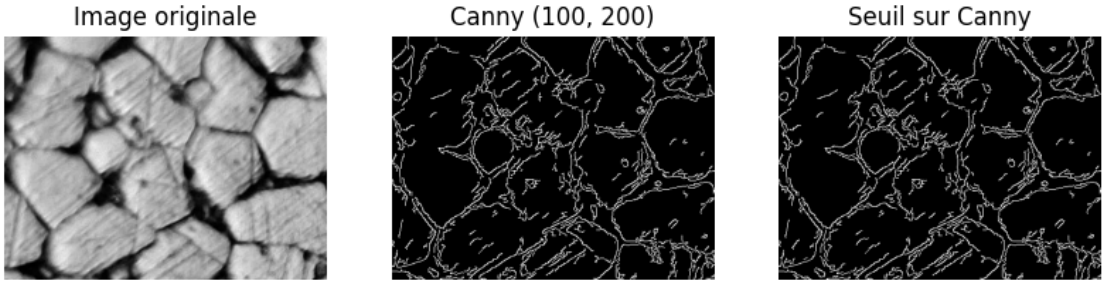
print("[GRAD] Shape image test Gradient :", img_grad_test.shape)
if img_grad_test.shape != IMG_SHAPE_REF:
    raise ValueError(
        f"Image test Gradient de taille {img_grad_test.shape}, "
        f"différente de la taille de référence {IMG_SHAPE_REF}"
    )

# 1) Détection de contours par Canny
canny_test = cv2.Canny(img_grad_test, 100, 200)

# 2) Seuil sur l'image Canny
_, threshCAN_test = cv2.threshold(canny_test, 90, 255, cv2.THRESH_BINARY)

plt.figure(figsize=(10, 10))
plt.subplot(1, 3, 1); plt.imshow(img_grad_test,  cmap="gray"); plt.title("Image␣
 ↪originale");  plt.axis("off")
plt.subplot(1, 3, 2); plt.imshow(canny_test,    cmap="gray"); plt.title("Canny␣
 ↪(100, 200)"); plt.axis("off")
plt.subplot(1, 3, 3); plt.imshow(threshCAN_test, cmap="gray"); plt.title("Seuil␣
 ↪sur Canny");  plt.axis("off")
plt.show()
```

```
[GRAD] Image test (Gradient) : /content/drive/MyDrive/Data/Grains/A_01_01.png
[GRAD] Shape image test Gradient : (300, 400)
```



Ce bloc illustre, sur une image test, le fonctionnement de la méthode Gradient : l'image originale est d'abord chargée et vérifiée (même taille que la référence), puis le détecteur de contours de Canny est appliqué avec les seuils (100, 200), ce qui fait ressortir les frontières des grains sous forme de traits fins ; un seuillage supplémentaire est ensuite appliqué sur la sortie de Canny pour obtenir une image binaire claire des contours, et les trois vues (originale, Canny, seuil sur Canny) permettent

de vérifier visuellement que les bords des grains sont correctement capturés avant de passer au comptage systématique des interceptions sur tout le jeu de données.

[ ]:
```
#  Étape 14 - Boucle Line Intercept (méthode Gradient/Canny)
#     VERSION COLAB + DRIVE + TOUTES LES IMAGES

N_LINES_GRID = 20  # même maillage que pour la méthode seuil manuel

for t, fname in enumerate(image_files[:n_grad]):

    img_path = IMG_DIR / fname
    print(f"[GRAD] Image {t+1}/{n_grad} :", img_path.name)

    # Lecture en niveaux de gris
    img = cv2.imread(str(img_path), cv2.IMREAD_GRAYSCALE)
    if img is None:
        raise FileNotFoundError(f"Impossible de lire l'image : {img_path}")

    # Vérification taille cohérente
    if img.shape != IMG_SHAPE_REF:
        raise ValueError(
            f"[GRAD] Image {fname} de taille {img.shape}, "
            f"différente de la taille de référence {IMG_SHAPE_REF}"
        )

    # 1) Détection de contours par Canny
    canny = cv2.Canny(img, 100, 200)

    # 2) Seuil sur l'image Canny
    _, threshCAN = cv2.threshold(canny, 90, 255, cv2.THRESH_BINARY)

    # 3) Création du maillage de 20 lignes horizontales et 20 verticales
    black = np.zeros_like(threshCAN)

    # Lignes horizontales
    for i in range(N_LINES_GRID):
        y = int(i * black.shape[0] / N_LINES_GRID)
        cv2.line(black, (0, y), (black.shape[1] - 1, y), 255, 1)

    # Lignes verticales
    for i in range(N_LINES_GRID):
        x = int(i * black.shape[1] / N_LINES_GRID)
        cv2.line(black, (x, 0), (x, black.shape[0] - 1), 255, 1)

    # 4) Intersection maillage × image segmentée (Canny + seuil)
    result = cv2.bitwise_and(black, threshCAN)
```

```
    # 5) Différences horizontales et verticales
    diff_x = np.diff(result, axis=1)
    diff_y = np.diff(result, axis=0)

    # 6) Comptage des interceptions sur les lignes (X)
    for x in range(numrows):
        count = 0
        for element in diff_x[x, :]:
            if element == VALUE_INTERCEPT:
                count += 1
        Total_Grains_X_grad[x, t] = count

    # 7) Comptage des interceptions sur les colonnes (Y)
    for y in range(numcols - 1):
        count = 0
        for element in diff_y[:, y]:
            if element == VALUE_INTERCEPT:
                count += 1
        Total_Grains_Y_grad[y, t] = count

print("  [GRAD] Boucle Gradient/Canny terminée.")
```

```
[GRAD] Image 1/336 : A_01_01.png
[GRAD] Image 2/336 : A_01_02.png
[GRAD] Image 3/336 : A_01_03.png
[GRAD] Image 4/336 : A_01_04.png
[GRAD] Image 5/336 : A_02_01.png
[GRAD] Image 6/336 : A_02_02.png
[GRAD] Image 7/336 : A_02_03.png
[GRAD] Image 8/336 : A_02_04.png
[GRAD] Image 9/336 : A_03_01.png
[GRAD] Image 10/336 : A_03_02.png
[GRAD] Image 11/336 : A_03_03.png
[GRAD] Image 12/336 : A_03_04.png
[GRAD] Image 13/336 : A_04_01.png
[GRAD] Image 14/336 : A_04_02.png
[GRAD] Image 15/336 : A_04_03.png
[GRAD] Image 16/336 : A_04_04.png
[GRAD] Image 17/336 : B_01_01.png
[GRAD] Image 18/336 : B_01_02.png
[GRAD] Image 19/336 : B_01_03.png
[GRAD] Image 20/336 : B_01_04.png
[GRAD] Image 21/336 : B_02_01.png
[GRAD] Image 22/336 : B_02_02.png
[GRAD] Image 23/336 : B_02_03.png
[GRAD] Image 24/336 : B_02_04.png
[GRAD] Image 25/336 : B_03_01.png
[GRAD] Image 26/336 : B_03_02.png
```

```
[GRAD] Image 27/336 : B_03_03.png
[GRAD] Image 28/336 : B_03_04.png
[GRAD] Image 29/336 : B_04_01.png
[GRAD] Image 30/336 : B_04_02.png
[GRAD] Image 31/336 : B_04_03.png
[GRAD] Image 32/336 : B_04_04.png
[GRAD] Image 33/336 : C_01_01.png
[GRAD] Image 34/336 : C_01_02.png
[GRAD] Image 35/336 : C_01_03.png
[GRAD] Image 36/336 : C_01_04.png
[GRAD] Image 37/336 : C_02_01.png
[GRAD] Image 38/336 : C_02_02.png
[GRAD] Image 39/336 : C_02_03.png
[GRAD] Image 40/336 : C_02_04.png
[GRAD] Image 41/336 : C_03_01.png
[GRAD] Image 42/336 : C_03_02.png
[GRAD] Image 43/336 : C_03_03.png
[GRAD] Image 44/336 : C_03_04.png
[GRAD] Image 45/336 : C_04_01.png
[GRAD] Image 46/336 : C_04_02.png
[GRAD] Image 47/336 : C_04_03.png
[GRAD] Image 48/336 : C_04_04.png
[GRAD] Image 49/336 : D_01_01.png
[GRAD] Image 50/336 : D_01_02.png
[GRAD] Image 51/336 : D_01_03.png
[GRAD] Image 52/336 : D_01_04.png
[GRAD] Image 53/336 : D_02_01.png
[GRAD] Image 54/336 : D_02_02.png
[GRAD] Image 55/336 : D_02_03.png
[GRAD] Image 56/336 : D_02_04.png
[GRAD] Image 57/336 : D_03_01.png
[GRAD] Image 58/336 : D_03_02.png
[GRAD] Image 59/336 : D_03_03.png
[GRAD] Image 60/336 : D_03_04.png
[GRAD] Image 61/336 : D_04_01.png
[GRAD] Image 62/336 : D_04_02.png
[GRAD] Image 63/336 : D_04_03.png
[GRAD] Image 64/336 : D_04_04.png
[GRAD] Image 65/336 : E_01_01.png
[GRAD] Image 66/336 : E_01_02.png
[GRAD] Image 67/336 : E_01_03.png
[GRAD] Image 68/336 : E_01_04.png
[GRAD] Image 69/336 : E_02_01.png
[GRAD] Image 70/336 : E_02_02.png
[GRAD] Image 71/336 : E_02_03.png
[GRAD] Image 72/336 : E_02_04.png
[GRAD] Image 73/336 : E_03_01.png
[GRAD] Image 74/336 : E_03_02.png
```

```
[GRAD] Image 75/336 : E_03_03.png
[GRAD] Image 76/336 : E_03_04.png
[GRAD] Image 77/336 : E_04_01.png
[GRAD] Image 78/336 : E_04_02.png
[GRAD] Image 79/336 : E_04_03.png
[GRAD] Image 80/336 : E_04_04.png
[GRAD] Image 81/336 : F_01_01.png
[GRAD] Image 82/336 : F_01_02.png
[GRAD] Image 83/336 : F_01_03.png
[GRAD] Image 84/336 : F_01_04.png
[GRAD] Image 85/336 : F_02_01.png
[GRAD] Image 86/336 : F_02_02.png
[GRAD] Image 87/336 : F_02_03.png
[GRAD] Image 88/336 : F_02_04.png
[GRAD] Image 89/336 : F_03_01.png
[GRAD] Image 90/336 : F_03_02.png
[GRAD] Image 91/336 : F_03_03.png
[GRAD] Image 92/336 : F_03_04.png
[GRAD] Image 93/336 : F_04_01.png
[GRAD] Image 94/336 : F_04_02.png
[GRAD] Image 95/336 : F_04_03.png
[GRAD] Image 96/336 : F_04_04.png
[GRAD] Image 97/336 : G_01_01.png
[GRAD] Image 98/336 : G_01_02.png
[GRAD] Image 99/336 : G_01_03.png
[GRAD] Image 100/336 : G_01_04.png
[GRAD] Image 101/336 : G_02_01.png
[GRAD] Image 102/336 : G_02_02.png
[GRAD] Image 103/336 : G_02_03.png
[GRAD] Image 104/336 : G_02_04.png
[GRAD] Image 105/336 : G_03_01.png
[GRAD] Image 106/336 : G_03_02.png
[GRAD] Image 107/336 : G_03_03.png
[GRAD] Image 108/336 : G_03_04.png
[GRAD] Image 109/336 : G_04_01.png
[GRAD] Image 110/336 : G_04_02.png
[GRAD] Image 111/336 : G_04_03.png
[GRAD] Image 112/336 : G_04_04.png
[GRAD] Image 113/336 : H_01_01.png
[GRAD] Image 114/336 : H_01_02.png
[GRAD] Image 115/336 : H_01_03.png
[GRAD] Image 116/336 : H_01_04.png
[GRAD] Image 117/336 : H_02_01.png
[GRAD] Image 118/336 : H_02_02.png
[GRAD] Image 119/336 : H_02_03.png
[GRAD] Image 120/336 : H_02_04.png
[GRAD] Image 121/336 : H_03_01.png
[GRAD] Image 122/336 : H_03_02.png
```

```
[GRAD] Image 123/336 : H_03_03.png
[GRAD] Image 124/336 : H_03_04.png
[GRAD] Image 125/336 : H_04_01.png
[GRAD] Image 126/336 : H_04_02.png
[GRAD] Image 127/336 : H_04_03.png
[GRAD] Image 128/336 : H_04_04.png
[GRAD] Image 129/336 : I_01_01.png
[GRAD] Image 130/336 : I_01_02.png
[GRAD] Image 131/336 : I_01_03.png
[GRAD] Image 132/336 : I_01_04.png
[GRAD] Image 133/336 : I_02_01.png
[GRAD] Image 134/336 : I_02_02.png
[GRAD] Image 135/336 : I_02_03.png
[GRAD] Image 136/336 : I_02_04.png
[GRAD] Image 137/336 : I_03_01.png
[GRAD] Image 138/336 : I_03_02.png
[GRAD] Image 139/336 : I_03_03.png
[GRAD] Image 140/336 : I_03_04.png
[GRAD] Image 141/336 : I_04_01.png
[GRAD] Image 142/336 : I_04_02.png
[GRAD] Image 143/336 : I_04_03.png
[GRAD] Image 144/336 : I_04_04.png
[GRAD] Image 145/336 : J_01_01.png
[GRAD] Image 146/336 : J_01_02.png
[GRAD] Image 147/336 : J_01_03.png
[GRAD] Image 148/336 : J_01_04.png
[GRAD] Image 149/336 : J_02_01.png
[GRAD] Image 150/336 : J_02_02.png
[GRAD] Image 151/336 : J_02_03.png
[GRAD] Image 152/336 : J_02_04.png
[GRAD] Image 153/336 : J_03_01.png
[GRAD] Image 154/336 : J_03_02.png
[GRAD] Image 155/336 : J_03_03.png
[GRAD] Image 156/336 : J_03_04.png
[GRAD] Image 157/336 : J_04_01.png
[GRAD] Image 158/336 : J_04_02.png
[GRAD] Image 159/336 : J_04_03.png
[GRAD] Image 160/336 : J_04_04.png
[GRAD] Image 161/336 : K_01_01.png
[GRAD] Image 162/336 : K_01_02.png
[GRAD] Image 163/336 : K_01_03.png
[GRAD] Image 164/336 : K_01_04.png
[GRAD] Image 165/336 : K_02_01.png
[GRAD] Image 166/336 : K_02_02.png
[GRAD] Image 167/336 : K_02_03.png
[GRAD] Image 168/336 : K_02_04.png
[GRAD] Image 169/336 : K_03_01.png
[GRAD] Image 170/336 : K_03_02.png
```

```
[GRAD] Image 171/336 : K_03_03.png
[GRAD] Image 172/336 : K_03_04.png
[GRAD] Image 173/336 : K_04_01.png
[GRAD] Image 174/336 : K_04_02.png
[GRAD] Image 175/336 : K_04_03.png
[GRAD] Image 176/336 : K_04_04.png
[GRAD] Image 177/336 : L_01_01.png
[GRAD] Image 178/336 : L_01_02.png
[GRAD] Image 179/336 : L_01_03.png
[GRAD] Image 180/336 : L_01_04.png
[GRAD] Image 181/336 : L_02_01.png
[GRAD] Image 182/336 : L_02_02.png
[GRAD] Image 183/336 : L_02_03.png
[GRAD] Image 184/336 : L_02_04.png
[GRAD] Image 185/336 : L_03_01.png
[GRAD] Image 186/336 : L_03_02.png
[GRAD] Image 187/336 : L_03_03.png
[GRAD] Image 188/336 : L_03_04.png
[GRAD] Image 189/336 : L_04_01.png
[GRAD] Image 190/336 : L_04_02.png
[GRAD] Image 191/336 : L_04_03.png
[GRAD] Image 192/336 : L_04_04.png
[GRAD] Image 193/336 : M_01_01.png
[GRAD] Image 194/336 : M_01_02.png
[GRAD] Image 195/336 : M_01_03.png
[GRAD] Image 196/336 : M_01_04.png
[GRAD] Image 197/336 : M_02_01.png
[GRAD] Image 198/336 : M_02_02.png
[GRAD] Image 199/336 : M_02_03.png
[GRAD] Image 200/336 : M_02_04.png
[GRAD] Image 201/336 : M_03_01.png
[GRAD] Image 202/336 : M_03_02.png
[GRAD] Image 203/336 : M_03_03.png
[GRAD] Image 204/336 : M_03_04.png
[GRAD] Image 205/336 : M_04_01.png
[GRAD] Image 206/336 : M_04_02.png
[GRAD] Image 207/336 : M_04_03.png
[GRAD] Image 208/336 : M_04_04.png
[GRAD] Image 209/336 : N_01_01.png
[GRAD] Image 210/336 : N_01_02.png
[GRAD] Image 211/336 : N_01_03.png
[GRAD] Image 212/336 : N_01_04.png
[GRAD] Image 213/336 : N_02_01.png
[GRAD] Image 214/336 : N_02_02.png
[GRAD] Image 215/336 : N_02_03.png
[GRAD] Image 216/336 : N_02_04.png
[GRAD] Image 217/336 : N_03_01.png
[GRAD] Image 218/336 : N_03_02.png
```

```
[GRAD] Image 219/336 : N_03_03.png
[GRAD] Image 220/336 : N_03_04.png
[GRAD] Image 221/336 : N_04_01.png
[GRAD] Image 222/336 : N_04_02.png
[GRAD] Image 223/336 : N_04_03.png
[GRAD] Image 224/336 : N_04_04.png
[GRAD] Image 225/336 : O_01_01.png
[GRAD] Image 226/336 : O_01_02.png
[GRAD] Image 227/336 : O_01_03.png
[GRAD] Image 228/336 : O_01_04.png
[GRAD] Image 229/336 : O_02_01.png
[GRAD] Image 230/336 : O_02_02.png
[GRAD] Image 231/336 : O_02_03.png
[GRAD] Image 232/336 : O_02_04.png
[GRAD] Image 233/336 : O_03_01.png
[GRAD] Image 234/336 : O_03_02.png
[GRAD] Image 235/336 : O_03_03.png
[GRAD] Image 236/336 : O_03_04.png
[GRAD] Image 237/336 : O_04_01.png
[GRAD] Image 238/336 : O_04_02.png
[GRAD] Image 239/336 : O_04_03.png
[GRAD] Image 240/336 : O_04_04.png
[GRAD] Image 241/336 : P_01_01.png
[GRAD] Image 242/336 : P_01_02.png
[GRAD] Image 243/336 : P_01_03.png
[GRAD] Image 244/336 : P_01_04.png
[GRAD] Image 245/336 : P_02_01.png
[GRAD] Image 246/336 : P_02_02.png
[GRAD] Image 247/336 : P_02_03.png
[GRAD] Image 248/336 : P_02_04.png
[GRAD] Image 249/336 : P_03_01.png
[GRAD] Image 250/336 : P_03_02.png
[GRAD] Image 251/336 : P_03_03.png
[GRAD] Image 252/336 : P_03_04.png
[GRAD] Image 253/336 : P_04_01.png
[GRAD] Image 254/336 : P_04_02.png
[GRAD] Image 255/336 : P_04_03.png
[GRAD] Image 256/336 : P_04_04.png
[GRAD] Image 257/336 : Q_01_01.png
[GRAD] Image 258/336 : Q_01_02.png
[GRAD] Image 259/336 : Q_01_03.png
[GRAD] Image 260/336 : Q_01_04.png
[GRAD] Image 261/336 : Q_02_01.png
[GRAD] Image 262/336 : Q_02_02.png
[GRAD] Image 263/336 : Q_02_03.png
[GRAD] Image 264/336 : Q_02_04.png
[GRAD] Image 265/336 : Q_03_01.png
[GRAD] Image 266/336 : Q_03_02.png
```

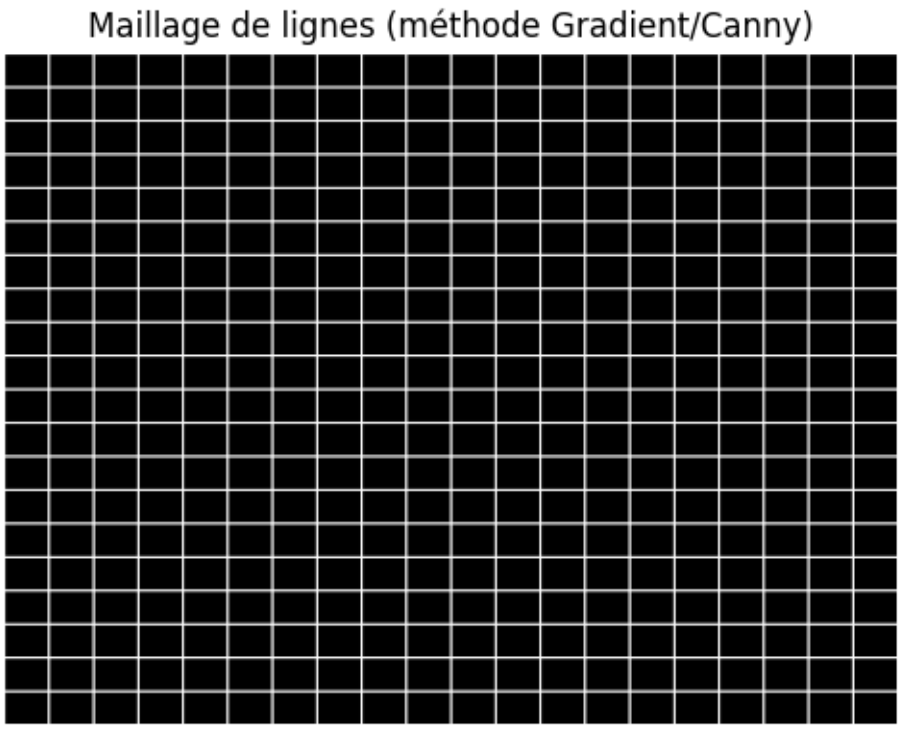
```
[GRAD] Image 267/336 : Q_03_03.png
[GRAD] Image 268/336 : Q_03_04.png
[GRAD] Image 269/336 : Q_04_01.png
[GRAD] Image 270/336 : Q_04_02.png
[GRAD] Image 271/336 : Q_04_03.png
[GRAD] Image 272/336 : Q_04_04.png
[GRAD] Image 273/336 : R_01_01.png
[GRAD] Image 274/336 : R_01_02.png
[GRAD] Image 275/336 : R_01_03.png
[GRAD] Image 276/336 : R_01_04.png
[GRAD] Image 277/336 : R_02_01.png
[GRAD] Image 278/336 : R_02_02.png
[GRAD] Image 279/336 : R_02_03.png
[GRAD] Image 280/336 : R_02_04.png
[GRAD] Image 281/336 : R_03_01.png
[GRAD] Image 282/336 : R_03_02.png
[GRAD] Image 283/336 : R_03_03.png
[GRAD] Image 284/336 : R_03_04.png
[GRAD] Image 285/336 : R_04_01.png
[GRAD] Image 286/336 : R_04_02.png
[GRAD] Image 287/336 : R_04_03.png
[GRAD] Image 288/336 : R_04_04.png
[GRAD] Image 289/336 : S_01_01.png
[GRAD] Image 290/336 : S_01_02.png
[GRAD] Image 291/336 : S_01_03.png
[GRAD] Image 292/336 : S_01_04.png
[GRAD] Image 293/336 : S_02_01.png
[GRAD] Image 294/336 : S_02_02.png
[GRAD] Image 295/336 : S_02_03.png
[GRAD] Image 296/336 : S_02_04.png
[GRAD] Image 297/336 : S_03_01.png
[GRAD] Image 298/336 : S_03_02.png
[GRAD] Image 299/336 : S_03_03.png
[GRAD] Image 300/336 : S_03_04.png
[GRAD] Image 301/336 : S_04_01.png
[GRAD] Image 302/336 : S_04_02.png
[GRAD] Image 303/336 : S_04_03.png
[GRAD] Image 304/336 : S_04_04.png
[GRAD] Image 305/336 : T_01_01.png
[GRAD] Image 306/336 : T_01_02.png
[GRAD] Image 307/336 : T_01_03.png
[GRAD] Image 308/336 : T_01_04.png
[GRAD] Image 309/336 : T_02_01.png
[GRAD] Image 310/336 : T_02_02.png
[GRAD] Image 311/336 : T_02_03.png
[GRAD] Image 312/336 : T_02_04.png
[GRAD] Image 313/336 : T_03_01.png
[GRAD] Image 314/336 : T_03_02.png
```

```
[GRAD] Image 315/336 : T_03_03.png
[GRAD] Image 316/336 : T_03_04.png
[GRAD] Image 317/336 : T_04_01.png
[GRAD] Image 318/336 : T_04_02.png
[GRAD] Image 319/336 : T_04_03.png
[GRAD] Image 320/336 : T_04_04.png
[GRAD] Image 321/336 : U_01_01.png
[GRAD] Image 322/336 : U_01_02.png
[GRAD] Image 323/336 : U_01_03.png
[GRAD] Image 324/336 : U_01_04.png
[GRAD] Image 325/336 : U_02_01.png
[GRAD] Image 326/336 : U_02_02.png
[GRAD] Image 327/336 : U_02_03.png
[GRAD] Image 328/336 : U_02_04.png
[GRAD] Image 329/336 : U_03_01.png
[GRAD] Image 330/336 : U_03_02.png
[GRAD] Image 331/336 : U_03_03.png
[GRAD] Image 332/336 : U_03_04.png
[GRAD] Image 333/336 : U_04_01.png
[GRAD] Image 334/336 : U_04_02.png
[GRAD] Image 335/336 : U_04_03.png
[GRAD] Image 336/336 : U_04_04.png
  [GRAD] Boucle Gradient/Canny terminée.
```

Ce bloc applique la méthode Gradient/Canny à l'ensemble des 336 micrographies : pour chaque image, il vérifie d'abord qu'elle a la même taille que l'image de référence, calcule les contours avec Canny, binarise cette carte de contours, superpose un maillage régulier de 20 lignes horizontales et 20 lignes verticales, puis intersecte ce maillage avec l'image segmentée ; à partir du résultat, il calcule les différences horizontales et verticales et compte, pour chaque ligne et chaque colonne, le nombre de transitions égales à `VALUE_INTERCEPT`, ce qui remplit progressivement les matrices `Total_Grains_X_grad` et `Total_Grains_Y_grad` avec les comptages d'interceptions pour toutes les images, comme le confirment les messages de progression et la validation finale indiquant que la boucle Gradient/Canny est terminée.

[ ]:
```
#  Étape 15 - Visualisation du maillage Gradient

plt.figure(figsize=(6, 6))
plt.imshow(black, cmap="gray")  # 'black' correspond à la dernière image de la␣
 ↪boucle précédente
plt.title("Maillage de lignes (méthode Gradient/Canny)")
plt.axis("off")
plt.show()
```



Cette figure montre le maillage utilisé pour la méthode Gradient/Canny : un réseau régulier de lignes horizontales et verticales superposé aux images, qui définit les trajectoires le long desquelles on compte les interceptions avec les contours des grains ; cette visualisation permet de vérifier que la grille couvre bien toute l'image et que le pas choisi (20 lignes dans chaque direction) est cohérent avant de s'appuyer sur ces lignes pour les calculs de tailles moyennes.

[ ]:
```
#  Étape 16 - Post-traitement & conversion en AVG_GrainX_Grad / AVG_GrainY_Grad

# Copie dans des variables dédiées à la méthode Gradient
TGX_grad = Total_Grains_X_grad.copy()
TGY_grad = Total_Grains_Y_grad.copy()

# Remplacer les zéros par 1 (éviter division par 0)
TGX_grad[TGX_grad == 0] = 1
TGY_grad[TGY_grad == 0] = 1

# Tailles moyennes des grains (méthode Gradient), en microns
AVG_GrainX_Grad = ROW_LENGTH_MICRONS / TGX_grad
AVG_GrainY_Grad = COL_LENGTH_MICRONS / TGY_grad

print("AVG_GrainX_Grad shape :", AVG_GrainX_Grad.shape)
print("AVG_GrainY_Grad shape :", AVG_GrainY_Grad.shape)
```

```
AVG_GrainX_Grad shape : (300, 336)
```

```
AVG_GrainY_Grad shape : (400, 336)
```

Ce bloc réalise le post-traitement des comptages obtenus avec la méthode Gradient/Canny : les matrices `Total_Grains_X_grad` et `Total_Grains_Y_grad` sont d'abord copiées dans `TGX_grad` et `TGY_grad`, puis tous les zéros y sont remplacés par 1 afin d'éviter toute division par zéro ; à partir des longueurs physiques de référence des lignes horizontales et verticales (`ROW_LENGTH_MICRONS` et `COL_LENGTH_MICRONS`), le code calcule ensuite les matrices `AVG_GrainX_Grad` et `AVG_GrainY_Grad`, qui contiennent pour chaque ligne/colonne de chaque image une taille moyenne de grain exprimée en microns, comme le confirment les dimensions finales (300 × 336 pour la direction X et 400 × 336 pour la direction Y).

[ ]:
```
#  Étape 17 - Statistiques globales (méthode Gradient)

meanXGrad = np.mean(AVG_GrainX_Grad)
STDXGrad  = np.std(AVG_GrainX_Grad)

meanYGrad = np.mean(AVG_GrainY_Grad)
STDYGrad  = np.std(AVG_GrainY_Grad)

print("[GRAD] meanXGrad :", meanXGrad)
print("[GRAD] meanYGrad :", meanYGrad)
print("[GRAD] STDXGrad  :", STDXGrad)
print("[GRAD] STDYGrad  :", STDYGrad)
```

```
[GRAD] meanXGrad : 68.47045
[GRAD] meanYGrad : 51.382805
[GRAD] STDXGrad  : 52.29283
[GRAD] STDYGrad  : 38.448326
```

Ce bloc synthétise les résultats de la méthode Gradient/Canny en calculant des statistiques globales sur les tailles moyennes de grains obtenues : la moyenne horizontale `meanXGrad` est d'environ 68,47 µm et la moyenne verticale `meanYGrad` d'environ 51,38 µm, tandis que les écarts-types `STDXGrad` et `STDYGrad` atteignent respectivement 52,29 µm et 38,45 µm ; ces valeurs indiquent que, selon cette méthode basée sur les contours Canny, les tailles estimées sont en moyenne sensiblement plus élevées et plus dispersées que celles issues du seuillage manuel, ce qui devra être discuté dans la comparaison des méthodes (sensibilité au paramétrage de Canny, à la structure du maillage et à la complexité des contours détectés).

[ ]:
```
#  Étape 18 - Sauvegarde des résultats Gradient en CSV (dans BASE_DIR)

out_TGX_grad = BASE_DIR / "Total_Grains_X_Grad.csv"
out_TGY_grad = BASE_DIR / "Total_Grains_Y_Grad.csv"
out_AX_grad  = BASE_DIR / "AVG_GrainX_Grad.csv"
out_AY_grad  = BASE_DIR / "AVG_GrainY_Grad.csv"

np.savetxt(str(out_TGX_grad), Total_Grains_X_grad, delimiter=",")
np.savetxt(str(out_TGY_grad), Total_Grains_Y_grad, delimiter=",")
np.savetxt(str(out_AX_grad),  AVG_GrainX_Grad,    delimiter=",")
np.savetxt(str(out_AY_grad),  AVG_GrainY_Grad,    delimiter=",")
```

```
print("  [GRAD] Fichiers CSV Gradient sauvegardés dans :")
print("   ", out_TGX_grad)
print("   ", out_TGY_grad)
print("   ", out_AX_grad)
print("   ", out_AY_grad)
```

```
[GRAD] Fichiers CSV Gradient sauvegardés dans :
   /content/drive/MyDrive/Data/Total_Grains_X_Grad.csv
   /content/drive/MyDrive/Data/Total_Grains_Y_Grad.csv
   /content/drive/MyDrive/Data/AVG_GrainX_Grad.csv
   /content/drive/MyDrive/Data/AVG_GrainY_Grad.csv
```

Ce bloc finalise la méthode Gradient/Canny en exportant tous les résultats vers des fichiers CSV dans le dossier du projet : les matrices de comptage des interceptions `Total_Grains_X_Grad` et `Total_Grains_Y_Grad` sont sauvegardées, ainsi que les matrices de tailles moyennes de grains `AVG_GrainX_Grad` et `AVG_GrainY_Grad`, ce qui permet de conserver une trace complète des calculs effectués pour chaque image et de réutiliser ces résultats ultérieurement dans le rapport, pour des analyses complémentaires ou des comparaisons systématiques avec les autres méthodes de mesure.

Méthode Gradient / Canny (version v2 avec masques filtrés)

Dans cette section, la méthode Gradient est améliorée en ajoutant une chaîne complète de filtrage morphologique pour produire des masques de contours plus propres avant le comptage des interceptions. L'objectif est d'obtenir des frontières de grains plus continues et moins bruitées que dans la version précédente, afin de rendre les estimations de taille de grain plus stables sur l'ensemble des images.

[ ]:
```
#  Étape 19 - Pipeline Gradient complet sur une image
#    (Canny + morphologie + seuil 190)

test_grad_full_path = IMG_DIR / image_files[0]
print("[GRAD FULL] Image test (pipeline Grad complet) :", test_grad_full_path)

img_grad_full = cv2.imread(str(test_grad_full_path), cv2.IMREAD_GRAYSCALE)
if img_grad_full is None:
    raise FileNotFoundError(f"Impossible de lire l'image :␣
 ↪{test_grad_full_path}")

if img_grad_full.shape != IMG_SHAPE_REF:
    raise ValueError(
        f"[GRAD FULL] Image test de taille {img_grad_full.shape}, "
        f"différente de la taille de référence {IMG_SHAPE_REF}"
    )

# 1) Canny
canny_full = cv2.Canny(img_grad_full, 100, 200)
```

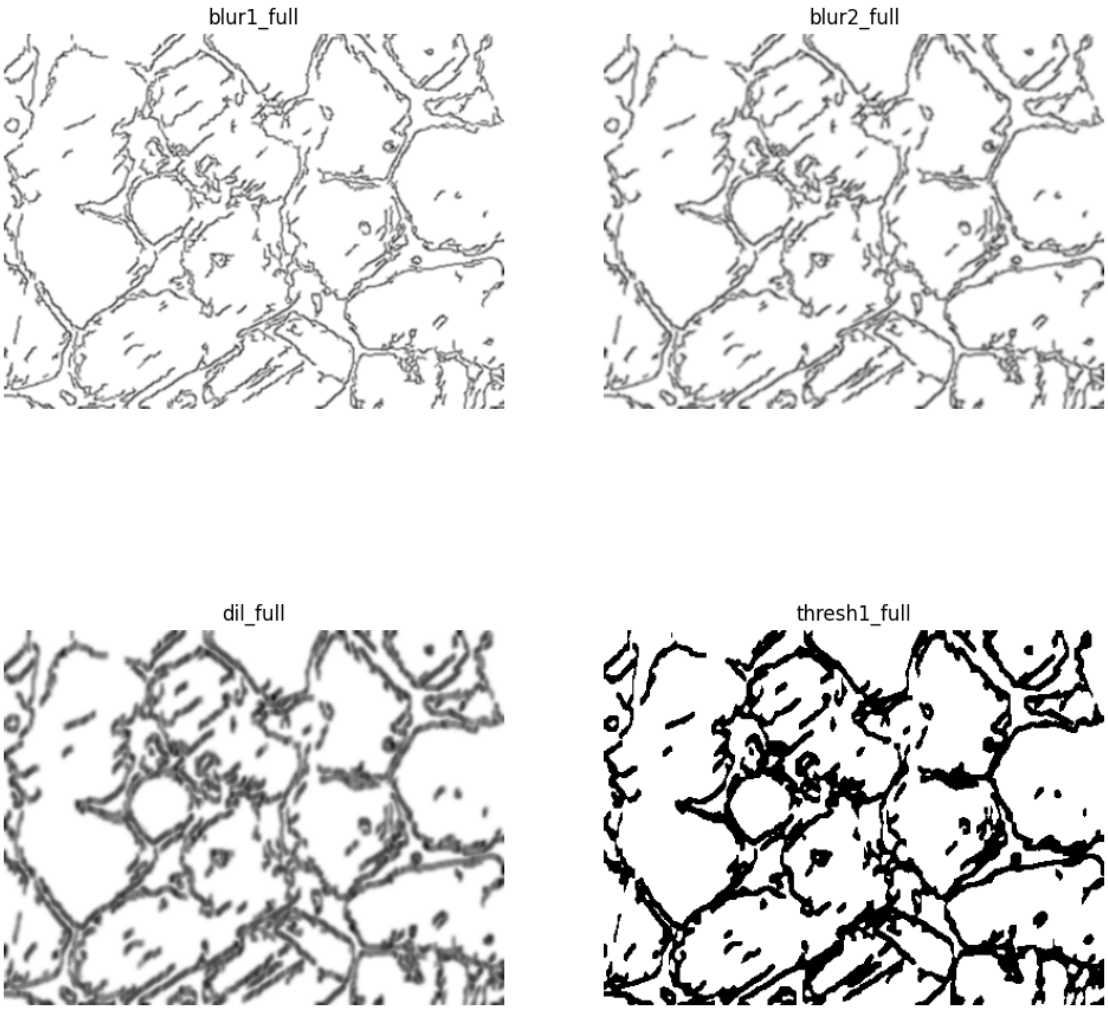
```
# 2) Seuil binaire inverse sur Canny
_, threshCAN_full = cv2.threshold(canny_full, 90, 255, cv2.THRESH_BINARY_INV)

# 3) Pipeline morphologique (comme notebook Grad)
blur1_full = cv2.GaussianBlur(threshCAN_full, (3, 3), 0)
blur2_full = cv2.GaussianBlur(blur1_full,     (3, 3), 0)
dil_full   = cv2.erode(blur2_full, (5, 5))
dil_full   = cv2.erode(dil_full,   (5, 5))
dil_full   = cv2.GaussianBlur(dil_full, (3, 3), 0)
dil_full   = cv2.GaussianBlur(dil_full, (3, 3), 0)
dil_full   = cv2.erode(dil_full,   (5, 5))

# 4) Seuil final à 190
thresh1_full = cv2.threshold(dil_full, 190, 255, cv2.THRESH_BINARY)[1]

plt.figure(figsize=(12, 12))
plt.subplot(2, 2, 1); plt.imshow(blur1_full,   cmap='gray'); plt.
 ↪title("blur1_full");   plt.axis("off")
plt.subplot(2, 2, 2); plt.imshow(blur2_full,   cmap='gray'); plt.
 ↪title("blur2_full");   plt.axis("off")
plt.subplot(2, 2, 3); plt.imshow(dil_full,     cmap='gray'); plt.
 ↪title("dil_full");     plt.axis("off")
plt.subplot(2, 2, 4); plt.imshow(thresh1_full, cmap='gray'); plt.
 ↪title("thresh1_full"); plt.axis("off")
plt.show()
```

```
[GRAD FULL] Image test (pipeline Grad complet) :
/content/drive/MyDrive/Data/Grains/A_01_01.png
```



Ce bloc illustre le pipeline Gradient complet sur une image test : après avoir chargé l'image et vérifié sa taille, il applique d'abord Canny pour détecter les contours, puis un seuillage binaire inverse pour isoler ces contours ; une série de filtrages morphologiques (flous gaussiens successifs puis érosions) est ensuite utilisée pour lisser les bords et combler les discontinuités, avant d'appliquer un seuil final à 190 qui produit un masque binaire net (`thresh1_full`) où les grains apparaissent sous forme de zones blanches bien délimitées, comme le montrent les quatre sous-figures (blur1_full, blur2_full, dil_full, thresh1_full) qui permettent de visualiser l'effet progressif de chaque étape de filtrage.

[ ]:
```
#  Étape 20 - Génération des masques de contours filtrés (area > 100)
#    VERSION COLAB + DRIVE - réutilise FILTERED_GRAD_DIR

GRAD_OUT_DIR = FILTERED_GRAD_DIR
GRAD_OUT_DIR.mkdir(exist_ok=True)

print("Dossier de sortie des masques Gradient :", GRAD_OUT_DIR)
```

```
black_mask_grad = None  # sera mis à jour à chaque itération

for t, fname in enumerate(image_files[:n_grad]):

    img_path = IMG_DIR / fname
    img = cv2.imread(str(img_path), cv2.IMREAD_GRAYSCALE)
    if img is None:
        raise FileNotFoundError(f"Impossible de lire l'image : {img_path}")

    # 1) Canny
    canny = cv2.Canny(img, 100, 200)

    # 2) Seuil binaire inverse sur Canny
    _, threshCAN = cv2.threshold(canny, 90, 255, cv2.THRESH_BINARY_INV)

    # 3) Pipeline morphologique (comme Étape 19)
    blur1 = cv2.GaussianBlur(threshCAN, (3, 3), 0)
    blur2 = cv2.GaussianBlur(blur1,     (3, 3), 0)
    dil   = cv2.erode(blur2, (5, 5))
    dil   = cv2.erode(dil,   (5, 5))
    dil   = cv2.GaussianBlur(dil, (3, 3), 0)
    dil   = cv2.GaussianBlur(dil, (3, 3), 0)
    dil   = cv2.erode(dil,   (5, 5))

    thresh1 = cv2.threshold(dil, 190, 255, cv2.THRESH_BINARY)[1]

    # 4) Contours externes sur l'image binaire propre
    _, binary = cv2.threshold(thresh1, 127, 255, cv2.THRESH_BINARY)
    contours, _ = cv2.findContours(binary, cv2.RETR_EXTERNAL, cv2.
 ↪CHAIN_APPROX_NONE)

    # 5) Masque noir puis remplissage des grains d'aire > 100
    black_mask_grad = np.zeros_like(img)

    for c in contours:
        if cv2.contourArea(c) > 100:  # seuil de surface
            cv2.drawContours(black_mask_grad, [c], -1, (255, 255, 255), -1)

    # 6) Sauvegarde du masque (nom FGRAD_###.png)
    out_name = f"FGRAD_{t+1:03d}.png"
    cv2.imwrite(str(GRAD_OUT_DIR / out_name), black_mask_grad)

print("  Masques filtrés Gradient générés et sauvegardés dans :", GRAD_OUT_DIR)
```

```
Dossier de sortie des masques Gradient :
/content/drive/MyDrive/Data/FilteredGradV2
  Masques filtrés Gradient générés et sauvegardés dans :
/content/drive/MyDrive/Data/FilteredGradV2
```

Ce bloc applique le pipeline Gradient complet à toutes les images pour générer des masques de contours filtrés : pour chaque micrographie, il calcule d'abord les contours avec Canny, applique un seuillage binaire inverse puis une série de filtrages morphologiques (flous gaussiens et érosions) avant un seuil final à 190, extrait ensuite les contours externes sur cette image binaire propre, et remplit en blanc uniquement les grains dont l'aire est supérieure à 100 pixels dans un masque noir ; chaque masque obtenu, beaucoup plus "propre" et débarrassé des petits artefacts, est sauvegardé dans le dossier `FilteredGradV2` sous la forme de fichiers `FGRAD_###.png`, ce qui constitue la base de la version Grad v2 utilisée pour le comptage d'interceptions.

[ ]:
```python
#  Étape 21 - Comptage des interceptions sur les masques filtrés (version Grad complète)

value = 255  # valeur à compter

Total_Grains_X_grad_v2 = np.zeros((numrows, n_grad), dtype=np.float32)
Total_Grains_Y_grad_v2 = np.zeros((numcols, n_grad), dtype=np.float32)

for t, fname in enumerate(image_files[:n_grad]):

    mask_path = GRAD_OUT_DIR / f"FGRAD_{t+1:03d}.png"
    mask = cv2.imread(str(mask_path), cv2.IMREAD_GRAYSCALE)
    if mask is None:
        raise FileNotFoundError(f"Impossible de lire le masque : {mask_path}")

    if mask.shape != IMG_SHAPE_REF:
        raise ValueError(
            f"[GRAD_v2] Masque {mask_path.name} de taille {mask.shape}, "
            f"différente de la taille de référence {IMG_SHAPE_REF}"
        )

    # Maillage 20×20 sur fond noir
    black = np.zeros_like(mask)

    for i in range(N_LINES_GRID):
        y = int(i * black.shape[0] / N_LINES_GRID)
        cv2.line(black, (0, y), (black.shape[1] - 1, y), 255, 1)

    for i in range(N_LINES_GRID):
        x = int(i * black.shape[1] / N_LINES_GRID)
        cv2.line(black, (x, 0), (x, black.shape[0] - 1), 255, 1)

    # Intersection maillage × masque filtré
    result = cv2.bitwise_and(black, mask)

    # Différences
    diff_x = np.diff(result, axis=1)
    diff_y = np.diff(result, axis=0)
```

```
    # Comptage sur les lignes
    for x in range(numrows):
        count = 0
        for element in diff_x[x, :]:
            if element == value:
                count += 1
        Total_Grains_X_grad_v2[x, t] = count

    # Comptage sur les colonnes
    for y in range(numcols - 1):
        count = 0
        for element in diff_y[:, y]:
            if element == value:
                count += 1
        Total_Grains_Y_grad_v2[y, t] = count

print(" Comptage des interceptions (version Grad complète) terminé.")
print("   Total_Grains_X_grad_v2 shape :", Total_Grains_X_grad_v2.shape)
print("   Total_Grains_Y_grad_v2 shape :", Total_Grains_Y_grad_v2.shape)
```

```
 Comptage des interceptions (version Grad complète) terminé.
   Total_Grains_X_grad_v2 shape : (300, 336)
   Total_Grains_Y_grad_v2 shape : (400, 336)
```

Ce bloc applique la méthode des interceptions linéaires aux masques filtrés générés par la version complète du pipeline Gradient : pour chaque masque `FGRAD_###.png`, il vérifie la taille, construit un maillage 20×20 de lignes horizontales et verticales sur un fond noir, intersecte ce maillage avec le masque binaire filtré, calcule ensuite les différences horizontales et verticales et compte, sur chaque ligne et chaque colonne, le nombre de transitions égales à 255 ; ces comptages remplissent les matrices `Total_Grains_X_grad_v2` (300 × 336) et `Total_Grains_Y_grad_v2` (400 × 336), ce qui fournit pour chaque image et pour chaque ligne/colonne le nombre d'interceptions observées sur les masques "propres" de la version Grad v2.

[ ]:
```
#  Étape 22 - Post-traitement & tailles moyennes (Grad v2)
# On part des matrices Total_Grains_X_grad_v2 / Y_grad_v2 déjà remplies␣
 ↪(version Grad complète)

# Copie pour post-traitement (on garde les noms v2 pour ne rien écraser)
Total_Grains_X_v2 = Total_Grains_X_grad_v2.copy()
Total_Grains_Y_v2 = Total_Grains_Y_grad_v2.copy()

# Remplacement des zéros par 1 (éviter division par 0)
Total_Grains_X_v2[Total_Grains_X_v2 == 0] = 1
Total_Grains_Y_v2[Total_Grains_Y_v2 == 0] = 1

# Conversion en tailles de grains (Grad v2), en microns
AVG_GrainX_Grad_v2 = ROW_LENGTH_MICRONS / Total_Grains_X_v2
```

```
AVG_GrainY_Grad_v2 = COL_LENGTH_MICRONS / Total_Grains_Y_v2

# Vérification de la dernière ligne Y (index -1)
print("Dernière ligne AVG_GrainY_Grad_v2 avant suppression :")
print(AVG_GrainY_Grad_v2[-1, :])

#   Option « comme dans le notebook Grad » :
#    on supprime la dernière ligne si on considère qu'elle est aberrante.
#    Tu peux commenter cette ligne si tu veux garder TOUT.
AVG_GrainY_Grad_v2 = np.delete(AVG_GrainY_Grad_v2, -1, axis=0)

print("Nouvelle shape AVG_GrainY_Grad_v2 :", AVG_GrainY_Grad_v2.shape)
```

```
Dernière ligne AVG_GrainY_Grad_v2 avant suppression :
[133.33333 133.33333 133.33333 133.33333 133.33333 133.33333 133.33333
 
```

```
 133.33333 133.33333 133.33333 133.33333 133.33333 133.33333 133.33333
 133.33333 133.33333 133.33333 133.33333 133.33333 133.33333 133.33333
 133.33333 133.33333 133.33333 133.33333 133.33333 133.33333 133.33333
 133.33333 133.33333 133.33333 133.33333 133.33333 133.33333 133.33333
 133.33333 133.33333 133.33333 133.33333 133.33333 133.33333 133.33333
 133.33333 133.33333 133.33333 133.33333 133.33333 133.33333 133.33333
 133.33333 133.33333 133.33333 133.33333 133.33333 133.33333 133.33333
 133.33333 133.33333 133.33333 133.33333 133.33333 133.33333 133.33333
 133.33333 133.33333 133.33333 133.33333 133.33333 133.33333 133.33333
 133.33333 133.33333 133.33333 133.33333 133.33333 133.33333 133.33333
 133.33333 133.33333 133.33333 133.33333 133.33333 133.33333 133.33333
 133.33333 133.33333 133.33333 133.33333 133.33333 133.33333 133.33333
 133.33333 133.33333 133.33333 133.33333 133.33333 133.33333 133.33333]
Nouvelle shape AVG_GrainY_Grad_v2 : (399, 336)
```

Ce bloc réalise le post-traitement de la version Grad v2 : les matrices de comptage `Total_Grains_X_grad_v2` et `Total_Grains_Y_grad_v2` sont copiées, les valeurs nulles sont remplacées par 1 pour éviter les divisions par zéro, puis converties en tailles moyennes de grains en microns (`AVG_GrainX_Grad_v2` et `AVG_GrainY_Grad_v2`) à partir des longueurs physiques de référence ; la dernière ligne de `AVG_GrainY_Grad_v2` est ensuite inspectée et montre des valeurs constantes très élevées (133,33 µm) sur toutes les images, interprétées comme aberrantes, et elle est donc supprimée, ce qui ramène la matrice verticale à une dimension de 399 × 336 et aligne la version Grad v2 sur le comportement observé dans le notebook de référence.

Initialisation & Line Intercept avec HED

Cette section ouvre la troisième chaîne de mesure basée sur le détecteur de contours HED. L'idée est de préparer une nouvelle famille de masques de contours, indépendante du seuillage manuel et de la méthode Gradient/Canny, puis d'appliquer à nouveau la méthode des interceptions linéaires pour obtenir des tailles moyennes de grains basées sur HED. Avant de lancer le traitement HED proprement dit, le notebook commence par figer et exporter les résultats de la version Gradient filtrée (Grad v2), qui serviront de référence pour la comparaison.

[ ]:
```python
#  Étape 23 - Statistiques globales & export CSV (Grad v2)

# Statistiques Grad v2
meanXGrad_v2 = np.mean(AVG_GrainX_Grad_v2)
STDXGrad_v2  = np.std(AVG_GrainX_Grad_v2)

meanYGrad_v2 = np.mean(AVG_GrainY_Grad_v2)
STDYGrad_v2  = np.std(AVG_GrainY_Grad_v2)

print("Grad v2 - meanXGrad_v2 :", meanXGrad_v2)
print("Grad v2 - meanYGrad_v2 :", meanYGrad_v2)
print("Grad v2 - STDXGrad_v2  :", STDXGrad_v2)
print("Grad v2 - STDYGrad_v2  :", STDYGrad_v2)
```

```
# Sauvegarde en CSV dans ton dossier projet sur Drive
np.savetxt(str(BASE_DIR / "Total_Grains_X_Grad_v2.csv"),␣
 ↪Total_Grains_X_grad_v2, delimiter=",")
np.savetxt(str(BASE_DIR / "Total_Grains_Y_Grad_v2.csv"),␣
 ↪Total_Grains_Y_grad_v2, delimiter=",")
np.savetxt(str(BASE_DIR / "AVG_GrainX_Grad_v2.csv"),     AVG_GrainX_Grad_v2,   ␣
 ↪ delimiter=",")
np.savetxt(str(BASE_DIR / "AVG_GrainY_Grad_v2.csv"),     AVG_GrainY_Grad_v2,   ␣
 ↪ delimiter=",")

print(" CSV Grad v2 sauvegardés dans :", BASE_DIR)
print(" - Total_Grains_X_Grad_v2.csv")
print(" - Total_Grains_Y_Grad_v2.csv")
print(" - AVG_GrainX_Grad_v2.csv")
print(" - AVG_GrainY_Grad_v2.csv")
```

```
Grad v2 - meanXGrad_v2 : 33.9847
Grad v2 - meanYGrad_v2 : 26.462593
Grad v2 - STDXGrad_v2  : 34.917866
Grad v2 - STDYGrad_v2  : 27.250618
 CSV Grad v2 sauvegardés dans : /content/drive/MyDrive/Data
 - Total_Grains_X_Grad_v2.csv
 - Total_Grains_Y_Grad_v2.csv
 - AVG_GrainX_Grad_v2.csv
 - AVG_GrainY_Grad_v2.csv
```

Ce bloc calcule d'abord les statistiques globales associées à la version Grad v2 en prenant la moyenne et l'écart-type des tailles moyennes de grains dans les directions horizontale et verticale (meanXGrad_v2 33,98 µm, meanYGrad_v2 26,46 µm, STDXGrad_v2 34,92 µm, STDYGrad_v2 27,25 µm), ce qui donne une vue d'ensemble du niveau et de la dispersion des tailles estimées avec les masques filtrés ; ces résultats sont ensuite figés en quatre fichiers CSV (Total_Grains_X_Grad_v2.csv, Total_Grains_Y_Grad_v2.csv, AVG_GrainX_Grad_v2.csv, AVG_GrainY_G enregistrés dans le dossier du projet, de manière à disposer d'une trace exploitable et directement comparable lorsque les tailles de grains issues de HED seront calculées.

[ ]:
```
#  Étape 24 - Initialisation modèle HED (CropLayer + chemins + réseau)

# Dossiers de sortie HED (doivent déjà exister selon ton bloc de config initial)
HED_RAW_DIR = BASE_DIR / "HED_raw"
HED_FILTERED_DIR = BASE_DIR / "HED_filtered"
HED_RAW_DIR.mkdir(parents=True, exist_ok=True)
HED_FILTERED_DIR.mkdir(parents=True, exist_ok=True)

# Chemins vers les poids HED (dossier réel de ton projet)
protoPath = str(WEIGHTS_HED_DIR / "deploy.prototxt")
modelPath = str(WEIGHTS_HED_DIR / "hed_pretrained_bsds.caffemodel")
```

```
print("protoPath :", protoPath)
print("modelPath :", modelPath)

# Définition de la CropLayer (copiée du notebook HED)
class CropLayer(object):
    def __init__(self, params, blobs):
        self.startX = 0
        self.startY = 0
        self.endX = 0
        self.endY = 0

    def getMemoryShapes(self, inputs):
        inputShape, targetShape = inputs[0], inputs[1]
        batchSize, numChannels = inputShape[0], inputShape[1]
        H, W = targetShape[2], targetShape[3]

        self.startX = int((inputShape[3] - W) / 2)
        self.startY = int((inputShape[2] - H) / 2)
        self.endX = self.startX + W
        self.endY = self.startY + H

        return [[batchSize, numChannels, H, W]]

    def forward(self, inputs):
        return [inputs[0][:, :, self.startY:self.endY,
                          self.startX:self.endX]]

# Enregistrement de la couche Crop dans OpenCV DNN
cv2.dnn_registerLayer("Crop", CropLayer)

# Chargement du réseau HED pré-entraîné
net_hed = cv2.dnn.readNetFromCaffe(protoPath, modelPath)

# Récupérer H, W de référence à partir de la première image de Grains
test_path = IMG_DIR / image_files[0]
img0 = cv2.imread(str(test_path), cv2.IMREAD_COLOR)
if img0 is None:
    raise FileNotFoundError(f"Impossible de lire l'image : {test_path}")

(H_ref, W_ref) = img0.shape[:2]
print("Taille de référence H×W :", H_ref, "x", W_ref)

print("  Modèle HED initialisé.")
```

```
protoPath : /content/drive/MyDrive/Data/weights HED/deploy.prototxt
modelPath : /content/drive/MyDrive/Data/weights
HED/hed_pretrained_bsds.caffemodel
Taille de référence H×W : 300 x 400
```

```
Modèle HED initialisé.
```

Ce bloc prépare l'utilisation du modèle HED pour la détection de contours : il crée (ou vérifie) les dossiers de sortie dédiés à HED (`HED_raw` et `HED_filtered`), renseigne les chemins vers les fichiers de poids Caffe (`deploy.prototxt` et `hed_pretrained_bsds.caffemodel`), définit puis enregistre dans OpenCV la couche spéciale `CropLayer` nécessaire au bon fonctionnement du réseau, charge le modèle HED pré-entraîné dans `net_hed`, et récupère enfin la taille de référence de la première image de grains (300×400), ce qui assure que le réseau sera appliqué de manière cohérente à toutes les micrographies ; le message final confirme que le modèle HED est correctement initialisé et prêt à être utilisé pour générer des cartes de contours.

[ ]:
```python
#  Étape 25 - Fonction apply_hed + test visuel

def apply_hed(img_bgr, net, W_ref, H_ref):
    """
    Reprend la chaîne du notebook HED :
    - blobFromImage(scalefactor=0.7, size=(W_ref, H_ref), mean=(105,117,123))
    - net.forward()
    - rescale 0-255
    - flou gaussien
    - seuil binaire inverse
    Retourne :
      - hed_map   : carte de contours 0-255
      - hed_blur  : carte floutée
      - hed_thresh: image binaire (0 / 255)
    """
    blob = cv2.dnn.blobFromImage(
        img_bgr,
        scalefactor=0.7,
        size=(W_ref, H_ref),
        mean=(105, 117, 123),
        swapRB=False,
        crop=False
    )

    net.setInput(blob)
    hed = net.forward()
    hed = hed[0, 0, :, :]       # (1,1,H,W) → (H,W)
    hed = (255 * hed).astype("uint8")  # 0-1 → 0-255

    hed_blur = cv2.GaussianBlur(hed, (3, 3), 0)
    hed_thresh = cv2.threshold(hed_blur, 40, 250, cv2.THRESH_BINARY_INV)[1]

    return hed, hed_blur, hed_thresh

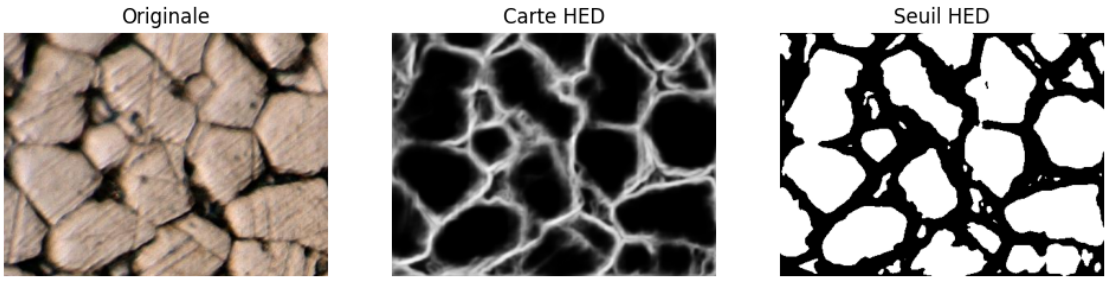
# Test sur une image
test_hed_path = IMG_DIR / image_files[0]
print("Image test HED :", test_hed_path)
```

```
img_test_bgr = cv2.imread(str(test_hed_path), cv2.IMREAD_COLOR)
if img_test_bgr is None:
    raise FileNotFoundError(f"Impossible de lire l'image : {test_hed_path}")

hed_map, hed_blur, hed_thresh = apply_hed(img_test_bgr, net_hed, W_ref=W_ref,␣
 ↪H_ref=H_ref)

plt.figure(figsize=(12, 4))
plt.subplot(1, 3, 1); plt.imshow(cv2.cvtColor(img_test_bgr, cv2.COLOR_BGR2RGB));
 ↪ plt.title("Originale");   plt.axis("off")
plt.subplot(1, 3, 2); plt.imshow(hed_map, cmap="gray");                    ␣
 ↪plt.title("Carte HED");   plt.axis("off")
plt.subplot(1, 3, 3); plt.imshow(hed_thresh, cmap="gray");                 ␣
 ↪plt.title("Seuil HED");   plt.axis("off")
plt.show()
```

```
Image test HED : /content/drive/MyDrive/Data/Grains/A_01_01.png
```



Ce bloc définit la fonction `apply_hed`, qui encapsule toute la chaîne HED pour une image couleur : l'image est d'abord transformée en « blob » et passée dans le réseau `net_hed` pour produire une carte de contours (`hed_map`) en niveaux de gris, puis cette carte est légèrement lissée par un flou gaussien (`hed_blur`) avant d'être seuillée pour obtenir une image binaire (`hed_thresh`) où les grains apparaissent comme des régions blanches bien séparées par des joints noirs ; le test visuel sur une micrographie de référence montre successivement l'image originale, la carte HED continue et le résultat seuillé, ce qui permet de vérifier que le modèle met bien en évidence les frontières des grains avant d'appliquer la méthode des interceptions.

[ ]:
```
#  Étape 26 - Génération et sauvegarde des cartes HED pour toutes les images␣
 ↪de Grains

N_HED = len(image_files)
print(f"[HED raw] Nombre d'images Grains disponibles : {N_HED}")

for t, fname in enumerate(image_files[:N_HED]):
    img_path = IMG_DIR / fname
```

```
    print(f"[HED raw] Image {t+1}/{N_HED} :", img_path.name)

    img_bgr = cv2.imread(str(img_path), cv2.IMREAD_COLOR)
    if img_bgr is None:
        raise FileNotFoundError(f"Impossible de lire l'image : {img_path}")

    _, _, hed_thresh = apply_hed(img_bgr, net_hed, W_ref=W_ref, H_ref=H_ref)

    out_name = f"{Path(fname).stem}_HED.png"
    cv2.imwrite(str(HED_RAW_DIR / out_name), hed_thresh)

print("  Cartes HED seuillées sauvegardées dans :", HED_RAW_DIR)
```

```
[HED raw] Nombre d'images Grains disponibles : 336
[HED raw] Image 1/336 : A_01_01.png
[HED raw] Image 2/336 : A_01_02.png
[HED raw] Image 3/336 : A_01_03.png
[HED raw] Image 4/336 : A_01_04.png
[HED raw] Image 5/336 : A_02_01.png
[HED raw] Image 6/336 : A_02_02.png
[HED raw] Image 7/336 : A_02_03.png
[HED raw] Image 8/336 : A_02_04.png
[HED raw] Image 9/336 : A_03_01.png
[HED raw] Image 10/336 : A_03_02.png
[HED raw] Image 11/336 : A_03_03.png
[HED raw] Image 12/336 : A_03_04.png
[HED raw] Image 13/336 : A_04_01.png
[HED raw] Image 14/336 : A_04_02.png
[HED raw] Image 15/336 : A_04_03.png
[HED raw] Image 16/336 : A_04_04.png
[HED raw] Image 17/336 : B_01_01.png
[HED raw] Image 18/336 : B_01_02.png
[HED raw] Image 19/336 : B_01_03.png
[HED raw] Image 20/336 : B_01_04.png
[HED raw] Image 21/336 : B_02_01.png
[HED raw] Image 22/336 : B_02_02.png
[HED raw] Image 23/336 : B_02_03.png
[HED raw] Image 24/336 : B_02_04.png
[HED raw] Image 25/336 : B_03_01.png
[HED raw] Image 26/336 : B_03_02.png
[HED raw] Image 27/336 : B_03_03.png
[HED raw] Image 28/336 : B_03_04.png
[HED raw] Image 29/336 : B_04_01.png
[HED raw] Image 30/336 : B_04_02.png
[HED raw] Image 31/336 : B_04_03.png
[HED raw] Image 32/336 : B_04_04.png
[HED raw] Image 33/336 : C_01_01.png
[HED raw] Image 34/336 : C_01_02.png
```

```
[HED raw] Image 35/336 : C_01_03.png
[HED raw] Image 36/336 : C_01_04.png
[HED raw] Image 37/336 : C_02_01.png
[HED raw] Image 38/336 : C_02_02.png
[HED raw] Image 39/336 : C_02_03.png
[HED raw] Image 40/336 : C_02_04.png
[HED raw] Image 41/336 : C_03_01.png
[HED raw] Image 42/336 : C_03_02.png
[HED raw] Image 43/336 : C_03_03.png
[HED raw] Image 44/336 : C_03_04.png
[HED raw] Image 45/336 : C_04_01.png
[HED raw] Image 46/336 : C_04_02.png
[HED raw] Image 47/336 : C_04_03.png
[HED raw] Image 48/336 : C_04_04.png
[HED raw] Image 49/336 : D_01_01.png
[HED raw] Image 50/336 : D_01_02.png
[HED raw] Image 51/336 : D_01_03.png
[HED raw] Image 52/336 : D_01_04.png
[HED raw] Image 53/336 : D_02_01.png
[HED raw] Image 54/336 : D_02_02.png
[HED raw] Image 55/336 : D_02_03.png
[HED raw] Image 56/336 : D_02_04.png
[HED raw] Image 57/336 : D_03_01.png
[HED raw] Image 58/336 : D_03_02.png
[HED raw] Image 59/336 : D_03_03.png
[HED raw] Image 60/336 : D_03_04.png
[HED raw] Image 61/336 : D_04_01.png
[HED raw] Image 62/336 : D_04_02.png
[HED raw] Image 63/336 : D_04_03.png
[HED raw] Image 64/336 : D_04_04.png
[HED raw] Image 65/336 : E_01_01.png
[HED raw] Image 66/336 : E_01_02.png
[HED raw] Image 67/336 : E_01_03.png
[HED raw] Image 68/336 : E_01_04.png
[HED raw] Image 69/336 : E_02_01.png
[HED raw] Image 70/336 : E_02_02.png
[HED raw] Image 71/336 : E_02_03.png
[HED raw] Image 72/336 : E_02_04.png
[HED raw] Image 73/336 : E_03_01.png
[HED raw] Image 74/336 : E_03_02.png
[HED raw] Image 75/336 : E_03_03.png
[HED raw] Image 76/336 : E_03_04.png
[HED raw] Image 77/336 : E_04_01.png
[HED raw] Image 78/336 : E_04_02.png
[HED raw] Image 79/336 : E_04_03.png
[HED raw] Image 80/336 : E_04_04.png
[HED raw] Image 81/336 : F_01_01.png
[HED raw] Image 82/336 : F_01_02.png
```

```
[HED raw] Image 83/336 : F_01_03.png
[HED raw] Image 84/336 : F_01_04.png
[HED raw] Image 85/336 : F_02_01.png
[HED raw] Image 86/336 : F_02_02.png
[HED raw] Image 87/336 : F_02_03.png
[HED raw] Image 88/336 : F_02_04.png
[HED raw] Image 89/336 : F_03_01.png
[HED raw] Image 90/336 : F_03_02.png
[HED raw] Image 91/336 : F_03_03.png
[HED raw] Image 92/336 : F_03_04.png
[HED raw] Image 93/336 : F_04_01.png
[HED raw] Image 94/336 : F_04_02.png
[HED raw] Image 95/336 : F_04_03.png
[HED raw] Image 96/336 : F_04_04.png
[HED raw] Image 97/336 : G_01_01.png
[HED raw] Image 98/336 : G_01_02.png
[HED raw] Image 99/336 : G_01_03.png
[HED raw] Image 100/336 : G_01_04.png
[HED raw] Image 101/336 : G_02_01.png
[HED raw] Image 102/336 : G_02_02.png
[HED raw] Image 103/336 : G_02_03.png
[HED raw] Image 104/336 : G_02_04.png
[HED raw] Image 105/336 : G_03_01.png
[HED raw] Image 106/336 : G_03_02.png
[HED raw] Image 107/336 : G_03_03.png
[HED raw] Image 108/336 : G_03_04.png
[HED raw] Image 109/336 : G_04_01.png
[HED raw] Image 110/336 : G_04_02.png
[HED raw] Image 111/336 : G_04_03.png
[HED raw] Image 112/336 : G_04_04.png
[HED raw] Image 113/336 : H_01_01.png
[HED raw] Image 114/336 : H_01_02.png
[HED raw] Image 115/336 : H_01_03.png
[HED raw] Image 116/336 : H_01_04.png
[HED raw] Image 117/336 : H_02_01.png
[HED raw] Image 118/336 : H_02_02.png
[HED raw] Image 119/336 : H_02_03.png
[HED raw] Image 120/336 : H_02_04.png
[HED raw] Image 121/336 : H_03_01.png
[HED raw] Image 122/336 : H_03_02.png
[HED raw] Image 123/336 : H_03_03.png
[HED raw] Image 124/336 : H_03_04.png
[HED raw] Image 125/336 : H_04_01.png
[HED raw] Image 126/336 : H_04_02.png
[HED raw] Image 127/336 : H_04_03.png
[HED raw] Image 128/336 : H_04_04.png
[HED raw] Image 129/336 : I_01_01.png
[HED raw] Image 130/336 : I_01_02.png
```

```
[HED raw] Image 131/336 : I_01_03.png
[HED raw] Image 132/336 : I_01_04.png
[HED raw] Image 133/336 : I_02_01.png
[HED raw] Image 134/336 : I_02_02.png
[HED raw] Image 135/336 : I_02_03.png
[HED raw] Image 136/336 : I_02_04.png
[HED raw] Image 137/336 : I_03_01.png
[HED raw] Image 138/336 : I_03_02.png
[HED raw] Image 139/336 : I_03_03.png
[HED raw] Image 140/336 : I_03_04.png
[HED raw] Image 141/336 : I_04_01.png
[HED raw] Image 142/336 : I_04_02.png
[HED raw] Image 143/336 : I_04_03.png
[HED raw] Image 144/336 : I_04_04.png
[HED raw] Image 145/336 : J_01_01.png
[HED raw] Image 146/336 : J_01_02.png
[HED raw] Image 147/336 : J_01_03.png
[HED raw] Image 148/336 : J_01_04.png
[HED raw] Image 149/336 : J_02_01.png
[HED raw] Image 150/336 : J_02_02.png
[HED raw] Image 151/336 : J_02_03.png
[HED raw] Image 152/336 : J_02_04.png
[HED raw] Image 153/336 : J_03_01.png
[HED raw] Image 154/336 : J_03_02.png
[HED raw] Image 155/336 : J_03_03.png
[HED raw] Image 156/336 : J_03_04.png
[HED raw] Image 157/336 : J_04_01.png
[HED raw] Image 158/336 : J_04_02.png
[HED raw] Image 159/336 : J_04_03.png
[HED raw] Image 160/336 : J_04_04.png
[HED raw] Image 161/336 : K_01_01.png
[HED raw] Image 162/336 : K_01_02.png
[HED raw] Image 163/336 : K_01_03.png
[HED raw] Image 164/336 : K_01_04.png
[HED raw] Image 165/336 : K_02_01.png
[HED raw] Image 166/336 : K_02_02.png
[HED raw] Image 167/336 : K_02_03.png
[HED raw] Image 168/336 : K_02_04.png
[HED raw] Image 169/336 : K_03_01.png
[HED raw] Image 170/336 : K_03_02.png
[HED raw] Image 171/336 : K_03_03.png
[HED raw] Image 172/336 : K_03_04.png
[HED raw] Image 173/336 : K_04_01.png
[HED raw] Image 174/336 : K_04_02.png
[HED raw] Image 175/336 : K_04_03.png
[HED raw] Image 176/336 : K_04_04.png
[HED raw] Image 177/336 : L_01_01.png
[HED raw] Image 178/336 : L_01_02.png
```

```
[HED raw] Image 179/336 : L_01_03.png
[HED raw] Image 180/336 : L_01_04.png
[HED raw] Image 181/336 : L_02_01.png
[HED raw] Image 182/336 : L_02_02.png
[HED raw] Image 183/336 : L_02_03.png
[HED raw] Image 184/336 : L_02_04.png
[HED raw] Image 185/336 : L_03_01.png
[HED raw] Image 186/336 : L_03_02.png
[HED raw] Image 187/336 : L_03_03.png
[HED raw] Image 188/336 : L_03_04.png
[HED raw] Image 189/336 : L_04_01.png
[HED raw] Image 190/336 : L_04_02.png
[HED raw] Image 191/336 : L_04_03.png
[HED raw] Image 192/336 : L_04_04.png
[HED raw] Image 193/336 : M_01_01.png
[HED raw] Image 194/336 : M_01_02.png
[HED raw] Image 195/336 : M_01_03.png
[HED raw] Image 196/336 : M_01_04.png
[HED raw] Image 197/336 : M_02_01.png
[HED raw] Image 198/336 : M_02_02.png
[HED raw] Image 199/336 : M_02_03.png
[HED raw] Image 200/336 : M_02_04.png
[HED raw] Image 201/336 : M_03_01.png
[HED raw] Image 202/336 : M_03_02.png
[HED raw] Image 203/336 : M_03_03.png
[HED raw] Image 204/336 : M_03_04.png
[HED raw] Image 205/336 : M_04_01.png
[HED raw] Image 206/336 : M_04_02.png
[HED raw] Image 207/336 : M_04_03.png
[HED raw] Image 208/336 : M_04_04.png
[HED raw] Image 209/336 : N_01_01.png
[HED raw] Image 210/336 : N_01_02.png
[HED raw] Image 211/336 : N_01_03.png
[HED raw] Image 212/336 : N_01_04.png
[HED raw] Image 213/336 : N_02_01.png
[HED raw] Image 214/336 : N_02_02.png
[HED raw] Image 215/336 : N_02_03.png
[HED raw] Image 216/336 : N_02_04.png
[HED raw] Image 217/336 : N_03_01.png
[HED raw] Image 218/336 : N_03_02.png
[HED raw] Image 219/336 : N_03_03.png
[HED raw] Image 220/336 : N_03_04.png
[HED raw] Image 221/336 : N_04_01.png
[HED raw] Image 222/336 : N_04_02.png
[HED raw] Image 223/336 : N_04_03.png
[HED raw] Image 224/336 : N_04_04.png
[HED raw] Image 225/336 : O_01_01.png
[HED raw] Image 226/336 : O_01_02.png
```

```
[HED raw] Image 227/336 : O_01_03.png
[HED raw] Image 228/336 : O_01_04.png
[HED raw] Image 229/336 : O_02_01.png
[HED raw] Image 230/336 : O_02_02.png
[HED raw] Image 231/336 : O_02_03.png
[HED raw] Image 232/336 : O_02_04.png
[HED raw] Image 233/336 : O_03_01.png
[HED raw] Image 234/336 : O_03_02.png
[HED raw] Image 235/336 : O_03_03.png
[HED raw] Image 236/336 : O_03_04.png
[HED raw] Image 237/336 : O_04_01.png
[HED raw] Image 238/336 : O_04_02.png
[HED raw] Image 239/336 : O_04_03.png
[HED raw] Image 240/336 : O_04_04.png
[HED raw] Image 241/336 : P_01_01.png
[HED raw] Image 242/336 : P_01_02.png
[HED raw] Image 243/336 : P_01_03.png
[HED raw] Image 244/336 : P_01_04.png
[HED raw] Image 245/336 : P_02_01.png
[HED raw] Image 246/336 : P_02_02.png
[HED raw] Image 247/336 : P_02_03.png
[HED raw] Image 248/336 : P_02_04.png
[HED raw] Image 249/336 : P_03_01.png
[HED raw] Image 250/336 : P_03_02.png
[HED raw] Image 251/336 : P_03_03.png
[HED raw] Image 252/336 : P_03_04.png
[HED raw] Image 253/336 : P_04_01.png
[HED raw] Image 254/336 : P_04_02.png
[HED raw] Image 255/336 : P_04_03.png
[HED raw] Image 256/336 : P_04_04.png
[HED raw] Image 257/336 : Q_01_01.png
[HED raw] Image 258/336 : Q_01_02.png
[HED raw] Image 259/336 : Q_01_03.png
[HED raw] Image 260/336 : Q_01_04.png
[HED raw] Image 261/336 : Q_02_01.png
[HED raw] Image 262/336 : Q_02_02.png
[HED raw] Image 263/336 : Q_02_03.png
[HED raw] Image 264/336 : Q_02_04.png
[HED raw] Image 265/336 : Q_03_01.png
[HED raw] Image 266/336 : Q_03_02.png
[HED raw] Image 267/336 : Q_03_03.png
[HED raw] Image 268/336 : Q_03_04.png
[HED raw] Image 269/336 : Q_04_01.png
[HED raw] Image 270/336 : Q_04_02.png
[HED raw] Image 271/336 : Q_04_03.png
[HED raw] Image 272/336 : Q_04_04.png
[HED raw] Image 273/336 : R_01_01.png
[HED raw] Image 274/336 : R_01_02.png
```

```
[HED raw] Image 275/336 : R_01_03.png
[HED raw] Image 276/336 : R_01_04.png
[HED raw] Image 277/336 : R_02_01.png
[HED raw] Image 278/336 : R_02_02.png
[HED raw] Image 279/336 : R_02_03.png
[HED raw] Image 280/336 : R_02_04.png
[HED raw] Image 281/336 : R_03_01.png
[HED raw] Image 282/336 : R_03_02.png
[HED raw] Image 283/336 : R_03_03.png
[HED raw] Image 284/336 : R_03_04.png
[HED raw] Image 285/336 : R_04_01.png
[HED raw] Image 286/336 : R_04_02.png
[HED raw] Image 287/336 : R_04_03.png
[HED raw] Image 288/336 : R_04_04.png
[HED raw] Image 289/336 : S_01_01.png
[HED raw] Image 290/336 : S_01_02.png
[HED raw] Image 291/336 : S_01_03.png
[HED raw] Image 292/336 : S_01_04.png
[HED raw] Image 293/336 : S_02_01.png
[HED raw] Image 294/336 : S_02_02.png
[HED raw] Image 295/336 : S_02_03.png
[HED raw] Image 296/336 : S_02_04.png
[HED raw] Image 297/336 : S_03_01.png
[HED raw] Image 298/336 : S_03_02.png
[HED raw] Image 299/336 : S_03_03.png
[HED raw] Image 300/336 : S_03_04.png
[HED raw] Image 301/336 : S_04_01.png
[HED raw] Image 302/336 : S_04_02.png
[HED raw] Image 303/336 : S_04_03.png
[HED raw] Image 304/336 : S_04_04.png
[HED raw] Image 305/336 : T_01_01.png
[HED raw] Image 306/336 : T_01_02.png
[HED raw] Image 307/336 : T_01_03.png
[HED raw] Image 308/336 : T_01_04.png
[HED raw] Image 309/336 : T_02_01.png
[HED raw] Image 310/336 : T_02_02.png
[HED raw] Image 311/336 : T_02_03.png
[HED raw] Image 312/336 : T_02_04.png
[HED raw] Image 313/336 : T_03_01.png
[HED raw] Image 314/336 : T_03_02.png
[HED raw] Image 315/336 : T_03_03.png
[HED raw] Image 316/336 : T_03_04.png
[HED raw] Image 317/336 : T_04_01.png
[HED raw] Image 318/336 : T_04_02.png
[HED raw] Image 319/336 : T_04_03.png
[HED raw] Image 320/336 : T_04_04.png
[HED raw] Image 321/336 : U_01_01.png
[HED raw] Image 322/336 : U_01_02.png
```

```
[HED raw] Image 323/336 : U_01_03.png
[HED raw] Image 324/336 : U_01_04.png
[HED raw] Image 325/336 : U_02_01.png
[HED raw] Image 326/336 : U_02_02.png
[HED raw] Image 327/336 : U_02_03.png
[HED raw] Image 328/336 : U_02_04.png
[HED raw] Image 329/336 : U_03_01.png
[HED raw] Image 330/336 : U_03_02.png
[HED raw] Image 331/336 : U_03_03.png
[HED raw] Image 332/336 : U_03_04.png
[HED raw] Image 333/336 : U_04_01.png
[HED raw] Image 334/336 : U_04_02.png
[HED raw] Image 335/336 : U_04_03.png
[HED raw] Image 336/336 : U_04_04.png
 Cartes HED seuillées sauvegardées dans : /content/drive/MyDrive/Data/HED_raw
```

Ce bloc applique la fonction HED à l'ensemble des 336 micrographies pour produire, pour chaque image de Grains, une carte de contours seuillée : on parcourt toutes les images une par une, on calcule leur version HED binaire (joints de grains noirs / grains blancs), puis on enregistre ces cartes dans le dossier `HED_raw` sous la forme de fichiers `*_HED.png`. Les logs confirment que toutes les images, de `A_01_01.png` jusqu'à `U_04_04.png`, ont bien été traitées et que l'on dispose maintenant d'un jeu complet de masques HED "bruts" prêt à être filtré puis exploité par la méthode des interceptions.

[ ]:
```python
#  Étape 27 - Filtrage des petites zones (aire < 100) et images HED filtrées

area_threshold = 100  #  seuil area en pixels²

raw_files = sorted([f for f in os.listdir(HED_RAW_DIR) if f.lower().endswith(".
 ↪png")])
N_HED_RAW = len(raw_files)
if N_HED_RAW == 0:
    raise RuntimeError(f"Aucun fichier HED_raw trouvé dans {HED_RAW_DIR}.␣
 ↪Vérifie l'étape 26.")

print(f"[HED filtered] Nombre d'images HED brutes trouvées : {N_HED_RAW}")

for t, fname in enumerate(raw_files[:N_HED_RAW]):
    img_path = HED_RAW_DIR / fname
    print(f"[HED filtered] Image {t+1}/{N_HED_RAW} :", img_path.name)

    img = cv2.imread(str(img_path))
    if img is None:
        raise FileNotFoundError(f"Impossible de lire l'image : {img_path}")

    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    _, binary = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)
```

```
    contours, _ = cv2.findContours(binary, cv2.RETR_EXTERNAL, cv2.
↪CHAIN_APPROX_NONE)

    # Image noire pour garder uniquement les grains assez grands
    black = np.zeros_like(img)

    for c in contours:
        if cv2.contourArea(c) > area_threshold:
            cv2.drawContours(black, [c], -1, (255, 255, 255), -1)
        else:
            # debug : en vert sur l'image originale
            cv2.drawContours(img, [c], -1, (0, 255, 0), 2)

    out_name = f"{Path(fname).stem}_FILT.png"
    cv2.imwrite(str(HED_FILTERED_DIR / out_name), black)

print("  Images HED filtrées sauvegardées dans :", HED_FILTERED_DIR)
```

```
[HED filtered] Nombre d'images HED brutes trouvées : 336
[HED filtered] Image 1/336 : A_01_01_HED.png
[HED filtered] Image 2/336 : A_01_02_HED.png
[HED filtered] Image 3/336 : A_01_03_HED.png
[HED filtered] Image 4/336 : A_01_04_HED.png
[HED filtered] Image 5/336 : A_02_01_HED.png
[HED filtered] Image 6/336 : A_02_02_HED.png
[HED filtered] Image 7/336 : A_02_03_HED.png
[HED filtered] Image 8/336 : A_02_04_HED.png
[HED filtered] Image 9/336 : A_03_01_HED.png
[HED filtered] Image 10/336 : A_03_02_HED.png
[HED filtered] Image 11/336 : A_03_03_HED.png
[HED filtered] Image 12/336 : A_03_04_HED.png
[HED filtered] Image 13/336 : A_04_01_HED.png
[HED filtered] Image 14/336 : A_04_02_HED.png
[HED filtered] Image 15/336 : A_04_03_HED.png
[HED filtered] Image 16/336 : A_04_04_HED.png
[HED filtered] Image 17/336 : B_01_01_HED.png
[HED filtered] Image 18/336 : B_01_02_HED.png
[HED filtered] Image 19/336 : B_01_03_HED.png
[HED filtered] Image 20/336 : B_01_04_HED.png
[HED filtered] Image 21/336 : B_02_01_HED.png
[HED filtered] Image 22/336 : B_02_02_HED.png
[HED filtered] Image 23/336 : B_02_03_HED.png
[HED filtered] Image 24/336 : B_02_04_HED.png
[HED filtered] Image 25/336 : B_03_01_HED.png
[HED filtered] Image 26/336 : B_03_02_HED.png
[HED filtered] Image 27/336 : B_03_03_HED.png
[HED filtered] Image 28/336 : B_03_04_HED.png
[HED filtered] Image 29/336 : B_04_01_HED.png
```

```
[HED filtered] Image 30/336 : B_04_02_HED.png
[HED filtered] Image 31/336 : B_04_03_HED.png
[HED filtered] Image 32/336 : B_04_04_HED.png
[HED filtered] Image 33/336 : C_01_01_HED.png
[HED filtered] Image 34/336 : C_01_02_HED.png
[HED filtered] Image 35/336 : C_01_03_HED.png
[HED filtered] Image 36/336 : C_01_04_HED.png
[HED filtered] Image 37/336 : C_02_01_HED.png
[HED filtered] Image 38/336 : C_02_02_HED.png
[HED filtered] Image 39/336 : C_02_03_HED.png
[HED filtered] Image 40/336 : C_02_04_HED.png
[HED filtered] Image 41/336 : C_03_01_HED.png
[HED filtered] Image 42/336 : C_03_02_HED.png
[HED filtered] Image 43/336 : C_03_03_HED.png
[HED filtered] Image 44/336 : C_03_04_HED.png
[HED filtered] Image 45/336 : C_04_01_HED.png
[HED filtered] Image 46/336 : C_04_02_HED.png
[HED filtered] Image 47/336 : C_04_03_HED.png
[HED filtered] Image 48/336 : C_04_04_HED.png
[HED filtered] Image 49/336 : D_01_01_HED.png
[HED filtered] Image 50/336 : D_01_02_HED.png
[HED filtered] Image 51/336 : D_01_03_HED.png
[HED filtered] Image 52/336 : D_01_04_HED.png
[HED filtered] Image 53/336 : D_02_01_HED.png
[HED filtered] Image 54/336 : D_02_02_HED.png
[HED filtered] Image 55/336 : D_02_03_HED.png
[HED filtered] Image 56/336 : D_02_04_HED.png
[HED filtered] Image 57/336 : D_03_01_HED.png
[HED filtered] Image 58/336 : D_03_02_HED.png
[HED filtered] Image 59/336 : D_03_03_HED.png
[HED filtered] Image 60/336 : D_03_04_HED.png
[HED filtered] Image 61/336 : D_04_01_HED.png
[HED filtered] Image 62/336 : D_04_02_HED.png
[HED filtered] Image 63/336 : D_04_03_HED.png
[HED filtered] Image 64/336 : D_04_04_HED.png
[HED filtered] Image 65/336 : E_01_01_HED.png
[HED filtered] Image 66/336 : E_01_02_HED.png
[HED filtered] Image 67/336 : E_01_03_HED.png
[HED filtered] Image 68/336 : E_01_04_HED.png
[HED filtered] Image 69/336 : E_02_01_HED.png
[HED filtered] Image 70/336 : E_02_02_HED.png
[HED filtered] Image 71/336 : E_02_03_HED.png
[HED filtered] Image 72/336 : E_02_04_HED.png
[HED filtered] Image 73/336 : E_03_01_HED.png
[HED filtered] Image 74/336 : E_03_02_HED.png
[HED filtered] Image 75/336 : E_03_03_HED.png
[HED filtered] Image 76/336 : E_03_04_HED.png
[HED filtered] Image 77/336 : E_04_01_HED.png
```

```
[HED filtered] Image 78/336 : E_04_02_HED.png
[HED filtered] Image 79/336 : E_04_03_HED.png
[HED filtered] Image 80/336 : E_04_04_HED.png
[HED filtered] Image 81/336 : F_01_01_HED.png
[HED filtered] Image 82/336 : F_01_02_HED.png
[HED filtered] Image 83/336 : F_01_03_HED.png
[HED filtered] Image 84/336 : F_01_04_HED.png
[HED filtered] Image 85/336 : F_02_01_HED.png
[HED filtered] Image 86/336 : F_02_02_HED.png
[HED filtered] Image 87/336 : F_02_03_HED.png
[HED filtered] Image 88/336 : F_02_04_HED.png
[HED filtered] Image 89/336 : F_03_01_HED.png
[HED filtered] Image 90/336 : F_03_02_HED.png
[HED filtered] Image 91/336 : F_03_03_HED.png
[HED filtered] Image 92/336 : F_03_04_HED.png
[HED filtered] Image 93/336 : F_04_01_HED.png
[HED filtered] Image 94/336 : F_04_02_HED.png
[HED filtered] Image 95/336 : F_04_03_HED.png
[HED filtered] Image 96/336 : F_04_04_HED.png
[HED filtered] Image 97/336 : G_01_01_HED.png
[HED filtered] Image 98/336 : G_01_02_HED.png
[HED filtered] Image 99/336 : G_01_03_HED.png
[HED filtered] Image 100/336 : G_01_04_HED.png
[HED filtered] Image 101/336 : G_02_01_HED.png
[HED filtered] Image 102/336 : G_02_02_HED.png
[HED filtered] Image 103/336 : G_02_03_HED.png
[HED filtered] Image 104/336 : G_02_04_HED.png
[HED filtered] Image 105/336 : G_03_01_HED.png
[HED filtered] Image 106/336 : G_03_02_HED.png
[HED filtered] Image 107/336 : G_03_03_HED.png
[HED filtered] Image 108/336 : G_03_04_HED.png
[HED filtered] Image 109/336 : G_04_01_HED.png
[HED filtered] Image 110/336 : G_04_02_HED.png
[HED filtered] Image 111/336 : G_04_03_HED.png
[HED filtered] Image 112/336 : G_04_04_HED.png
[HED filtered] Image 113/336 : H_01_01_HED.png
[HED filtered] Image 114/336 : H_01_02_HED.png
[HED filtered] Image 115/336 : H_01_03_HED.png
[HED filtered] Image 116/336 : H_01_04_HED.png
[HED filtered] Image 117/336 : H_02_01_HED.png
[HED filtered] Image 118/336 : H_02_02_HED.png
[HED filtered] Image 119/336 : H_02_03_HED.png
[HED filtered] Image 120/336 : H_02_04_HED.png
[HED filtered] Image 121/336 : H_03_01_HED.png
[HED filtered] Image 122/336 : H_03_02_HED.png
[HED filtered] Image 123/336 : H_03_03_HED.png
[HED filtered] Image 124/336 : H_03_04_HED.png
[HED filtered] Image 125/336 : H_04_01_HED.png
```

```
[HED filtered] Image 126/336 : H_04_02_HED.png
[HED filtered] Image 127/336 : H_04_03_HED.png
[HED filtered] Image 128/336 : H_04_04_HED.png
[HED filtered] Image 129/336 : I_01_01_HED.png
[HED filtered] Image 130/336 : I_01_02_HED.png
[HED filtered] Image 131/336 : I_01_03_HED.png
[HED filtered] Image 132/336 : I_01_04_HED.png
[HED filtered] Image 133/336 : I_02_01_HED.png
[HED filtered] Image 134/336 : I_02_02_HED.png
[HED filtered] Image 135/336 : I_02_03_HED.png
[HED filtered] Image 136/336 : I_02_04_HED.png
[HED filtered] Image 137/336 : I_03_01_HED.png
[HED filtered] Image 138/336 : I_03_02_HED.png
[HED filtered] Image 139/336 : I_03_03_HED.png
[HED filtered] Image 140/336 : I_03_04_HED.png
[HED filtered] Image 141/336 : I_04_01_HED.png
[HED filtered] Image 142/336 : I_04_02_HED.png
[HED filtered] Image 143/336 : I_04_03_HED.png
[HED filtered] Image 144/336 : I_04_04_HED.png
[HED filtered] Image 145/336 : J_01_01_HED.png
[HED filtered] Image 146/336 : J_01_02_HED.png
[HED filtered] Image 147/336 : J_01_03_HED.png
[HED filtered] Image 148/336 : J_01_04_HED.png
[HED filtered] Image 149/336 : J_02_01_HED.png
[HED filtered] Image 150/336 : J_02_02_HED.png
[HED filtered] Image 151/336 : J_02_03_HED.png
[HED filtered] Image 152/336 : J_02_04_HED.png
[HED filtered] Image 153/336 : J_03_01_HED.png
[HED filtered] Image 154/336 : J_03_02_HED.png
[HED filtered] Image 155/336 : J_03_03_HED.png
[HED filtered] Image 156/336 : J_03_04_HED.png
[HED filtered] Image 157/336 : J_04_01_HED.png
[HED filtered] Image 158/336 : J_04_02_HED.png
[HED filtered] Image 159/336 : J_04_03_HED.png
[HED filtered] Image 160/336 : J_04_04_HED.png
[HED filtered] Image 161/336 : K_01_01_HED.png
[HED filtered] Image 162/336 : K_01_02_HED.png
[HED filtered] Image 163/336 : K_01_03_HED.png
[HED filtered] Image 164/336 : K_01_04_HED.png
[HED filtered] Image 165/336 : K_02_01_HED.png
[HED filtered] Image 166/336 : K_02_02_HED.png
[HED filtered] Image 167/336 : K_02_03_HED.png
[HED filtered] Image 168/336 : K_02_04_HED.png
[HED filtered] Image 169/336 : K_03_01_HED.png
[HED filtered] Image 170/336 : K_03_02_HED.png
[HED filtered] Image 171/336 : K_03_03_HED.png
[HED filtered] Image 172/336 : K_03_04_HED.png
[HED filtered] Image 173/336 : K_04_01_HED.png
```

```
[HED filtered] Image 174/336 : K_04_02_HED.png
[HED filtered] Image 175/336 : K_04_03_HED.png
[HED filtered] Image 176/336 : K_04_04_HED.png
[HED filtered] Image 177/336 : L_01_01_HED.png
[HED filtered] Image 178/336 : L_01_02_HED.png
[HED filtered] Image 179/336 : L_01_03_HED.png
[HED filtered] Image 180/336 : L_01_04_HED.png
[HED filtered] Image 181/336 : L_02_01_HED.png
[HED filtered] Image 182/336 : L_02_02_HED.png
[HED filtered] Image 183/336 : L_02_03_HED.png
[HED filtered] Image 184/336 : L_02_04_HED.png
[HED filtered] Image 185/336 : L_03_01_HED.png
[HED filtered] Image 186/336 : L_03_02_HED.png
[HED filtered] Image 187/336 : L_03_03_HED.png
[HED filtered] Image 188/336 : L_03_04_HED.png
[HED filtered] Image 189/336 : L_04_01_HED.png
[HED filtered] Image 190/336 : L_04_02_HED.png
[HED filtered] Image 191/336 : L_04_03_HED.png
[HED filtered] Image 192/336 : L_04_04_HED.png
[HED filtered] Image 193/336 : M_01_01_HED.png
[HED filtered] Image 194/336 : M_01_02_HED.png
[HED filtered] Image 195/336 : M_01_03_HED.png
[HED filtered] Image 196/336 : M_01_04_HED.png
[HED filtered] Image 197/336 : M_02_01_HED.png
[HED filtered] Image 198/336 : M_02_02_HED.png
[HED filtered] Image 199/336 : M_02_03_HED.png
[HED filtered] Image 200/336 : M_02_04_HED.png
[HED filtered] Image 201/336 : M_03_01_HED.png
[HED filtered] Image 202/336 : M_03_02_HED.png
[HED filtered] Image 203/336 : M_03_03_HED.png
[HED filtered] Image 204/336 : M_03_04_HED.png
[HED filtered] Image 205/336 : M_04_01_HED.png
[HED filtered] Image 206/336 : M_04_02_HED.png
[HED filtered] Image 207/336 : M_04_03_HED.png
[HED filtered] Image 208/336 : M_04_04_HED.png
[HED filtered] Image 209/336 : N_01_01_HED.png
[HED filtered] Image 210/336 : N_01_02_HED.png
[HED filtered] Image 211/336 : N_01_03_HED.png
[HED filtered] Image 212/336 : N_01_04_HED.png
[HED filtered] Image 213/336 : N_02_01_HED.png
[HED filtered] Image 214/336 : N_02_02_HED.png
[HED filtered] Image 215/336 : N_02_03_HED.png
[HED filtered] Image 216/336 : N_02_04_HED.png
[HED filtered] Image 217/336 : N_03_01_HED.png
[HED filtered] Image 218/336 : N_03_02_HED.png
[HED filtered] Image 219/336 : N_03_03_HED.png
[HED filtered] Image 220/336 : N_03_04_HED.png
[HED filtered] Image 221/336 : N_04_01_HED.png
```

```
[HED filtered] Image 222/336 : N_04_02_HED.png
[HED filtered] Image 223/336 : N_04_03_HED.png
[HED filtered] Image 224/336 : N_04_04_HED.png
[HED filtered] Image 225/336 : O_01_01_HED.png
[HED filtered] Image 226/336 : O_01_02_HED.png
[HED filtered] Image 227/336 : O_01_03_HED.png
[HED filtered] Image 228/336 : O_01_04_HED.png
[HED filtered] Image 229/336 : O_02_01_HED.png
[HED filtered] Image 230/336 : O_02_02_HED.png
[HED filtered] Image 231/336 : O_02_03_HED.png
[HED filtered] Image 232/336 : O_02_04_HED.png
[HED filtered] Image 233/336 : O_03_01_HED.png
[HED filtered] Image 234/336 : O_03_02_HED.png
[HED filtered] Image 235/336 : O_03_03_HED.png
[HED filtered] Image 236/336 : O_03_04_HED.png
[HED filtered] Image 237/336 : O_04_01_HED.png
[HED filtered] Image 238/336 : O_04_02_HED.png
[HED filtered] Image 239/336 : O_04_03_HED.png
[HED filtered] Image 240/336 : O_04_04_HED.png
[HED filtered] Image 241/336 : P_01_01_HED.png
[HED filtered] Image 242/336 : P_01_02_HED.png
[HED filtered] Image 243/336 : P_01_03_HED.png
[HED filtered] Image 244/336 : P_01_04_HED.png
[HED filtered] Image 245/336 : P_02_01_HED.png
[HED filtered] Image 246/336 : P_02_02_HED.png
[HED filtered] Image 247/336 : P_02_03_HED.png
[HED filtered] Image 248/336 : P_02_04_HED.png
[HED filtered] Image 249/336 : P_03_01_HED.png
[HED filtered] Image 250/336 : P_03_02_HED.png
[HED filtered] Image 251/336 : P_03_03_HED.png
[HED filtered] Image 252/336 : P_03_04_HED.png
[HED filtered] Image 253/336 : P_04_01_HED.png
[HED filtered] Image 254/336 : P_04_02_HED.png
[HED filtered] Image 255/336 : P_04_03_HED.png
[HED filtered] Image 256/336 : P_04_04_HED.png
[HED filtered] Image 257/336 : Q_01_01_HED.png
[HED filtered] Image 258/336 : Q_01_02_HED.png
[HED filtered] Image 259/336 : Q_01_03_HED.png
[HED filtered] Image 260/336 : Q_01_04_HED.png
[HED filtered] Image 261/336 : Q_02_01_HED.png
[HED filtered] Image 262/336 : Q_02_02_HED.png
[HED filtered] Image 263/336 : Q_02_03_HED.png
[HED filtered] Image 264/336 : Q_02_04_HED.png
[HED filtered] Image 265/336 : Q_03_01_HED.png
[HED filtered] Image 266/336 : Q_03_02_HED.png
[HED filtered] Image 267/336 : Q_03_03_HED.png
[HED filtered] Image 268/336 : Q_03_04_HED.png
[HED filtered] Image 269/336 : Q_04_01_HED.png
```

```
[HED filtered] Image 270/336 : Q_04_02_HED.png
[HED filtered] Image 271/336 : Q_04_03_HED.png
[HED filtered] Image 272/336 : Q_04_04_HED.png
[HED filtered] Image 273/336 : R_01_01_HED.png
[HED filtered] Image 274/336 : R_01_02_HED.png
[HED filtered] Image 275/336 : R_01_03_HED.png
[HED filtered] Image 276/336 : R_01_04_HED.png
[HED filtered] Image 277/336 : R_02_01_HED.png
[HED filtered] Image 278/336 : R_02_02_HED.png
[HED filtered] Image 279/336 : R_02_03_HED.png
[HED filtered] Image 280/336 : R_02_04_HED.png
[HED filtered] Image 281/336 : R_03_01_HED.png
[HED filtered] Image 282/336 : R_03_02_HED.png
[HED filtered] Image 283/336 : R_03_03_HED.png
[HED filtered] Image 284/336 : R_03_04_HED.png
[HED filtered] Image 285/336 : R_04_01_HED.png
[HED filtered] Image 286/336 : R_04_02_HED.png
[HED filtered] Image 287/336 : R_04_03_HED.png
[HED filtered] Image 288/336 : R_04_04_HED.png
[HED filtered] Image 289/336 : S_01_01_HED.png
[HED filtered] Image 290/336 : S_01_02_HED.png
[HED filtered] Image 291/336 : S_01_03_HED.png
[HED filtered] Image 292/336 : S_01_04_HED.png
[HED filtered] Image 293/336 : S_02_01_HED.png
[HED filtered] Image 294/336 : S_02_02_HED.png
[HED filtered] Image 295/336 : S_02_03_HED.png
[HED filtered] Image 296/336 : S_02_04_HED.png
[HED filtered] Image 297/336 : S_03_01_HED.png
[HED filtered] Image 298/336 : S_03_02_HED.png
[HED filtered] Image 299/336 : S_03_03_HED.png
[HED filtered] Image 300/336 : S_03_04_HED.png
[HED filtered] Image 301/336 : S_04_01_HED.png
[HED filtered] Image 302/336 : S_04_02_HED.png
[HED filtered] Image 303/336 : S_04_03_HED.png
[HED filtered] Image 304/336 : S_04_04_HED.png
[HED filtered] Image 305/336 : T_01_01_HED.png
[HED filtered] Image 306/336 : T_01_02_HED.png
[HED filtered] Image 307/336 : T_01_03_HED.png
[HED filtered] Image 308/336 : T_01_04_HED.png
[HED filtered] Image 309/336 : T_02_01_HED.png
[HED filtered] Image 310/336 : T_02_02_HED.png
[HED filtered] Image 311/336 : T_02_03_HED.png
[HED filtered] Image 312/336 : T_02_04_HED.png
[HED filtered] Image 313/336 : T_03_01_HED.png
[HED filtered] Image 314/336 : T_03_02_HED.png
[HED filtered] Image 315/336 : T_03_03_HED.png
[HED filtered] Image 316/336 : T_03_04_HED.png
[HED filtered] Image 317/336 : T_04_01_HED.png
```

```
[HED filtered] Image 318/336 : T_04_02_HED.png
[HED filtered] Image 319/336 : T_04_03_HED.png
[HED filtered] Image 320/336 : T_04_04_HED.png
[HED filtered] Image 321/336 : U_01_01_HED.png
[HED filtered] Image 322/336 : U_01_02_HED.png
[HED filtered] Image 323/336 : U_01_03_HED.png
[HED filtered] Image 324/336 : U_01_04_HED.png
[HED filtered] Image 325/336 : U_02_01_HED.png
[HED filtered] Image 326/336 : U_02_02_HED.png
[HED filtered] Image 327/336 : U_02_03_HED.png
[HED filtered] Image 328/336 : U_02_04_HED.png
[HED filtered] Image 329/336 : U_03_01_HED.png
[HED filtered] Image 330/336 : U_03_02_HED.png
[HED filtered] Image 331/336 : U_03_03_HED.png
[HED filtered] Image 332/336 : U_03_04_HED.png
[HED filtered] Image 333/336 : U_04_01_HED.png
[HED filtered] Image 334/336 : U_04_02_HED.png
[HED filtered] Image 335/336 : U_04_03_HED.png
[HED filtered] Image 336/336 : U_04_04_HED.png
 Images HED filtrées sauvegardées dans :
/content/drive/MyDrive/Data/HED_filtered
```

Ce bloc prend les 336 cartes HED brutes produites à l'étape précédente et les nettoie en éliminant les plus petites régions, considérées comme du bruit de segmentation : pour chaque image *_HED.png, on repasse en niveaux de gris, on binarise puis on détecte les contours ; on crée ensuite une image noire sur laquelle on ne redessine en blanc que les contours dont l'aire dépasse 100 pixels², tandis que les petits objets sont simplement marqués en vert sur l'image originale à titre de debug. Les nouvelles cartes HED ainsi filtrées, qui ne contiennent plus que des grains suffisamment grands et cohérents, sont sauvegardées dans le dossier `HED_filtered` avec le suffixe `_FILT.png`. Les logs montrent que l'opération a bien été appliquée à l'ensemble des micrographies, de `A_01_01_HED.png` à `U_04_04_HED.png`, ce qui prépare un jeu de masques HED "propres" pour le comptage d'interceptions.

[ ]:
```python
#  Étape 28 - Initialisation des matrices d'interceptions HED

value = 255   # valeur de transition à compter dans diff_x / diff_y

# On se basera sur le nombre d'images HED filtrées, pas sur l'ancien n=40
filt_files = sorted([f for f in os.listdir(HED_FILTERED_DIR) if f.lower().
 ↪endswith(".png")])
N_HED_FILT = len(filt_files)
if N_HED_FILT == 0:
    raise RuntimeError(f"Aucun fichier HED filtré trouvé dans␣
 ↪{HED_FILTERED_DIR}. Vérifie l'étape 27.")

Total_Grains_X_HED = np.zeros((numrows, N_HED_FILT), dtype=np.float32)
Total_Grains_Y_HED = np.zeros((numcols, N_HED_FILT), dtype=np.float32)
```

```
print("Total_Grains_X_HED shape :", Total_Grains_X_HED.shape)
print("Total_Grains_Y_HED shape :", Total_Grains_Y_HED.shape)
print("Nombre d'images HED filtrées (N_HED_FILT) :", N_HED_FILT)
```

```
Total_Grains_X_HED shape : (300, 336)
Total_Grains_Y_HED shape : (400, 336)
Nombre d'images HED filtrées (N_HED_FILT) : 336
```

Ce bloc prépare le terrain pour le Line Intercept appliqué aux masques HED filtrés : on commence par lister tous les fichiers *_FILT.png présents dans `HED_FILTERED_DIR` et on vérifie qu'il y en a bien au moins un (sinon on déclenche une erreur, ce qui sécurise le pipeline). Le nombre d'images trouvées `N_HED_FILT` sert directement à dimensionner deux matrices de comptage, `Total_Grains_X_HED` et `Total_Grains_Y_HED`, respectivement de taille (300, 336) et (400, 336) en float32 : chaque ligne représente une position de maillage (300 lignes horizontales, 400 colonnes verticales) et chaque colonne correspond à une micrographie HED filtrée. Initialisées à zéro, ces matrices accueilleront ensuite, pour chaque image, le nombre d'interceptions ligne/grain (X) et colonne/grain (Y) détectées, en comptant les transitions de valeur 255 dans les dérivées `diff_x` et `diff_y`. Les impressions en sortie confirment que les dimensions sont cohérentes avec la grille et que les 336 masques HED filtrés seront bien pris en compte dans la suite de l'analyse.

[ ]:
```
#  Étape 29 - Line Intercept sur les images HED filtrées

filt_files = sorted([f for f in os.listdir(HED_FILTERED_DIR) if f.lower().
 ↪endswith(".png")])
N_HED_FILT = len(filt_files)
if N_HED_FILT == 0:
    raise RuntimeError(f"Aucun fichier HED filtré trouvé dans␣
 ↪{HED_FILTERED_DIR}. Vérifie l'étape 27.")

for t, fname in enumerate(filt_files[:N_HED_FILT]):

    img_path = HED_FILTERED_DIR / fname
    print(f"[Line Intercept HED] Image {t+1}/{N_HED_FILT} :", img_path.name)

    img = cv2.imread(str(img_path), cv2.IMREAD_GRAYSCALE)
    if img is None:
        raise FileNotFoundError(f"Impossible de lire l'image : {img_path}")

    # Re-seuil comme dans le notebook (90, 255, THRESH_BINARY)
    _, thresh = cv2.threshold(img, 90, 255, cv2.THRESH_BINARY)

    diff_x = np.diff(thresh, axis=1)
    diff_y = np.diff(thresh, axis=0)

    # Sécurité sur la taille réelle de l'image
    h, w = thresh.shape
    h_eff = min(h, numrows)
    w_eff = min(w, numcols)
```

```
    # Lignes (X)
    for x in range(h_eff):
        count = 0
        for element in diff_x[x, :]:
            if element == value:
                count += 1
        Total_Grains_X_HED[x, t] = count

    # Colonnes (Y)
    for y in range(w_eff - 1):
        count = 0
        for element in diff_y[:, y]:
            if element == value:
                count += 1
        Total_Grains_Y_HED[y, t] = count

print("  Boucle Line Intercept HED terminée.")
```

```
[Line Intercept HED] Image 1/336 : A_01_01_HED_FILT.png
[Line Intercept HED] Image 2/336 : A_01_02_HED_FILT.png
[Line Intercept HED] Image 3/336 : A_01_03_HED_FILT.png
[Line Intercept HED] Image 4/336 : A_01_04_HED_FILT.png
[Line Intercept HED] Image 5/336 : A_02_01_HED_FILT.png
[Line Intercept HED] Image 6/336 : A_02_02_HED_FILT.png
[Line Intercept HED] Image 7/336 : A_02_03_HED_FILT.png
[Line Intercept HED] Image 8/336 : A_02_04_HED_FILT.png
[Line Intercept HED] Image 9/336 : A_03_01_HED_FILT.png
[Line Intercept HED] Image 10/336 : A_03_02_HED_FILT.png
[Line Intercept HED] Image 11/336 : A_03_03_HED_FILT.png
[Line Intercept HED] Image 12/336 : A_03_04_HED_FILT.png
[Line Intercept HED] Image 13/336 : A_04_01_HED_FILT.png
[Line Intercept HED] Image 14/336 : A_04_02_HED_FILT.png
[Line Intercept HED] Image 15/336 : A_04_03_HED_FILT.png
[Line Intercept HED] Image 16/336 : A_04_04_HED_FILT.png
[Line Intercept HED] Image 17/336 : B_01_01_HED_FILT.png
[Line Intercept HED] Image 18/336 : B_01_02_HED_FILT.png
[Line Intercept HED] Image 19/336 : B_01_03_HED_FILT.png
[Line Intercept HED] Image 20/336 : B_01_04_HED_FILT.png
[Line Intercept HED] Image 21/336 : B_02_01_HED_FILT.png
[Line Intercept HED] Image 22/336 : B_02_02_HED_FILT.png
[Line Intercept HED] Image 23/336 : B_02_03_HED_FILT.png
[Line Intercept HED] Image 24/336 : B_02_04_HED_FILT.png
[Line Intercept HED] Image 25/336 : B_03_01_HED_FILT.png
[Line Intercept HED] Image 26/336 : B_03_02_HED_FILT.png
[Line Intercept HED] Image 27/336 : B_03_03_HED_FILT.png
[Line Intercept HED] Image 28/336 : B_03_04_HED_FILT.png
[Line Intercept HED] Image 29/336 : B_04_01_HED_FILT.png
```

```
[Line Intercept HED] Image 30/336 : B_04_02_HED_FILT.png
[Line Intercept HED] Image 31/336 : B_04_03_HED_FILT.png
[Line Intercept HED] Image 32/336 : B_04_04_HED_FILT.png
[Line Intercept HED] Image 33/336 : C_01_01_HED_FILT.png
[Line Intercept HED] Image 34/336 : C_01_02_HED_FILT.png
[Line Intercept HED] Image 35/336 : C_01_03_HED_FILT.png
[Line Intercept HED] Image 36/336 : C_01_04_HED_FILT.png
[Line Intercept HED] Image 37/336 : C_02_01_HED_FILT.png
[Line Intercept HED] Image 38/336 : C_02_02_HED_FILT.png
[Line Intercept HED] Image 39/336 : C_02_03_HED_FILT.png
[Line Intercept HED] Image 40/336 : C_02_04_HED_FILT.png
[Line Intercept HED] Image 41/336 : C_03_01_HED_FILT.png
[Line Intercept HED] Image 42/336 : C_03_02_HED_FILT.png
[Line Intercept HED] Image 43/336 : C_03_03_HED_FILT.png
[Line Intercept HED] Image 44/336 : C_03_04_HED_FILT.png
[Line Intercept HED] Image 45/336 : C_04_01_HED_FILT.png
[Line Intercept HED] Image 46/336 : C_04_02_HED_FILT.png
[Line Intercept HED] Image 47/336 : C_04_03_HED_FILT.png
[Line Intercept HED] Image 48/336 : C_04_04_HED_FILT.png
[Line Intercept HED] Image 49/336 : D_01_01_HED_FILT.png
[Line Intercept HED] Image 50/336 : D_01_02_HED_FILT.png
[Line Intercept HED] Image 51/336 : D_01_03_HED_FILT.png
[Line Intercept HED] Image 52/336 : D_01_04_HED_FILT.png
[Line Intercept HED] Image 53/336 : D_02_01_HED_FILT.png
[Line Intercept HED] Image 54/336 : D_02_02_HED_FILT.png
[Line Intercept HED] Image 55/336 : D_02_03_HED_FILT.png
[Line Intercept HED] Image 56/336 : D_02_04_HED_FILT.png
[Line Intercept HED] Image 57/336 : D_03_01_HED_FILT.png
[Line Intercept HED] Image 58/336 : D_03_02_HED_FILT.png
[Line Intercept HED] Image 59/336 : D_03_03_HED_FILT.png
[Line Intercept HED] Image 60/336 : D_03_04_HED_FILT.png
[Line Intercept HED] Image 61/336 : D_04_01_HED_FILT.png
[Line Intercept HED] Image 62/336 : D_04_02_HED_FILT.png
[Line Intercept HED] Image 63/336 : D_04_03_HED_FILT.png
[Line Intercept HED] Image 64/336 : D_04_04_HED_FILT.png
[Line Intercept HED] Image 65/336 : E_01_01_HED_FILT.png
[Line Intercept HED] Image 66/336 : E_01_02_HED_FILT.png
[Line Intercept HED] Image 67/336 : E_01_03_HED_FILT.png
[Line Intercept HED] Image 68/336 : E_01_04_HED_FILT.png
[Line Intercept HED] Image 69/336 : E_02_01_HED_FILT.png
[Line Intercept HED] Image 70/336 : E_02_02_HED_FILT.png
[Line Intercept HED] Image 71/336 : E_02_03_HED_FILT.png
[Line Intercept HED] Image 72/336 : E_02_04_HED_FILT.png
[Line Intercept HED] Image 73/336 : E_03_01_HED_FILT.png
[Line Intercept HED] Image 74/336 : E_03_02_HED_FILT.png
[Line Intercept HED] Image 75/336 : E_03_03_HED_FILT.png
[Line Intercept HED] Image 76/336 : E_03_04_HED_FILT.png
[Line Intercept HED] Image 77/336 : E_04_01_HED_FILT.png
```

```
[Line Intercept HED] Image 78/336 : E_04_02_HED_FILT.png
[Line Intercept HED] Image 79/336 : E_04_03_HED_FILT.png
[Line Intercept HED] Image 80/336 : E_04_04_HED_FILT.png
[Line Intercept HED] Image 81/336 : F_01_01_HED_FILT.png
[Line Intercept HED] Image 82/336 : F_01_02_HED_FILT.png
[Line Intercept HED] Image 83/336 : F_01_03_HED_FILT.png
[Line Intercept HED] Image 84/336 : F_01_04_HED_FILT.png
[Line Intercept HED] Image 85/336 : F_02_01_HED_FILT.png
[Line Intercept HED] Image 86/336 : F_02_02_HED_FILT.png
[Line Intercept HED] Image 87/336 : F_02_03_HED_FILT.png
[Line Intercept HED] Image 88/336 : F_02_04_HED_FILT.png
[Line Intercept HED] Image 89/336 : F_03_01_HED_FILT.png
[Line Intercept HED] Image 90/336 : F_03_02_HED_FILT.png
[Line Intercept HED] Image 91/336 : F_03_03_HED_FILT.png
[Line Intercept HED] Image 92/336 : F_03_04_HED_FILT.png
[Line Intercept HED] Image 93/336 : F_04_01_HED_FILT.png
[Line Intercept HED] Image 94/336 : F_04_02_HED_FILT.png
[Line Intercept HED] Image 95/336 : F_04_03_HED_FILT.png
[Line Intercept HED] Image 96/336 : F_04_04_HED_FILT.png
[Line Intercept HED] Image 97/336 : G_01_01_HED_FILT.png
[Line Intercept HED] Image 98/336 : G_01_02_HED_FILT.png
[Line Intercept HED] Image 99/336 : G_01_03_HED_FILT.png
[Line Intercept HED] Image 100/336 : G_01_04_HED_FILT.png
[Line Intercept HED] Image 101/336 : G_02_01_HED_FILT.png
[Line Intercept HED] Image 102/336 : G_02_02_HED_FILT.png
[Line Intercept HED] Image 103/336 : G_02_03_HED_FILT.png
[Line Intercept HED] Image 104/336 : G_02_04_HED_FILT.png
[Line Intercept HED] Image 105/336 : G_03_01_HED_FILT.png
[Line Intercept HED] Image 106/336 : G_03_02_HED_FILT.png
[Line Intercept HED] Image 107/336 : G_03_03_HED_FILT.png
[Line Intercept HED] Image 108/336 : G_03_04_HED_FILT.png
[Line Intercept HED] Image 109/336 : G_04_01_HED_FILT.png
[Line Intercept HED] Image 110/336 : G_04_02_HED_FILT.png
[Line Intercept HED] Image 111/336 : G_04_03_HED_FILT.png
[Line Intercept HED] Image 112/336 : G_04_04_HED_FILT.png
[Line Intercept HED] Image 113/336 : H_01_01_HED_FILT.png
[Line Intercept HED] Image 114/336 : H_01_02_HED_FILT.png
[Line Intercept HED] Image 115/336 : H_01_03_HED_FILT.png
[Line Intercept HED] Image 116/336 : H_01_04_HED_FILT.png
[Line Intercept HED] Image 117/336 : H_02_01_HED_FILT.png
[Line Intercept HED] Image 118/336 : H_02_02_HED_FILT.png
[Line Intercept HED] Image 119/336 : H_02_03_HED_FILT.png
[Line Intercept HED] Image 120/336 : H_02_04_HED_FILT.png
[Line Intercept HED] Image 121/336 : H_03_01_HED_FILT.png
[Line Intercept HED] Image 122/336 : H_03_02_HED_FILT.png
[Line Intercept HED] Image 123/336 : H_03_03_HED_FILT.png
[Line Intercept HED] Image 124/336 : H_03_04_HED_FILT.png
[Line Intercept HED] Image 125/336 : H_04_01_HED_FILT.png
```

```
[Line Intercept HED] Image 126/336 : H_04_02_HED_FILT.png
[Line Intercept HED] Image 127/336 : H_04_03_HED_FILT.png
[Line Intercept HED] Image 128/336 : H_04_04_HED_FILT.png
[Line Intercept HED] Image 129/336 : I_01_01_HED_FILT.png
[Line Intercept HED] Image 130/336 : I_01_02_HED_FILT.png
[Line Intercept HED] Image 131/336 : I_01_03_HED_FILT.png
[Line Intercept HED] Image 132/336 : I_01_04_HED_FILT.png
[Line Intercept HED] Image 133/336 : I_02_01_HED_FILT.png
[Line Intercept HED] Image 134/336 : I_02_02_HED_FILT.png
[Line Intercept HED] Image 135/336 : I_02_03_HED_FILT.png
[Line Intercept HED] Image 136/336 : I_02_04_HED_FILT.png
[Line Intercept HED] Image 137/336 : I_03_01_HED_FILT.png
[Line Intercept HED] Image 138/336 : I_03_02_HED_FILT.png
[Line Intercept HED] Image 139/336 : I_03_03_HED_FILT.png
[Line Intercept HED] Image 140/336 : I_03_04_HED_FILT.png
[Line Intercept HED] Image 141/336 : I_04_01_HED_FILT.png
[Line Intercept HED] Image 142/336 : I_04_02_HED_FILT.png
[Line Intercept HED] Image 143/336 : I_04_03_HED_FILT.png
[Line Intercept HED] Image 144/336 : I_04_04_HED_FILT.png
[Line Intercept HED] Image 145/336 : J_01_01_HED_FILT.png
[Line Intercept HED] Image 146/336 : J_01_02_HED_FILT.png
[Line Intercept HED] Image 147/336 : J_01_03_HED_FILT.png
[Line Intercept HED] Image 148/336 : J_01_04_HED_FILT.png
[Line Intercept HED] Image 149/336 : J_02_01_HED_FILT.png
[Line Intercept HED] Image 150/336 : J_02_02_HED_FILT.png
[Line Intercept HED] Image 151/336 : J_02_03_HED_FILT.png
[Line Intercept HED] Image 152/336 : J_02_04_HED_FILT.png
[Line Intercept HED] Image 153/336 : J_03_01_HED_FILT.png
[Line Intercept HED] Image 154/336 : J_03_02_HED_FILT.png
[Line Intercept HED] Image 155/336 : J_03_03_HED_FILT.png
[Line Intercept HED] Image 156/336 : J_03_04_HED_FILT.png
[Line Intercept HED] Image 157/336 : J_04_01_HED_FILT.png
[Line Intercept HED] Image 158/336 : J_04_02_HED_FILT.png
[Line Intercept HED] Image 159/336 : J_04_03_HED_FILT.png
[Line Intercept HED] Image 160/336 : J_04_04_HED_FILT.png
[Line Intercept HED] Image 161/336 : K_01_01_HED_FILT.png
[Line Intercept HED] Image 162/336 : K_01_02_HED_FILT.png
[Line Intercept HED] Image 163/336 : K_01_03_HED_FILT.png
[Line Intercept HED] Image 164/336 : K_01_04_HED_FILT.png
[Line Intercept HED] Image 165/336 : K_02_01_HED_FILT.png
[Line Intercept HED] Image 166/336 : K_02_02_HED_FILT.png
[Line Intercept HED] Image 167/336 : K_02_03_HED_FILT.png
[Line Intercept HED] Image 168/336 : K_02_04_HED_FILT.png
[Line Intercept HED] Image 169/336 : K_03_01_HED_FILT.png
[Line Intercept HED] Image 170/336 : K_03_02_HED_FILT.png
[Line Intercept HED] Image 171/336 : K_03_03_HED_FILT.png
[Line Intercept HED] Image 172/336 : K_03_04_HED_FILT.png
[Line Intercept HED] Image 173/336 : K_04_01_HED_FILT.png
```

```
[Line Intercept HED] Image 174/336 : K_04_02_HED_FILT.png
[Line Intercept HED] Image 175/336 : K_04_03_HED_FILT.png
[Line Intercept HED] Image 176/336 : K_04_04_HED_FILT.png
[Line Intercept HED] Image 177/336 : L_01_01_HED_FILT.png
[Line Intercept HED] Image 178/336 : L_01_02_HED_FILT.png
[Line Intercept HED] Image 179/336 : L_01_03_HED_FILT.png
[Line Intercept HED] Image 180/336 : L_01_04_HED_FILT.png
[Line Intercept HED] Image 181/336 : L_02_01_HED_FILT.png
[Line Intercept HED] Image 182/336 : L_02_02_HED_FILT.png
[Line Intercept HED] Image 183/336 : L_02_03_HED_FILT.png
[Line Intercept HED] Image 184/336 : L_02_04_HED_FILT.png
[Line Intercept HED] Image 185/336 : L_03_01_HED_FILT.png
[Line Intercept HED] Image 186/336 : L_03_02_HED_FILT.png
[Line Intercept HED] Image 187/336 : L_03_03_HED_FILT.png
[Line Intercept HED] Image 188/336 : L_03_04_HED_FILT.png
[Line Intercept HED] Image 189/336 : L_04_01_HED_FILT.png
[Line Intercept HED] Image 190/336 : L_04_02_HED_FILT.png
[Line Intercept HED] Image 191/336 : L_04_03_HED_FILT.png
[Line Intercept HED] Image 192/336 : L_04_04_HED_FILT.png
[Line Intercept HED] Image 193/336 : M_01_01_HED_FILT.png
[Line Intercept HED] Image 194/336 : M_01_02_HED_FILT.png
[Line Intercept HED] Image 195/336 : M_01_03_HED_FILT.png
[Line Intercept HED] Image 196/336 : M_01_04_HED_FILT.png
[Line Intercept HED] Image 197/336 : M_02_01_HED_FILT.png
[Line Intercept HED] Image 198/336 : M_02_02_HED_FILT.png
[Line Intercept HED] Image 199/336 : M_02_03_HED_FILT.png
[Line Intercept HED] Image 200/336 : M_02_04_HED_FILT.png
[Line Intercept HED] Image 201/336 : M_03_01_HED_FILT.png
[Line Intercept HED] Image 202/336 : M_03_02_HED_FILT.png
[Line Intercept HED] Image 203/336 : M_03_03_HED_FILT.png
[Line Intercept HED] Image 204/336 : M_03_04_HED_FILT.png
[Line Intercept HED] Image 205/336 : M_04_01_HED_FILT.png
[Line Intercept HED] Image 206/336 : M_04_02_HED_FILT.png
[Line Intercept HED] Image 207/336 : M_04_03_HED_FILT.png
[Line Intercept HED] Image 208/336 : M_04_04_HED_FILT.png
[Line Intercept HED] Image 209/336 : N_01_01_HED_FILT.png
[Line Intercept HED] Image 210/336 : N_01_02_HED_FILT.png
[Line Intercept HED] Image 211/336 : N_01_03_HED_FILT.png
[Line Intercept HED] Image 212/336 : N_01_04_HED_FILT.png
[Line Intercept HED] Image 213/336 : N_02_01_HED_FILT.png
[Line Intercept HED] Image 214/336 : N_02_02_HED_FILT.png
[Line Intercept HED] Image 215/336 : N_02_03_HED_FILT.png
[Line Intercept HED] Image 216/336 : N_02_04_HED_FILT.png
[Line Intercept HED] Image 217/336 : N_03_01_HED_FILT.png
[Line Intercept HED] Image 218/336 : N_03_02_HED_FILT.png
[Line Intercept HED] Image 219/336 : N_03_03_HED_FILT.png
[Line Intercept HED] Image 220/336 : N_03_04_HED_FILT.png
[Line Intercept HED] Image 221/336 : N_04_01_HED_FILT.png
```

```
[Line Intercept HED] Image 222/336 : N_04_02_HED_FILT.png
[Line Intercept HED] Image 223/336 : N_04_03_HED_FILT.png
[Line Intercept HED] Image 224/336 : N_04_04_HED_FILT.png
[Line Intercept HED] Image 225/336 : O_01_01_HED_FILT.png
[Line Intercept HED] Image 226/336 : O_01_02_HED_FILT.png
[Line Intercept HED] Image 227/336 : O_01_03_HED_FILT.png
[Line Intercept HED] Image 228/336 : O_01_04_HED_FILT.png
[Line Intercept HED] Image 229/336 : O_02_01_HED_FILT.png
[Line Intercept HED] Image 230/336 : O_02_02_HED_FILT.png
[Line Intercept HED] Image 231/336 : O_02_03_HED_FILT.png
[Line Intercept HED] Image 232/336 : O_02_04_HED_FILT.png
[Line Intercept HED] Image 233/336 : O_03_01_HED_FILT.png
[Line Intercept HED] Image 234/336 : O_03_02_HED_FILT.png
[Line Intercept HED] Image 235/336 : O_03_03_HED_FILT.png
[Line Intercept HED] Image 236/336 : O_03_04_HED_FILT.png
[Line Intercept HED] Image 237/336 : O_04_01_HED_FILT.png
[Line Intercept HED] Image 238/336 : O_04_02_HED_FILT.png
[Line Intercept HED] Image 239/336 : O_04_03_HED_FILT.png
[Line Intercept HED] Image 240/336 : O_04_04_HED_FILT.png
[Line Intercept HED] Image 241/336 : P_01_01_HED_FILT.png
[Line Intercept HED] Image 242/336 : P_01_02_HED_FILT.png
[Line Intercept HED] Image 243/336 : P_01_03_HED_FILT.png
[Line Intercept HED] Image 244/336 : P_01_04_HED_FILT.png
[Line Intercept HED] Image 245/336 : P_02_01_HED_FILT.png
[Line Intercept HED] Image 246/336 : P_02_02_HED_FILT.png
[Line Intercept HED] Image 247/336 : P_02_03_HED_FILT.png
[Line Intercept HED] Image 248/336 : P_02_04_HED_FILT.png
[Line Intercept HED] Image 249/336 : P_03_01_HED_FILT.png
[Line Intercept HED] Image 250/336 : P_03_02_HED_FILT.png
[Line Intercept HED] Image 251/336 : P_03_03_HED_FILT.png
[Line Intercept HED] Image 252/336 : P_03_04_HED_FILT.png
[Line Intercept HED] Image 253/336 : P_04_01_HED_FILT.png
[Line Intercept HED] Image 254/336 : P_04_02_HED_FILT.png
[Line Intercept HED] Image 255/336 : P_04_03_HED_FILT.png
[Line Intercept HED] Image 256/336 : P_04_04_HED_FILT.png
[Line Intercept HED] Image 257/336 : Q_01_01_HED_FILT.png
[Line Intercept HED] Image 258/336 : Q_01_02_HED_FILT.png
[Line Intercept HED] Image 259/336 : Q_01_03_HED_FILT.png
[Line Intercept HED] Image 260/336 : Q_01_04_HED_FILT.png
[Line Intercept HED] Image 261/336 : Q_02_01_HED_FILT.png
[Line Intercept HED] Image 262/336 : Q_02_02_HED_FILT.png
[Line Intercept HED] Image 263/336 : Q_02_03_HED_FILT.png
[Line Intercept HED] Image 264/336 : Q_02_04_HED_FILT.png
[Line Intercept HED] Image 265/336 : Q_03_01_HED_FILT.png
[Line Intercept HED] Image 266/336 : Q_03_02_HED_FILT.png
[Line Intercept HED] Image 267/336 : Q_03_03_HED_FILT.png
[Line Intercept HED] Image 268/336 : Q_03_04_HED_FILT.png
[Line Intercept HED] Image 269/336 : Q_04_01_HED_FILT.png
```

```
[Line Intercept HED] Image 270/336 : Q_04_02_HED_FILT.png
[Line Intercept HED] Image 271/336 : Q_04_03_HED_FILT.png
[Line Intercept HED] Image 272/336 : Q_04_04_HED_FILT.png
[Line Intercept HED] Image 273/336 : R_01_01_HED_FILT.png
[Line Intercept HED] Image 274/336 : R_01_02_HED_FILT.png
[Line Intercept HED] Image 275/336 : R_01_03_HED_FILT.png
[Line Intercept HED] Image 276/336 : R_01_04_HED_FILT.png
[Line Intercept HED] Image 277/336 : R_02_01_HED_FILT.png
[Line Intercept HED] Image 278/336 : R_02_02_HED_FILT.png
[Line Intercept HED] Image 279/336 : R_02_03_HED_FILT.png
[Line Intercept HED] Image 280/336 : R_02_04_HED_FILT.png
[Line Intercept HED] Image 281/336 : R_03_01_HED_FILT.png
[Line Intercept HED] Image 282/336 : R_03_02_HED_FILT.png
[Line Intercept HED] Image 283/336 : R_03_03_HED_FILT.png
[Line Intercept HED] Image 284/336 : R_03_04_HED_FILT.png
[Line Intercept HED] Image 285/336 : R_04_01_HED_FILT.png
[Line Intercept HED] Image 286/336 : R_04_02_HED_FILT.png
[Line Intercept HED] Image 287/336 : R_04_03_HED_FILT.png
[Line Intercept HED] Image 288/336 : R_04_04_HED_FILT.png
[Line Intercept HED] Image 289/336 : S_01_01_HED_FILT.png
[Line Intercept HED] Image 290/336 : S_01_02_HED_FILT.png
[Line Intercept HED] Image 291/336 : S_01_03_HED_FILT.png
[Line Intercept HED] Image 292/336 : S_01_04_HED_FILT.png
[Line Intercept HED] Image 293/336 : S_02_01_HED_FILT.png
[Line Intercept HED] Image 294/336 : S_02_02_HED_FILT.png
[Line Intercept HED] Image 295/336 : S_02_03_HED_FILT.png
[Line Intercept HED] Image 296/336 : S_02_04_HED_FILT.png
[Line Intercept HED] Image 297/336 : S_03_01_HED_FILT.png
[Line Intercept HED] Image 298/336 : S_03_02_HED_FILT.png
[Line Intercept HED] Image 299/336 : S_03_03_HED_FILT.png
[Line Intercept HED] Image 300/336 : S_03_04_HED_FILT.png
[Line Intercept HED] Image 301/336 : S_04_01_HED_FILT.png
[Line Intercept HED] Image 302/336 : S_04_02_HED_FILT.png
[Line Intercept HED] Image 303/336 : S_04_03_HED_FILT.png
[Line Intercept HED] Image 304/336 : S_04_04_HED_FILT.png
[Line Intercept HED] Image 305/336 : T_01_01_HED_FILT.png
[Line Intercept HED] Image 306/336 : T_01_02_HED_FILT.png
[Line Intercept HED] Image 307/336 : T_01_03_HED_FILT.png
[Line Intercept HED] Image 308/336 : T_01_04_HED_FILT.png
[Line Intercept HED] Image 309/336 : T_02_01_HED_FILT.png
[Line Intercept HED] Image 310/336 : T_02_02_HED_FILT.png
[Line Intercept HED] Image 311/336 : T_02_03_HED_FILT.png
[Line Intercept HED] Image 312/336 : T_02_04_HED_FILT.png
[Line Intercept HED] Image 313/336 : T_03_01_HED_FILT.png
[Line Intercept HED] Image 314/336 : T_03_02_HED_FILT.png
[Line Intercept HED] Image 315/336 : T_03_03_HED_FILT.png
[Line Intercept HED] Image 316/336 : T_03_04_HED_FILT.png
[Line Intercept HED] Image 317/336 : T_04_01_HED_FILT.png
```

```
[Line Intercept HED] Image 318/336 : T_04_02_HED_FILT.png
[Line Intercept HED] Image 319/336 : T_04_03_HED_FILT.png
[Line Intercept HED] Image 320/336 : T_04_04_HED_FILT.png
[Line Intercept HED] Image 321/336 : U_01_01_HED_FILT.png
[Line Intercept HED] Image 322/336 : U_01_02_HED_FILT.png
[Line Intercept HED] Image 323/336 : U_01_03_HED_FILT.png
[Line Intercept HED] Image 324/336 : U_01_04_HED_FILT.png
[Line Intercept HED] Image 325/336 : U_02_01_HED_FILT.png
[Line Intercept HED] Image 326/336 : U_02_02_HED_FILT.png
[Line Intercept HED] Image 327/336 : U_02_03_HED_FILT.png
[Line Intercept HED] Image 328/336 : U_02_04_HED_FILT.png
[Line Intercept HED] Image 329/336 : U_03_01_HED_FILT.png
[Line Intercept HED] Image 330/336 : U_03_02_HED_FILT.png
[Line Intercept HED] Image 331/336 : U_03_03_HED_FILT.png
[Line Intercept HED] Image 332/336 : U_03_04_HED_FILT.png
[Line Intercept HED] Image 333/336 : U_04_01_HED_FILT.png
[Line Intercept HED] Image 334/336 : U_04_02_HED_FILT.png
[Line Intercept HED] Image 335/336 : U_04_03_HED_FILT.png
[Line Intercept HED] Image 336/336 : U_04_04_HED_FILT.png
 Boucle Line Intercept HED terminée.
```

Dans cette étape, on applique la méthode des interceptions linéaires directement sur les masques HED déjà filtrés (images *_HED_FILT.png issues de l'étape 27). On commence par relister tous les fichiers présents dans `HED_FILTERED_DIR` et vérifier qu'il y en a bien `N_HED_FILT`. Pour chaque masque, l'image est relue en niveaux de gris puis reseuillée à 90 (`THRESH_BINARY`) afin d'obtenir une carte parfaitement binaire (0 / 255). On calcule ensuite les dérivées discrètes `diff_x` (variation horizontale) et `diff_y` (variation verticale), qui font apparaître les transitions de fond noir vers contour blanc. Par sécurité, on borne la hauteur et la largeur effectivement utilisées (`h_eff`, `w_eff`) à `numrows` et `numcols` pour rester cohérent avec le maillage global. La boucle remplit alors les matrices `Total_Grains_X_HED` et `Total_Grains_Y_HED` : pour chaque ligne, on compte le nombre de transitions égales à 255 dans `diff_x` (interceptions ligne/grain) et on stocke ce nombre dans `Total_Grains_X_HED[x, t]`; de même, pour chaque colonne, on compte les transitions dans `diff_y` et on renseigne `Total_Grains_Y_HED[y, t]`. Au final, pour chaque image filtrée HED, on obtient un profil complet d'interceptions horizontales et verticales compatible avec la grille 20×20, prêt pour le post-traitement (conversion en longueurs moyennes de grains à l'étape 30).

[ ]:
```python
#  Étape 30 - Post-traitement HED

Total_Grains_X = Total_Grains_X_HED.copy()
Total_Grains_Y = Total_Grains_Y_HED.copy()

Total_Grains_X[Total_Grains_X == 0] = 1
Total_Grains_Y[Total_Grains_Y == 0] = 1

AVG_GrainX_HED = ROW_LENGTH_MICRONS / Total_Grains_X
AVG_GrainY_HED = COL_LENGTH_MICRONS / Total_Grains_Y

print("AVG_GrainX_HED shape :", AVG_GrainX_HED.shape)
```

```
print("AVG_GrainY_HED shape :", AVG_GrainY_HED.shape)
```

```
AVG_GrainX_HED shape : (300, 336)
AVG_GrainY_HED shape : (400, 336)
```

Après la boucle Line Intercept HED, les matrices `Total_Grains_X_HED` et `Total_Grains_Y_HED` contiennent, pour chaque image et pour chaque ligne/colonne du maillage, le nombre d'interceptions entre les lignes de mesure et les contours issus du modèle HED. Dans cette étape de post-traitement, on commence par dupliquer ces matrices dans `Total_Grains_X` et `Total_Grains_Y` afin de préserver les résultats bruts. On remplace ensuite toutes les valeurs nulles par 1 pour éviter toute division par zéro lors du passage aux longueurs moyennes (une ligne ou colonne sans interception est traitée comme une seule "pseudo-interception"). La conversion en tailles de grains se fait alors en divisant la longueur physique d'une ligne du maillage (`ROW_LENGTH_MICRONS` pour les lignes, `COL_LENGTH_MICRONS` pour les colonnes) par le nombre d'interceptions correspondantes. On obtient ainsi deux cartes complètes, `AVG_GrainX_HED` et `AVG_GrainY_HED`, qui donnent pour chaque position du maillage une estimation de la taille moyenne de grain selon les directions X et Y, entièrement basée sur la segmentation HED filtrée.

Planimétrie HED & mesures géométriques des grains

Dans cette section, on passe de la logique **Line Intercept** (tailles moyennes de grains déduites des nombres d'interceptions sur un maillage) à une approche **planimétrique complète** appliquée aux masques HED filtrés. Concrètement, il s'agit de reconstruire des **labels de grains** à partir des masques HED, puis d'extraire, pour chaque grain individuel, des grandeurs géométriques détaillées : aire, périmètre, diamètres de Feret max/min, circularité, aspect ratio, etc.
Les étapes 31 à ~38 construisent ainsi une "brique planimétrique HED" qui vient **compléter** les tailles moyennes issues des intercepts : - d'abord en **résumant et sauvegardant** les résultats HED Line Intercept (Étape 31), - puis en calculant, pour chaque grain, des **features géométriques** (aire, périmètre, Feret…), - enfin en produisant des **statistiques globales** et des visualisations (histogrammes, boxplots, distributions) permettant de comparer les différentes métriques de taille et de forme.
L'étape 31 sert donc de **pont** : elle clôt proprement la partie Line Intercept HED (en figeant les matrices et statistiques globales) avant de basculer vers l'analyse planimétrique par grain.

[ ]:
```
#  Étape 31 - Statistiques globales HED + sauvegarde CSV

meanX_HED = np.mean(AVG_GrainX_HED)
meanY_HED = np.mean(AVG_GrainY_HED)
STDX_HED  = np.std(AVG_GrainX_HED)
STDY_HED  = np.std(AVG_GrainY_HED)

print("HED - meanX_HED :", meanX_HED)
print("HED - meanY_HED :", meanY_HED)
print("HED - STDX_HED  :", STDX_HED)
print("HED - STDY_HED  :", STDY_HED)

# Sauvegarde CSV dans BASE_DIR
```

```
np.savetxt(str(BASE_DIR / "Total_Grains_X_HED.csv"), Total_Grains_X_HED,␣
 ↪delimiter=",")
np.savetxt(str(BASE_DIR / "Total_Grains_Y_HED.csv"), Total_Grains_Y_HED,␣
 ↪delimiter=",")
np.savetxt(str(BASE_DIR / "AVG_GrainX_HED.csv"),     AVG_GrainX_HED,    ␣
 ↪delimiter=",")
np.savetxt(str(BASE_DIR / "AVG_GrainY_HED.csv"),     AVG_GrainY_HED,    ␣
 ↪delimiter=",")

print(" Fichiers CSV HED sauvegardés dans :", BASE_DIR)
print(" - Total_Grains_X_HED.csv")
print(" - Total_Grains_Y_HED.csv")
print(" - AVG_GrainX_HED.csv")
print(" - AVG_GrainY_HED.csv")
```

```
HED - meanX_HED : 41.564625
HED - meanY_HED : 43.752647
HED - STDX_HED  : 18.404299
HED - STDY_HED  : 22.618967
 Fichiers CSV HED sauvegardés dans : /content/drive/MyDrive/Data
 - Total_Grains_X_HED.csv
 - Total_Grains_Y_HED.csv
 - AVG_GrainX_HED.csv
 - AVG_GrainY_HED.csv
```

Concrètement, le code commence par résumer les matrices `AVG_GrainX_HED` et `AVG_GrainY_HED` en calculant les moyennes globales et les écarts-types des tailles de grains selon X et Y. On obtient ainsi un ordre de grandeur global de la taille des grains vue par HED, ainsi qu'une mesure de la dispersion spatiale de ces tailles dans le maillage. Ensuite, toutes les matrices de la méthode HED Line Intercept (`Total_Grains_X_HED`, `Total_Grains_Y_HED`, `AVG_GrainX_HED`, `AVG_GrainY_HED`) sont sauvegardées en CSV dans `BASE_DIR`. Cette sauvegarde fige les résultats de la phase Line Intercept (pour comparaison future, analyses statistiques ou export vers d'autres outils) et prépare le terrain pour la suite de la section, qui exploitera les masques HED en mode planimétrique pour extraire des mesures géométriques détaillées par grain.

[ ]:
```
# Étape 32 - Fonction planimétrique générale

import math

def measure_grains_planimetric(binary_img, p2m, area_min_pixels=0):
    """
    binary_img      : image binaire 0/255 (uint8)
    p2m             : facteur pixels -> microns (P2M)
    area_min_pixels : seuil d'aire pour ignorer les petits objets (en pixels²)
    """
    # Assurer une image binaire propre
    _, bin_img = cv2.threshold(binary_img, 127, 255, cv2.THRESH_BINARY)
```

```
    # Contours externes
    contours, _ = cv2.findContours(bin_img, cv2.RETR_EXTERNAL, cv2.
↪CHAIN_APPROX_SIMPLE)

    max_dists   = []  # max Feret (µm)
    min_dists   = []  # min Feret (µm)
    x_dias      = []  # diamètre X (µm)
    y_dias      = []  # diamètre Y (µm)
    aspect_rat  = []  # aspect ratio X/Y
    areas       = []  # aire (µm²)
    perimeters  = []  # périmètre (µm)
    circularity = []  # circularité 4 A/P²

    for cnt in contours:
        area_px = cv2.contourArea(cnt)
        if area_px <= area_min_pixels:
            continue  # ignorer les petites particules parasites

        per_px = cv2.arcLength(cnt, True)

        # Aire / périmètre en unités physiques (µm, µm²)
        area_um2 = area_px * (p2m ** 2)
        per_um   = per_px * p2m

        # Circularité (éviter division par zéro)
        if per_um > 0:
            circ = (4.0 * math.pi * area_um2) / (per_um ** 2)
        else:
            circ = 0.0

        # Bounding box → diamètres X/Y (en µm) + aspect ratio
        x, y, w, h = cv2.boundingRect(cnt)
        x_um = w * p2m
        y_um = h * p2m
        ar   = x_um / y_um if y_um > 0 else 0.0

        # Feret min / max à partir du convex hull
        hull = cv2.convexHull(cnt)
        hull_pts = hull[:, 0, :]  # (N, 2)

        max_d = 0.0
        min_d = float("inf")

        n_pts = hull_pts.shape[0]
        for i in range(n_pts):
            for j in range(i + 1, n_pts):
                dx = float(hull_pts[i, 0] - hull_pts[j, 0])
```

```
                dy = float(hull_pts[i, 1] - hull_pts[j, 1])
                d_pix = math.sqrt(dx * dx + dy * dy)
                if d_pix > max_d:
                    max_d = d_pix
                if 0.0 < d_pix < min_d:
                    min_d = d_pix

        # Conversion des Feret en µm
        max_um = max_d * p2m
        min_um = min_d * p2m if min_d < float("inf") else 0.0

        # Stockage
        max_dists.append(max_um)
        min_dists.append(min_um)
        x_dias.append(x_um)
        y_dias.append(y_um)
        aspect_rat.append(ar)
        areas.append(area_um2)
        perimeters.append(per_um)
        circularity.append(circ)

    return {
        "max_dists":   np.array(max_dists),
        "min_dists":   np.array(min_dists),
        "x_dias":      np.array(x_dias),
        "y_dias":      np.array(y_dias),
        "aspect_rat":  np.array(aspect_rat),
        "areas":       np.array(areas),
        "perimeters":  np.array(perimeters),
        "circularity": np.array(circularity),
    }
```

La fonction `measure_grains_planimetric` prend en entrée une image binaire de grains (valeurs 0/255) et un facteur de calibration `p2m` (pixels → microns) pour extraire, pour chaque grain, un ensemble complet de mesures géométriques. Elle commence par nettoyer l'image via un seuillage binaire, puis détecte les contours externes des grains. Un premier filtrage élimine les petites particules parasites dont l'aire en pixels est inférieure à `area_min_pixels`. Pour chaque contour conservé, on calcule l'aire et le périmètre en unités physiques (µm² et µm) grâce au facteur `p2m`. La circularité est ensuite évaluée avec la formule :

$$C = \frac{4\pi A}{P^2}$$

où (A) est l'aire du grain et (P) son périmètre (en µm). Une boîte englobante est ajustée sur chaque contour afin d'obtenir les diamètres projetés selon l'axe (X) et l'axe (Y), exprimés en microns ; leur rapport fournit un premier indicateur d'allongement (aspect ratio (X/Y)). En parallèle, l'enveloppe convexe du contour est utilisée pour calculer les distances de Feret minimale et maximale en parcourant toutes les paires de points de l'enveloppe, puis en les convertissant en microns. Toutes

ces grandeurs (Feret max/min, diamètres (X) et (Y), aspect ratio, aire, périmètre et circularité) sont finalement stockées dans des tableaux NumPy et renvoyées sous forme d'un dictionnaire. La fonction encapsule ainsi toute la brique planimétrique nécessaire pour caractériser finement la géométrie des grains à partir d'une image segmentée.

[ ]:
```
# Étape 33 - Test planimétrique sur une image FHED

hed_filt_files = sorted([
    f for f in os.listdir(HED_FILTERED_DIR)
    if f.lower().endswith((".png", ".jpg", ".jpeg", ".tif", ".tiff"))
])

if len(hed_filt_files) == 0:
    raise RuntimeError(f"Aucune image trouvée dans {HED_FILTERED_DIR}. Vérifie␣
 ↪les étapes HED 26-27.")

test_planim_path = HED_FILTERED_DIR / hed_filt_files[0]
print("Image test planimétrique (HED filtrée) :", test_planim_path)

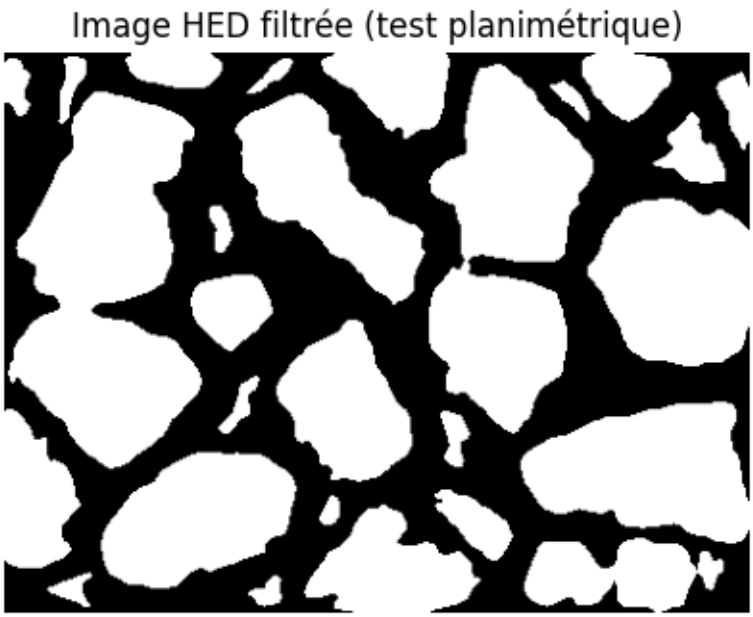
img_filt = cv2.imread(str(test_planim_path), 0)
if img_filt is None:
    raise FileNotFoundError(f"Impossible de lire l'image : {test_planim_path}")

planim_res_test = measure_grains_planimetric(img_filt, p2m=P2M,␣
 ↪area_min_pixels=100)

for k, v in planim_res_test.items():
    print(k, "→", v.shape)

plt.figure(figsize=(5, 5))
plt.imshow(img_filt, cmap="gray")
plt.title("Image HED filtrée (test planimétrique)")
plt.axis("off")
plt.show()
```

```
Image test planimétrique (HED filtrée) :
/content/drive/MyDrive/Data/HED_filtered/A_01_01_HED_FILT.png
max_dists → (27,)
min_dists → (27,)
x_dias → (27,)
y_dias → (27,)
aspect_rat → (27,)
areas → (27,)
perimeters → (27,)
circularity → (27,)
```



Cette étape sert de **sanity check** pour vérifier que la fonction planimétrique fonctionne correctement avant de la lancer sur tout le dataset. On commence par lister tous les fichiers d'images dans le dossier `HED_FILTERED_DIR` (les masques HED déjà filtrés). Si aucune image n'est trouvée, on lève une erreur pour signaler que les étapes 26–27 n'ont pas produit de résultats. On sélectionne ensuite la première image filtrée comme **image test**, on la lit en niveaux de gris et on l'envoie à la fonction `measure_grains_planimetric` avec la calibration `P2M` et un seuil d'aire minimale de 100 pixels^2 pour ignorer les petits objets parasites. Le dictionnaire retourné contient les tableaux des différentes grandeurs par grain (Feret max/min, diamètres projetés X/Y, aspect ratio, aire, périmètre, circularité) et, pour chaque clé, on affiche la **taille du vecteur** (par exemple `(27,)` signifie qu'on a détecté 27 grains mesurés). Enfin, l'image HED filtrée utilisée pour ce test est affichée, ce qui permet de visualiser les grains analysés et de confirmer que le masque binaire est cohérent avec les mesures obtenues.

[ ]:
```
# Étape 34 - Boucle planimétrique HED (toutes les images filtrées disponibles)

hed_filt_files = sorted([
    f for f in os.listdir(HED_FILTERED_DIR)
    if f.lower().endswith((".png", ".jpg", ".jpeg", ".tif", ".tiff"))
])

N_HED_FILT = len(hed_filt_files)
if N_HED_FILT == 0:
    raise RuntimeError(f"Aucune image trouvée dans {HED_FILTERED_DIR}. Vérifie␣
 ↪les étapes HED 26-27.")

print(f"[Planimétrique HED] Nombre d'images filtrées : {N_HED_FILT}")
```

```
MAX_HED   = []
MIN_HED   = []
X_DIA_HED = []
Y_DIA_HED = []
AR_HED    = []
AREA_HED  = []
PERIM_HED = []
CR_HED    = []

for t, fname in enumerate(hed_filt_files[:N_HED_FILT]):
    img_path = HED_FILTERED_DIR / fname
    print(f"[Planimétrique HED] Image {t+1}/{N_HED_FILT} :", img_path.name)

    img_filt = cv2.imread(str(img_path), cv2.IMREAD_GRAYSCALE)
    if img_filt is None:
        raise FileNotFoundError(f"Impossible de lire l'image : {img_path}")

    res = measure_grains_planimetric(img_filt, p2m=P2M, area_min_pixels=100)

    MAX_HED.append(res["max_dists"])
    MIN_HED.append(res["min_dists"])
    X_DIA_HED.append(res["x_dias"])
    Y_DIA_HED.append(res["y_dias"])
    AR_HED.append(res["aspect_rat"])
    AREA_HED.append(res["areas"])
    PERIM_HED.append(res["perimeters"])
    CR_HED.append(res["circularity"])

# Concaténation globale
MAX_HED   = np.concatenate(MAX_HED)   if len(MAX_HED)   > 0 else np.array([])
MIN_HED   = np.concatenate(MIN_HED)   if len(MIN_HED)   > 0 else np.array([])
X_DIA_HED = np.concatenate(X_DIA_HED) if len(X_DIA_HED) > 0 else np.array([])
Y_DIA_HED = np.concatenate(Y_DIA_HED) if len(Y_DIA_HED) > 0 else np.array([])
AR_HED    = np.concatenate(AR_HED)    if len(AR_HED)    > 0 else np.array([])
AREA_HED  = np.concatenate(AREA_HED)  if len(AREA_HED)  > 0 else np.array([])
PERIM_HED = np.concatenate(PERIM_HED) if len(PERIM_HED) > 0 else np.array([])
CR_HED    = np.concatenate(CR_HED)    if len(CR_HED)    > 0 else np.array([])

print("Taille MAX_HED   :", MAX_HED.shape)
print("Taille MIN_HED   :", MIN_HED.shape)
print("Taille X_DIA_HED :", X_DIA_HED.shape)
print("Taille Y_DIA_HED :", Y_DIA_HED.shape)
print("Taille AR_HED    :", AR_HED.shape)
print("Taille AREA_HED  :", AREA_HED.shape)
print("Taille PERIM_HED :", PERIM_HED.shape)
print("Taille CR_HED    :", CR_HED.shape)
```

```
[Planimétrique HED] Nombre d'images filtrées : 336
[Planimétrique HED] Image 1/336 : A_01_01_HED_FILT.png
[Planimétrique HED] Image 2/336 : A_01_02_HED_FILT.png
[Planimétrique HED] Image 3/336 : A_01_03_HED_FILT.png
[Planimétrique HED] Image 4/336 : A_01_04_HED_FILT.png
[Planimétrique HED] Image 5/336 : A_02_01_HED_FILT.png
[Planimétrique HED] Image 6/336 : A_02_02_HED_FILT.png
[Planimétrique HED] Image 7/336 : A_02_03_HED_FILT.png
[Planimétrique HED] Image 8/336 : A_02_04_HED_FILT.png
[Planimétrique HED] Image 9/336 : A_03_01_HED_FILT.png
[Planimétrique HED] Image 10/336 : A_03_02_HED_FILT.png
[Planimétrique HED] Image 11/336 : A_03_03_HED_FILT.png
[Planimétrique HED] Image 12/336 : A_03_04_HED_FILT.png
[Planimétrique HED] Image 13/336 : A_04_01_HED_FILT.png
[Planimétrique HED] Image 14/336 : A_04_02_HED_FILT.png
[Planimétrique HED] Image 15/336 : A_04_03_HED_FILT.png
[Planimétrique HED] Image 16/336 : A_04_04_HED_FILT.png
[Planimétrique HED] Image 17/336 : B_01_01_HED_FILT.png
[Planimétrique HED] Image 18/336 : B_01_02_HED_FILT.png
[Planimétrique HED] Image 19/336 : B_01_03_HED_FILT.png
[Planimétrique HED] Image 20/336 : B_01_04_HED_FILT.png
[Planimétrique HED] Image 21/336 : B_02_01_HED_FILT.png
[Planimétrique HED] Image 22/336 : B_02_02_HED_FILT.png
[Planimétrique HED] Image 23/336 : B_02_03_HED_FILT.png
[Planimétrique HED] Image 24/336 : B_02_04_HED_FILT.png
[Planimétrique HED] Image 25/336 : B_03_01_HED_FILT.png
[Planimétrique HED] Image 26/336 : B_03_02_HED_FILT.png
[Planimétrique HED] Image 27/336 : B_03_03_HED_FILT.png
[Planimétrique HED] Image 28/336 : B_03_04_HED_FILT.png
[Planimétrique HED] Image 29/336 : B_04_01_HED_FILT.png
[Planimétrique HED] Image 30/336 : B_04_02_HED_FILT.png
[Planimétrique HED] Image 31/336 : B_04_03_HED_FILT.png
[Planimétrique HED] Image 32/336 : B_04_04_HED_FILT.png
[Planimétrique HED] Image 33/336 : C_01_01_HED_FILT.png
[Planimétrique HED] Image 34/336 : C_01_02_HED_FILT.png
[Planimétrique HED] Image 35/336 : C_01_03_HED_FILT.png
[Planimétrique HED] Image 36/336 : C_01_04_HED_FILT.png
[Planimétrique HED] Image 37/336 : C_02_01_HED_FILT.png
[Planimétrique HED] Image 38/336 : C_02_02_HED_FILT.png
[Planimétrique HED] Image 39/336 : C_02_03_HED_FILT.png
[Planimétrique HED] Image 40/336 : C_02_04_HED_FILT.png
[Planimétrique HED] Image 41/336 : C_03_01_HED_FILT.png
[Planimétrique HED] Image 42/336 : C_03_02_HED_FILT.png
[Planimétrique HED] Image 43/336 : C_03_03_HED_FILT.png
[Planimétrique HED] Image 44/336 : C_03_04_HED_FILT.png
[Planimétrique HED] Image 45/336 : C_04_01_HED_FILT.png
[Planimétrique HED] Image 46/336 : C_04_02_HED_FILT.png
[Planimétrique HED] Image 47/336 : C_04_03_HED_FILT.png
```

```
[Planimétrique HED] Image 48/336 : C_04_04_HED_FILT.png
[Planimétrique HED] Image 49/336 : D_01_01_HED_FILT.png
[Planimétrique HED] Image 50/336 : D_01_02_HED_FILT.png
[Planimétrique HED] Image 51/336 : D_01_03_HED_FILT.png
[Planimétrique HED] Image 52/336 : D_01_04_HED_FILT.png
[Planimétrique HED] Image 53/336 : D_02_01_HED_FILT.png
[Planimétrique HED] Image 54/336 : D_02_02_HED_FILT.png
[Planimétrique HED] Image 55/336 : D_02_03_HED_FILT.png
[Planimétrique HED] Image 56/336 : D_02_04_HED_FILT.png
[Planimétrique HED] Image 57/336 : D_03_01_HED_FILT.png
[Planimétrique HED] Image 58/336 : D_03_02_HED_FILT.png
[Planimétrique HED] Image 59/336 : D_03_03_HED_FILT.png
[Planimétrique HED] Image 60/336 : D_03_04_HED_FILT.png
[Planimétrique HED] Image 61/336 : D_04_01_HED_FILT.png
[Planimétrique HED] Image 62/336 : D_04_02_HED_FILT.png
[Planimétrique HED] Image 63/336 : D_04_03_HED_FILT.png
[Planimétrique HED] Image 64/336 : D_04_04_HED_FILT.png
[Planimétrique HED] Image 65/336 : E_01_01_HED_FILT.png
[Planimétrique HED] Image 66/336 : E_01_02_HED_FILT.png
[Planimétrique HED] Image 67/336 : E_01_03_HED_FILT.png
[Planimétrique HED] Image 68/336 : E_01_04_HED_FILT.png
[Planimétrique HED] Image 69/336 : E_02_01_HED_FILT.png
[Planimétrique HED] Image 70/336 : E_02_02_HED_FILT.png
[Planimétrique HED] Image 71/336 : E_02_03_HED_FILT.png
[Planimétrique HED] Image 72/336 : E_02_04_HED_FILT.png
[Planimétrique HED] Image 73/336 : E_03_01_HED_FILT.png
[Planimétrique HED] Image 74/336 : E_03_02_HED_FILT.png
[Planimétrique HED] Image 75/336 : E_03_03_HED_FILT.png
[Planimétrique HED] Image 76/336 : E_03_04_HED_FILT.png
[Planimétrique HED] Image 77/336 : E_04_01_HED_FILT.png
[Planimétrique HED] Image 78/336 : E_04_02_HED_FILT.png
[Planimétrique HED] Image 79/336 : E_04_03_HED_FILT.png
[Planimétrique HED] Image 80/336 : E_04_04_HED_FILT.png
[Planimétrique HED] Image 81/336 : F_01_01_HED_FILT.png
[Planimétrique HED] Image 82/336 : F_01_02_HED_FILT.png
[Planimétrique HED] Image 83/336 : F_01_03_HED_FILT.png
[Planimétrique HED] Image 84/336 : F_01_04_HED_FILT.png
[Planimétrique HED] Image 85/336 : F_02_01_HED_FILT.png
[Planimétrique HED] Image 86/336 : F_02_02_HED_FILT.png
[Planimétrique HED] Image 87/336 : F_02_03_HED_FILT.png
[Planimétrique HED] Image 88/336 : F_02_04_HED_FILT.png
[Planimétrique HED] Image 89/336 : F_03_01_HED_FILT.png
[Planimétrique HED] Image 90/336 : F_03_02_HED_FILT.png
[Planimétrique HED] Image 91/336 : F_03_03_HED_FILT.png
[Planimétrique HED] Image 92/336 : F_03_04_HED_FILT.png
[Planimétrique HED] Image 93/336 : F_04_01_HED_FILT.png
[Planimétrique HED] Image 94/336 : F_04_02_HED_FILT.png
[Planimétrique HED] Image 95/336 : F_04_03_HED_FILT.png
```

```
[Planimétrique HED] Image 96/336 : F_04_04_HED_FILT.png
[Planimétrique HED] Image 97/336 : G_01_01_HED_FILT.png
[Planimétrique HED] Image 98/336 : G_01_02_HED_FILT.png
[Planimétrique HED] Image 99/336 : G_01_03_HED_FILT.png
[Planimétrique HED] Image 100/336 : G_01_04_HED_FILT.png
[Planimétrique HED] Image 101/336 : G_02_01_HED_FILT.png
[Planimétrique HED] Image 102/336 : G_02_02_HED_FILT.png
[Planimétrique HED] Image 103/336 : G_02_03_HED_FILT.png
[Planimétrique HED] Image 104/336 : G_02_04_HED_FILT.png
[Planimétrique HED] Image 105/336 : G_03_01_HED_FILT.png
[Planimétrique HED] Image 106/336 : G_03_02_HED_FILT.png
[Planimétrique HED] Image 107/336 : G_03_03_HED_FILT.png
[Planimétrique HED] Image 108/336 : G_03_04_HED_FILT.png
[Planimétrique HED] Image 109/336 : G_04_01_HED_FILT.png
[Planimétrique HED] Image 110/336 : G_04_02_HED_FILT.png
[Planimétrique HED] Image 111/336 : G_04_03_HED_FILT.png
[Planimétrique HED] Image 112/336 : G_04_04_HED_FILT.png
[Planimétrique HED] Image 113/336 : H_01_01_HED_FILT.png
[Planimétrique HED] Image 114/336 : H_01_02_HED_FILT.png
[Planimétrique HED] Image 115/336 : H_01_03_HED_FILT.png
[Planimétrique HED] Image 116/336 : H_01_04_HED_FILT.png
[Planimétrique HED] Image 117/336 : H_02_01_HED_FILT.png
[Planimétrique HED] Image 118/336 : H_02_02_HED_FILT.png
[Planimétrique HED] Image 119/336 : H_02_03_HED_FILT.png
[Planimétrique HED] Image 120/336 : H_02_04_HED_FILT.png
[Planimétrique HED] Image 121/336 : H_03_01_HED_FILT.png
[Planimétrique HED] Image 122/336 : H_03_02_HED_FILT.png
[Planimétrique HED] Image 123/336 : H_03_03_HED_FILT.png
[Planimétrique HED] Image 124/336 : H_03_04_HED_FILT.png
[Planimétrique HED] Image 125/336 : H_04_01_HED_FILT.png
[Planimétrique HED] Image 126/336 : H_04_02_HED_FILT.png
[Planimétrique HED] Image 127/336 : H_04_03_HED_FILT.png
[Planimétrique HED] Image 128/336 : H_04_04_HED_FILT.png
[Planimétrique HED] Image 129/336 : I_01_01_HED_FILT.png
[Planimétrique HED] Image 130/336 : I_01_02_HED_FILT.png
[Planimétrique HED] Image 131/336 : I_01_03_HED_FILT.png
[Planimétrique HED] Image 132/336 : I_01_04_HED_FILT.png
[Planimétrique HED] Image 133/336 : I_02_01_HED_FILT.png
[Planimétrique HED] Image 134/336 : I_02_02_HED_FILT.png
[Planimétrique HED] Image 135/336 : I_02_03_HED_FILT.png
[Planimétrique HED] Image 136/336 : I_02_04_HED_FILT.png
[Planimétrique HED] Image 137/336 : I_03_01_HED_FILT.png
[Planimétrique HED] Image 138/336 : I_03_02_HED_FILT.png
[Planimétrique HED] Image 139/336 : I_03_03_HED_FILT.png
[Planimétrique HED] Image 140/336 : I_03_04_HED_FILT.png
[Planimétrique HED] Image 141/336 : I_04_01_HED_FILT.png
[Planimétrique HED] Image 142/336 : I_04_02_HED_FILT.png
[Planimétrique HED] Image 143/336 : I_04_03_HED_FILT.png
```

```
[Planimétrique HED] Image 144/336 : I_04_04_HED_FILT.png
[Planimétrique HED] Image 145/336 : J_01_01_HED_FILT.png
[Planimétrique HED] Image 146/336 : J_01_02_HED_FILT.png
[Planimétrique HED] Image 147/336 : J_01_03_HED_FILT.png
[Planimétrique HED] Image 148/336 : J_01_04_HED_FILT.png
[Planimétrique HED] Image 149/336 : J_02_01_HED_FILT.png
[Planimétrique HED] Image 150/336 : J_02_02_HED_FILT.png
[Planimétrique HED] Image 151/336 : J_02_03_HED_FILT.png
[Planimétrique HED] Image 152/336 : J_02_04_HED_FILT.png
[Planimétrique HED] Image 153/336 : J_03_01_HED_FILT.png
[Planimétrique HED] Image 154/336 : J_03_02_HED_FILT.png
[Planimétrique HED] Image 155/336 : J_03_03_HED_FILT.png
[Planimétrique HED] Image 156/336 : J_03_04_HED_FILT.png
[Planimétrique HED] Image 157/336 : J_04_01_HED_FILT.png
[Planimétrique HED] Image 158/336 : J_04_02_HED_FILT.png
[Planimétrique HED] Image 159/336 : J_04_03_HED_FILT.png
[Planimétrique HED] Image 160/336 : J_04_04_HED_FILT.png
[Planimétrique HED] Image 161/336 : K_01_01_HED_FILT.png
[Planimétrique HED] Image 162/336 : K_01_02_HED_FILT.png
[Planimétrique HED] Image 163/336 : K_01_03_HED_FILT.png
[Planimétrique HED] Image 164/336 : K_01_04_HED_FILT.png
[Planimétrique HED] Image 165/336 : K_02_01_HED_FILT.png
[Planimétrique HED] Image 166/336 : K_02_02_HED_FILT.png
[Planimétrique HED] Image 167/336 : K_02_03_HED_FILT.png
[Planimétrique HED] Image 168/336 : K_02_04_HED_FILT.png
[Planimétrique HED] Image 169/336 : K_03_01_HED_FILT.png
[Planimétrique HED] Image 170/336 : K_03_02_HED_FILT.png
[Planimétrique HED] Image 171/336 : K_03_03_HED_FILT.png
[Planimétrique HED] Image 172/336 : K_03_04_HED_FILT.png
[Planimétrique HED] Image 173/336 : K_04_01_HED_FILT.png
[Planimétrique HED] Image 174/336 : K_04_02_HED_FILT.png
[Planimétrique HED] Image 175/336 : K_04_03_HED_FILT.png
[Planimétrique HED] Image 176/336 : K_04_04_HED_FILT.png
[Planimétrique HED] Image 177/336 : L_01_01_HED_FILT.png
[Planimétrique HED] Image 178/336 : L_01_02_HED_FILT.png
[Planimétrique HED] Image 179/336 : L_01_03_HED_FILT.png
[Planimétrique HED] Image 180/336 : L_01_04_HED_FILT.png
[Planimétrique HED] Image 181/336 : L_02_01_HED_FILT.png
[Planimétrique HED] Image 182/336 : L_02_02_HED_FILT.png
[Planimétrique HED] Image 183/336 : L_02_03_HED_FILT.png
[Planimétrique HED] Image 184/336 : L_02_04_HED_FILT.png
[Planimétrique HED] Image 185/336 : L_03_01_HED_FILT.png
[Planimétrique HED] Image 186/336 : L_03_02_HED_FILT.png
[Planimétrique HED] Image 187/336 : L_03_03_HED_FILT.png
[Planimétrique HED] Image 188/336 : L_03_04_HED_FILT.png
[Planimétrique HED] Image 189/336 : L_04_01_HED_FILT.png
[Planimétrique HED] Image 190/336 : L_04_02_HED_FILT.png
[Planimétrique HED] Image 191/336 : L_04_03_HED_FILT.png
```

```
[Planimétrique HED] Image 192/336 : L_04_04_HED_FILT.png
[Planimétrique HED] Image 193/336 : M_01_01_HED_FILT.png
[Planimétrique HED] Image 194/336 : M_01_02_HED_FILT.png
[Planimétrique HED] Image 195/336 : M_01_03_HED_FILT.png
[Planimétrique HED] Image 196/336 : M_01_04_HED_FILT.png
[Planimétrique HED] Image 197/336 : M_02_01_HED_FILT.png
[Planimétrique HED] Image 198/336 : M_02_02_HED_FILT.png
[Planimétrique HED] Image 199/336 : M_02_03_HED_FILT.png
[Planimétrique HED] Image 200/336 : M_02_04_HED_FILT.png
[Planimétrique HED] Image 201/336 : M_03_01_HED_FILT.png
[Planimétrique HED] Image 202/336 : M_03_02_HED_FILT.png
[Planimétrique HED] Image 203/336 : M_03_03_HED_FILT.png
[Planimétrique HED] Image 204/336 : M_03_04_HED_FILT.png
[Planimétrique HED] Image 205/336 : M_04_01_HED_FILT.png
[Planimétrique HED] Image 206/336 : M_04_02_HED_FILT.png
[Planimétrique HED] Image 207/336 : M_04_03_HED_FILT.png
[Planimétrique HED] Image 208/336 : M_04_04_HED_FILT.png
[Planimétrique HED] Image 209/336 : N_01_01_HED_FILT.png
[Planimétrique HED] Image 210/336 : N_01_02_HED_FILT.png
[Planimétrique HED] Image 211/336 : N_01_03_HED_FILT.png
[Planimétrique HED] Image 212/336 : N_01_04_HED_FILT.png
[Planimétrique HED] Image 213/336 : N_02_01_HED_FILT.png
[Planimétrique HED] Image 214/336 : N_02_02_HED_FILT.png
[Planimétrique HED] Image 215/336 : N_02_03_HED_FILT.png
[Planimétrique HED] Image 216/336 : N_02_04_HED_FILT.png
[Planimétrique HED] Image 217/336 : N_03_01_HED_FILT.png
[Planimétrique HED] Image 218/336 : N_03_02_HED_FILT.png
[Planimétrique HED] Image 219/336 : N_03_03_HED_FILT.png
[Planimétrique HED] Image 220/336 : N_03_04_HED_FILT.png
[Planimétrique HED] Image 221/336 : N_04_01_HED_FILT.png
[Planimétrique HED] Image 222/336 : N_04_02_HED_FILT.png
[Planimétrique HED] Image 223/336 : N_04_03_HED_FILT.png
[Planimétrique HED] Image 224/336 : N_04_04_HED_FILT.png
[Planimétrique HED] Image 225/336 : O_01_01_HED_FILT.png
[Planimétrique HED] Image 226/336 : O_01_02_HED_FILT.png
[Planimétrique HED] Image 227/336 : O_01_03_HED_FILT.png
[Planimétrique HED] Image 228/336 : O_01_04_HED_FILT.png
[Planimétrique HED] Image 229/336 : O_02_01_HED_FILT.png
[Planimétrique HED] Image 230/336 : O_02_02_HED_FILT.png
[Planimétrique HED] Image 231/336 : O_02_03_HED_FILT.png
[Planimétrique HED] Image 232/336 : O_02_04_HED_FILT.png
[Planimétrique HED] Image 233/336 : O_03_01_HED_FILT.png
[Planimétrique HED] Image 234/336 : O_03_02_HED_FILT.png
[Planimétrique HED] Image 235/336 : O_03_03_HED_FILT.png
[Planimétrique HED] Image 236/336 : O_03_04_HED_FILT.png
[Planimétrique HED] Image 237/336 : O_04_01_HED_FILT.png
[Planimétrique HED] Image 238/336 : O_04_02_HED_FILT.png
[Planimétrique HED] Image 239/336 : O_04_03_HED_FILT.png
```

```
[Planimétrique HED] Image 240/336 : O_04_04_HED_FILT.png
[Planimétrique HED] Image 241/336 : P_01_01_HED_FILT.png
[Planimétrique HED] Image 242/336 : P_01_02_HED_FILT.png
[Planimétrique HED] Image 243/336 : P_01_03_HED_FILT.png
[Planimétrique HED] Image 244/336 : P_01_04_HED_FILT.png
[Planimétrique HED] Image 245/336 : P_02_01_HED_FILT.png
[Planimétrique HED] Image 246/336 : P_02_02_HED_FILT.png
[Planimétrique HED] Image 247/336 : P_02_03_HED_FILT.png
[Planimétrique HED] Image 248/336 : P_02_04_HED_FILT.png
[Planimétrique HED] Image 249/336 : P_03_01_HED_FILT.png
[Planimétrique HED] Image 250/336 : P_03_02_HED_FILT.png
[Planimétrique HED] Image 251/336 : P_03_03_HED_FILT.png
[Planimétrique HED] Image 252/336 : P_03_04_HED_FILT.png
[Planimétrique HED] Image 253/336 : P_04_01_HED_FILT.png
[Planimétrique HED] Image 254/336 : P_04_02_HED_FILT.png
[Planimétrique HED] Image 255/336 : P_04_03_HED_FILT.png
[Planimétrique HED] Image 256/336 : P_04_04_HED_FILT.png
[Planimétrique HED] Image 257/336 : Q_01_01_HED_FILT.png
[Planimétrique HED] Image 258/336 : Q_01_02_HED_FILT.png
[Planimétrique HED] Image 259/336 : Q_01_03_HED_FILT.png
[Planimétrique HED] Image 260/336 : Q_01_04_HED_FILT.png
[Planimétrique HED] Image 261/336 : Q_02_01_HED_FILT.png
[Planimétrique HED] Image 262/336 : Q_02_02_HED_FILT.png
[Planimétrique HED] Image 263/336 : Q_02_03_HED_FILT.png
[Planimétrique HED] Image 264/336 : Q_02_04_HED_FILT.png
[Planimétrique HED] Image 265/336 : Q_03_01_HED_FILT.png
[Planimétrique HED] Image 266/336 : Q_03_02_HED_FILT.png
[Planimétrique HED] Image 267/336 : Q_03_03_HED_FILT.png
[Planimétrique HED] Image 268/336 : Q_03_04_HED_FILT.png
[Planimétrique HED] Image 269/336 : Q_04_01_HED_FILT.png
[Planimétrique HED] Image 270/336 : Q_04_02_HED_FILT.png
[Planimétrique HED] Image 271/336 : Q_04_03_HED_FILT.png
[Planimétrique HED] Image 272/336 : Q_04_04_HED_FILT.png
[Planimétrique HED] Image 273/336 : R_01_01_HED_FILT.png
[Planimétrique HED] Image 274/336 : R_01_02_HED_FILT.png
[Planimétrique HED] Image 275/336 : R_01_03_HED_FILT.png
[Planimétrique HED] Image 276/336 : R_01_04_HED_FILT.png
[Planimétrique HED] Image 277/336 : R_02_01_HED_FILT.png
[Planimétrique HED] Image 278/336 : R_02_02_HED_FILT.png
[Planimétrique HED] Image 279/336 : R_02_03_HED_FILT.png
[Planimétrique HED] Image 280/336 : R_02_04_HED_FILT.png
[Planimétrique HED] Image 281/336 : R_03_01_HED_FILT.png
[Planimétrique HED] Image 282/336 : R_03_02_HED_FILT.png
[Planimétrique HED] Image 283/336 : R_03_03_HED_FILT.png
[Planimétrique HED] Image 284/336 : R_03_04_HED_FILT.png
[Planimétrique HED] Image 285/336 : R_04_01_HED_FILT.png
[Planimétrique HED] Image 286/336 : R_04_02_HED_FILT.png
[Planimétrique HED] Image 287/336 : R_04_03_HED_FILT.png
```

```
[Planimétrique HED] Image 288/336 : R_04_04_HED_FILT.png
[Planimétrique HED] Image 289/336 : S_01_01_HED_FILT.png
[Planimétrique HED] Image 290/336 : S_01_02_HED_FILT.png
[Planimétrique HED] Image 291/336 : S_01_03_HED_FILT.png
[Planimétrique HED] Image 292/336 : S_01_04_HED_FILT.png
[Planimétrique HED] Image 293/336 : S_02_01_HED_FILT.png
[Planimétrique HED] Image 294/336 : S_02_02_HED_FILT.png
[Planimétrique HED] Image 295/336 : S_02_03_HED_FILT.png
[Planimétrique HED] Image 296/336 : S_02_04_HED_FILT.png
[Planimétrique HED] Image 297/336 : S_03_01_HED_FILT.png
[Planimétrique HED] Image 298/336 : S_03_02_HED_FILT.png
[Planimétrique HED] Image 299/336 : S_03_03_HED_FILT.png
[Planimétrique HED] Image 300/336 : S_03_04_HED_FILT.png
[Planimétrique HED] Image 301/336 : S_04_01_HED_FILT.png
[Planimétrique HED] Image 302/336 : S_04_02_HED_FILT.png
[Planimétrique HED] Image 303/336 : S_04_03_HED_FILT.png
[Planimétrique HED] Image 304/336 : S_04_04_HED_FILT.png
[Planimétrique HED] Image 305/336 : T_01_01_HED_FILT.png
[Planimétrique HED] Image 306/336 : T_01_02_HED_FILT.png
[Planimétrique HED] Image 307/336 : T_01_03_HED_FILT.png
[Planimétrique HED] Image 308/336 : T_01_04_HED_FILT.png
[Planimétrique HED] Image 309/336 : T_02_01_HED_FILT.png
[Planimétrique HED] Image 310/336 : T_02_02_HED_FILT.png
[Planimétrique HED] Image 311/336 : T_02_03_HED_FILT.png
[Planimétrique HED] Image 312/336 : T_02_04_HED_FILT.png
[Planimétrique HED] Image 313/336 : T_03_01_HED_FILT.png
[Planimétrique HED] Image 314/336 : T_03_02_HED_FILT.png
[Planimétrique HED] Image 315/336 : T_03_03_HED_FILT.png
[Planimétrique HED] Image 316/336 : T_03_04_HED_FILT.png
[Planimétrique HED] Image 317/336 : T_04_01_HED_FILT.png
[Planimétrique HED] Image 318/336 : T_04_02_HED_FILT.png
[Planimétrique HED] Image 319/336 : T_04_03_HED_FILT.png
[Planimétrique HED] Image 320/336 : T_04_04_HED_FILT.png
[Planimétrique HED] Image 321/336 : U_01_01_HED_FILT.png
[Planimétrique HED] Image 322/336 : U_01_02_HED_FILT.png
[Planimétrique HED] Image 323/336 : U_01_03_HED_FILT.png
[Planimétrique HED] Image 324/336 : U_01_04_HED_FILT.png
[Planimétrique HED] Image 325/336 : U_02_01_HED_FILT.png
[Planimétrique HED] Image 326/336 : U_02_02_HED_FILT.png
[Planimétrique HED] Image 327/336 : U_02_03_HED_FILT.png
[Planimétrique HED] Image 328/336 : U_02_04_HED_FILT.png
[Planimétrique HED] Image 329/336 : U_03_01_HED_FILT.png
[Planimétrique HED] Image 330/336 : U_03_02_HED_FILT.png
[Planimétrique HED] Image 331/336 : U_03_03_HED_FILT.png
[Planimétrique HED] Image 332/336 : U_03_04_HED_FILT.png
[Planimétrique HED] Image 333/336 : U_04_01_HED_FILT.png
[Planimétrique HED] Image 334/336 : U_04_02_HED_FILT.png
[Planimétrique HED] Image 335/336 : U_04_03_HED_FILT.png
```

```
[Planimétrique HED] Image 336/336 : U_04_04_HED_FILT.png
Taille MAX_HED   : (7623,)
Taille MIN_HED   : (7623,)
Taille X_DIA_HED : (7623,)
Taille Y_DIA_HED : (7623,)
Taille AR_HED    : (7623,)
Taille AREA_HED  : (7623,)
Taille PERIM_HED : (7623,)
Taille CR_HED    : (7623,)
```

Cette étape applique la **mesure planimétrique à l'ensemble du jeu d'images HED filtrées**. On commence par lister toutes les images du dossier `HED_FILTERED_DIR` et à vérifier qu'il y en a bien 336, sinon on lève une erreur. Pour chaque image, le masque HED filtré est chargé en niveaux de gris puis envoyé à la fonction `measure_grains_planimetric` avec la calibration spatiale `P2M` et un seuil d'aire minimale de 100 pixels² afin d'éliminer les petits objets parasites. La fonction renvoie, pour chaque grain de l'image, un ensemble de grandeurs géométriques : **Feret max/min** (`max_dists`, `min_dists`), **diamètres projetés** selon X et Y (`x_dias`, `y_dias`), **aspect ratio** (`aspect_rat`), **aire** (`areas`), **périmètre** (`perimeters`) et **circularité** (`circularity`). Ces vecteurs sont ajoutés à des listes globales (`MAX_HED`, `MIN_HED`, `X_DIA_HED`, `Y_DIA_HED`, `AR_HED`, `AREA_HED`, `PERIM_HED`, `CR_HED`). À la fin de la boucle, on concatène toutes ces listes pour obtenir des **vecteurs globaux** qui regroupent les mesures de tous les grains de toutes les images (ici 7 623 grains). Les impressions finales des tailles montrent que chaque vecteur possède la même longueur (7623), ce qui confirme que l'on dispose maintenant d'une **base planimétrique complète** pour la population de grains HED filtrés, prête pour les statistiques descriptives et les analyses ultérieures.

[ ]:
```python
# Étape 35 - Statistiques planimétriques HED

def describe_array(name, arr):
    if arr.size == 0:
        print(f"{name} : (vide)")
        return
    print(f"{name} : mean = {np.mean(arr):.3f}, std = {np.std(arr):.3f}, N =␣
 ↪{arr.size}")

describe_array("MAX_HED (Feret max, µm)", MAX_HED)
describe_array("MIN_HED (Feret min, µm)", MIN_HED)
describe_array("X_DIA_HED (diamètre X, µm)", X_DIA_HED)
describe_array("Y_DIA_HED (diamètre Y, µm)", Y_DIA_HED)
describe_array("AR_HED (aspect ratio X/Y)", AR_HED)
describe_array("AREA_HED (aire, µm²)", AREA_HED)
describe_array("PERIM_HED (périmètre, µm)", PERIM_HED)
describe_array("CR_HED (circularité)", CR_HED)

# Étape 37 - Aspect ratio basé sur Feret

AR_FERET_HED = np.full_like(MAX_HED, fill_value=np.nan, dtype=float)
valid_mask = (MIN_HED > 0)
AR_FERET_HED[valid_mask] = MAX_HED[valid_mask] / MIN_HED[valid_mask]
```

```
print("AR_FERET_HED shape :", AR_FERET_HED.shape)
print("Nombre de valeurs valides (MIN_HED > 0) :", np.sum(valid_mask))

# Étape 38 - Stats rapides "style notebook originel"

def quick_stats(name, arr):
    arr = np.asarray(arr)
    valid = np.isfinite(arr)
    if np.sum(valid) == 0:
        print(f"{name} : (aucune valeur valide)")
        return
    vals = arr[valid]
    print(f"{name}")
    print("  N    :", len(vals))
    print("  mean :", np.mean(vals))
    print("  std  :", np.std(vals))
    print("  max  :", np.max(vals))
    print()

quick_stats("MAX_HED   (Feret max, µm)",   MAX_HED)
quick_stats("MIN_HED   (Feret min, µm)",   MIN_HED)
quick_stats("AR_FERET_HED (Feret max/min)", AR_FERET_HED)
```

```
MAX_HED (Feret max, µm) : mean = 31.509, std = 24.581, N = 7623
MIN_HED (Feret min, µm) : mean = 0.586, std = 0.177, N = 7623
X_DIA_HED (diamètre X, µm) : mean = 25.535, std = 20.995, N = 7623
Y_DIA_HED (diamètre Y, µm) : mean = 25.324, std = 20.268, N = 7623
AR_HED (aspect ratio X/Y) : mean = 1.138, std = 0.650, N = 7623
AREA_HED (aire, µm²) : mean = 569.921, std = 1035.546, N = 7623
PERIM_HED (périmètre, µm) : mean = 101.682, std = 108.527, N = 7623
CR_HED (circularité) : mean = 0.530, std = 0.171, N = 7623
AR_FERET_HED shape : (7623,)
Nombre de valeurs valides (MIN_HED > 0) : 7623
MAX_HED   (Feret max, µm)
  N    : 7623
  mean : 31.50949178924843
  std  : 24.580978219665027
  max  : 221.06811418906867

MIN_HED   (Feret min, µm)
  N    : 7623
  mean : 0.5856891266296181
  std  : 0.17655148219094255
  max  : 1.8856180831641265

AR_FERET_HED (Feret max/min)
  N    : 7623
```

```
mean : 56.02861931175783
std  : 44.53476893156342
max  : 426.76574370490425
```

Les fonctions `describe_array` et `quick_stats` calculent des statistiques descriptives sur toutes les grandeurs planimétriques mesurées sur les 7 623 grains détectés. On obtient d'abord, pour chaque variable (Feret max/min, diamètres projetés X et Y, aire, périmètre, circularité, aspect ratio X/Y), la moyenne, l'écart-type et le nombre de grains. En moyenne, les grains présentent un Feret maximal d'environ **31,5 µm**, des diamètres projetés proches de **25 µm** dans les deux directions, et une aire moyenne autour de **570 µm²**, avec une dispersion assez forte (écarts-types élevés), ce qui confirme une distribution de tailles large. La circularité moyenne d'environ **0,53** indique des formes clairement irrégulières (loin du disque parfait de circularité 1), tandis que l'aspect ratio X/Y moyen de **1,14** suggère une légère allongation moyenne des grains.
L'étape 37 construit ensuite un **aspect ratio basé sur les Feret** (`AR_FERET_HED`) en prenant, pour chaque grain, le rapport Feret max / Feret min, uniquement lorsque le Feret min est strictement positif. Le masque de validité montre que toutes les 7 623 valeurs sont exploitables. Les statistiques rapides (Étape 38) révèlent que `AR_FERET_HED` a une moyenne d'environ **56** avec un écart-type très élevé (~45) et des valeurs maximales supérieures à **400**, ce qui traduit l'existence de grains ou de contours très allongés selon certaines directions du convexe (ou de grains aux géométries complexes).
Au final, ce bloc fournit une **vue synthétique de la population de grains** en termes de taille et de forme, et met en évidence une grande hétérogénéité morphologique dans les microstructures analysées.

[ ]:
```
# Étape 36 - Sauvegarde CSV des mesures planimétriques HED

np.savetxt(str(BASE_DIR / "MAX_HED_planimetric.csv"),   MAX_HED,  ␣
 ↪delimiter=",")
np.savetxt(str(BASE_DIR / "MIN_HED_planimetric.csv"),   MIN_HED,  ␣
 ↪delimiter=",")
np.savetxt(str(BASE_DIR / "X_DIA_HED_planimetric.csv"), X_DIA_HED,␣
 ↪delimiter=",")
np.savetxt(str(BASE_DIR / "Y_DIA_HED_planimetric.csv"), Y_DIA_HED,␣
 ↪delimiter=",")
np.savetxt(str(BASE_DIR / "AR_HED_planimetric.csv"),    AR_HED,   ␣
 ↪delimiter=",")
np.savetxt(str(BASE_DIR / "AREA_HED_planimetric.csv"),  AREA_HED, ␣
 ↪delimiter=",")
np.savetxt(str(BASE_DIR / "PERIM_HED_planimetric.csv"), PERIM_HED,␣
 ↪delimiter=",")
np.savetxt(str(BASE_DIR / "CR_HED_planimetric.csv"),    CR_HED,   ␣
 ↪delimiter=",")

print("  Fichiers CSV planimétriques HED sauvegardés dans :", BASE_DIR)
print(" - MAX_HED_planimetric.csv")
print(" - MIN_HED_planimetric.csv")
print(" - X_DIA_HED_planimetric.csv")
```

```
print(" - Y_DIA_HED_planimetric.csv")
print(" - AR_HED_planimetric.csv")
print(" - AREA_HED_planimetric.csv")
print(" - PERIM_HED_planimetric.csv")
print(" - CR_HED_planimetric.csv")
```

```
 Fichiers CSV planimétriques HED sauvegardés dans : /content/drive/MyDrive/Data
 - MAX_HED_planimetric.csv
 - MIN_HED_planimetric.csv
 - X_DIA_HED_planimetric.csv
 - Y_DIA_HED_planimetric.csv
 - AR_HED_planimetric.csv
 - AREA_HED_planimetric.csv
 - PERIM_HED_planimetric.csv
 - CR_HED_planimetric.csv
```

Cette étape ne réalise plus de nouveaux calculs sur les grains, mais organise **toute la brique planimétrique HED** en fichiers exploitables. Les vecteurs globaux issus de la boucle planimétrique (MAX_HED, MIN_HED, X_DIA_HED, Y_DIA_HED, AR_HED, AREA_HED, PERIM_HED, CR_HED), chacun contenant les mesures pour l'ensemble des 7 623 grains, sont sauvegardés séparément au format **CSV** dans le répertoire `BASE_DIR`. Concrètement, on obtient un fichier par grandeur : Feret maximal et minimal, diamètres projetés X/Y, aspect ratio, aire, périmètre et circularité. Ces fichiers constituent la **sortie brute structurée** de la planimétrie HED : ils pourront être réimportés ensuite dans Python, R ou tout autre outil (Excel, logiciel de métallographie, etc.) pour produire des statistiques avancées, des histogrammes, des corrélations ou des comparaisons entre zones et conditions de traitement thermique.

[ ]:
```
#  Étape 37 - Planimétrie Gradient : collecte des aires de grains

areas_grad = []      # aires des grains en pixels²
num_grains_grad = 0  # compteur global de grains

N_IMAGES = len(image_files)
print(f"[Planimétrie Gradient] Nombre d'images : {N_IMAGES}")

for t, fname in enumerate(image_files[:N_IMAGES]):
    img_path = IMG_DIR / fname
    print(f"[Planimétrie Gradient] Image {t+1}/{N_IMAGES} :", img_path.name)

    img = cv2.imread(str(img_path), 0)
    if img is None:
        raise FileNotFoundError(f"Impossible de lire l'image : {img_path}")

    # 1) Canny
    canny = cv2.Canny(img, 100, 200)

    # 2) Seuil binaire
    _, threshCAN = cv2.threshold(canny, 90, 255, cv2.THRESH_BINARY)
```

```
    # 3) Contours externes
    contours, _ = cv2.findContours(threshCAN, cv2.RETR_EXTERNAL, cv2.
 ↪CHAIN_APPROX_NONE)

    for c in contours:
        area_px = cv2.contourArea(c)
        if area_px <= 0:
            continue
        num_grains_grad += 1
        areas_grad.append(area_px)

areas_grad = np.array(areas_grad, dtype=float)

print("Taille du vecteur areas_grad :", areas_grad.size)
print("Nombre total de grains (num_grains_grad) :", num_grains_grad)
```

```
[Planimétrie Gradient] Nombre d'images : 336
[Planimétrie Gradient] Image 1/336 : A_01_01.png
[Planimétrie Gradient] Image 2/336 : A_01_02.png
[Planimétrie Gradient] Image 3/336 : A_01_03.png
[Planimétrie Gradient] Image 4/336 : A_01_04.png
[Planimétrie Gradient] Image 5/336 : A_02_01.png
[Planimétrie Gradient] Image 6/336 : A_02_02.png
[Planimétrie Gradient] Image 7/336 : A_02_03.png
[Planimétrie Gradient] Image 8/336 : A_02_04.png
[Planimétrie Gradient] Image 9/336 : A_03_01.png
[Planimétrie Gradient] Image 10/336 : A_03_02.png
[Planimétrie Gradient] Image 11/336 : A_03_03.png
[Planimétrie Gradient] Image 12/336 : A_03_04.png
[Planimétrie Gradient] Image 13/336 : A_04_01.png
[Planimétrie Gradient] Image 14/336 : A_04_02.png
[Planimétrie Gradient] Image 15/336 : A_04_03.png
[Planimétrie Gradient] Image 16/336 : A_04_04.png
[Planimétrie Gradient] Image 17/336 : B_01_01.png
[Planimétrie Gradient] Image 18/336 : B_01_02.png
[Planimétrie Gradient] Image 19/336 : B_01_03.png
[Planimétrie Gradient] Image 20/336 : B_01_04.png
[Planimétrie Gradient] Image 21/336 : B_02_01.png
[Planimétrie Gradient] Image 22/336 : B_02_02.png
[Planimétrie Gradient] Image 23/336 : B_02_03.png
[Planimétrie Gradient] Image 24/336 : B_02_04.png
[Planimétrie Gradient] Image 25/336 : B_03_01.png
[Planimétrie Gradient] Image 26/336 : B_03_02.png
[Planimétrie Gradient] Image 27/336 : B_03_03.png
[Planimétrie Gradient] Image 28/336 : B_03_04.png
[Planimétrie Gradient] Image 29/336 : B_04_01.png
[Planimétrie Gradient] Image 30/336 : B_04_02.png
```

```
[Planimétrie Gradient] Image 31/336 : B_04_03.png
[Planimétrie Gradient] Image 32/336 : B_04_04.png
[Planimétrie Gradient] Image 33/336 : C_01_01.png
[Planimétrie Gradient] Image 34/336 : C_01_02.png
[Planimétrie Gradient] Image 35/336 : C_01_03.png
[Planimétrie Gradient] Image 36/336 : C_01_04.png
[Planimétrie Gradient] Image 37/336 : C_02_01.png
[Planimétrie Gradient] Image 38/336 : C_02_02.png
[Planimétrie Gradient] Image 39/336 : C_02_03.png
[Planimétrie Gradient] Image 40/336 : C_02_04.png
[Planimétrie Gradient] Image 41/336 : C_03_01.png
[Planimétrie Gradient] Image 42/336 : C_03_02.png
[Planimétrie Gradient] Image 43/336 : C_03_03.png
[Planimétrie Gradient] Image 44/336 : C_03_04.png
[Planimétrie Gradient] Image 45/336 : C_04_01.png
[Planimétrie Gradient] Image 46/336 : C_04_02.png
[Planimétrie Gradient] Image 47/336 : C_04_03.png
[Planimétrie Gradient] Image 48/336 : C_04_04.png
[Planimétrie Gradient] Image 49/336 : D_01_01.png
[Planimétrie Gradient] Image 50/336 : D_01_02.png
[Planimétrie Gradient] Image 51/336 : D_01_03.png
[Planimétrie Gradient] Image 52/336 : D_01_04.png
[Planimétrie Gradient] Image 53/336 : D_02_01.png
[Planimétrie Gradient] Image 54/336 : D_02_02.png
[Planimétrie Gradient] Image 55/336 : D_02_03.png
[Planimétrie Gradient] Image 56/336 : D_02_04.png
[Planimétrie Gradient] Image 57/336 : D_03_01.png
[Planimétrie Gradient] Image 58/336 : D_03_02.png
[Planimétrie Gradient] Image 59/336 : D_03_03.png
[Planimétrie Gradient] Image 60/336 : D_03_04.png
[Planimétrie Gradient] Image 61/336 : D_04_01.png
[Planimétrie Gradient] Image 62/336 : D_04_02.png
[Planimétrie Gradient] Image 63/336 : D_04_03.png
[Planimétrie Gradient] Image 64/336 : D_04_04.png
[Planimétrie Gradient] Image 65/336 : E_01_01.png
[Planimétrie Gradient] Image 66/336 : E_01_02.png
[Planimétrie Gradient] Image 67/336 : E_01_03.png
[Planimétrie Gradient] Image 68/336 : E_01_04.png
[Planimétrie Gradient] Image 69/336 : E_02_01.png
[Planimétrie Gradient] Image 70/336 : E_02_02.png
[Planimétrie Gradient] Image 71/336 : E_02_03.png
[Planimétrie Gradient] Image 72/336 : E_02_04.png
[Planimétrie Gradient] Image 73/336 : E_03_01.png
[Planimétrie Gradient] Image 74/336 : E_03_02.png
[Planimétrie Gradient] Image 75/336 : E_03_03.png
[Planimétrie Gradient] Image 76/336 : E_03_04.png
[Planimétrie Gradient] Image 77/336 : E_04_01.png
[Planimétrie Gradient] Image 78/336 : E_04_02.png
```

```
[Planimétrie Gradient] Image 79/336 : E_04_03.png
[Planimétrie Gradient] Image 80/336 : E_04_04.png
[Planimétrie Gradient] Image 81/336 : F_01_01.png
[Planimétrie Gradient] Image 82/336 : F_01_02.png
[Planimétrie Gradient] Image 83/336 : F_01_03.png
[Planimétrie Gradient] Image 84/336 : F_01_04.png
[Planimétrie Gradient] Image 85/336 : F_02_01.png
[Planimétrie Gradient] Image 86/336 : F_02_02.png
[Planimétrie Gradient] Image 87/336 : F_02_03.png
[Planimétrie Gradient] Image 88/336 : F_02_04.png
[Planimétrie Gradient] Image 89/336 : F_03_01.png
[Planimétrie Gradient] Image 90/336 : F_03_02.png
[Planimétrie Gradient] Image 91/336 : F_03_03.png
[Planimétrie Gradient] Image 92/336 : F_03_04.png
[Planimétrie Gradient] Image 93/336 : F_04_01.png
[Planimétrie Gradient] Image 94/336 : F_04_02.png
[Planimétrie Gradient] Image 95/336 : F_04_03.png
[Planimétrie Gradient] Image 96/336 : F_04_04.png
[Planimétrie Gradient] Image 97/336 : G_01_01.png
[Planimétrie Gradient] Image 98/336 : G_01_02.png
[Planimétrie Gradient] Image 99/336 : G_01_03.png
[Planimétrie Gradient] Image 100/336 : G_01_04.png
[Planimétrie Gradient] Image 101/336 : G_02_01.png
[Planimétrie Gradient] Image 102/336 : G_02_02.png
[Planimétrie Gradient] Image 103/336 : G_02_03.png
[Planimétrie Gradient] Image 104/336 : G_02_04.png
[Planimétrie Gradient] Image 105/336 : G_03_01.png
[Planimétrie Gradient] Image 106/336 : G_03_02.png
[Planimétrie Gradient] Image 107/336 : G_03_03.png
[Planimétrie Gradient] Image 108/336 : G_03_04.png
[Planimétrie Gradient] Image 109/336 : G_04_01.png
[Planimétrie Gradient] Image 110/336 : G_04_02.png
[Planimétrie Gradient] Image 111/336 : G_04_03.png
[Planimétrie Gradient] Image 112/336 : G_04_04.png
[Planimétrie Gradient] Image 113/336 : H_01_01.png
[Planimétrie Gradient] Image 114/336 : H_01_02.png
[Planimétrie Gradient] Image 115/336 : H_01_03.png
[Planimétrie Gradient] Image 116/336 : H_01_04.png
[Planimétrie Gradient] Image 117/336 : H_02_01.png
[Planimétrie Gradient] Image 118/336 : H_02_02.png
[Planimétrie Gradient] Image 119/336 : H_02_03.png
[Planimétrie Gradient] Image 120/336 : H_02_04.png
[Planimétrie Gradient] Image 121/336 : H_03_01.png
[Planimétrie Gradient] Image 122/336 : H_03_02.png
[Planimétrie Gradient] Image 123/336 : H_03_03.png
[Planimétrie Gradient] Image 124/336 : H_03_04.png
[Planimétrie Gradient] Image 125/336 : H_04_01.png
[Planimétrie Gradient] Image 126/336 : H_04_02.png
```

```
[Planimétrie Gradient] Image 127/336 : H_04_03.png
[Planimétrie Gradient] Image 128/336 : H_04_04.png
[Planimétrie Gradient] Image 129/336 : I_01_01.png
[Planimétrie Gradient] Image 130/336 : I_01_02.png
[Planimétrie Gradient] Image 131/336 : I_01_03.png
[Planimétrie Gradient] Image 132/336 : I_01_04.png
[Planimétrie Gradient] Image 133/336 : I_02_01.png
[Planimétrie Gradient] Image 134/336 : I_02_02.png
[Planimétrie Gradient] Image 135/336 : I_02_03.png
[Planimétrie Gradient] Image 136/336 : I_02_04.png
[Planimétrie Gradient] Image 137/336 : I_03_01.png
[Planimétrie Gradient] Image 138/336 : I_03_02.png
[Planimétrie Gradient] Image 139/336 : I_03_03.png
[Planimétrie Gradient] Image 140/336 : I_03_04.png
[Planimétrie Gradient] Image 141/336 : I_04_01.png
[Planimétrie Gradient] Image 142/336 : I_04_02.png
[Planimétrie Gradient] Image 143/336 : I_04_03.png
[Planimétrie Gradient] Image 144/336 : I_04_04.png
[Planimétrie Gradient] Image 145/336 : J_01_01.png
[Planimétrie Gradient] Image 146/336 : J_01_02.png
[Planimétrie Gradient] Image 147/336 : J_01_03.png
[Planimétrie Gradient] Image 148/336 : J_01_04.png
[Planimétrie Gradient] Image 149/336 : J_02_01.png
[Planimétrie Gradient] Image 150/336 : J_02_02.png
[Planimétrie Gradient] Image 151/336 : J_02_03.png
[Planimétrie Gradient] Image 152/336 : J_02_04.png
[Planimétrie Gradient] Image 153/336 : J_03_01.png
[Planimétrie Gradient] Image 154/336 : J_03_02.png
[Planimétrie Gradient] Image 155/336 : J_03_03.png
[Planimétrie Gradient] Image 156/336 : J_03_04.png
[Planimétrie Gradient] Image 157/336 : J_04_01.png
[Planimétrie Gradient] Image 158/336 : J_04_02.png
[Planimétrie Gradient] Image 159/336 : J_04_03.png
[Planimétrie Gradient] Image 160/336 : J_04_04.png
[Planimétrie Gradient] Image 161/336 : K_01_01.png
[Planimétrie Gradient] Image 162/336 : K_01_02.png
[Planimétrie Gradient] Image 163/336 : K_01_03.png
[Planimétrie Gradient] Image 164/336 : K_01_04.png
[Planimétrie Gradient] Image 165/336 : K_02_01.png
[Planimétrie Gradient] Image 166/336 : K_02_02.png
[Planimétrie Gradient] Image 167/336 : K_02_03.png
[Planimétrie Gradient] Image 168/336 : K_02_04.png
[Planimétrie Gradient] Image 169/336 : K_03_01.png
[Planimétrie Gradient] Image 170/336 : K_03_02.png
[Planimétrie Gradient] Image 171/336 : K_03_03.png
[Planimétrie Gradient] Image 172/336 : K_03_04.png
[Planimétrie Gradient] Image 173/336 : K_04_01.png
[Planimétrie Gradient] Image 174/336 : K_04_02.png
```

```
[Planimétrie Gradient] Image 175/336 : K_04_03.png
[Planimétrie Gradient] Image 176/336 : K_04_04.png
[Planimétrie Gradient] Image 177/336 : L_01_01.png
[Planimétrie Gradient] Image 178/336 : L_01_02.png
[Planimétrie Gradient] Image 179/336 : L_01_03.png
[Planimétrie Gradient] Image 180/336 : L_01_04.png
[Planimétrie Gradient] Image 181/336 : L_02_01.png
[Planimétrie Gradient] Image 182/336 : L_02_02.png
[Planimétrie Gradient] Image 183/336 : L_02_03.png
[Planimétrie Gradient] Image 184/336 : L_02_04.png
[Planimétrie Gradient] Image 185/336 : L_03_01.png
[Planimétrie Gradient] Image 186/336 : L_03_02.png
[Planimétrie Gradient] Image 187/336 : L_03_03.png
[Planimétrie Gradient] Image 188/336 : L_03_04.png
[Planimétrie Gradient] Image 189/336 : L_04_01.png
[Planimétrie Gradient] Image 190/336 : L_04_02.png
[Planimétrie Gradient] Image 191/336 : L_04_03.png
[Planimétrie Gradient] Image 192/336 : L_04_04.png
[Planimétrie Gradient] Image 193/336 : M_01_01.png
[Planimétrie Gradient] Image 194/336 : M_01_02.png
[Planimétrie Gradient] Image 195/336 : M_01_03.png
[Planimétrie Gradient] Image 196/336 : M_01_04.png
[Planimétrie Gradient] Image 197/336 : M_02_01.png
[Planimétrie Gradient] Image 198/336 : M_02_02.png
[Planimétrie Gradient] Image 199/336 : M_02_03.png
[Planimétrie Gradient] Image 200/336 : M_02_04.png
[Planimétrie Gradient] Image 201/336 : M_03_01.png
[Planimétrie Gradient] Image 202/336 : M_03_02.png
[Planimétrie Gradient] Image 203/336 : M_03_03.png
[Planimétrie Gradient] Image 204/336 : M_03_04.png
[Planimétrie Gradient] Image 205/336 : M_04_01.png
[Planimétrie Gradient] Image 206/336 : M_04_02.png
[Planimétrie Gradient] Image 207/336 : M_04_03.png
[Planimétrie Gradient] Image 208/336 : M_04_04.png
[Planimétrie Gradient] Image 209/336 : N_01_01.png
[Planimétrie Gradient] Image 210/336 : N_01_02.png
[Planimétrie Gradient] Image 211/336 : N_01_03.png
[Planimétrie Gradient] Image 212/336 : N_01_04.png
[Planimétrie Gradient] Image 213/336 : N_02_01.png
[Planimétrie Gradient] Image 214/336 : N_02_02.png
[Planimétrie Gradient] Image 215/336 : N_02_03.png
[Planimétrie Gradient] Image 216/336 : N_02_04.png
[Planimétrie Gradient] Image 217/336 : N_03_01.png
[Planimétrie Gradient] Image 218/336 : N_03_02.png
[Planimétrie Gradient] Image 219/336 : N_03_03.png
[Planimétrie Gradient] Image 220/336 : N_03_04.png
[Planimétrie Gradient] Image 221/336 : N_04_01.png
[Planimétrie Gradient] Image 222/336 : N_04_02.png
```

```
[Planimétrie Gradient] Image 223/336 : N_04_03.png
[Planimétrie Gradient] Image 224/336 : N_04_04.png
[Planimétrie Gradient] Image 225/336 : O_01_01.png
[Planimétrie Gradient] Image 226/336 : O_01_02.png
[Planimétrie Gradient] Image 227/336 : O_01_03.png
[Planimétrie Gradient] Image 228/336 : O_01_04.png
[Planimétrie Gradient] Image 229/336 : O_02_01.png
[Planimétrie Gradient] Image 230/336 : O_02_02.png
[Planimétrie Gradient] Image 231/336 : O_02_03.png
[Planimétrie Gradient] Image 232/336 : O_02_04.png
[Planimétrie Gradient] Image 233/336 : O_03_01.png
[Planimétrie Gradient] Image 234/336 : O_03_02.png
[Planimétrie Gradient] Image 235/336 : O_03_03.png
[Planimétrie Gradient] Image 236/336 : O_03_04.png
[Planimétrie Gradient] Image 237/336 : O_04_01.png
[Planimétrie Gradient] Image 238/336 : O_04_02.png
[Planimétrie Gradient] Image 239/336 : O_04_03.png
[Planimétrie Gradient] Image 240/336 : O_04_04.png
[Planimétrie Gradient] Image 241/336 : P_01_01.png
[Planimétrie Gradient] Image 242/336 : P_01_02.png
[Planimétrie Gradient] Image 243/336 : P_01_03.png
[Planimétrie Gradient] Image 244/336 : P_01_04.png
[Planimétrie Gradient] Image 245/336 : P_02_01.png
[Planimétrie Gradient] Image 246/336 : P_02_02.png
[Planimétrie Gradient] Image 247/336 : P_02_03.png
[Planimétrie Gradient] Image 248/336 : P_02_04.png
[Planimétrie Gradient] Image 249/336 : P_03_01.png
[Planimétrie Gradient] Image 250/336 : P_03_02.png
[Planimétrie Gradient] Image 251/336 : P_03_03.png
[Planimétrie Gradient] Image 252/336 : P_03_04.png
[Planimétrie Gradient] Image 253/336 : P_04_01.png
[Planimétrie Gradient] Image 254/336 : P_04_02.png
[Planimétrie Gradient] Image 255/336 : P_04_03.png
[Planimétrie Gradient] Image 256/336 : P_04_04.png
[Planimétrie Gradient] Image 257/336 : Q_01_01.png
[Planimétrie Gradient] Image 258/336 : Q_01_02.png
[Planimétrie Gradient] Image 259/336 : Q_01_03.png
[Planimétrie Gradient] Image 260/336 : Q_01_04.png
[Planimétrie Gradient] Image 261/336 : Q_02_01.png
[Planimétrie Gradient] Image 262/336 : Q_02_02.png
[Planimétrie Gradient] Image 263/336 : Q_02_03.png
[Planimétrie Gradient] Image 264/336 : Q_02_04.png
[Planimétrie Gradient] Image 265/336 : Q_03_01.png
[Planimétrie Gradient] Image 266/336 : Q_03_02.png
[Planimétrie Gradient] Image 267/336 : Q_03_03.png
[Planimétrie Gradient] Image 268/336 : Q_03_04.png
[Planimétrie Gradient] Image 269/336 : Q_04_01.png
[Planimétrie Gradient] Image 270/336 : Q_04_02.png
```

```
[Planimétrie Gradient] Image 271/336 : Q_04_03.png
[Planimétrie Gradient] Image 272/336 : Q_04_04.png
[Planimétrie Gradient] Image 273/336 : R_01_01.png
[Planimétrie Gradient] Image 274/336 : R_01_02.png
[Planimétrie Gradient] Image 275/336 : R_01_03.png
[Planimétrie Gradient] Image 276/336 : R_01_04.png
[Planimétrie Gradient] Image 277/336 : R_02_01.png
[Planimétrie Gradient] Image 278/336 : R_02_02.png
[Planimétrie Gradient] Image 279/336 : R_02_03.png
[Planimétrie Gradient] Image 280/336 : R_02_04.png
[Planimétrie Gradient] Image 281/336 : R_03_01.png
[Planimétrie Gradient] Image 282/336 : R_03_02.png
[Planimétrie Gradient] Image 283/336 : R_03_03.png
[Planimétrie Gradient] Image 284/336 : R_03_04.png
[Planimétrie Gradient] Image 285/336 : R_04_01.png
[Planimétrie Gradient] Image 286/336 : R_04_02.png
[Planimétrie Gradient] Image 287/336 : R_04_03.png
[Planimétrie Gradient] Image 288/336 : R_04_04.png
[Planimétrie Gradient] Image 289/336 : S_01_01.png
[Planimétrie Gradient] Image 290/336 : S_01_02.png
[Planimétrie Gradient] Image 291/336 : S_01_03.png
[Planimétrie Gradient] Image 292/336 : S_01_04.png
[Planimétrie Gradient] Image 293/336 : S_02_01.png
[Planimétrie Gradient] Image 294/336 : S_02_02.png
[Planimétrie Gradient] Image 295/336 : S_02_03.png
[Planimétrie Gradient] Image 296/336 : S_02_04.png
[Planimétrie Gradient] Image 297/336 : S_03_01.png
[Planimétrie Gradient] Image 298/336 : S_03_02.png
[Planimétrie Gradient] Image 299/336 : S_03_03.png
[Planimétrie Gradient] Image 300/336 : S_03_04.png
[Planimétrie Gradient] Image 301/336 : S_04_01.png
[Planimétrie Gradient] Image 302/336 : S_04_02.png
[Planimétrie Gradient] Image 303/336 : S_04_03.png
[Planimétrie Gradient] Image 304/336 : S_04_04.png
[Planimétrie Gradient] Image 305/336 : T_01_01.png
[Planimétrie Gradient] Image 306/336 : T_01_02.png
[Planimétrie Gradient] Image 307/336 : T_01_03.png
[Planimétrie Gradient] Image 308/336 : T_01_04.png
[Planimétrie Gradient] Image 309/336 : T_02_01.png
[Planimétrie Gradient] Image 310/336 : T_02_02.png
[Planimétrie Gradient] Image 311/336 : T_02_03.png
[Planimétrie Gradient] Image 312/336 : T_02_04.png
[Planimétrie Gradient] Image 313/336 : T_03_01.png
[Planimétrie Gradient] Image 314/336 : T_03_02.png
[Planimétrie Gradient] Image 315/336 : T_03_03.png
[Planimétrie Gradient] Image 316/336 : T_03_04.png
[Planimétrie Gradient] Image 317/336 : T_04_01.png
[Planimétrie Gradient] Image 318/336 : T_04_02.png
```

```
[Planimétrie Gradient] Image 319/336 : T_04_03.png
[Planimétrie Gradient] Image 320/336 : T_04_04.png
[Planimétrie Gradient] Image 321/336 : U_01_01.png
[Planimétrie Gradient] Image 322/336 : U_01_02.png
[Planimétrie Gradient] Image 323/336 : U_01_03.png
[Planimétrie Gradient] Image 324/336 : U_01_04.png
[Planimétrie Gradient] Image 325/336 : U_02_01.png
[Planimétrie Gradient] Image 326/336 : U_02_02.png
[Planimétrie Gradient] Image 327/336 : U_02_03.png
[Planimétrie Gradient] Image 328/336 : U_02_04.png
[Planimétrie Gradient] Image 329/336 : U_03_01.png
[Planimétrie Gradient] Image 330/336 : U_03_02.png
[Planimétrie Gradient] Image 331/336 : U_03_03.png
[Planimétrie Gradient] Image 332/336 : U_03_04.png
[Planimétrie Gradient] Image 333/336 : U_04_01.png
[Planimétrie Gradient] Image 334/336 : U_04_02.png
[Planimétrie Gradient] Image 335/336 : U_04_03.png
[Planimétrie Gradient] Image 336/336 : U_04_04.png
Taille du vecteur areas_grad : 187599
Nombre total de grains (num_grains_grad) : 187599
```

Cette étape applique une **approche planimétrique basée sur le gradient** directement sur les images originales (`image_files`). Pour chaque micrographie (336 au total), le code commence par calculer les contours des joints de grains via l'algorithme de **Canny**, puis binarise le résultat et extrait les **contours externes**. Pour chaque contour, il calcule l'aire en pixels^2 (`cv2.contourArea`) et ne conserve que les objets ayant une aire strictement positive. Toutes ces aires sont empilées dans un grand vecteur `areas_grad`, tandis que `num_grains_grad` compte le nombre total de grains détectés. Au final, on obtient **187 599 grains** identifiés sur l'ensemble du jeu d'images, chacun associé à une aire mesurée. Ce vecteur `areas_grad` constitue donc la base de la **distribution des tailles de grains dérivée du filtrage par gradient**, indépendante du pipeline HED, et permettra de comparer les deux méthodes de segmentation (HED vs gradient) en termes de statistiques d'aire.

[ ]:
```python
#  Étape 38 - Conversion pixels² → µm² et statistiques globales d'aire (Gradient)

if areas_grad.size == 0:
    raise RuntimeError("areas_grad est vide : vérifie l'Étape 37 (aucun grain détecté).")

conversion_area = P2M ** 2
areas_grad_um2 = areas_grad * conversion_area

avg_grain_area_grad = np.mean(areas_grad_um2)
std_grain_area_grad = np.std(areas_grad_um2)

print("Gradient - aire moyenne de grain (µm²)     :", avg_grain_area_grad)
print("Gradient - écart-type aire de grain (µm²) :", std_grain_area_grad)
print("Gradient - nombre total de grains         :", areas_grad_um2.size)
```

```
# (optionnel) Sauvegarde des aires Gradient en CSV
np.savetxt(str(BASE_DIR / "areas_grad_um2.csv"), areas_grad_um2, delimiter=",")
print("  Aires planimétriques Gradient sauvegardées dans areas_grad_um2.csv")
```

```
Gradient - aire moyenne de grain (µm²)     : 3.80594516054371
Gradient - écart-type aire de grain (µm²) : 27.25046858919984
Gradient - nombre total de grains         : 187599
  Aires planimétriques Gradient sauvegardées dans areas_grad_um2.csv
```

Dans cette étape, les aires de grains obtenues avec la méthode **Gradient**, initialement exprimées en pixels² (`areas_grad`), sont converties en unités physiques (**µm²**) grâce au facteur de calibration (P2M^2), ce qui donne le vecteur `areas_grad_um2`. À partir de ces valeurs, le code calcule l'aire **moyenne** d'un grain (3,81 µm²) et son **écart-type** (27,25 µm²) sur un total de **187 599 grains** détectés. Le contraste entre une moyenne très faible et un écart-type beaucoup plus élevé indique une distribution d'aires **fortement dispersée**, avec une grande majorité de petits grains et quelques grains nettement plus gros qui augmentent la variance. Enfin, l'ensemble des aires converties est sauvegardé dans le fichier `areas_grad_um2.csv`, ce qui permet de tracer des histogrammes, comparer cette distribution aux mesures HED et documenter précisément la morphologie globale du matériau.

Segmentation modulaire des grains et étiquetage automatique

Dans cette section, on met en place une **boîte à outils générique** pour la segmentation des grains, indépendante de la méthode utilisée (seuillage manuel, gradient, HED). L'idée est de factoriser tout ce que faisait l'ingénieur dans des blocs séparés et parfois redondants, en le regroupant dans des **fonctions réutilisables**.

[ ]:
```
#  Étape 39 - Utilitaire connected components + fausses couleurs

def label_components(binary_img, connectivity=4):
    """
    binary_img : image binaire 0/255 (uint8)
    connectivity : 4 ou 8

    Retourne :
      - n_labels   : nombre total de labels (incluant le fond)
      - labels     : matrice de labels (même taille que binary_img)
      - stats      : tableau (n_labels, 5) -> [x, y, w, h, area]
      - centroids  : tableau (n_labels, 2) -> [cx, cy]
      - false_colors            : image RGB fausses couleurs
      - false_colors_centroids  : fausses couleurs + croix aux centroïdes
    """
    # S'assurer du bon type
    if binary_img.dtype != np.uint8:
        bin_img = binary_img.astype(np.uint8)
    else:
        bin_img = binary_img.copy()
```

```
    n_labels, labels, stats, centroids = cv2.connectedComponentsWithStats(
        bin_img, connectivity=connectivity
    )

    # Palette aléatoire (label 0 = fond noir)
    colors = np.random.randint(0, 255, size=(n_labels, 3), dtype=np.uint8)
    colors[0] = [0, 0, 0]
    false_colors = colors[labels]

    # Ajout des centroïdes (croix blanches)
    false_colors_centroid = false_colors.copy()
    for (cx, cy) in centroids:
        cv2.drawMarker(
            false_colors_centroid,
            (int(cx), int(cy)),
            color=(255, 255, 255),
            markerType=cv2.MARKER_CROSS,
            markerSize=10,
            thickness=1,
        )

    return n_labels, labels, stats, centroids, false_colors,␣
↪false_colors_centroid
```

ce bloc, la fonction `label_components`, joue le rôle de brique de base commune après n'importe quelle segmentation binaire. On part d'une image 0/255, que l'on convertit si nécessaire en `uint8`, puis on applique `cv2.connectedComponentsWithStats` pour identifier toutes les composantes connexes. On obtient ainsi : le nombre de labels (fond inclus), une carte de labels de même taille que l'image d'origine, un tableau de statistiques (x, y, largeur, hauteur, aire) pour chaque composante, et les centroïdes associés. À partir de cette carte de labels, la fonction génère une image en fausses couleurs en attribuant une couleur aléatoire à chaque grain (le fond restant noir), puis ajoute une croix blanche sur chaque centroïde pour faciliter la lecture visuelle. Au final, `label_components` renvoie à la fois les objets analytiques (labels, stats, centroïdes, compteur de labels) et les objets visuels (fausses couleurs avec et sans centroïdes), qui seront réutilisés de manière uniforme par les modules de seuillage manuel, de gradient et de HED.

[ ]:
```
#  Étape 40 - Module de segmentation : thresholding manuel

def segment_manual_threshold(
    gray_img,
    adaptive_block_size=55,
    adaptive_C=2,
    median_kernel_sequence=(5, 5, 7, 7),
    connectivity=4,
):
    """
```

```
    gray_img : image en niveaux de gris (uint8)
    Retour :
      dict avec :
        - 'binary'
        - 'hist' (256,)
        - 'n_labels', 'labels', 'stats', 'centroids'
        - 'false_colors', 'false_colors_centroid'
    """

    # Histogramme global (0-255) - comme dans le script original
    hist, _ = np.histogram(gray_img.flatten(), bins=256, range=(0, 255))

    # Seuillage adaptatif (version ingénieur)
    th2 = cv2.adaptiveThreshold(
        gray_img,
        255,
        cv2.ADAPTIVE_THRESH_MEAN_C,
        cv2.THRESH_BINARY,
        adaptive_block_size,
        adaptive_C,
    )

    # Enchaînement de flous médians
    blurred = th2.copy()
    for k in median_kernel_sequence:
        blurred = cv2.medianBlur(blurred, k)

    binary = blurred  # masque final (0/255)

    # Connected components + fausses couleurs
    (
        n_labels,
        labels,
        stats,
        centroids,
        false_colors,
        false_colors_centroid,
    ) = label_components(binary, connectivity=connectivity)

    return {
        "binary": binary,
        "hist": hist,
        "n_labels": n_labels,
        "labels": labels,
        "stats": stats,
        "centroids": centroids,
        "false_colors": false_colors,
```

```
        "false_colors_centroid": false_colors_centroid,
    }
```

ce bloc, `segment_manual_threshold`, encapsule entièrement la logique de la méthode de seuillage manuel. À partir d'une image en niveaux de gris, la fonction commence par calculer l'histogramme global des intensités (256 classes), ce qui documente la distribution des niveaux de gris et peut servir à analyser la qualité de la micrographie. Ensuite, elle applique un seuillage adaptatif de type `ADAPTIVE_THRESH_MEAN_C`, comme dans le script original de l'ingénieur, afin de produire un premier masque binaire qui s'adapte aux variations locales de luminosité. Ce masque est ensuite régularisé par une séquence de flous médians successifs (noyaux 5×5 puis 7×7), ce qui permet de réduire le bruit, combler de petits trous et lisser les frontières des grains. Le résultat flouté constitue le masque final, qui est immédiatement passé à `label_components` pour obtenir labels, statistiques, centroïdes et fausses couleurs. La fonction renvoie enfin un dictionnaire structuré contenant le masque binaire, l'histogramme et toutes les informations de connected components, ce qui transforme la section "MANUAL IMAGE THRESHOLDING METHODS" en un module propre, paramétrable et facilement comparable aux méthodes basées sur le gradient et sur HED.

[ ]:
```
#  Étape 41 - Module de segmentation : gradient

def segment_gradient(
    gray_img,
    canny_threshold1=100,
    canny_threshold2=200,
    canny_inv_thresh=90,
    gauss_kernel=(3, 3),
    final_thresh=190,
    connectivity=4,
):
    """
    gray_img : image en niveaux de gris (uint8)
    Retour :
      dict avec :
        - 'prewittx', 'prewitty'
        - 'sobelx', 'sobely'
        - 'canny'
        - 'binary'
        - 'n_labels', 'labels', 'stats', 'centroids'
        - 'false_colors', 'false_colors_centroid'
    """

    # Kernels Prewitt (comme dans le script)
    kernelx = np.array([[1, 1, 1],
                        [0, 0, 0],
                        [-1, -1, -1]], dtype=np.float32)
    kernely = np.array([[-1, 0, 1],
                        [-1, 0, 1],
                        [-1, 0, 1]], dtype=np.float32)
```

```
    prewittx = cv2.filter2D(gray_img, -1, kernelx)
    prewitty = cv2.filter2D(gray_img, -1, kernely)

    # Sobel (built-in OpenCV)
    sobelx = cv2.Sobel(gray_img, cv2.CV_64F, 1, 0)
    sobely = cv2.Sobel(gray_img, cv2.CV_64F, 0, 1)

    # Canny
    canny = cv2.Canny(gray_img, canny_threshold1, canny_threshold2)

    # Inversion + flous + seuillage (pipeline de l'ingénieur)
    _, threshCAN = cv2.threshold(canny, canny_inv_thresh, 255, cv2.
↪THRESH_BINARY_INV)

    blur1 = cv2.GaussianBlur(threshCAN, gauss_kernel, 0)
    blur2 = cv2.GaussianBlur(blur1, gauss_kernel, 0)
    _, binary = cv2.threshold(blur2, final_thresh, 255, cv2.THRESH_BINARY)

    # Connected components
    (
        n_labels,
        labels,
        stats,
        centroids,
        false_colors,
        false_colors_centroid,
    ) = label_components(binary, connectivity=connectivity)

    return {
        "prewittx": prewittx,
        "prewitty": prewitty,
        "sobelx": sobelx,
        "sobely": sobely,
        "canny": canny,
        "binary": binary,
        "n_labels": n_labels,
        "labels": labels,
        "stats": stats,
        "centroids": centroids,
        "false_colors": false_colors,
        "false_colors_centroid": false_colors_centroid,
    }
```

La fonction `segment_gradient` implémente une chaîne complète de segmentation basée sur les gradients, en reprenant et en structurant proprement la logique de la section « GRADIENT EDGE DETECTION ». À partir d'une image en niveaux de gris, elle commence par calculer les contours avec deux familles de filtres : Prewitt et Sobel. Les noyaux Prewitt sont codés “à la main” via

`cv2.filter2D`, l'un sensible aux variations horizontales (`prewittx`), l'autre aux variations verticales (`prewitty`). En parallèle, la fonction applique les filtres Sobel intégrés d'OpenCV (`cv2.Sobel`) pour obtenir `sobelx` et `sobely`, qui fournissent une approximation plus lissée du gradient. Ces quatre cartes (Prewitt/Sobel horizontales et verticales) sont conservées dans le dictionnaire de sortie pour l'analyse et la comparaison des différentes signatures de gradient, mais ne sont pas directement utilisées pour produire le masque final. Le cœur de la segmentation repose ensuite sur Canny et un pipeline de post-traitement inspiré du script de l'ingénieur. On calcule d'abord une carte de contours avec `cv2.Canny`, pilotée par deux seuils (`canny_threshold1`, `canny_threshold2`). Cette carte est ensuite inversée par un seuillage binaire (`cv2.threshold` avec `THRESH_BINARY_INV`) afin de transformer les contours en régions pleines, ce qui prépare le terrain pour une segmentation en régions. Deux flous gaussiens successifs (`GaussianBlur`) viennent lisser cette image binaire inversée, combler les petits trous et réduire le bruit de contour. Enfin, un dernier seuillage global (`final_thresh`) produit le masque binaire "propre" `binary`, qui représente la segmentation finale issue de la méthode gradient. Ce masque est passé à `label_components`, qui réalise le connected components, calcule les stats et les centroïdes, et génère les images en fausses couleurs. Au final, `segment_gradient` retourne un dictionnaire très complet qui regroupe à la fois les cartes de gradient (Prewitt, Sobel), la carte Canny brute, le masque binaire final et toutes les informations de labeling. Cette structure permet de comparer directement la segmentation gradient aux autres méthodes (thresholding manuel, HED), tout en gardant une traçabilité claire des étapes intermédiaires.

[ ]:
```python
#  Étape 42 - Module HED : edges + segmentation

def compute_hed_edges(img_rgb, net, scalefactor=0.7):
    """
    img_rgb : image couleur (H, W, 3) en RGB (uint8)
    net     : réseau HED déjà chargé via cv2.dnn.readNetFromCaffe(...)
    Retour  : carte HED (uint8, 0-255)
    """
    (H, W) = img_rgb.shape[:2]

    # Moyenne par canal (comme dans le code d'origine)
    mean_pixel_values = np.average(img_rgb, axis=(0, 1))

    blob = cv2.dnn.blobFromImage(
        img_rgb,
        scalefactor=scalefactor,
        size=(W, H),
        mean=(
            float(mean_pixel_values[0]),
            float(mean_pixel_values[1]),
            float(mean_pixel_values[2]),
        ),
        swapRB=False,
        crop=False,
    )

    net.setInput(blob)
```

```
    hed = net.forward()
    hed = hed[0, 0, :, :]           # enlever batch & channel
    hed = (255 * hed).astype("uint8")

    return hed


def segment_hed_from_edges(
    hed_map,
    adaptive_block_size=49,
    adaptive_C=2,
    global_inv_thresh=25,
    connectivity=4,
):
    """
    hed_map : carte HED uint8 (0-255)
    Retour :
      dict avec :
        - 'hed' (edge map)
        - 'adapt_binary' (adaptiveThreshold INV)
        - 'global_binary' (thresh inverse)
        - 'n_labels', 'labels', 'stats', 'centroids'
        - 'false_colors', 'false_colors_centroid'
    """

    # Seuillage adaptatif (comme AdThreshHED)
    adapt_binary = cv2.adaptiveThreshold(
        hed_map,
        255,
        cv2.ADAPTIVE_THRESH_MEAN_C,
        cv2.THRESH_BINARY_INV,
        adaptive_block_size,
        adaptive_C,
    )

    # Seuillage global inverse (comme threshCAN sur hed)
    _, global_binary = cv2.threshold(
        hed_map, global_inv_thresh, 255, cv2.THRESH_BINARY_INV
    )

    # Connected components sur la version adaptative (choix par défaut)
    (
        n_labels,
        labels,
        stats,
        centroids,
        false_colors,
```

```
        false_colors_centroid,
    ) = label_components(adapt_binary, connectivity=connectivity)

    return {
        "hed": hed_map,
        "adapt_binary": adapt_binary,
        "global_binary": global_binary,
        "n_labels": n_labels,
        "labels": labels,
        "stats": stats,
        "centroids": centroids,
        "false_colors": false_colors,
        "false_colors_centroid": false_colors_centroid,
    }
```

Le module HED découpe la logique en deux briques : d'abord la production d'une carte de contours par le réseau profond HED, puis la conversion de cette carte en segmentation de grains via des seuillages et du connected components. La fonction `compute_hed_edges` prend une image couleur RGB et un réseau déjà chargé avec `cv2.dnn.readNetFromCaffe`. Elle commence par récupérer la hauteur et la largeur de l'image, puis calcule la moyenne des valeurs de pixels par canal (R, G, B). Ces moyennes servent de termes de normalisation dans `cv2.dnn.blobFromImage`, qui construit un "blob" adapté au réseau : bonne taille spatiale (W, H), échelle (`scalefactor`) et soustraction de la moyenne par canal. On injecte ensuite ce blob dans le réseau (`net.setInput(blob)`), on fait une passe avant (`net.forward()`), puis on extrait le premier canal de sortie (en supprimant les dimensions batch et channel). Comme la sortie du réseau est typiquement dans [0,1], on la remap sur [0,255] et on la convertit en `uint8`. Le résultat `hed` est donc une carte de contours HED dense, au même format qu'une image en niveaux de gris classique, prête à être seuillée. La fonction `segment_hed_from_edges` prend cette carte HED et la transforme en segmentation exploitable. Elle applique d'abord un seuillage adaptatif inversé (`cv2.adaptiveThreshold` avec `THRESH_BINARY_INV`) pour produire `adapt_binary` : là où la carte HED est forte, on obtient des zones blanches dans le masque binaire, en tenant compte des variations locales grâce au bloc `adaptive_block_size` et au paramètre `adaptive_C`. En parallèle, un seuillage global inverse (`cv2.threshold` avec `THRESH_BINARY_INV`) fournit `global_binary`, qui est une version plus simple, pilotée par un seul seuil `global_inv_thresh`. Dans cette implémentation, le connected components est réalisé sur la version adaptative (choix par défaut jugé plus robuste sur des contrastes non homogènes). La fonction `label_components` est appelée sur `adapt_binary`, ce qui produit le nombre total de labels, la matrice de labels, les statistiques géométriques (x, y, w, h, area), les centroïdes, ainsi que deux images en fausses couleurs (avec et sans croix aux centroïdes). Au final, `segment_hed_from_edges` renvoie un dictionnaire complet contenant la carte HED d'origine, les deux masques binaires (adaptatif et global) et tous les objets de segmentation. Cela permet de comparer finement la méthode HED aux autres approches (gradient, thresholding manuel), tout en gardant la trace de chaque étape de transformation de la carte de contours en grains segmentés.

[ ]:
```
#  Étape 43 - Test multi-méthodes sur une image

test_img_path = IMG_DIR / image_files[0]
print("Image de test pour la segmentation :", test_img_path)
```

```
img_bgr = cv2.imread(str(test_img_path), cv2.IMREAD_COLOR)
if img_bgr is None:
    raise FileNotFoundError(f"Impossible de lire l'image : {test_img_path}")

img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
img_gray = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY)

seg_results = {}

# 1) Thresholding manuel
seg_results["manual"] = segment_manual_threshold(img_gray)

# 2) Gradient
seg_results["gradient"] = segment_gradient(img_gray)

# 3) HED (si le réseau est disponible)
hed_net = None
if "HED_NET" in globals():
    hed_net = HED_NET
elif "net_hed" in globals():
    hed_net = net_hed

if hed_net is not None:
    hed_map = compute_hed_edges(img_rgb, hed_net)
    seg_results["hed"] = segment_hed_from_edges(hed_map)
else:
    print("  Réseau HED non trouvé (HED_NET ou net_hed). Étape HED ignorée pour␣
 ↪ce test.")

# Visualisation
n_methods = len(seg_results)
plt.figure(figsize=(5 * n_methods, 5))

for idx, (method_name, res) in enumerate(seg_results.items(), start=1):
    plt.subplot(1, n_methods, idx)
    plt.imshow(res["false_colors"])
    plt.title(f"Segmentation {method_name}", fontsize=14)
    plt.axis("off")

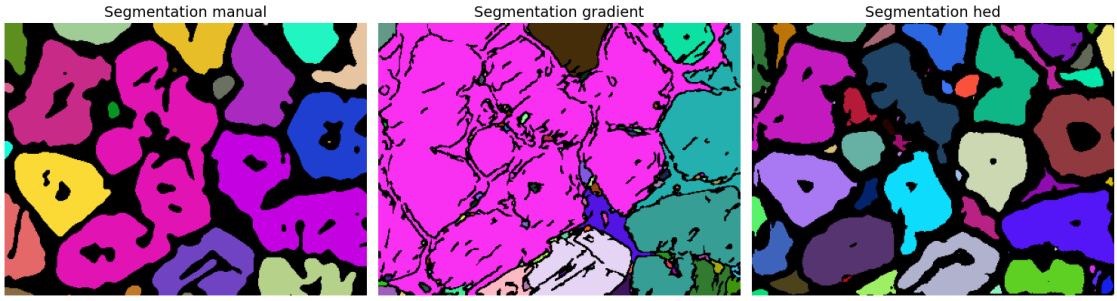
plt.tight_layout()
plt.show()

# Résumé du nombre de grains
rows = []
for method_name, res in seg_results.items():
    n_labels = int(res["n_labels"])
```

```
    n_grains = max(n_labels - 1, 0)
    rows.append({"method": method_name, "n_labels": n_labels, "n_grains":␣
 ↪n_grains})

seg_summary_test = pd.DataFrame(rows)
print("Résumé segmentation (image de test) :")
print(seg_summary_test)
```

```
Image de test pour la segmentation :
/content/drive/MyDrive/Data/Grains/A_01_01.png
```




```
Résumé segmentation (image de test) :
     method  n_labels  n_grains
0    manual        27        26
1  gradient       153       152
2       hed        63        62
```

Cette étape joue le rôle de banc d'essai comparatif entre les trois familles de segmentation développées dans le notebook. On commence par charger une image de référence en couleur puis on en dérive sa version en niveaux de gris. Sur cette même image test, on applique successivement : (i) la méthode de seuillage manuel (`segment_manual_threshold`), (ii) la méthode à base de gradient (`segment_gradient`), et (iii) la méthode HED (`compute_hed_edges` + `segment_hed_from_edges`), à condition que le réseau HED ait bien été chargé en mémoire. Les résultats sont stockés dans un dictionnaire `seg_results` puis affichés côte à côte sous forme de fausses couleurs, ce qui permet d'inspecter visuellement la qualité des contours, la séparation entre grains voisins et la présence éventuelle de bruit ou de sur-segmentation. Enfin, un petit tableau récapitulatif calcule pour chaque méthode le nombre total de labels et le nombre de grains (labels non nuls). Dans l'exemple illustré, le seuillage manuel détecte 26 grains, le gradient monte à 152 grains (forte sur-segmentation liée aux fragments de frontières), alors que HED détecte 62 grains, ce qui reflète un compromis intéressant : plus détaillé que la méthode manuelle, mais moins fragmenté que le gradient pur. Cette étape sert donc à valider qualitativement et quantitativement le comportement de chaque pipeline avant d'aller plus loin dans le nettoyage des labels et l'extraction de mesures.

[ ]:
```
#  Étape 44 - Nettoyage des labels par aire

def filter_components_by_area(labels, stats, min_area_px=100):
    """
```

```
    labels : matrice de labels (sortie de connectedComponentsWithStats)
    stats  : tableau stats (n_labels, 5) associé aux labels
             (colonnes OpenCV : x, y, w, h, area)
    min_area_px : aire minimale (en pixels) pour garder un objet

    Retour :
      - new_labels                : labels filtrés (fond = 0)
      - kept_indices              : indices des labels gardés (  0)
      - false_colors              : fausses couleurs après filtrage
      - false_colors_centroid     : fausses couleurs + centroïdes
    """
    # Indices des labels que l'on garde (sauf le fond 0)
    areas = stats[:, cv2.CC_STAT_AREA]
    kept = np.where(areas >= min_area_px)[0]
    kept = kept[kept != 0]  # ne jamais garder le fond comme "grain"

    # Masque : True là où le label est gardé
    mask_keep = np.isin(labels, kept)

    # On met à 0 (fond) tout ce qui n'est pas gardé
    new_labels = np.where(mask_keep, labels, 0).astype(np.int32)

    # Recalcule fausses couleurs + centroïdes à partir des labels filtrés
    n_labels_f, labels_f, stats_f, centroids_f = cv2.
↪connectedComponentsWithStats(
        (new_labels > 0).astype(np.uint8), connectivity=4
    )

    colors = np.random.randint(0, 255, size=(n_labels_f, 3), dtype=np.uint8)
    colors[0] = [0, 0, 0]
    false_colors = colors[labels_f]

    false_colors_centroid = false_colors.copy()
    for (cx, cy) in centroids_f:
        cv2.drawMarker(
            false_colors_centroid,
            (int(cx), int(cy)),
            color=(255, 255, 255),
            markerType=cv2.MARKER_CROSS,
            markerSize=10,
            thickness=1,
        )

    return {
        "new_labels": new_labels,
        "kept_indices": kept,
        "n_labels_f": n_labels_f,
```

```
        "stats_f": stats_f,
        "centroids_f": centroids_f,
        "false_colors": false_colors,
        "false_colors_centroid": false_colors_centroid,
    }
```

Cette étape introduit un filtre morphologique simple mais essentiel pour nettoyer les cartes de labels produites par la segmentation. À partir de la matrice de labels et du tableau `stats` renvoyés par `connectedComponentsWithStats`, la fonction commence par calculer l'aire de chaque composante puis sélectionne uniquement les objets dont l'aire dépasse un seuil `min_area_px`, en excluant systématiquement le label 0 qui correspond au fond. Tous les pixels appartenant à des labels trop petits (micro-fragments de frontières, bruit, artefacts issus du gradient) sont remis à 0 dans `new_labels`, ce qui revient à les supprimer de la segmentation. On relance ensuite un `connectedComponentsWithStats` sur cette version binaire nettoyée pour obtenir un nouveau jeu de labels compacts (`labels_f`) ainsi que des statistiques et centroïdes cohérents avec les objets filtrés. Enfin, on génère une image en fausses couleurs et une version annotée avec des croix blanches aux centroïdes, ce qui permet de visualiser facilement l'effet du filtrage. En pratique, cette fonction sert de "post-traitement universel" que l'on applique surtout sur la méthode gradient, très sujette à la sur-segmentation, pour se rapprocher d'un nombre de grains réaliste avant de passer à la mesure géométrique.

[ ]:
```
#  Étape 45 - Filtrage de la méthode gradient sur l'image de test

grad_res = seg_results["gradient"]

#  à ajuster : seuil d'aire minimale en pixels
#   - commence par 300-500 px
#   - augmente si tu vois encore beaucoup de petits morceaux
filtered_grad = filter_components_by_area(
    labels=grad_res["labels"],
    stats=grad_res["stats"],
    min_area_px=500,   # à tuner !
)

# Visualisation avant / après
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.imshow(grad_res["false_colors"])
plt.title(f"Gradient - brut (N={grad_res['n_labels']-1} grains)", fontsize=12)
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(filtered_grad["false_colors"])
plt.title(
    f"Gradient - filtré (N={filtered_grad['n_labels_f']-1} grains)",
    fontsize=12,
```

```
)
plt.axis("off")

plt.tight_layout()
plt.show()

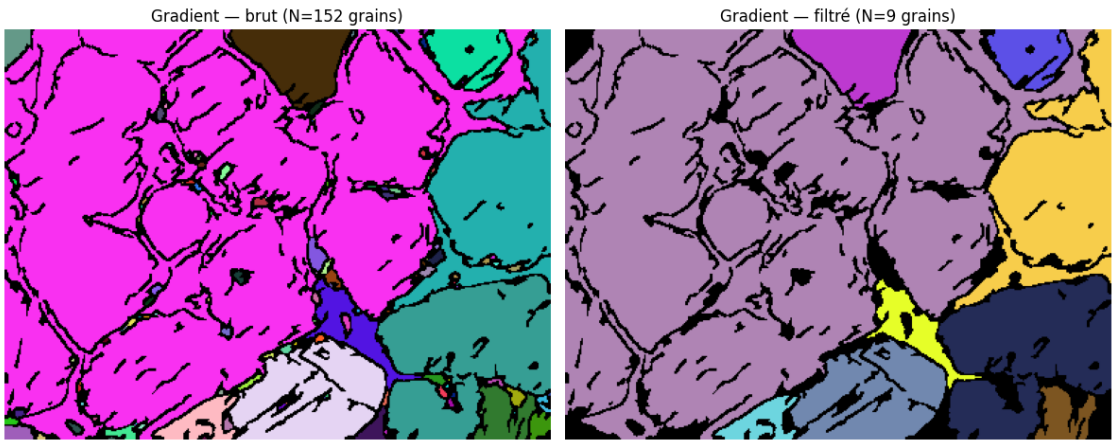
# Nouveau petit résumé
rows = []

# Manual
rows.append({
    "method": "manual",
    "n_grains": int(seg_results["manual"]["n_labels"] - 1),
})

# Gradient brut
rows.append({
    "method": "gradient_raw",
    "n_grains": int(seg_results["gradient"]["n_labels"] - 1),
})

# Gradient filtré
rows.append({
    "method": "gradient_filtered",
    "n_grains": int(filtered_grad["n_labels_f"] - 1),
})

seg_summary_test2 = pd.DataFrame(rows)
print("Résumé segmentation (image de test, avec filtrage gradient) :")
print(seg_summary_test2)
```




```
Résumé segmentation (image de test, avec filtrage gradient) :
```

```
              method  n_grains
0             manual        26
1      gradient_raw       152
2  gradient_filtered         9
```

Cette étape applique concrètement le filtre par aire à la segmentation issue du gradient et en montre l'impact visuel et numérique. À partir des labels bruts de la méthode gradient, `filter_components_by_area` supprime toutes les composantes dont l'aire est inférieure à `min_area_px = 500` pixels, ce qui revient à éliminer la multitude de petits segments parasites le long des frontières. La figure avant/après illustre bien ce nettoyage : à gauche, la segmentation gradient brute présente de très nombreuses petites taches colorées accrochées aux joints de grains (152 "grains" détectés), typiques d'une sur-segmentation par détection de contours ; à droite, après filtrage, la plupart de ces fragments disparaissent au profit de quelques grands amas cohérents (9 grains retenus), beaucoup plus proches de la morphologie des vrais grains mais encore en dessous du comptage manuel (26 grains). Le tableau récapitulatif confirme ce comportement : la méthode manuelle reste un point de référence réaliste, le gradient brut surestime fortement le nombre de grains, tandis que le gradient filtré les regroupe de manière agressive, ce qui suggère que le seuil de 500 px est volontairement "dur" pour illustrer le principe et devra ensuite être ajusté (diminué par exemple) pour atteindre un compromis entre élimination du bruit et préservation des petits grains réels.

[ ]:
```python
# Étape 45 - Extraction des caractéristiques à partir d'une carte de labels

from skimage.measure import regionprops_table

def compute_grain_features_from_labels(
    labels: np.ndarray,
    image_id: str,
    min_area_px: int = 50,
    pixel_size_um: float | None = None
) -> tuple[np.ndarray, pd.DataFrame]:
    """
    labels      : image de labels (2D, int), avec 0 = fond, 1..N = grains
    image_id    : identifiant de l'image (ex: 'A_01_01')
    min_area_px : seuil d'aire minimale pour garder un grain
    pixel_size_um : taille d'un pixel en µm (si connue) pour convertir les␣
 ↪mesures
    """

    labels = np.asarray(labels, dtype=np.int32)

    props = regionprops_table(
        labels,
        properties=[
            "label",
            "area",
            "perimeter",
            "equivalent_diameter",
```

```
            "major_axis_length",
            "minor_axis_length",
            "eccentricity",
            "centroid",
        ],
    )
    df = pd.DataFrame(props)

    # Suppression du fond
    df = df[df["label"] != 0].copy()
    # Filtre sur l'aire
    df = df[df["area"] >= min_area_px].copy()

    if df.empty:
        labels_clean = np.zeros_like(labels, dtype=np.int32)
        empty_df = pd.DataFrame(
            columns=[
                "image_id", "area_px", "perimeter_px", "equiv_diameter_px",
                "major_axis_px", "minor_axis_px", "eccentricity",
                "centroid_row", "centroid_col", "grain_id",
                "axis_ratio", "circularity",
                "area_um2", "perimeter_um",
                "equivalent_diameter_um", "major_axis_um", "minor_axis_um",
            ]
        )
        return labels_clean, empty_df

    df = df.sort_values("label").reset_index(drop=True)
    df["grain_id"] = np.arange(1, len(df) + 1, dtype=int)

    labels_clean = np.zeros_like(labels, dtype=np.int32)
    for old_label, new_id in zip(df["label"].astype(int), df["grain_id"].
↪astype(int)):
        labels_clean[labels == old_label] = new_id

    df = df.rename(
        columns={
            "area": "area_px",
            "perimeter": "perimeter_px",
            "equivalent_diameter": "equiv_diameter_px",
            "major_axis_length": "major_axis_px",
            "minor_axis_length": "minor_axis_px",
            "eccentricity": "eccentricity",
            "centroid-0": "centroid_row",
            "centroid-1": "centroid_col",
        }
    )
```

```
    df["axis_ratio"] = df["major_axis_px"] / (df["minor_axis_px"] + 1e-8)
    df["circularity"] = 4.0 * np.pi * df["area_px"] / (df["perimeter_px"]**2 +␣
↪1e-8)

    if pixel_size_um is not None:
        df["area_um2"] = df["area_px"] * (pixel_size_um**2)
        df["equiv_diameter_um"] = df["equiv_diameter_px"] * pixel_size_um
        df["major_axis_um"] = df["major_axis_px"] * pixel_size_um
        df["minor_axis_um"] = df["minor_axis_px"] * pixel_size_um

    df.insert(0, "image_id", image_id)
    df = df.drop(columns=["label"])

    return labels_clean, df
```

Cette étape transforme une simple carte de labels en un véritable tableau de mesures géométriques exploitables pour chaque grain. À partir d'une image de labels où 0 correspond au fond et les autres valeurs aux grains, la fonction commence par extraire, via `regionprops_table`, les propriétés de chaque région connexe : aire, périmètre, diamètre équivalent, longueurs des axes majeur et mineur, excentricité et coordonnées du centroïde. On supprime ensuite le fond (label 0) et les grains trop petits en imposant un seuil minimal d'aire `min_area_px`, ce qui évite de conserver du bruit ou des micro-fragments non représentatifs. Si aucun grain ne reste, la fonction renvoie une carte de labels vide et un DataFrame vide déjà structuré avec toutes les colonnes attendues. Dans le cas général, les grains sont triés, puis re-numérotés de 1 à N pour obtenir une carte `labels_clean` cohérente et compacte, où chaque grain possède un identifiant unique `grain_id`. Le DataFrame est alors nettoyé et renommé avec des noms explicites (`area_px`, `perimeter_px`, `equiv_diameter_px`, etc.), les coordonnées des centroïdes sont mises en forme (`centroid_row`, `centroid_col`), et deux indicateurs de forme sont ajoutés : le rapport d'axes (`axis_ratio`) et la circularité, qui mesure à quel point le grain est proche d'un cercle. Enfin, si la taille physique du pixel `pixel_size_um` est fournie, toutes les grandeurs sont converties en unités réelles (µm, µm²), avant d'insérer l'identifiant de l'image `image_id` en première colonne. Au final, la fonction retourne donc d'un côté une carte de labels nettoyée et re-indexée, et de l'autre un tableau structuré des caractéristiques morphologiques de chaque grain, prêt pour l'analyse statistique ou la corrélation avec les propriétés mécaniques du matériau.

[ ]:
```
# Étape 46 - Application sur une image (ex: image de référence déjà traitée)

def draw_grain_ids_on_image(
    img_rgb: np.ndarray,
    features_df: pd.DataFrame,
    id_col: str = "grain_id",
) -> np.ndarray:
    vis = img_rgb.copy()
    for _, row in features_df.iterrows():
        gid = int(row[id_col])
        r = int(row["centroid_row"])
```

```
        c = int(row["centroid_col"])
        cv2.putText(
            vis,
            str(gid),
            (c, r),
            fontFace=cv2.FONT_HERSHEY_SIMPLEX,
            fontScale=0.4,
            color=(255, 0, 0),
            thickness=1,
            lineType=cv2.LINE_AA,
        )
    return vis

# Si P2M n'est pas déjà défini, on le recalcule (fallback)
try:
    P2M
except NameError:
    pixels = 225
    microns = 100
    scale = pixels / microns
    P2M = 1.0 / scale

def add_physical_units(df: pd.DataFrame, p2m: float) -> pd.DataFrame:
    df = df.copy()
    if "area_px" in df.columns:
        df["area_um2"] = df["area_px"] * (p2m ** 2)

    px_um_pairs = [
        ("perimeter_px",          "perimeter_um"),
        ("equiv_diameter_px",     "equiv_diameter_um"),
        ("major_axis_px",         "major_axis_um"),
        ("minor_axis_px",         "minor_axis_um"),
    ]
    for col_px, col_um in px_um_pairs:
        if col_px in df.columns:
            df[col_um] = df[col_px] * p2m
    return df

# 1) Choix de la méthode (hed si dispo, sinon gradient, sinon manual)
if "hed" in seg_results:
    PREFERRED_METHOD = "hed"
elif "gradient" in seg_results:
    PREFERRED_METHOD = "gradient"
else:
    PREFERRED_METHOD = "manual"

seg_ref = seg_results[PREFERRED_METHOD]
```

```
labels_ref = seg_ref["labels"]

# 2) ID de l'image
idx_ref = 0
fname_ref = image_files[idx_ref]
image_id_ref = Path(fname_ref).stem

# 3) Features en pixels
labels_clean_ref, features_ref = compute_grain_features_from_labels(
    labels_ref,
    image_id=image_id_ref,
    min_area_px=80,
    pixel_size_um=None,
)

# 3-bis) Ajout unités physiques
features_ref = add_physical_units(features_ref, P2M)

print(f"Méthode de référence utilisée : {PREFERRED_METHOD}")
print("Aperçu des caractéristiques pour l'image de référence :")
display(features_ref.head())

# 4) Image RGB de référence
try:
    img_rgb
except NameError:
    img_bgr_ref = cv2.imread(str(IMG_DIR / fname_ref))
    img_rgb = cv2.cvtColor(img_bgr_ref, cv2.COLOR_BGR2RGB)

# 5) Visualisation
vis_ids_ref = draw_grain_ids_on_image(img_rgb, features_ref)

plt.figure(figsize=(15, 5))

plt.subplot(1, 3, 1)
plt.title("Image originale")
plt.imshow(img_rgb)
plt.axis("off")

plt.subplot(1, 3, 2)
plt.title(f"Labels nettoyés ({PREFERRED_METHOD})")
plt.imshow(labels_clean_ref, cmap="nipy_spectral")
plt.axis("off")

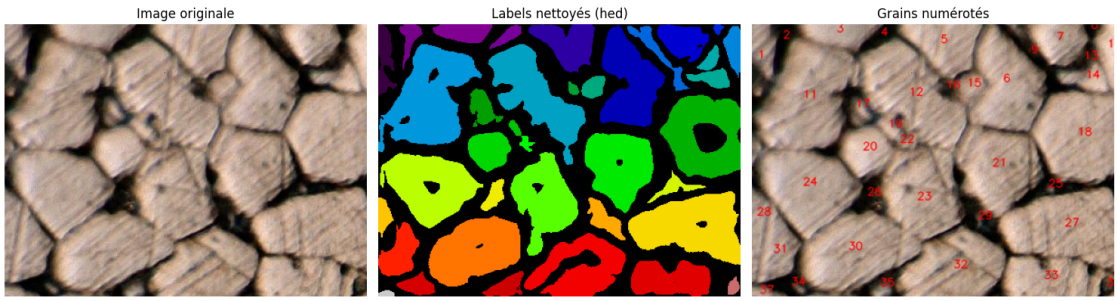
plt.subplot(1, 3, 3)
plt.title("Grains numérotés")
plt.imshow(vis_ids_ref)
```

```
plt.axis("off")

plt.tight_layout()
plt.show()
```

```
Méthode de référence utilisée : hed
Aperçu des caractéristiques pour l'image de référence :
```

	image_id	area_px	perimeter_px	equiv_diameter_px	major_axis_px	minor_axis_px	eccentricity	centroid_row	centroid_col	grain_id	axis_ratio	circularity	area_um2	perimeter_um	equiv_diameter_um	major_axis_um	minor_axis_um
0	A_01_01	965.0	219.195959	35.052477	86.537242	18.450218	0.977007	37.994819	6.848705	1	4.690310	0.252390	190.617284	97.420426	15.578879	38.460997	8.200097
1	A_01_01	525.0	131.497475	25.854415	48.132709	20.611472	0.903674	15.102857	34.329524	2	2.335239	0.381535	103.703704	58.443322	11.490851	21.392315	9.160654
2	A_01_01	1062.0	153.012193	36.772006	65.980106	21.691865	0.944412	8.436911	93.967043	3	3.041698	0.570010	209.777778	68.005419	16.343114	29.324492	9.640829
3	A_01_01	374.0	97.189863	21.821815	41.644608	12.324281	0.955207	12.558824	142.652406	4	3.379070	0.497553	73.876543	43.195495	9.698584	18.508714	5.477458
4	A_01_01	2339.0	242.308658	54.572038	73.019071	49.720637	0.732352	20.925182	208.728516	5	1.468587	0.500613	462.024691	107.692737	24.254239	32.452921	22.098061



Cette étape applique l'ensemble du pipeline à une image de référence pour vérifier, de façon visuelle

et numérique, que tout fonctionne de bout en bout. On commence par une petite fonction utilitaire `draw_grain_ids_on_image` qui projette, sur l'image RGB originale, les identifiants de grains issus du DataFrame de caractéristiques, en utilisant les centroïdes pour positionner les numéros ; cela permet de faire le lien direct entre chaque grain visible et sa ligne dans le tableau. Ensuite, si la conversion pixel → micromètre `P2M` n'existe pas encore, elle est recalculée à partir d'une échelle connue (225 pixels pour 100 µm), puis la fonction `add_physical_units` enrichit le DataFrame avec les grandeurs physiques associées (aire en µm², périmètre et diamètres en µm). La méthode de segmentation « préférée » est choisie automatiquement (HED si disponible, sinon gradient, sinon manuel), et l'image de référence `A_01_01` est transformée en carte de labels nettoyée avec `compute_grain_features_from_labels`, puis convertie en unités réelles. L'aperçu du DataFrame montre que chaque grain possède désormais un identifiant, une position, des mesures géométriques en pixels et en micromètres, ce qui valide la calibration. Enfin, la figure à trois volets juxtapose l'image brute, les labels nettoyés en fausses couleurs et les grains numérotés sur l'image originale : on voit ainsi que la segmentation HED sélectionnée isole correctement les grains, que le nettoyage supprime le bruit, et que chaque numéro inscrit correspond bien à un grain réellement mesuré dans le tableau de caractéristiques.

[ ]:
```
# ============================================================
#   STEP 47 - Dataset U-Net (Unified) + 3 inputs (X,G,H)
#     X = image         (256,256,1) float32 [0,1]
#     G = grad_prior    (Canny 100/200 -> threshold 90) (256,256,1) float32 [0,1]
#     H = hed_prior     (HED pre/raw if exists else edge-like)      (256,256,1)␣
 ↪float32 [0,1]
#     Y = mask          (GT mask OR pseudo-label via HED auto)      (256,256,1)␣
 ↪float32 {0,1}
#
#   Corrections intégrées (finales) :
#    (1) Audit dossiers priors + matching robuste "HEDPre/GradPre/ThreshPre"␣
 ↪pour RG (stems)
#    (2) NEW_SUBSET = "RG"|"AG"|"both" (pour filtrer la collecte Kaggle si␣
 ↪besoin)
#    (3) NEW_MASK_INVERT = "auto"|True|False (fix pos_rate trop élevé =>␣
 ↪boundaries)
#    (4) Split anti-leakage : SPLIT_MODE="random"|"grouped" (GroupShuffleSplit)
#
# Sorties :
#    X_train, G_train, H_train, Y_train
#    train_ds, val_ds : ((X,G,H), Y) + batch + prefetch
#    + résumé lisible comme UNE SEULE base (avec breakdown optionnel par␣
 ↪sous-collections)
# ============================================================

from pathlib import Path
import re
import numpy as np
import cv2
import matplotlib.pyplot as plt
```

```
import tensorflow as tf
from sklearn.model_selection import train_test_split, GroupShuffleSplit

# ----------------------------
# Config
# ----------------------------
IMG_HEIGHT, IMG_WIDTH, IMG_CHANNELS = 256, 256, 1
MIN_AREA_PX = 80
MAX_AREA_RATIO = 0.60
RANDOM_SEED = 42

DATASET_MODE   = "both"     # "old" | "new" | "both"  (gardé pour compat␣
 ↪notebook, mais logs "unified")
NEW_SUBSET     = "both"     # "RG" | "AG" | "both"
USE_NEW_SPLITS = True       # True: split global ; False: split par␣
 ↪sous-collection puis concat
SPLIT_MODE     = "grouped"  # "random" | "grouped"

VAL_FRACTION   = 0.20
BATCH_SIZE     = 8

NEW_MASK_INVERT = "auto"    # "auto" | True | False (pour masques boundaries)

# 2.25 px = 1 micron (constant conservée)
P2M = globals().get("P2M", 2.25)

# Variables attendues par le notebook
IMG_DIR = Path(IMG_DIR)
HED_RAW_DIR = Path(HED_RAW_DIR)

# ----------------------------
# Dataset directories (same root)
# ----------------------------
BASE_DIR = Path(globals().get("BASE_DIR", IMG_DIR.parent))
DEFAULT_GRAIN = BASE_DIR / "GRAIN DATA SET"
GRAIN_DS_DIR = Path(globals().get("GRAIN_DS_DIR", DEFAULT_GRAIN if␣
 ↪DEFAULT_GRAIN.exists() else BASE_DIR))

AG_DIR         = Path(globals().get("AG_DIR",          GRAIN_DS_DIR / "AG"))
AGMASK_DIR     = Path(globals().get("AGMASK_DIR",      GRAIN_DS_DIR / "AGMask"))
RG_DIR         = Path(globals().get("RG_DIR",          GRAIN_DS_DIR / "RG"))
RGMASK_DIR     = Path(globals().get("RGMASK_DIR",      GRAIN_DS_DIR / "RGMask"))
HED_PRE_DIR    = Path(globals().get("HED_PRE_DIR",     GRAIN_DS_DIR /␣
 ↪"HED_PRE"))
GRAD_PRE_DIR   = Path(globals().get("GRAD_PRE_DIR",    GRAIN_DS_DIR /␣
 ↪"GRAD_PRE"))
```

```
THRESH_PRE_DIR = Path(globals().get("THRESH_PRE_DIR",  GRAIN_DS_DIR /␣
 ↪"THRESH_PRE"))  # optionnel

# ----------------------------
# IO helpers / matching
# ----------------------------
IMG_EXTS = (".png", ".jpg", ".jpeg", ".bmp", ".tif", ".tiff")

def _read_gray(path: Path):
    if path is None:
        return None
    return cv2.imread(str(path), cv2.IMREAD_GRAYSCALE)

def _resize_gray(im: np.ndarray, w=IMG_WIDTH, h=IMG_HEIGHT, inter=cv2.
 ↪INTER_AREA):
    return cv2.resize(im, (w, h), interpolation=inter)

def _to_float01(im_uint8: np.ndarray):
    return im_uint8.astype(np.float32) / 255.0

def _binarize01(mask_uint8: np.ndarray, thr=127):
    m = mask_uint8.astype(np.uint8)
    uniq = np.unique(m)
    if len(uniq) <= 3:
        return (m > thr).astype(np.float32)
    _, mb = cv2.threshold(m, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
    return (mb > 0).astype(np.float32)

def _list_files(dirp: Path):
    if not dirp.is_dir():
        return []
    files = [p for p in dirp.iterdir() if p.is_file() and p.suffix.lower() in␣
 ↪IMG_EXTS]
    files.sort(key=lambda p: (p.stem.lower(), p.suffix.lower(), p.name.lower()))
    return files

def _glob_any_ext(directory: Path, stem: str):
    """Exact match stem + known extensions."""
    if not directory.is_dir():
        return None
    for ext in IMG_EXTS:
        p = directory / f"{stem}{ext}"
        if p.is_file():
            return p
    hits = [p for p in directory.glob(f"{stem}.*") if p.is_file() and p.suffix.
 ↪lower() in IMG_EXTS]
    hits.sort(key=lambda p: p.name.lower())
```

```
    return hits[0] if hits else None

def _audit_dir(dirp: Path, name: str, k=5):
    if not dirp.exists():
        print(f"[AUDIT] {name}: {dirp} -> NOT FOUND")
        return 0
    if not dirp.is_dir():
        print(f"[AUDIT] {name}: {dirp} -> NOT A DIR")
        return 0
    files = _list_files(dirp)
    print(f"[AUDIT] {name}: {dirp} -> {len(files)} files")
    if files:
        print("        examples:", [p.name for p in files[:k]])
    return len(files)

# ----------------------------
# Priors
# ----------------------------
def grad_prior_from_img_pdf(img_uint8: np.ndarray) -> np.ndarray:
    canny = cv2.Canny(img_uint8, 100, 200)
    _, grad_bin = cv2.threshold(canny, 90, 255, cv2.THRESH_BINARY)
    k = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (3, 3))
    grad_bin = cv2.morphologyEx(grad_bin, cv2.MORPH_CLOSE, k, iterations=1)
    return grad_bin

def edge_like_fallback(img_uint8: np.ndarray) -> np.ndarray:
    can = cv2.Canny(img_uint8, 100, 200)
    sx = cv2.Sobel(img_uint8, cv2.CV_32F, 1, 0, ksize=3)
    sy = cv2.Sobel(img_uint8, cv2.CV_32F, 0, 1, ksize=3)
    mag = cv2.magnitude(sx, sy)
    mag_u8 = cv2.normalize(mag, None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)
    out = np.maximum(can, mag_u8)
    if out.mean() < 1.0:
        out = mag_u8
    return out

# ----------------------------
# Prior filename mapping (RG stems -> HEDPre/GradPre/ThreshPre)
# ----------------------------
_re_digits = re.compile(r"(\d+)")

def _prior_stem_candidates(stem: str, subset: str, kind: str):
    """
    kind in {"hed","grad","thr"}.
    RG example:
      image stem = RG10_1_1
      priors stems = HEDPre10_1_1 / GradPre10_1_1 / ThreshPre10_1_1
```

```
    """
    kind_map = {"hed": "HEDPre", "grad": "GradPre", "thr": "ThreshPre"}
    pref = kind_map[kind]

    cands = [stem]

    if subset == "RG":
        core = stem[2:] if stem.startswith("RG") else stem
        core = core.lstrip("_")
        cands += [core, f"{pref}{core}", f"{pref}{stem}"]

    elif subset == "AG":
        # AG_NOISE229 -> 229 ; AG229 -> 229
        m = _re_digits.search(stem)
        if m:
            core = m.group(1)
            cands += [core, f"{pref}{core}", f"{pref}{stem}"]
        else:
            cands += [f"{pref}{stem}"]

    # unique stable
    out, seen = [], set()
    for x in cands:
        xl = x.lower()
        if xl not in seen:
            seen.add(xl)
            out.append(x)
    return out

def _find_prior_with_candidates(directory: Path, stem: str, subset: str, kind:␣
 ↪str):
    """
    1) exact match over candidates
    2) fuzzy contains (case-insensitive) over candidates
    """
    if not directory.is_dir():
        return None

    # exact
    for cand in _prior_stem_candidates(stem, subset, kind):
        p = _glob_any_ext(directory, cand)
        if p is not None:
            return p

    # fuzzy contains
    files = _list_files(directory)
    if not files:
```

```
        return None
    cand_l = [c.lower() for c in _prior_stem_candidates(stem, subset, kind)]
    for f in files:
        fs = f.stem.lower()
        if any(c in fs for c in cand_l):
            return f
    return None

# ============================================================
#  Pseudo-labeling function (fallback define if missing)
# ============================================================
if "hed_to_labels_auto_best" not in globals():

    def relabel_sequential(labels: np.ndarray) -> np.ndarray:
        labels = labels.astype(np.int32)
        uniq = np.unique(labels)
        uniq = uniq[uniq != 0]
        out = np.zeros_like(labels, dtype=np.int32)
        for new_id, old_id in enumerate(uniq, start=1):
            out[labels == old_id] = new_id
        return out

    def filter_by_area(labels: np.ndarray, min_area_px: int, max_area_ratio:␣
 ↪float) -> np.ndarray:
        h, w = labels.shape
        max_area = int(max_area_ratio * h * w)
        labels = labels.astype(np.int32)
        counts = np.bincount(labels.ravel())
        out = np.zeros_like(labels, dtype=np.int32)
        for lid in range(1, len(counts)):
            a = counts[lid]
            if a >= min_area_px and a <= max_area:
                out[labels == lid] = lid
        return relabel_sequential(out)

    def make_labels_from_region_mask(region_mask_255: np.ndarray,
                                     min_area_px: int,
                                     max_area_ratio: float,
                                     close_ks: int = 7,
                                     open_ks: int = 5,
                                     it_close: int = 1,
                                     it_open: int = 1) -> np.ndarray:
        region_mask_255 = region_mask_255.astype(np.uint8)
        k_close = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (close_ks,␣
 ↪close_ks))
        k_open  = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (open_ks,␣
 ↪open_ks))
```

```
        m = cv2.morphologyEx(region_mask_255, cv2.MORPH_CLOSE, k_close,␣
↪iterations=it_close)
        m = cv2.morphologyEx(m,             cv2.MORPH_OPEN,  k_open, ␣
↪iterations=it_open)
        _, lab = cv2.connectedComponents((m > 0).astype(np.uint8))
        return filter_by_area(lab, min_area_px=min_area_px,␣
↪max_area_ratio=max_area_ratio)

    def make_labels_from_edges(edge_gray: np.ndarray,
                               min_area_px: int,
                               max_area_ratio: float,
                               p: int = 85,
                               close_ks: int = 9,
                               close_iter: int = 2,
                               dilate_ks: int = 5,
                               dilate_iter: int = 1):
        g = edge_gray.astype(np.uint8)
        thr = float(np.percentile(g, p))
        thr = max(thr, 10.0)
        ebin = (g >= thr).astype(np.uint8) * 255
        k_close = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (close_ks,␣
↪close_ks))
        ebin = cv2.morphologyEx(ebin, cv2.MORPH_CLOSE, k_close,␣
↪iterations=close_iter)
        k_dil = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (dilate_ks,␣
↪dilate_ks))
        ebin = cv2.dilate(ebin, k_dil, iterations=dilate_iter)
        interiors = cv2.bitwise_not(ebin)
        _, lab = cv2.connectedComponents(interiors)
        return filter_by_area(lab, min_area_px=min_area_px,␣
↪max_area_ratio=max_area_ratio)

    def score_labels(labels: np.ndarray) -> dict:
        m = (labels > 0)
        pos = float(m.mean())
        n_labels = int(labels.max())
        if pos < 0.03 or pos > 0.85: pos_pen = 3.0
        elif pos < 0.08 or pos > 0.80: pos_pen = 1.2
        else: pos_pen = 0.0
        if n_labels < 2: n_pen = 3.0
        elif n_labels > 400: n_pen = 2.0
        elif n_labels > 250: n_pen = 1.0
        else: n_pen = 0.0
        score = -abs(pos - 0.50) - pos_pen - 0.7 * n_pen
        return {"score": float(score), "pos": float(pos), "n_labels":␣
↪int(n_labels)}
```

```
    def hed_to_labels_auto_best(hed_gray: np.ndarray,
                                min_area_px: int,
                                max_area_ratio: float,
                                debug: bool = False):
        g = hed_gray.astype(np.uint8)
        _, otsu = cv2.threshold(g, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
        otsu_inv = cv2.bitwise_not(otsu)

        lab_r1 = make_labels_from_region_mask(otsu,     min_area_px,␣
↪max_area_ratio)
        lab_r2 = make_labels_from_region_mask(otsu_inv, min_area_px,␣
↪max_area_ratio)

        lab_e  = make_labels_from_edges(g, min_area_px, max_area_ratio, p=85)

        s_r1 = score_labels(lab_r1)
        s_r2 = score_labels(lab_r2)
        s_e  = score_labels(lab_e)

        candidates = [("region", lab_r1, s_r1), ("region_inv", lab_r2, s_r2),␣
↪("edge", lab_e, s_e)]
        mode, labels_best, stats_best = max(candidates, key=lambda x:␣
↪x[2]["score"])

        if debug:
            print(f"[DEBUG] best={mode} | score={stats_best['score']:.3f} |␣
↪pos={100*stats_best['pos']:.2f}% | n={stats_best['n_labels']}")

        return labels_best, mode, stats_best, {}

# ----------------------------
# Kaggle mask naming
# ----------------------------
def infer_new_mask_stem(img_stem: str, subset: str) -> str:
    if subset == "RG":
        # RG10_1_1 -> RGMask10_1_1
        if img_stem.startswith("RG"):
            return "RGMask" + img_stem[2:]
        return "RGMask" + img_stem

    # subset == "AG": AG_NOISE1 -> AG1
    if img_stem.startswith("AG_NOISE"):
        m = _re_digits.search(img_stem)
        return "AG" + (m.group(1) if m else img_stem.replace("AG_NOISE", ""))
    if img_stem.startswith("AG"):
        m = _re_digits.search(img_stem)
```

```
        return "AG" + (m.group(1) if m else img_stem.replace("AG", ""))
    return img_stem

# ----------------------------
# AUDIT (info only)
# ----------------------------
_ = _audit_dir(HED_PRE_DIR,    "HED_PRE")
_ = _audit_dir(GRAD_PRE_DIR,   "GRAD_PRE")
_ = _audit_dir(THRESH_PRE_DIR, "THRESH_PRE")

# ----------------------------
# 1) Build unified samples list (ONE database)
# ----------------------------
samples = []
stats = {
    "missing_images": 0,
    "missing_masks": 0,
    "missing_hed_maps": 0,
    "missing_grad_maps": 0,
    "masks_inverted": 0,
}

# ---- Collection A (IMG_DIR + HED_RAW_DIR + pseudo-label)
if DATASET_MODE in ("old", "both"):
    if "image_files" not in globals() or image_files is None:
        raise RuntimeError("image_files est requis pour DATASET_MODE='old'/
 ↪'both'.")

    valid_imgdir = []
    for fname in image_files:
        p = IMG_DIR / fname
        if not p.is_file():
            matches = list(IMG_DIR.rglob(fname))
            if matches:
                p = sorted(matches, key=lambda x: x.as_posix())[0]
            else:
                stats["missing_images"] += 1
                continue
        valid_imgdir.append(p)

    valid_imgdir = sorted(valid_imgdir, key=lambda p: p.stem)

    for p in valid_imgdir:
        stem = p.stem
        hed_p = HED_RAW_DIR / f"{stem}_HED.png"
        if not hed_p.is_file():
            hed_p = None
```

```
            stats["missing_hed_maps"] += 1

        samples.append({
            "stem": stem,
            "img_path": p,
            "mask_path": None,      # pseudo-label
            "hed_path": hed_p,      # raw hed if exists
            "grad_path": None,      # computed from img
            "subset": "IMG"         # neutral label (no "old")
        })

# ---- Collection B (Kaggle RG/AG)
if DATASET_MODE in ("new", "both"):

    def collect_kaggle(img_dir: Path, mask_dir: Path, subset: str):
        if not img_dir.is_dir() or not mask_dir.is_dir():
            return []

        img_files = sorted([p for p in img_dir.iterdir() if p.is_file() and p.
 ↪suffix.lower() in IMG_EXTS],
                           key=lambda x: x.stem)
        out = []
        for ip in img_files:
            stem = ip.stem
            mstem = infer_new_mask_stem(stem, subset=subset)
            mp = _glob_any_ext(mask_dir, mstem)
            if mp is None:
                stats["missing_masks"] += 1
                continue

            hed_pre  = _find_prior_with_candidates(HED_PRE_DIR,  stem,␣
 ↪subset=subset, kind="hed")
            grad_pre = _find_prior_with_candidates(GRAD_PRE_DIR, stem,␣
 ↪subset=subset, kind="grad")

            if hed_pre is None:
                stats["missing_hed_maps"] += 1
            if grad_pre is None:
                stats["missing_grad_maps"] += 1

            out.append({
                "stem": stem,
                "img_path": ip,
                "mask_path": mp,
                "hed_path": hed_pre,
                "grad_path": grad_pre,
                "subset": subset  # "RG" or "AG" (neutral, no "new")
```

```
            })
        return out

    if NEW_SUBSET in ("RG", "both"):
        samples += collect_kaggle(RG_DIR, RGMASK_DIR, subset="RG")
    if NEW_SUBSET in ("AG", "both"):
        samples += collect_kaggle(AG_DIR, AGMASK_DIR, subset="AG")

# stable sort
samples = sorted(samples, key=lambda s: (s["subset"], s["stem"]))
N = len(samples)
if N == 0:
    raise RuntimeError("Aucun échantillon trouvé. Vérifie DATASET_MODE/
 ↪NEW_SUBSET et les chemins.")

# ----------------------------
# 2) Summary (unified, readable)
# ----------------------------
def _count_subset(tag):
    return sum(1 for s in samples if s["subset"] == tag)

n_img = _count_subset("IMG")
n_rg  = _count_subset("RG")
n_ag  = _count_subset("AG")

# matched priors (physically existing)
matched_hed  = sum(1 for s in samples if s.get("hed_path")  is not None and␣
 ↪Path(s["hed_path"]).is_file())
matched_grad = sum(1 for s in samples if s.get("grad_path") is not None and␣
 ↪Path(s["grad_path"]).is_file())

print("============================================================")
print("  [BUILD] Unified dataset ready")
print(f"[INFO] Total samples: {N}  (IMG={n_img} | RG={n_rg} | AG={n_ag})")
print(f"[INFO] Mode: DATASET_MODE={DATASET_MODE} | NEW_SUBSET={NEW_SUBSET} |␣
 ↪SPLIT_MODE={SPLIT_MODE}")
print(f"[WARN] Missing: images={stats['missing_images']} |␣
 ↪masks={stats['missing_masks']}")
print(f"[INFO] Priors found on-disk: HED={matched_hed} | GRAD={matched_grad}")
print(f"[INFO] Priors missing (will fallback calc):␣
 ↪HED_missing={stats['missing_hed_maps']} |␣
 ↪GRAD_missing={stats['missing_grad_maps']}")
print("============================================================")

# ----------------------------
# 3) Allocate arrays
```

```
# ----------------------------
X_train = np.zeros((N, IMG_HEIGHT, IMG_WIDTH, 1), dtype=np.float32)
G_train = np.zeros((N, IMG_HEIGHT, IMG_WIDTH, 1), dtype=np.float32)
H_train = np.zeros((N, IMG_HEIGHT, IMG_WIDTH, 1), dtype=np.float32)
Y_train = np.zeros((N, IMG_HEIGHT, IMG_WIDTH, 1), dtype=np.float32)

image_ids = []
subsets = []

# ----------------------------
# 4) Fill tensors
# ----------------------------
pos_rates = np.zeros((N,), dtype=np.float32)

for i, s in enumerate(samples):
    subset = s["subset"]
    stem = s["stem"]

    img = _read_gray(s["img_path"])
    if img is None:
        raise FileNotFoundError(f"Image illisible: {s['img_path']}")
    img_r = _resize_gray(img, inter=cv2.INTER_AREA)

    # --- G prior: file if exists else PDF canny-threshold
    if s["grad_path"] is not None and Path(s["grad_path"]).is_file():
        g0 = _read_gray(s["grad_path"])
        if g0 is None:
            g0 = grad_prior_from_img_pdf(img)
    else:
        g0 = grad_prior_from_img_pdf(img)
    g_r = _resize_gray(g0, inter=cv2.INTER_AREA)

    # --- H prior: file if exists else edge-like fallback
    if s["hed_path"] is not None and Path(s["hed_path"]).is_file():
        h0 = _read_gray(s["hed_path"])
        if h0 is None:
            h0 = edge_like_fallback(img)
    else:
        h0 = edge_like_fallback(img)
    h_r = _resize_gray(h0, inter=cv2.INTER_AREA)

    # --- Y mask
    if subset == "IMG":
        # pseudo-label via auto-labeling from HED-like (h0)
        labels, mode, stats_best, extra = hed_to_labels_auto_best(
            h0.astype(np.uint8), MIN_AREA_PX, MAX_AREA_RATIO, debug=False
        )
```

```
        lab_r = _resize_gray(labels.astype(np.int32), inter=cv2.INTER_NEAREST)
        y = (lab_r > 0).astype(np.float32)
    else:
        m = _read_gray(s["mask_path"])
        if m is None:
            raise FileNotFoundError(f"Mask illisible: {s['mask_path']}")
        m_r = _resize_gray(m, inter=cv2.INTER_NEAREST)
        y = _binarize01(m_r, thr=127)

        # inversion auto (boundaries)
        if NEW_MASK_INVERT == "auto":
            if float(y.mean()) > 0.5:
                y = 1.0 - y
                stats["masks_inverted"] += 1
        elif NEW_MASK_INVERT is True:
            y = 1.0 - y
            stats["masks_inverted"] += 1

    # store
    X_train[i, ..., 0] = _to_float01(img_r)
    G_train[i, ..., 0] = _to_float01(g_r)
    H_train[i, ..., 0] = _to_float01(h_r)
    Y_train[i, ..., 0] = y

    image_ids.append(stem)
    subsets.append(subset)
    pos_rates[i] = 100.0 * float(y.mean())

# ----------------------------
# 5) Positivity stats (unified + breakdown)
# ----------------------------
def _pos_stats(label, idxs):
    if len(idxs) == 0:
        return
    pr = pos_rates[idxs]
    print(f"[POS] {label}: mean={pr.mean():.2f}% | min={pr.min():.2f}% |␣
 ↪max={pr.max():.2f}% | n={len(idxs)}")

idx_all = np.arange(N)
idx_img = np.array([i for i,ss in enumerate(subsets) if ss=="IMG"], dtype=int)
idx_rg  = np.array([i for i,ss in enumerate(subsets) if ss=="RG"], dtype=int)
idx_ag  = np.array([i for i,ss in enumerate(subsets) if ss=="AG"], dtype=int)

print("============================================================")
print("  Arrays ready (3-inputs) - Unified dataset")
print("X_train:", X_train.shape, X_train.dtype, "range [0,1]")
print("G_train:", G_train.shape, G_train.dtype, "range [0,1]")
```

```
print("H_train:", H_train.shape, H_train.dtype, "range [0,1]")
print("Y_train:", Y_train.shape, Y_train.dtype, "values {0,1}")
print(f"[INFO] Masks inverted (mode={NEW_MASK_INVERT}):␣
 ↪{stats['masks_inverted']}")
_pos_stats("ALL", idx_all)
_pos_stats("IMG", idx_img)
_pos_stats("RG",  idx_rg)
_pos_stats("AG",  idx_ag)
print("============================================================")

# ----------------------------
# 6) Split train/val (random or grouped)
# ----------------------------
def group_id(subset: str, stem: str):
    """
    Group key pour éviter leakage tiles (surtout RG/AG).
    """
    if subset == "IMG":
        parts = stem.split("_")
        return "_".join(parts[:2]) if len(parts) >= 2 else stem

    if subset == "RG":
        m = re.match(r"^(RG\d+)", stem)
        return m.group(1) if m else stem.split("_")[0]

    if subset == "AG":
        m = re.search(r"(AG\d+)", stem)
        if m:
            return m.group(1)
        m2 = re.search(r"(\d+)", stem)
        return "AG"+m2.group(1) if m2 else stem.split("_")[0]

    return stem.split("_")[0]

idx_all = np.arange(N)

if SPLIT_MODE == "grouped":
    groups = np.array([group_id(subsets[i], image_ids[i]) for i in idx_all])
    gss = GroupShuffleSplit(n_splits=1, test_size=VAL_FRACTION,␣
 ↪random_state=RANDOM_SEED)
    idx_tr, idx_va = next(gss.split(idx_all, groups=groups))
else:
    if USE_NEW_SPLITS:
        idx_tr, idx_va = train_test_split(idx_all, test_size=VAL_FRACTION,␣
 ↪random_state=RANDOM_SEED, shuffle=True)
    else:
        # split par sous-collection puis concat (compatible)
```

```
        idx_tr_list, idx_va_list = [], []
        for tag, idxs in [("IMG", idx_img), ("RG", idx_rg), ("AG", idx_ag)]:
            if len(idxs) > 0:
                tr, va = train_test_split(idxs, test_size=VAL_FRACTION,␣
 ↪random_state=RANDOM_SEED, shuffle=True)
                idx_tr_list.append(tr); idx_va_list.append(va)
        idx_tr = np.concatenate(idx_tr_list) if idx_tr_list else np.array([],␣
 ↪dtype=int)
        idx_va = np.concatenate(idx_va_list) if idx_va_list else np.array([],␣
 ↪dtype=int)

X_tr, X_va = X_train[idx_tr], X_train[idx_va]
G_tr, G_va = G_train[idx_tr], G_train[idx_va]
H_tr, H_va = H_train[idx_tr], H_train[idx_va]
Y_tr, Y_va = Y_train[idx_tr], Y_train[idx_va]

print("============================================================")
print("  Split ready")
print(f"[INFO] Train: {len(idx_tr)} | Val: {len(idx_va)} | batch={BATCH_SIZE} |␣
 ↪split_mode={SPLIT_MODE}")
print(f"[INFO] Train pos% mean={100*np.mean(Y_tr):.2f}% | Val pos% mean={100*np.
 ↪mean(Y_va):.2f}%")
if SPLIT_MODE == "grouped":
    tr_groups = set([group_id(subsets[i], image_ids[i]) for i in idx_tr])
    va_groups = set([group_id(subsets[i], image_ids[i]) for i in idx_va])
    inter = tr_groups.intersection(va_groups)
    print(f"[INFO] Group leakage check: intersect_groups={len(inter)} (should␣
 ↪be 0).")
print("============================================================")

# ----------------------------
# 7) TFDS: ((X,G,H),Y)
# ----------------------------
train_ds = tf.data.Dataset.from_tensor_slices(((X_tr, G_tr, H_tr), Y_tr))
train_ds = train_ds.batch(BATCH_SIZE).prefetch(tf.data.AUTOTUNE)

val_ds = tf.data.Dataset.from_tensor_slices(((X_va, G_va, H_va), Y_va))
val_ds = val_ds.batch(BATCH_SIZE).prefetch(tf.data.AUTOTUNE)

print("  TF datasets created for 3-input model: train_ds / val_ds")
print(f"[INFO] Train batches: {tf.data.experimental.cardinality(train_ds).
 ↪numpy()} | Val batches: {tf.data.experimental.cardinality(val_ds).numpy()}")

# ----------------------------
# 8) Sanity check (3 random samples): img / grad / hed / mask
# ----------------------------
```

```
rng = np.random.default_rng(RANDOM_SEED)
pick = rng.choice(N, size=min(3, N), replace=False)

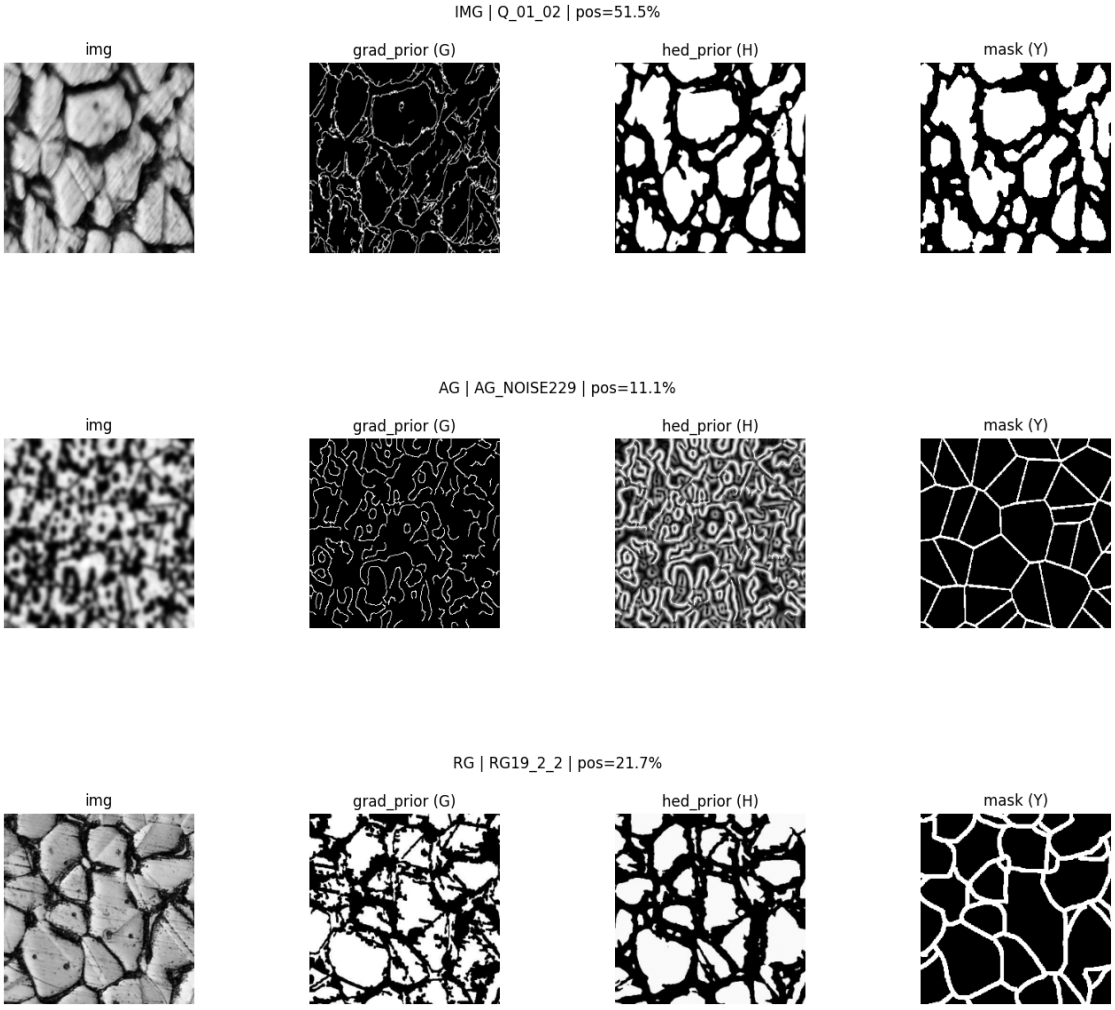
for k in pick:
    pos = float(pos_rates[k])
    tag = subsets[k]
    title = f"{tag} | {image_ids[k]} | pos={pos:.1f}%"
    plt.figure(figsize=(14, 3))
    plt.suptitle(title)

    plt.subplot(1,4,1); plt.imshow(X_train[k, ..., 0], cmap="gray"); plt.
 ↪title("img"); plt.axis("off")
    plt.subplot(1,4,2); plt.imshow(G_train[k, ..., 0], cmap="gray"); plt.
 ↪title("grad_prior (G)"); plt.axis("off")
    plt.subplot(1,4,3); plt.imshow(H_train[k, ..., 0], cmap="gray"); plt.
 ↪title("hed_prior (H)"); plt.axis("off")
    plt.subplot(1,4,4); plt.imshow(Y_train[k, ..., 0], cmap="gray"); plt.
 ↪title("mask (Y)"); plt.axis("off")

    plt.tight_layout()
    plt.show()
```

```
[AUDIT] HED_PRE: /content/drive/MyDrive/Data/GRAIN DATA SET/HED_PRE -> 480 files
        examples: ['HEDPre10_1_1.jpg', 'HEDPre10_1_2.jpg', 'HEDPre10_1_3.jpg',
'HEDPre10_1_4.jpg', 'HEDPre10_2_1.jpg']
[AUDIT] GRAD_PRE: /content/drive/MyDrive/Data/GRAIN DATA SET/GRAD_PRE -> 480
files
        examples: ['GradPre10_1_1.jpg', 'GradPre10_1_2.jpg',
'GradPre10_1_3.jpg', 'GradPre10_1_4.jpg', 'GradPre10_2_1.jpg']
[AUDIT] THRESH_PRE: /content/drive/MyDrive/Data/GRAIN DATA SET/THRESH_PRE -> 480
files
        examples: ['ThreshPre10_1_1.jpg', 'ThreshPre10_1_2.jpg',
'ThreshPre10_1_3.jpg', 'ThreshPre10_1_4.jpg', 'ThreshPre10_2_1.jpg']
============================================================
  [BUILD] Unified dataset ready
[INFO] Total samples: 1616  (IMG=336 | RG=480 | AG=800)
[INFO] Mode: DATASET_MODE=both | NEW_SUBSET=both | SPLIT_MODE=grouped
[WARN] Missing: images=0 | masks=0
[INFO] Priors found on-disk: HED=856 | GRAD=520
[INFO] Priors missing (will fallback calc): HED_missing=760 | GRAD_missing=760
============================================================
============================================================
  Arrays ready (3-inputs) - Unified dataset
X_train: (1616, 256, 256, 1) float32 range [0,1]
G_train: (1616, 256, 256, 1) float32 range [0,1]
H_train: (1616, 256, 256, 1) float32 range [0,1]
Y_train: (1616, 256, 256, 1) float32 values {0,1}
```

```
[INFO] Masks inverted (mode=auto): 1280
[POS] ALL: mean=20.60% | min=7.97% | max=63.93% | n=1616
[POS] IMG: mean=43.86% | min=12.94% | max=63.93% | n=336
[POS] RG: mean=20.85% | min=14.88% | max=28.55% | n=480
[POS] AG: mean=10.68% | min=7.97% | max=13.36% | n=800
============================================================
============================================================
 Split ready
[INFO] Train: 1299 | Val: 317 | batch=8 | split_mode=grouped
[INFO] Train pos% mean=20.22% | Val pos% mean=22.14%
[INFO] Group leakage check: intersect_groups=0 (should be 0).
============================================================
 TF datasets created for 3-input model: train_ds / val_ds
[INFO] Train batches: 163 | Val batches: 40
```



Cette étape **construit un dataset unifié "prêt U-Net" à 3 entrées** en fusionnant trois sources : (i) un lot **IMG** (images internes) où le masque est généré par **pseudo-labellisation** à partir d'un

HED raw quand il existe, (ii) la sous-collection **RG** (images + RGMask), et (iii) la sous-collection **AG** (images + AGMask). Le pipeline réalise d'abord un **audit des dossiers de priors** (`HED_PRE`, `GRAD_PRE`, `THRESH_PRE`) pour confirmer la disponibilité sur disque (ici **480 fichiers** dans chacun), puis construit une liste unique `samples` triée (ici **N=1616** échantillons : **IMG=336 | RG=480 | AG=800**). Pour chaque échantillon, il prépare systématiquement quatre tenseurs au format Keras attendu : **X** (image grayscale normalisée `[0,1]`), **G** (prior `grad_prior`), **H** (prior `hed_prior`) et **Y** (mask). Les priors **G** et **H** sont récupérés sur disque quand disponibles, sinon **reconstruits de manière déterministe** à partir de l'image : `grad_prior` via Canny + seuillage + morphologie, et `hed_prior` via un fallback "edge-like" (Canny + Sobel magnitude), ce qui garantit que le modèle reçoit toujours des entrées cohérentes (pas de zéros "silencieux"). Le masque **Y** est soit lu depuis les masques Kaggle (AG/RG) et binarisé (avec une **inversion automatique** optionnelle pour corriger les masques de type "boundaries" — ici **1280 masques inversés** en mode `auto`), soit généré pour **IMG** par une routine `hed_to_labels_auto_best` qui choisit automatiquement la meilleure stratégie (régions vs edges) selon des critères de qualité (taux de pixels positifs, nombre d'objets, etc.). À la fin du remplissage, les tableaux **`X_train, G_train, H_train, Y_train`** sont alloués en **float32** et ont tous la forme **(1616, 256, 256, 1)**, avec un résumé clair des distributions de positivité (ici **ALL 20.60%**, **IMG 43.86%**, **RG 20.85%**, **AG 10.68%**), ce qui permet de vérifier l'équilibre global et les différences structurelles entre sous-collections. Le split **train/val** est ensuite réalisé en mode **anti-leakage** (`SPLIT_MODE="grouped"`) via `GroupShuffleSplit`, en regroupant les images par identifiant logique (ex. tuiles d'un même RGxx), et le contrôle de fuite confirme **0 groupe en intersection** ; on obtient ici **1299** images train et **317** val. Enfin, les jeux `train_ds` et `val_ds` sont créés au format TensorFlow **`((X,G,H), Y)`**, batchés (8) et préfetchés, avec un décompte stable (**163 batches train**, **40 val**), puis un **sanity-check visuel** affiche quelques exemples aléatoires sous forme de quadruplets **img / grad_prior / hed_prior / mask** avec le `pos_rate`, afin de valider qualitativement l'alignement des entrées (priors) et la cohérence des masques avant l'entraînement.

[ ]:
```
import warnings, re
warnings.filterwarnings("ignore", message="Transparent hugepages are not␣
 ↪enabled", category=UserWarning)

import numpy as np
import cv2
import tensorflow as tf
from tensorflow.keras import layers, models

# ============================================================
# 1) Constantes + Seeds
# ============================================================
IMG_HEIGHT, IMG_WIDTH = 256, 256
IMG_CH_IMG  = 1
IMG_CH_HED  = 1
SEED = 42

tf.keras.utils.set_random_seed(SEED)
np.random.seed(SEED)
```

```
# ============================================================
# 2) Utils (nom TF safe)
# ============================================================
def tf_safe_name(name: str) -> str:
    """Nom compatible TF root scope. Évite + espaces etc."""
    name = (name or "Model").strip()
    name = name.replace("+", "_plus_")
    name = re.sub(r"[^A-Za-z0-9._\/>\-]", "_", name)
    if not re.match(r"^[A-Za-z0-9.]", name):
        name = "M_" + name
    return name

# ============================================================
# 3) Post-process morphologique (utile pour checks/metrics externes)
# ============================================================
def postprocess_binary(mask_bool, k_open=3, k_close=5, iterations=1):
    mb = (mask_bool.astype(np.uint8) * 255)
    kO = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (k_open, k_open))
    kC = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (k_close, k_close))
    mb = cv2.morphologyEx(mb, cv2.MORPH_OPEN, kO, iterations=iterations)
    mb = cv2.morphologyEx(mb, cv2.MORPH_CLOSE, kC, iterations=iterations)
    return (mb > 0)

# ============================================================
# 4) Transitions robustes + overlay + scan threshold
#    (on les garde pour l'étape des checks/validation ensuite)
# ============================================================
def compute_transitions_from_mask(mask, thr=0.5, do_post=True, k_open=3,␣
 ↪k_close=5, iterations=1):
    m = np.asarray(mask)

    if m.ndim == 4:
        m = m[0]
    if m.ndim == 3:
        m = m[..., 0]
    if m.ndim != 2:
        raise ValueError(f"mask doit être 2D après squeeze, trouvé shape={m.
 ↪shape}")

    m_bool = (m > thr)

    if do_post:
        m_bool = postprocess_binary(m_bool, k_open=k_open, k_close=k_close,␣
 ↪iterations=iterations)

    fg_ratio = float(m_bool.mean())
```

```
    dx = (m_bool[:, 1:] != m_bool[:, :-1])   # (H, W-1)
    dy = (m_bool[1:, :] != m_bool[:-1, :])   # (H-1, W)

    Tx = dx.sum(axis=1).astype(np.float32)   # (H,)
    Ty = dy.sum(axis=0).astype(np.float32)   # (W,)

    return dx, dy, Tx, Ty, m_bool, fg_ratio

def overlay_transitions_bool(img_gray, dx_bool, dy_bool, alpha=0.75):
    img = np.asarray(img_gray)
    if img.ndim == 3:
        img = img[..., 0]
    if img.ndim != 2:
        raise ValueError(f"img_gray doit être 2D, trouvé shape={img.shape}")

    if img.dtype != np.uint8:
        img_u8 = np.clip(img, 0, 255).astype(np.uint8)
    else:
        img_u8 = img.copy()

    rgb = np.stack([img_u8, img_u8, img_u8], axis=-1)

    H, W = img_u8.shape[:2]
    mx_full = np.zeros((H, W), dtype=bool)
    my_full = np.zeros((H, W), dtype=bool)

    mx_full[:, :W-1] = np.asarray(dx_bool, dtype=bool)
    my_full[:H-1, :] = np.asarray(dy_bool, dtype=bool)

    overlay = rgb.astype(np.float32)
    overlay[mx_full, 0] = 255  # R
    overlay[my_full, 1] = 255  # G

    out = (alpha * overlay + (1.0 - alpha) * rgb).astype(np.uint8)
    return out, mx_full, my_full

def robust_grain_estimate(Tx, Ty):
    Tx = np.asarray(Tx, dtype=np.float32)
    Ty = np.asarray(Ty, dtype=np.float32)

    mean_x, mean_y = float(Tx.mean()), float(Ty.mean())
    med_x,  med_y  = float(np.percentile(Tx, 50)), float(np.percentile(Ty, 50))

    est_mean = 0.5 * ((mean_x/2.0) + (mean_y/2.0))
    est_med  = 0.5 * ((med_x/2.0)  + (med_y/2.0))

    return {
```

```
        "mean_Tx": mean_x, "mean_Ty": mean_y,
        "med_Tx":  med_x,  "med_Ty":  med_y,
        "est_grains_mean": est_mean,
        "est_grains_med":  est_med,
        "ratio_Tx_Ty":     mean_x / (mean_y + 1e-9),
    }

def scan_thresholds(mask_prob, img_gray=None, thrs=(0.4, 0.5, 0.6, 0.65, 0.7, 0.
↪75),
                    k_open=3, k_close=5, iterations=1,
                    min_fg=0.01, max_fg=0.99,
                    verbose=True):
    rows = []
    for t in thrs:
        dx, dy, Tx, Ty, _, fg = compute_transitions_from_mask(mask_prob, thr=t,␣
↪do_post=False)
        st = robust_grain_estimate(Tx, Ty)
        rows.append({
            "thr": float(t),
            "fg_ratio": float(fg),
            "ratio_Tx_Ty": float(st["ratio_Tx_Ty"]),
            "grains_mean": float(st["est_grains_mean"]),
            "grains_med":  float(st["est_grains_med"]),
            "pct_dx": float(100.0 * dx.mean()),
            "pct_dy": float(100.0 * dy.mean()),
        })

    def score(r):
        if (r["fg_ratio"] <= min_fg) or (r["fg_ratio"] >= max_fg):
            return 1e9
        ratio_pen = abs(r["ratio_Tx_Ty"] - 1.0)
        trans_pen = 0.5*(r["pct_dx"] + r["pct_dy"]) / 100.0
        thr_pen   = 0.05 * (1.0 - r["thr"])
        return ratio_pen + trans_pen + thr_pen

    best = min(rows, key=score)

    if verbose:
        print("\n========== [THRESH SCAN] ==========")
        print("thr | fg% | Tx/Ty | grains_med | grains_mean | %dx | %dy")
        for r in rows:
            print(f"{r['thr']:.2f} | {100*r['fg_ratio']:.2f}% |␣
↪{r['ratio_Tx_Ty']:.2f} | "
                  f"{r['grains_med']:.2f} | {r['grains_mean']:.2f} |␣
↪{r['pct_dx']:.2f}% | {r['pct_dy']:.2f}%")
        print("[BEST] thr =", best["thr"], "| fg% =",␣
↪round(100*best["fg_ratio"], 2),
```

```
            "| Tx/Ty =", round(best["ratio_Tx_Ty"], 2),
            "| grains_med =", round(best["grains_med"], 2),
            "| %dx =", round(best["pct_dx"], 2), "| %dy =",␣
 ↪round(best["pct_dy"], 2))

    if img_gray is not None:
        dx, dy, _, _, _, _ = compute_transitions_from_mask(
            mask_prob, thr=best["thr"],
            do_post=True, k_open=k_open, k_close=k_close, iterations=iterations
        )
        vis, mx_full, my_full = overlay_transitions_bool(img_gray, dx, dy,␣
 ↪alpha=0.75)
        return {"rows": rows, "best": best, "best_vis": vis, "best_mx":␣
 ↪mx_full, "best_my": my_full}

    return {"rows": rows, "best": best}

# ============================================================
# 5) Losses stables (sans @tf.function)
# ============================================================
def dice_coef(y_true, y_pred, smooth=1.0):
    y_true = tf.cast(y_true, tf.float32)
    y_pred = tf.cast(tf.clip_by_value(y_pred, 0.0, 1.0), tf.float32)

    y_true_f = tf.reshape(y_true, [-1])
    y_pred_f = tf.reshape(y_pred, [-1])

    inter = tf.reduce_sum(y_true_f * y_pred_f)
    denom = tf.reduce_sum(y_true_f) + tf.reduce_sum(y_pred_f)
    return (2.0 * inter + smooth) / (denom + smooth)

def dice_loss(y_true, y_pred):
    return 1.0 - dice_coef(y_true, y_pred)

def bce_dice_loss(y_true, y_pred, bce_weight=0.5):
    y_true = tf.cast(y_true, tf.float32)
    y_pred = tf.cast(tf.clip_by_value(y_pred, 0.0, 1.0), tf.float32)

    bce = tf.keras.losses.binary_crossentropy(y_true, y_pred)
    bce = tf.reduce_mean(bce)
    d   = dice_loss(y_true, y_pred)
    return bce_weight * bce + (1.0 - bce_weight) * d

# ============================================================
# 6) Metric IoU (sérialisable)
# ============================================================
@tf.keras.utils.register_keras_serializable()
```

```
class MeanIoUThreshold(tf.keras.metrics.Metric):
    def __init__(self, name="iou_score", threshold=0.5, smooth=1.0, **kwargs):
        super().__init__(name=name, **kwargs)
        self.threshold = float(threshold)
        self.smooth = float(smooth)
        self.total = self.add_weight(name="total", initializer="zeros")
        self.count = self.add_weight(name="count", initializer="zeros")

    def update_state(self, y_true, y_pred, sample_weight=None):
        y_true = tf.cast(y_true > 0.5, tf.float32)
        y_pred = tf.cast(y_pred > self.threshold, tf.float32)

        y_true_f = tf.reshape(y_true, [-1])
        y_pred_f = tf.reshape(y_pred, [-1])

        inter = tf.reduce_sum(y_true_f * y_pred_f)
        union = tf.reduce_sum(y_true_f) + tf.reduce_sum(y_pred_f) - inter
        iou   = (inter + self.smooth) / (union + self.smooth)

        self.total.assign_add(iou)
        self.count.assign_add(1.0)

    def result(self):
        return tf.math.divide_no_nan(self.total, self.count)

    def reset_state(self):
        self.total.assign(0.0)
        self.count.assign(0.0)

# ============================================================
# 7) Blocks réseau (BN + ReLU) + U-Net
# ============================================================
def conv_block(x, n_filters, dropout=0.0):
    x = layers.Conv2D(n_filters, 3, padding="same", use_bias=False)(x)
    x = layers.BatchNormalization()(x)
    x = layers.Activation("relu")(x)

    x = layers.Conv2D(n_filters, 3, padding="same", use_bias=False)(x)
    x = layers.BatchNormalization()(x)
    x = layers.Activation("relu")(x)

    if dropout and dropout > 0:
        x = layers.Dropout(dropout)(x)
    return x

def build_unet_single_input(input_shape=(IMG_HEIGHT, IMG_WIDTH, 1),␣
 ↪base_filters=32, name="U_Net_GrainSeg"):
```

```
    inp = layers.Input(shape=input_shape, name="img")

    c1 = conv_block(inp, base_filters, dropout=0.0); p1 = layers.
↪MaxPooling2D(2)(c1)
    c2 = conv_block(p1, base_filters*2, dropout=0.0); p2 = layers.
↪MaxPooling2D(2)(c2)
    c3 = conv_block(p2, base_filters*4, dropout=0.1); p3 = layers.
↪MaxPooling2D(2)(c3)
    c4 = conv_block(p3, base_filters*8, dropout=0.1); p4 = layers.
↪MaxPooling2D(2)(c4)
    c5 = conv_block(p4, base_filters*16, dropout=0.2)

    u6 = layers.Conv2DTranspose(base_filters*8, 2, strides=2,␣
↪padding="same")(c5)
    u6 = layers.Concatenate()([u6, c4]); c6 = conv_block(u6, base_filters*8,␣
↪dropout=0.1)

    u7 = layers.Conv2DTranspose(base_filters*4, 2, strides=2,␣
↪padding="same")(c6)
    u7 = layers.Concatenate()([u7, c3]); c7 = conv_block(u7, base_filters*4,␣
↪dropout=0.1)

    u8 = layers.Conv2DTranspose(base_filters*2, 2, strides=2,␣
↪padding="same")(c7)
    u8 = layers.Concatenate()([u8, c2]); c8 = conv_block(u8, base_filters*2,␣
↪dropout=0.0)

    u9 = layers.Conv2DTranspose(base_filters, 2, strides=2, padding="same")(c8)
    u9 = layers.Concatenate()([u9, c1]); c9 = conv_block(u9, base_filters,␣
↪dropout=0.0)

    out = layers.Conv2D(1, 1, activation="sigmoid", name="mask")(c9)
    return models.Model(inp, out, name=tf_safe_name(name))

def build_unet_two_inputs(img_shape=(IMG_HEIGHT, IMG_WIDTH, 1),␣
↪prior_shape=(IMG_HEIGHT, IMG_WIDTH, 1),
                          base_filters=32, name="U_Net_GrainSeg_HEDPrior"):
    inp_img   = layers.Input(shape=img_shape,   name="img")
    inp_prior = layers.Input(shape=prior_shape, name="hed_prior")

    x = layers.Concatenate(name="concat_img_prior")([inp_img, inp_prior])  #␣
↪(H,W,2)

    c1 = conv_block(x, base_filters, dropout=0.0); p1 = layers.
↪MaxPooling2D(2)(c1)
```

```
    c2 = conv_block(p1, base_filters*2, dropout=0.0); p2 = layers.
↪MaxPooling2D(2)(c2)
    c3 = conv_block(p2, base_filters*4, dropout=0.1); p3 = layers.
↪MaxPooling2D(2)(c3)
    c4 = conv_block(p3, base_filters*8, dropout=0.1); p4 = layers.
↪MaxPooling2D(2)(c4)
    c5 = conv_block(p4, base_filters*16, dropout=0.2)

    u6 = layers.Conv2DTranspose(base_filters*8, 2, strides=2,␣
↪padding="same")(c5)
    u6 = layers.Concatenate()([u6, c4]); c6 = conv_block(u6, base_filters*8,␣
↪dropout=0.1)

    u7 = layers.Conv2DTranspose(base_filters*4, 2, strides=2,␣
↪padding="same")(c6)
    u7 = layers.Concatenate()([u7, c3]); c7 = conv_block(u7, base_filters*4,␣
↪dropout=0.1)

    u8 = layers.Conv2DTranspose(base_filters*2, 2, strides=2,␣
↪padding="same")(c7)
    u8 = layers.Concatenate()([u8, c2]); c8 = conv_block(u8, base_filters*2,␣
↪dropout=0.0)

    u9 = layers.Conv2DTranspose(base_filters, 2, strides=2, padding="same")(c8)
    u9 = layers.Concatenate()([u9, c1]); c9 = conv_block(u9, base_filters,␣
↪dropout=0.0)

    out = layers.Conv2D(1, 1, activation="sigmoid", name="mask")(c9)
    return models.Model([inp_img, inp_prior], out, name=tf_safe_name(name))

#  NEW: 3-input U-Net (img + grad_prior + hed_prior)
def build_unet_three_inputs(img_shape=(IMG_HEIGHT, IMG_WIDTH, 1),
                            grad_shape=(IMG_HEIGHT, IMG_WIDTH, 1),
                            hed_shape=(IMG_HEIGHT, IMG_WIDTH, 1),
                            base_filters=32,
                            name="U_Net_GrainSeg_GradHED"):
    inp_img  = layers.Input(shape=img_shape,  name="img")
    inp_grad = layers.Input(shape=grad_shape, name="grad_prior")
    inp_hed  = layers.Input(shape=hed_shape,  name="hed_prior")

    x = layers.Concatenate(name="concat_img_grad_hed")([inp_img, inp_grad,␣
↪inp_hed])  # (H,W,3)

    c1 = conv_block(x, base_filters, dropout=0.0); p1 = layers.
↪MaxPooling2D(2)(c1)
```

```
    c2 = conv_block(p1, base_filters*2, dropout=0.0); p2 = layers.
 ↪MaxPooling2D(2)(c2)
    c3 = conv_block(p2, base_filters*4, dropout=0.1); p3 = layers.
 ↪MaxPooling2D(2)(c3)
    c4 = conv_block(p3, base_filters*8, dropout=0.1); p4 = layers.
 ↪MaxPooling2D(2)(c4)
    c5 = conv_block(p4, base_filters*16, dropout=0.2)

    u6 = layers.Conv2DTranspose(base_filters*8, 2, strides=2,␣
 ↪padding="same")(c5)
    u6 = layers.Concatenate()([u6, c4]); c6 = conv_block(u6, base_filters*8,␣
 ↪dropout=0.1)

    u7 = layers.Conv2DTranspose(base_filters*4, 2, strides=2,␣
 ↪padding="same")(c6)
    u7 = layers.Concatenate()([u7, c3]); c7 = conv_block(u7, base_filters*4,␣
 ↪dropout=0.1)

    u8 = layers.Conv2DTranspose(base_filters*2, 2, strides=2,␣
 ↪padding="same")(c7)
    u8 = layers.Concatenate()([u8, c2]); c8 = conv_block(u8, base_filters*2,␣
 ↪dropout=0.0)

    u9 = layers.Conv2DTranspose(base_filters, 2, strides=2, padding="same")(c8)
    u9 = layers.Concatenate()([u9, c1]); c9 = conv_block(u9, base_filters,␣
 ↪dropout=0.0)

    out = layers.Conv2D(1, 1, activation="sigmoid", name="mask")(c9)
    return models.Model([inp_img, inp_grad, inp_hed], out,␣
 ↪name=tf_safe_name(name))

# ============================================================
# 8) Build + Compile (UNE SEULE FOIS) + Summary + Sanity check
# ============================================================
MODEL_INPUTS = 3  # 1 | 2 | 3

if MODEL_INPUTS == 3:
    unet_model = build_unet_three_inputs(
        img_shape=(IMG_HEIGHT, IMG_WIDTH, IMG_CH_IMG),
        grad_shape=(IMG_HEIGHT, IMG_WIDTH, 1),
        hed_shape=(IMG_HEIGHT, IMG_WIDTH, IMG_CH_HED),
        base_filters=32,
        name="U_Net_GrainSeg_GradHED"
    )
elif MODEL_INPUTS == 2:
    unet_model = build_unet_two_inputs(
```

```
        img_shape=(IMG_HEIGHT, IMG_WIDTH, IMG_CH_IMG),
        prior_shape=(IMG_HEIGHT, IMG_WIDTH, IMG_CH_HED),
        base_filters=32,
        name="U_Net_GrainSeg_HEDPrior"
    )
else:
    unet_model = build_unet_single_input(
        input_shape=(IMG_HEIGHT, IMG_WIDTH, IMG_CH_IMG),
        base_filters=32,
        name="U_Net_GrainSeg"
    )

unet_model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3, clipnorm=1.0),
    loss=bce_dice_loss,
    metrics=[
        dice_coef,
        MeanIoUThreshold(name="iou_score", threshold=0.5),
        tf.keras.metrics.BinaryAccuracy(name="bin_acc", threshold=0.5),
        tf.keras.metrics.Precision(name="precision", thresholds=0.5),
        tf.keras.metrics.Recall(name="recall", thresholds=0.5),
    ],
)

print("[INFO] Model ready:", unet_model.name)
unet_model.summary()

# ============================================================
# 9) Sanity check obligatoire (1 batch -> predict -> shapes)
# ============================================================
if "train_ds" in globals() and train_ds is not None:
    try:
        (inputs, yb) = next(iter(train_ds.take(1)))
        if MODEL_INPUTS == 3:
            xb, gb, hb = inputs
            print("[SANITY] batch shapes:",
                  "img", tuple(xb.shape),
                  "| grad", tuple(gb.shape),
                  "| hed", tuple(hb.shape),
                  "| y", tuple(yb.shape))
            pred = unet_model.predict([xb, gb, hb], verbose=0)  #  list, pas␣
 ↪tuple
            print("[SANITY] pred shape:", tuple(pred.shape))
        elif MODEL_INPUTS == 2:
            xb, hb = inputs
            print("[SANITY] batch shapes:",
                  "img", tuple(xb.shape),
```

```
                  "| hed", tuple(hb.shape),
                  "| y", tuple(yb.shape))
            pred = unet_model.predict([xb, hb], verbose=0)
            print("[SANITY] pred shape:", tuple(pred.shape))
        else:
            xb = inputs
            print("[SANITY] batch shapes:",
                  "img", tuple(xb.shape),
                  "| y", tuple(yb.shape))
            pred = unet_model.predict(xb, verbose=0)
            print("[SANITY] pred shape:", tuple(pred.shape))
    except Exception as e:
        print("[SANITY][WARN] Predict sanity-check skipped due to:", repr(e))
```

```
[INFO] Model ready: U_Net_GrainSeg_GradHED
```

Model: "U_Net_GrainSeg_GradHED"

Layer (type)	Output Shape	Param #	Connected to
img (InputLayer)	(None, 256, 256, 1)	0	-
grad_prior (InputLayer)	(None, 256, 256, 1)	0	-
hed_prior (InputLayer)	(None, 256, 256, 1)	0	-
concat_img_grad_hed (Concatenate)	(None, 256, 256, 3)	0	img[0][0], grad_prior[0][0], hed_prior[0][0]
conv2d (Conv2D)	(None, 256, 256, 32)	864	concat_img_grad_…
batch_normalization (BatchNormalizatio…	(None, 256, 256, 32)	128	conv2d[0][0]
activation (Activation)	(None, 256, 256, 32)	0	batch_normalizat…
conv2d_1 (Conv2D)	(None, 256, 256, 32)	9,216	activation[0][0]
batch_normalizatio… (BatchNormalizatio…	(None, 256, 256, 32)	128	conv2d_1[0][0]

```
activation_1          (None, 256, 256,            0   batch_normalizat…
(Activation)          32)

max_pooling2d         (None, 128, 128,            0   activation_1[0][…
(MaxPooling2D)        32)

conv2d_2 (Conv2D)     (None, 128, 128,       18,432   max_pooling2d[0]…
                      64)

batch_normalizatio…   (None, 128, 128,          256   conv2d_2[0][0]
(BatchNormalizatio…   64)

activation_2          (None, 128, 128,            0   batch_normalizat…
(Activation)          64)

conv2d_3 (Conv2D)     (None, 128, 128,       36,864   activation_2[0][…
                      64)

batch_normalizatio…   (None, 128, 128,          256   conv2d_3[0][0]
(BatchNormalizatio…   64)

activation_3          (None, 128, 128,            0   batch_normalizat…
(Activation)          64)

max_pooling2d_1       (None, 64, 64,              0   activation_3[0][…
(MaxPooling2D)        64)

conv2d_4 (Conv2D)     (None, 64, 64,         73,728   max_pooling2d_1[…
                      128)

batch_normalizatio…   (None, 64, 64,            512   conv2d_4[0][0]
(BatchNormalizatio…   128)

activation_4          (None, 64, 64,              0   batch_normalizat…
(Activation)          128)

conv2d_5 (Conv2D)     (None, 64, 64,        147,456   activation_4[0][…
                      128)

batch_normalizatio…   (None, 64, 64,            512   conv2d_5[0][0]
(BatchNormalizatio…   128)

activation_5          (None, 64, 64,              0   batch_normalizat…
(Activation)          128)

dropout (Dropout)     (None, 64, 64,              0   activation_5[0][…
                      128)
```

```
max_pooling2d_2 (MaxPooling2D)       (None, 32, 32, 128)               0   dropout[0][0]

conv2d_6 (Conv2D)                    (None, 32, 32, 256)         294,912   max_pooling2d_2[…

batch_normalizatio… (BatchNormalizatio… (None, 32, 32, 256)        1,024   conv2d_6[0][0]

activation_6 (Activation)            (None, 32, 32, 256)               0   batch_normalizat…

conv2d_7 (Conv2D)                    (None, 32, 32, 256)         589,824   activation_6[0][…

batch_normalizatio… (BatchNormalizatio… (None, 32, 32, 256)        1,024   conv2d_7[0][0]

activation_7 (Activation)            (None, 32, 32, 256)               0   batch_normalizat…

dropout_1 (Dropout)                  (None, 32, 32, 256)               0   activation_7[0][…

max_pooling2d_3 (MaxPooling2D)       (None, 16, 16, 256)               0   dropout_1[0][0]

conv2d_8 (Conv2D)                    (None, 16, 16, 512)       1,179,648   max_pooling2d_3[…

batch_normalizatio… (BatchNormalizatio… (None, 16, 16, 512)        2,048   conv2d_8[0][0]

activation_8 (Activation)            (None, 16, 16, 512)               0   batch_normalizat…

conv2d_9 (Conv2D)                    (None, 16, 16, 512)       2,359,296   activation_8[0][…

batch_normalizatio… (BatchNormalizatio… (None, 16, 16, 512)        2,048   conv2d_9[0][0]

activation_9 (Activation)            (None, 16, 16, 512)               0   batch_normalizat…

dropout_2 (Dropout)                  (None, 16, 16, 512)               0   activation_9[0][…
```

```
conv2d_transpose      (None, 32, 32,        524,544   dropout_2[0][0]
(Conv2DTranspose)     256)

concatenate           (None, 32, 32,              0   conv2d_transpose…
(Concatenate)         512)                            dropout_1[0][0]

conv2d_10 (Conv2D)    (None, 32, 32,      1,179,648   concatenate[0][0]
                      256)

batch_normalizatio…   (None, 32, 32,          1,024   conv2d_10[0][0]
(BatchNormalizatio…   256)

activation_10         (None, 32, 32,              0   batch_normalizat…
(Activation)          256)

conv2d_11 (Conv2D)    (None, 32, 32,        589,824   activation_10[0]…
                      256)

batch_normalizatio…   (None, 32, 32,          1,024   conv2d_11[0][0]
(BatchNormalizatio…   256)

activation_11         (None, 32, 32,              0   batch_normalizat…
(Activation)          256)

dropout_3 (Dropout)   (None, 32, 32,              0   activation_11[0]…
                      256)

conv2d_transpose_1    (None, 64, 64,        131,200   dropout_3[0][0]
(Conv2DTranspose)     128)

concatenate_1         (None, 64, 64,              0   conv2d_transpose…
(Concatenate)         256)                            dropout[0][0]

conv2d_12 (Conv2D)    (None, 64, 64,        294,912   concatenate_1[0]…
                      128)

batch_normalizatio…   (None, 64, 64,            512   conv2d_12[0][0]
(BatchNormalizatio…   128)

activation_12         (None, 64, 64,              0   batch_normalizat…
(Activation)          128)

conv2d_13 (Conv2D)    (None, 64, 64,        147,456   activation_12[0]…
                      128)

batch_normalizatio…   (None, 64, 64,            512   conv2d_13[0][0]
(BatchNormalizatio…   128)
```

```
activation_13 (Activation)      (None, 64, 64, 128)          0      batch_normalizat…

dropout_4 (Dropout)             (None, 64, 64, 128)          0      activation_13[0]…

conv2d_transpose_2              (None, 128, 128,         32,832     dropout_4[0][0]
(Conv2DTranspose)               64)

concatenate_2 (Concatenate)     (None, 128, 128,             0      conv2d_transpose…
                                128)                                activation_3[0][…

conv2d_14 (Conv2D)              (None, 128, 128,         73,728     concatenate_2[0]…
                                64)

batch_normalizatio…             (None, 128, 128,            256     conv2d_14[0][0]
(BatchNormalizatio…             64)

activation_14 (Activation)      (None, 128, 128,             0      batch_normalizat…
                                64)

conv2d_15 (Conv2D)              (None, 128, 128,         36,864     activation_14[0]…
                                64)

batch_normalizatio…             (None, 128, 128,            256     conv2d_15[0][0]
(BatchNormalizatio…             64)

activation_15 (Activation)      (None, 128, 128,             0      batch_normalizat…
                                64)

conv2d_transpose_3              (None, 256, 256,          8,224     activation_15[0]…
(Conv2DTranspose)               32)

concatenate_3 (Concatenate)     (None, 256, 256,             0      conv2d_transpose…
                                64)                                 activation_1[0][…

conv2d_16 (Conv2D)              (None, 256, 256,         18,432     concatenate_3[0]…
                                32)

batch_normalizatio…             (None, 256, 256,            128     conv2d_16[0][0]
(BatchNormalizatio…             32)

activation_16 (Activation)      (None, 256, 256,             0      batch_normalizat…
                                32)

conv2d_17 (Conv2D)              (None, 256, 256,          9,216     activation_16[0]…
                                32)
```

```
batch_normalizatio…  (None, 256, 256,          128   conv2d_17[0][0]
(BatchNormalizatio…  32)

activation_17        (None, 256, 256,            0   batch_normalizat…
(Activation)         32)

mask (Conv2D)        (None, 256, 256,           33   activation_17[0]…
                     1)


Total params: 7,768,929 (29.64 MB)

Trainable params: 7,763,041 (29.61 MB)

Non-trainable params: 5,888 (23.00 KB)

[SANITY] batch shapes: img (8, 256, 256, 1) | grad (8, 256, 256, 1) | hed (8,
256, 256, 1) | y (8, 256, 256, 1)
[SANITY] pred shape: (8, 256, 256, 1)
```

Ce script définit et instancie le **modèle de segmentation U-Net** dans sa version **à 3 entrées**, alignée avec le dataset unifié construit à l'étape précédente. Au lieu de consommer uniquement l'image, le réseau reçoit désormais **trois canaux d'information complémentaires** : **`img`** (micrographie grayscale normalisée), **`grad_prior`** (carte de gradients/contours issue du prior "Canny + morphologie"), et **`hed_prior`** (carte edge-like ou HED pré-calculée). Ces trois tenseurs `(256×256×1)` sont concaténés dès l'entrée via `concat_img_grad_hed`, produisant un volume `(256×256×3)` qui fournit au réseau à la fois la texture brute et deux "guides" explicites sur les transitions de grains. L'architecture suit un **encoder–decoder classique avec skip connections**, mais avec une implémentation "stable" : chaque bloc `conv_block` applique **deux convolutions 3×3** avec **Batch Normalization + ReLU**, ce qui améliore la stabilité d'apprentissage et la robustesse aux variations d'intensité. L'encodeur effectue **4 niveaux de downsampling** (MaxPooling2D) avec une progression de filtres **32 → 64 → 128 → 256**, puis un **bottleneck à 512 filtres** (avec dropout plus fort) pour capturer une représentation riche à faible résolution. Le décodeur reconstruit ensuite la prédiction à pleine résolution au moyen de `Conv2DTranspose` (upsampling ×2), et **concatène à chaque étage** les features correspondantes de l'encodeur (skip connections) afin de préserver la précision des bords et des interfaces entre grains. La sortie est une carte `mask` en **sigmoïde** `(256×256×1)` représentant une probabilité pixel-wise. La compilation est conçue pour la segmentation binaire avec un compromis stabilité/qualité : l'optimiseur est **Adam** avec **clipnorm=1.0** pour limiter les gradients instables, et la perte est une combinaison **BCE + Dice** (`bce_dice_loss`) qui corrige l'effet du déséquilibre de classes (les joints de grains occupent souvent une faible proportion de pixels) tout en encourageant directement le recouvrement global. Les métriques suivies couvrent à la fois l'accord global et la qualité de segmentation : **Dice**, un **IoU à seuil** sérialisable (`MeanIoUThreshold`), ainsi que **BinaryAccuracy**, **Precision** et **Recall** à `threshold=0.5`. Le résumé confirme un modèle d'environ **7.77 M paramètres** (29.6 MB), cohérent avec un U-Net 32-base filtres enrichi par BN. Enfin, le **sanity check** sur un batch extrait

de `train_ds` valide l'intégration complète "dataset modèle" : les entrées sont bien des triplets `(img, grad, hed)` de forme `(8,256,256,1)` et la sortie `pred` a exactement la forme attendue `(8,256,256,1)`. Cette vérification garantit que la signature du modèle (**3 inputs**) correspond au format TFDS `((X,G,H), Y)` et que l'inférence est opérationnelle avant de lancer l'entraînement.

[ ]:
```
# ============================
#  MAX POST-CHECK (robuste + couvre un max de cas)
# Objectif : vérifier rapidement que
#   - le batch est OK (shapes/dtypes/ranges)
#   - le modèle prédit autre chose que ~0.5 constant
#   - le seuillage est raisonnable (scan + garde-fous fg%)
#   - transitions + overlay sont cohérents
#   - option : postprocess morpho + comparaison "raw vs post"
# ============================

import numpy as np
import cv2
import matplotlib.pyplot as plt

# ----------------------------
# Utils robustes
# ----------------------------
def to_numpy(x):
    """Convertit tf.Tensor / torch / np -> np.ndarray sans casser."""
    if hasattr(x, "numpy"):
        return x.numpy()
    return np.asarray(x)

def squeeze_hw(x):
    """
    Ramène un tableau (B,H,W,C) / (H,W,C) / (H,W) vers (H,W).
    Accepte aussi (B,H,W) ou (H,W,1).
    """
    x = to_numpy(x)
    if x.ndim == 4:   # (B,H,W,C)
        x = x[0]
    if x.ndim == 3:   # (H,W,C) ou (B,H,W)
        if x.shape[-1] in (1, 3, 4):
            x = x[..., 0]
        else:
            x = x[0]
    if x.ndim != 2:
        raise ValueError(f"Attendu (H,W) après squeeze, got shape={x.shape}")
    return x

def img_to_u8(img, assume_01=True):
    """
```

```
    Normalise une image en uint8.
    - si assume_01 True : considère que valeurs sont dans [0,1]
    - sinon : clip [0,255]
    """
    img = squeeze_hw(img)
    if img.dtype == np.uint8:
        return img
    if assume_01:
        return np.clip(img * 255.0, 0, 255).astype(np.uint8)
    return np.clip(img, 0, 255).astype(np.uint8)

def postprocess_binary(mask_bool, k_open=3, k_close=5, iterations=1):
    """Open + close pour réduire bruit et stabiliser frontières."""
    mb = (mask_bool.astype(np.uint8) * 255)
    kO = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (k_open, k_open))
    kC = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (k_close, k_close))
    mb = cv2.morphologyEx(mb, cv2.MORPH_OPEN,  kO, iterations=iterations)
    mb = cv2.morphologyEx(mb, cv2.MORPH_CLOSE, kC, iterations=iterations)
    return (mb > 0)

def compute_transitions_from_mask(mask, thr=0.5, do_post=False, k_open=3,␣
↪k_close=5, iterations=1):
    """
    - mask: prob (0..1) ou 0/255 ou 0/1, shape flexible
    - thr : seuil sur la version float
    - do_post: morpho après binarisation
    Retour:
      dx_bool (H,W-1), dy_bool (H-1,W), Tx (H,), Ty (W,), m_bool (H,W), fg_ratio
    """
    m = squeeze_hw(mask)

    # Si mask en 0..255, ramener en 0..1 pour thr cohérent
    if m.dtype != np.bool_ and m.max() > 1.5:
        m = m.astype(np.float32) / 255.0
    else:
        m = m.astype(np.float32)

    m_bool = (m > float(thr))

    if do_post:
        m_bool = postprocess_binary(m_bool, k_open=k_open, k_close=k_close,␣
↪iterations=iterations)

    fg_ratio = float(m_bool.mean())

    dx = (m_bool[:, 1:] != m_bool[:, :-1])
    dy = (m_bool[1:, :] != m_bool[:-1, :])
```

```
    Tx = dx.sum(axis=1).astype(np.float32)
    Ty = dy.sum(axis=0).astype(np.float32)

    return dx, dy, Tx, Ty, m_bool, fg_ratio

def robust_grain_estimate(Tx, Ty):
    Tx = np.asarray(Tx, dtype=np.float32)
    Ty = np.asarray(Ty, dtype=np.float32)

    mean_x, mean_y = float(Tx.mean()), float(Ty.mean())
    med_x,  med_y  = float(np.percentile(Tx, 50)), float(np.percentile(Ty, 50))

    est_mean = 0.5 * ((mean_x / 2.0) + (mean_y / 2.0))
    est_med  = 0.5 * ((med_x  / 2.0) + (med_y  / 2.0))

    return {
        "mean_Tx": mean_x, "mean_Ty": mean_y,
        "med_Tx":  med_x,  "med_Ty":  med_y,
        "ratio_Tx_Ty": mean_x / (mean_y + 1e-9),
        "est_grains_mean": est_mean,
        "est_grains_med":  est_med,
    }

def overlay_transitions_bool(img_gray_u8, dx_bool, dy_bool, alpha=0.75):
    img_u8 = img_to_u8(img_gray_u8, assume_01=False) if img_gray_u8.dtype != np.
 ↪uint8 else img_gray_u8
    H, W = img_u8.shape[:2]

    rgb = np.stack([img_u8, img_u8, img_u8], axis=-1)

    mx_full = np.zeros((H, W), dtype=bool)
    my_full = np.zeros((H, W), dtype=bool)

    mx_full[:, :W-1] = np.asarray(dx_bool, dtype=bool)
    my_full[:H-1, :] = np.asarray(dy_bool, dtype=bool)

    overlay = rgb.astype(np.float32)
    overlay[mx_full, 0] = 255  # Rouge: dx
    overlay[my_full, 1] = 255  # Vert : dy

    out = (alpha * overlay + (1.0 - alpha) * rgb).astype(np.uint8)
    return out, mx_full, my_full

def prob_stats(p, tag=""):
    p = squeeze_hw(p).astype(np.float32)
    print(tag, "min/max/mean:", float(p.min()), float(p.max()), float(p.mean()))
```

```
    for q in [1, 5, 10, 50, 90, 95, 99]:
        print(tag, f"{q:>2} pct:", float(np.percentile(p, q)))

def scan_thresholds(mask_prob, thrs=(0.35, 0.40, 0.45, 0.50, 0.55, 0.60, 0.65),
                    min_fg=0.01, max_fg=0.99, verbose=True):
    """
    Scan simple (SANS postprocess) pour éviter d'écraser le signal.
    Choisit un thr "raisonnable" selon:
      - fg_ratio dans [min_fg, max_fg]
      - ratio Tx/Ty proche de 1
      - transitions pas trop denses (penalité légère)
    """
    rows = []
    for t in thrs:
        dx, dy, Tx, Ty, _, fg = compute_transitions_from_mask(mask_prob, thr=t,␣
↪do_post=False)
        st = robust_grain_estimate(Tx, Ty)
        rows.append({
            "thr": float(t),
            "fg_ratio": float(fg),
            "ratio_Tx_Ty": float(st["ratio_Tx_Ty"]),
            "grains_med": float(st["est_grains_med"]),
            "pct_dx": float(100.0 * dx.mean()),
            "pct_dy": float(100.0 * dy.mean()),
        })

    def score(r):
        if (r["fg_ratio"] <= min_fg) or (r["fg_ratio"] >= max_fg):
            return 1e9
        ratio_pen = abs(r["ratio_Tx_Ty"] - 1.0)
        dens_pen  = 0.5 * (r["pct_dx"] + r["pct_dy"]) / 100.0
        thr_pen   = 0.02 * abs(r["thr"] - 0.5)
        return ratio_pen + dens_pen + thr_pen

    best = min(rows, key=score)

    if verbose:
        print("\n========== [THRESH SCAN] ==========")
        print("thr | fg% | Tx/Ty | grains_med | %dx | %dy")
        for r in rows:
            print(f"{r['thr']:.2f} | {100*r['fg_ratio']:.2f}% |␣
↪{r['ratio_Tx_Ty']:.2f} | "
                  f"{r['grains_med']:.2f} | {r['pct_dx']:.2f}% | {r['pct_dy']:.
↪2f}%")
        print("[BEST] thr =", best["thr"], "| fg% =",␣
↪round(100*best["fg_ratio"], 2),
              "| Tx/Ty =", round(best["ratio_Tx_Ty"], 2),
```

```
            "| grains_med =", round(best["grains_med"], 2))

    return {"rows": rows, "best": best}

# ----------------------------
# Helpers: déduction du nombre d'inputs + extraction batch
# ----------------------------
def _infer_model_inputs_from_batch(inputs):
    """
    Retourne 1/2/3 en se basant sur le batch `inputs`.
    Supporte:
      - tuple/list: (xb, hb) ou (xb, gb, hb)
      - dict: {"img":..., "grad_prior":..., "hed_prior":...} (ordre quelconque)
      - tensor/array: xb seul
    """
    if isinstance(inputs, dict):
        keys = set(inputs.keys())
        has_img  = ("img" in keys)
        has_g    = ("grad_prior" in keys) or ("grad" in keys)
        has_h    = ("hed_prior" in keys)  or ("hed"  in keys) or ("prior" in␣
 ↪keys)
        if has_img and has_g and has_h:
            return 3
        if has_img and has_h:
            return 2
        # sinon on considère 1 (img)
        return 1

    if isinstance(inputs, (tuple, list)):
        if len(inputs) == 3:
            return 3
        if len(inputs) == 2:
            return 2
        return 1

    return 1

def _get_from_dict(d, names):
    for k in names:
        if k in d:
            return d[k]
    return None

def _batch_extract(batch, MODEL_INPUTS="auto"):
    """
    Retourne (mode, xb, gb, hb, yb, raw_inputs)
      - mode: 1/2/3
```

```
      - gb/hb peuvent être None
      - raw_inputs: inputs originaux (dict/tuple/tensor) pour predict dict si␣
↪besoin
    """
    if isinstance(batch, (tuple, list)) and len(batch) == 2:
        inputs, yb = batch
    else:
        raise ValueError("Batch inattendu. Attendu un tuple/list (inputs, yb).")

    mode = _infer_model_inputs_from_batch(inputs) if MODEL_INPUTS == "auto"␣
↪else int(MODEL_INPUTS)

    xb = gb = hb = None

    if isinstance(inputs, dict):
        xb = _get_from_dict(inputs, ["img", "image", "x", "xb"])
        gb = _get_from_dict(inputs, ["grad_prior", "grad", "g", "gb"])
        hb = _get_from_dict(inputs, ["hed_prior", "hed", "prior", "h", "hb"])
        # si dict mais mode forcé 2/3 et clés manquantes -> on garde None et on␣
↪tombera en fallback
    elif isinstance(inputs, (tuple, list)):
        if mode == 3 and len(inputs) >= 3:
            xb, gb, hb = inputs[0], inputs[1], inputs[2]
        elif mode == 2 and len(inputs) >= 2:
            xb, hb = inputs[0], inputs[1]
        else:
            xb = inputs[0] if isinstance(inputs, (tuple, list)) and len(inputs)␣
↪>= 1 else inputs
    else:
        xb = inputs

    return mode, xb, gb, hb, yb, inputs

def _print_array_stats(arr, name):
    a = to_numpy(arr)
    print(f"[{name}] shape:", a.shape, "| dtype:", a.dtype,
          "| min/max/mean:", float(a.min()), float(a.max()), float(a.mean()))

def _safe_placeholder_like(x_u8, text="N/A"):
    # Crée une image placeholder (H,W) uint8
    H, W = x_u8.shape[:2]
    ph = np.zeros((H, W), dtype=np.uint8)
    cv2.putText(ph, text, (10, max(25, H//2)),
                cv2.FONT_HERSHEY_SIMPLEX, 0.8, (255,), 2, cv2.LINE_AA)
    return ph

# ------------------------------
```

```
#  CHECK COMPLET (1/2/3 inputs)
# ----------------------------
def run_post_check(model, train_ds, MODEL_INPUTS="auto",
                   thr_candidates=(0.35,0.40,0.45,0.50,0.55,0.60,0.65),
                   do_post=True, k_open=3, k_close=5, iterations=1,
                   show=True):
    """
    - model: ton modèle final (unet_model)
    - train_ds: tf.data dataset
    - MODEL_INPUTS: "auto" | 1 | 2 | 3
        * "auto" déduit depuis le batch (tuple/list/dict)
    - do_post: applique morpho uniquement au thr choisi (pour overlay + stats␣
 ↪finales)
    """
    if train_ds is None:
        raise ValueError("train_ds est requis")

    batch = next(iter(train_ds.take(1)))
    mode, xb, gb, hb, yb, raw_inputs = _batch_extract(batch,␣
 ↪MODEL_INPUTS=MODEL_INPUTS)

    if xb is None:
        raise ValueError("Impossible d'extraire `xb` (img) depuis le batch.")

    xb_np = to_numpy(xb)
    yb_np = to_numpy(yb)
    gb_np = to_numpy(gb) if gb is not None else None
    hb_np = to_numpy(hb) if hb is not None else None

    # --------- Checks shapes/dtypes ----------
    print("\n========== [BATCH CHECKS] ==========")
    print("[INFO] Inferred MODEL_INPUTS =", mode, "(requested =", MODEL_INPUTS,␣
 ↪")")

    _print_array_stats(xb, "xb(img)")
    if gb_np is not None:
        _print_array_stats(gb, "gb(grad_prior)")
    else:
        print("[gb(grad_prior)] not present")

    if hb_np is not None:
        _print_array_stats(hb, "hb(hed_prior)")
    else:
        print("[hb(hed_prior)] not present")

    _print_array_stats(yb, "yb(mask)")
    print("[yb] fg%:", float(100.0 * (yb_np > 0.5).mean()))
```

```
    # --------- Prediction ----------
    print("\n========== [PRED CHECKS] ==========")

    # IMPORTANT: Keras préfère list pour multi-inputs.
    pred = None
    if isinstance(raw_inputs, dict):
        # Si le dataset renvoie dict, on tente d'abord predict(dict) (si modèle
↪a des noms d'inputs compatibles)
        try:
            pred = model.predict(raw_inputs, verbose=0)
        except Exception:
            # fallback: on compose selon mode
            if mode == 3 and (gb is not None) and (hb is not None):
                pred = model.predict([xb, gb, hb], verbose=0)
            elif mode == 2 and (hb is not None):
                pred = model.predict([xb, hb], verbose=0)
            else:
                pred = model.predict(xb, verbose=0)
    else:
        if mode == 3:
            if (gb is None) or (hb is None):
                # fallback si batch incomplet
                if hb is not None:
                    pred = model.predict([xb, hb], verbose=0)
                else:
                    pred = model.predict(xb, verbose=0)
            else:
                pred = model.predict([xb, gb, hb], verbose=0)
        elif mode == 2:
            if hb is None:
                pred = model.predict(xb, verbose=0)
            else:
                pred = model.predict([xb, hb], verbose=0)
        else:
            pred = model.predict(xb, verbose=0)

    # (B,H,W,1) -> (H,W)
    mask_prob = squeeze_hw(pred).astype(np.float32)

    prob_stats(mask_prob, tag="[pred]")

    # Heuristic: détecter sortie ~constante (mauvais signe)
    std_pred = float(mask_prob.std())
    print("[pred] std:", std_pred)
    if std_pred < 1e-3:
        print("  Attention: prédiction quasi-constante (std très faible).")
```

```
    # --------- Threshold scan (sans postprocess) ----------
    scan = scan_thresholds(mask_prob, thrs=thr_candidates, verbose=True)
    best_thr = float(scan["best"]["thr"])

    # --------- Transitions @ best_thr ----------
    dx_raw, dy_raw, Tx_raw, Ty_raw, mb_raw, fg_raw =␣
↪compute_transitions_from_mask(
        mask_prob, thr=best_thr, do_post=False
    )
    st_raw = robust_grain_estimate(Tx_raw, Ty_raw)

    print("\n========== [TRANSITIONS @ BEST_THR | RAW] ==========")
    print(f"[INFO] thr={best_thr:.2f} | fg%={100*fg_raw:.2f}% | "
          f"mean Tx={st_raw['mean_Tx']:.2f} | mean Ty={st_raw['mean_Ty']:.2f} |␣
↪ratio={st_raw['ratio_Tx_Ty']:.2f}")
    print(f"[INFO] med  Tx={st_raw['med_Tx']:.2f} | med  Ty={st_raw['med_Ty']:.
↪2f}")
    print(f"[EST] grains (med)  {st_raw['est_grains_med']:.2f}")

    # --------- Postprocess (optionnel) ----------
    if do_post:
        dx_pp, dy_pp, Tx_pp, Ty_pp, mb_pp, fg_pp =␣
↪compute_transitions_from_mask(
            mask_prob, thr=best_thr, do_post=True, k_open=k_open,␣
↪k_close=k_close, iterations=iterations
        )
        st_pp = robust_grain_estimate(Tx_pp, Ty_pp)

        print("\n========== [TRANSITIONS @ BEST_THR | POST] ==========")
        print(f"[INFO] post(kO={k_open},kC={k_close},it={iterations}) |␣
↪fg%={100*fg_pp:.2f}% | "
              f"mean Tx={st_pp['mean_Tx']:.2f} | mean Ty={st_pp['mean_Ty']:.2f}␣
↪| ratio={st_pp['ratio_Tx_Ty']:.2f}")
        print(f"[INFO] med  Tx={st_pp['med_Tx']:.2f} | med  Ty={st_pp['med_Ty']:
↪.2f}")
        print(f"[EST] grains (med)  {st_pp['est_grains_med']:.2f}")

        dx_use, dy_use = dx_pp, dy_pp
    else:
        dx_use, dy_use = dx_raw, dy_raw

    # --------- Overlay ----------
    # xb est (B,H,W,1) float [0,1] -> u8
    img_u8 = img_to_u8(xb_np[0, ..., 0], assume_01=True)
```

```
    vis, mx_full, my_full = overlay_transitions_bool(img_u8, dx_use, dy_use,␣
↪alpha=0.75)
    print("[VIS] % transitions X:", float(100.0 * mx_full.mean()),
          "| % transitions Y:", float(100.0 * my_full.mean()))

    # --------- Visualisations ----------
    if show:
        yb_hw = squeeze_hw(yb_np).astype(np.float32)

        # priors pour affichage
        if gb_np is not None:
            gb_u8 = img_to_u8(gb_np[0, ..., 0], assume_01=True)
        else:
            gb_u8 = _safe_placeholder_like(img_u8, "grad_prior N/A")

        if hb_np is not None:
            hb_u8 = img_to_u8(hb_np[0, ..., 0], assume_01=True)
        else:
            hb_u8 = _safe_placeholder_like(img_u8, "hed_prior N/A")

        # Figure 2x3 (6 panneaux)
        plt.figure(figsize=(16, 8))

        plt.subplot(2, 3, 1)
        plt.imshow(img_u8, cmap="gray")
        plt.title("Image (xb)")
        plt.axis("off")

        plt.subplot(2, 3, 2)
        plt.imshow(gb_u8, cmap="gray")
        plt.title("Grad prior (gb)")
        plt.axis("off")

        plt.subplot(2, 3, 3)
        plt.imshow(hb_u8, cmap="gray")
        plt.title("HED prior (hb)")
        plt.axis("off")

        plt.subplot(2, 3, 4)
        plt.imshow(yb_hw, cmap="gray")
        plt.title("GT mask (yb)")
        plt.axis("off")

        plt.subplot(2, 3, 5)
        plt.imshow(mask_prob, cmap="gray")
        plt.title(f"Pred prob (thr={best_thr:.2f})")
        plt.axis("off")
```

```
        plt.subplot(2, 3, 6)
        plt.imshow(vis)
        plt.title("Overlay transitions (R=dx, G=dy)")
        plt.axis("off")

        plt.tight_layout()
        plt.show()

    return {
        "best_thr": best_thr,
        "pred_std": std_pred,
        "scan": scan,
        "MODEL_INPUTS_inferred": mode,
    }

# ------------------------------
#  Usage (auto recommandé)
# ------------------------------
# - model = unet_model
# - MODEL_INPUTS="auto" : déduit depuis train_ds (supporte 1/2/3 + dict)
if ("unet_model" in globals()) and ("train_ds" in globals()):
    _ = run_post_check(
        model=unet_model,
        train_ds=train_ds,
        MODEL_INPUTS="auto",        # ou 3 pour forcer
        thr_candidates=(0.35,0.40,0.45,0.50,0.55,0.60,0.65),
        do_post=True,
        k_open=3, k_close=5, iterations=1,
        show=True
    )
```

```
========== [BATCH CHECKS] ==========
[INFO] Inferred MODEL_INPUTS = 3 (requested = auto )
[xb(img)] shape: (8, 256, 256, 1) | dtype: float32 | min/max/mean: 0.0
0.6313725709915161 0.2840977907180786
[gb(grad_prior)] shape: (8, 256, 256, 1) | dtype: float32 | min/max/mean: 0.0
1.0 0.2292175590991974
[hb(hed_prior)] shape: (8, 256, 256, 1) | dtype: float32 | min/max/mean: 0.0 1.0
0.2849670648574829
[yb(mask)] shape: (8, 256, 256, 1) | dtype: float32 | min/max/mean: 0.0 1.0
0.10704994201660156
[yb] fg%: 10.704994201660156

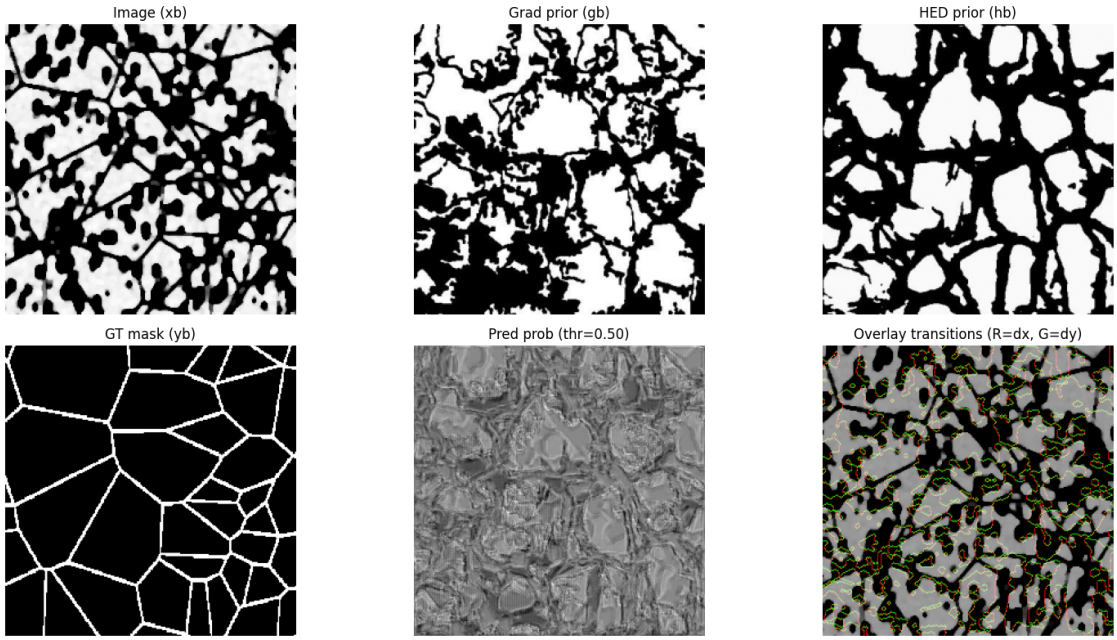
========== [PRED CHECKS] ==========
[pred] min/max/mean: 0.48406991362571716 0.5161095261573792 0.4994337558746338
[pred]  1 pct: 0.4903862774372101
```

```
[pred]  5 pct: 0.49324336647987366
[pred] 10 pct: 0.49458175897598267
[pred] 50 pct: 0.4996924102306366
[pred] 90 pct: 0.5038497447967529
[pred] 95 pct: 0.5050215125083923
[pred] 99 pct: 0.5074055790901184
[pred] std: 0.003632092149928212

========== [THRESH SCAN] ==========
thr | fg% | Tx/Ty | grains_med | %dx | %dy
0.35 | 100.00% | 0.00 | 0.00 | 0.00% | 0.00%
0.40 | 100.00% | 0.00 | 0.00 | 0.00% | 0.00%
0.45 | 100.00% | 0.00 | 0.00 | 0.00% | 0.00%
0.50 | 46.67% | 1.10 | 30.25 | 25.05% | 22.75%
0.55 | 0.00% | 0.00 | 0.00 | 0.00% | 0.00%
0.60 | 0.00% | 0.00 | 0.00 | 0.00% | 0.00%
0.65 | 0.00% | 0.00 | 0.00 | 0.00% | 0.00%
[BEST] thr = 0.5 | fg% = 46.67 | Tx/Ty = 1.1 | grains_med = 30.25

========== [TRANSITIONS @ BEST_THR | RAW] ==========
[INFO] thr=0.50 | fg%=46.67% | mean Tx=63.89 | mean Ty=58.01 | ratio=1.10
[INFO] med  Tx=63.00 | med  Ty=58.00
[EST] grains (med)  30.25

========== [TRANSITIONS @ BEST_THR | POST] ==========
[INFO] post(kO=3,kC=5,it=1) | fg%=43.97% | mean Tx=21.37 | mean Ty=20.36 |
ratio=1.05
[INFO] med  Tx=21.00 | med  Ty=21.00
[EST] grains (med)  10.50
[VIS] % transitions X: 8.34808349609375 | % transitions Y: 7.95440673828125
```



Le post-check confirme que la chaîne de données est saine : le pipeline opère bien en **3 entrées** (xb/gb/hb) avec des tenseurs cohérents en **(8, 256, 256, 1)**, et une vérité terrain **peu dense** (**fg 10,7%**), typique de joints fins. En revanche, la sortie du modèle est **quasi plate** autour de **0,5** (0,484–0,516 ; moyenne 0,499 ; **std 0,0036**), ce qui indique une **absence de pouvoir discriminant** sur ce batch (modèle non convergé, logits/loss incohérents ou ordre d'inputs à vérifier). Cette "platitude" se voit directement dans le scan de seuils : **thr 0,45 fg=100%** et **thr 0,55 fg=0%**, donc **thr=0,50** devient un choix par défaut mais **non fiable** (fg 46,7%) car il découpe principalement du bruit autour du seuil. Les transitions calculées à **thr=0,5** reflètent donc surtout une binarisation instable : l'estimation "raw" (**grains_med 30,25**) chute fortement après morphologie (**fg 44% ; grains_med 10,5**), ce qui montre que l'open/close supprime des micro-îlots générés par une carte de probabilité trop proche de 0,5. Visuellement, l'overlay (dx rouge, dy vert) sert ici surtout de **contrôle de cohérence géométrique**, mais tant que la proba reste centrée sur 0,5, **les métriques de grains/transitions ne doivent pas être interprétées comme des résultats physiques**.

[ ]:
```
# ============================================================
# PATCH: ton modèle attend 3 entrées (img, grad, hed)
# => on adapte le dataset pour produire ((img, grad, hed), y)
# Sans toucher au modèle (NE PAS REBUILD).
# ============================================================

import numpy as np
import tensorflow as tf

assert "unet_model" in globals(), "unet_model introuvable."
```

```
assert len(unet_model.inputs) == 3, f"Ce patch vise un modèle 3-inputs. Got:␣
 ↪{len(unet_model.inputs)} inputs."
assert "train_ds" in globals() and "val_ds" in globals(), "train_ds/val_ds␣
 ↪introuvables."

AUTOTUNE = tf.data.AUTOTUNE

# ---- utilitaires robustes (supporte tuple imbriqué / dict / extras) ----
def _is_tensorlike(z):
    return tf.is_tensor(z)

def _flatten_tuple(x):
    if isinstance(x, (tuple, list)):
        out = []
        for e in x:
            out.extend(_flatten_tuple(e))
        return out
    return [x]

def _ensure_channels(t, c=1):
    t = tf.convert_to_tensor(t)
    if t.shape.rank == 2:
        t = t[..., tf.newaxis]
    if t.shape.rank == 3:
        ch = tf.shape(t)[-1]
        t = tf.cond(ch > c, lambda: t[..., :c], lambda: t)
    return t

def _pick_img_grad_hed_from_x(x):
    """
    Accepte:
      - (img, grad, hed)
      - ((img, grad, hed),)
      - (img, something, ..., hed) -> first=img, middle=grad (best-effort),␣
 ↪last=hed
      - dict keys: img/image, grad/gradient, hed/hed_prior
      - si grad absent: grad = zeros_like(img)
    """
    img = grad = hed = None

    if isinstance(x, dict):
        for k in ("img", "image", "x"):
            if k in x:
                img = x[k]; break
        for k in ("grad", "gradient", "sobel", "canny", "g"):
            if k in x:
                grad = x[k]; break
```

```
        for k in ("hed", "hed_prior", "hedp"):
            if k in x:
                hed = x[k]; break

        if img is None or hed is None:
            vals = []
            for v in x.values():
                vals.extend(_flatten_tuple(v))
            tens = [v for v in vals if _is_tensorlike(v)]
            if len(tens) >= 2:
                img = img if img is not None else tens[0]
                hed = hed if hed is not None else tens[-1]
                if len(tens) >= 3 and grad is None:
                    grad = tens[1]
        # grad optional
        if img is None or hed is None:
            raise ValueError(f"Dict inputs non reconnus. Clés dispo: {list(x.
 ↪keys())}")

    else:
        parts = _flatten_tuple(x)
        tens = [p for p in parts if _is_tensorlike(p)]
        if len(tens) < 2:
            raise ValueError(f"Entrée invalide: pas assez de tenseurs dans x.␣
 ↪type={type(x)}")
        img = tens[0]
        hed = tens[-1]
        if len(tens) >= 3:
            grad = tens[1]  # best-effort
        else:
            grad = None

    # grad fallback
    if grad is None:
        grad = tf.zeros_like(img)

    return img, grad, hed

# ---- map: normalize + shapes -> ((img, grad, hed), y) ----
IMG_H, IMG_W = 256, 256
IMG_C = 1

def _ensure_example_3in(x, y):
    img, grad, hed = _pick_img_grad_hed_from_x(x)

    img  = tf.cast(_ensure_channels(img,  IMG_C), tf.float32)
    grad = tf.cast(_ensure_channels(grad, 1),     tf.float32)
```

```
    hed  = tf.cast(_ensure_channels(hed,  1),     tf.float32)
    y    = tf.cast(_ensure_channels(y,    1),     tf.float32)

    img  = tf.clip_by_value(img,  0.0, 1.0)
    grad = tf.clip_by_value(grad, 0.0, 1.0)
    hed  = tf.clip_by_value(hed,  0.0, 1.0)
    y    = tf.clip_by_value(y,    0.0, 1.0)

    # force shapes (modifie si tes dims diffèrent)
    img  = tf.ensure_shape(img,  [IMG_H, IMG_W, IMG_C])
    grad = tf.ensure_shape(grad, [IMG_H, IMG_W, 1])
    hed  = tf.ensure_shape(hed,  [IMG_H, IMG_W, 1])
    y    = tf.ensure_shape(y,    [IMG_H, IMG_W, 1])

    return (img, grad, hed), y

def _make_epoch_ds_3in(ds, training: bool, batch_size: int, shuffle_buffer: int␣
 ↪= 2048, seed: int = 1337):
    ds2 = ds.unbatch().map(_ensure_example_3in, num_parallel_calls=AUTOTUNE)
    if training:
        ds2 = ds2.shuffle(shuffle_buffer, seed=seed,␣
 ↪reshuffle_each_iteration=True)
    ds2 = ds2.batch(batch_size, drop_remainder=True).prefetch(AUTOTUNE)
    return ds2

# ---- build epoch ds + warmup (3 inputs now) ----
BATCH_SIZE = int(globals().get("BATCH_SIZE", 8))
train_epoch_ds = _make_epoch_ds_3in(train_ds, training=True, ␣
 ↪batch_size=BATCH_SIZE)
val_epoch_ds   = _make_epoch_ds_3in(val_ds,   training=False,␣
 ↪batch_size=BATCH_SIZE)

((xb_w, gb_w, hb_w), yb_w) = next(iter(train_epoch_ds.take(1)))
_ = unet_model([xb_w, gb_w, hb_w], training=False)

print("[OK] Dataset adapté en 3 entrées: (img, grad, hed). Warmup forward pass␣
 ↪réussi.")
```

```
[OK] Dataset adapté en 3 entrées: (img, grad, hed). Warmup forward pass réussi.
```

[ ]:
```
import tensorflow as tf

# ---- FIX Keras3: add_weight(name=...) not positional ----
@tf.keras.utils.register_keras_serializable()
class MeanIoUThreshold(tf.keras.metrics.Metric):
    def __init__(self, threshold=0.5, name="iou", **kwargs):
        super().__init__(name=name, **kwargs)
```

```
        self.threshold = float(threshold)

        self.tp = self.add_weight(name="tp", shape=(), initializer="zeros",␣
 ↪dtype=tf.float32)
        self.fp = self.add_weight(name="fp", shape=(), initializer="zeros",␣
 ↪dtype=tf.float32)
        self.fn = self.add_weight(name="fn", shape=(), initializer="zeros",␣
 ↪dtype=tf.float32)

    def update_state(self, y_true, y_pred, sample_weight=None):
        yt = tf.cast(y_true > 0.5, tf.float32)
        yp = tf.cast(y_pred >= self.threshold, tf.float32)

        yt = tf.reshape(yt, [-1])
        yp = tf.reshape(yp, [-1])

        tp = tf.reduce_sum(yt * yp)
        fp = tf.reduce_sum((1.0 - yt) * yp)
        fn = tf.reduce_sum(yt * (1.0 - yp))

        self.tp.assign_add(tp)
        self.fp.assign_add(fp)
        self.fn.assign_add(fn)

    def result(self):
        return tf.math.divide_no_nan(self.tp, self.tp + self.fp + self.fn)

    def reset_state(self):
        self.tp.assign(0.0); self.fp.assign(0.0); self.fn.assign(0.0)

# ---- Prefer built-in if available, else use fixed custom ----
try:
    iou_05 = tf.keras.metrics.BinaryIoU(name="iou@0.5", threshold=0.5)
except Exception:
    iou_05 = MeanIoUThreshold(name="iou@0.5", threshold=0.5)

print("[OK] iou_05 =", type(iou_05).__name__)
```

```
[OK] iou_05 = BinaryIoU
```

[ ]:
```
# ========================
# CONFIG
# ========================
import os
import time
import datetime as _dt
import numpy as np
```

```
import tensorflow as tf

assert "unet_model" in globals(), "unet_model introuvable (NE PAS REBUILD)."
assert "train_ds" in globals(), "train_ds introuvable."
assert "val_ds" in globals(), "val_ds introuvable."

CFG = {
    # Data
    "IMG_H": 256,
    "IMG_W": 256,
    "IMG_C": 1,
    "BATCH_SIZE": int(globals().get("BATCH_SIZE", 8)),
    "SHUFFLE_BUFFER": 2048,
    "SEED": 1337,

    # Phase 1 (TRAIN)
    "EPOCHS_PHASE1": 50,
    "LR_PHASE1": 1e-3,
    "BCE_WEIGHT_PHASE1": 0.35,
    "MONITOR_PHASE1": "val_dice",

    # Phase 2 (FINETUNE precision-first)
    "EPOCHS_PHASE2": 25,
    "LR_PHASE2": 3e-5,
    "STRICT_THR": 0.95,
    "TARGET_PRECISION": 0.90,
    "MONITOR_PHASE2": None,
    "TV_ALPHA": 0.85,
    "TV_GAMMA": 1.33,
    "BCE_WEIGHT_PHASE2": 0.05,

    # Threshold calibration
    "CALIB_OBJECTIVE": "dice",
    "THR_GRID_COARSE": np.round(np.linspace(0.50, 0.99, 11), 3),
    "THR_GRID_DENSE": np.round(np.concatenate([np.linspace(0.90, 0.99, 10),
                                               np.linspace(0.99, 0.999, 10)]),␣
↪3),

    # Optional postprocess
    "ENABLE_MORPH_POST": False,
    "MORPH_K_OPEN": 3,
    "MORPH_K_CLOSE": 5,

    # Logging / checkpoints
    "OUT_DIR": os.path.join(globals().get("BASE_DIR", "."), "models"),
    "SAVE_BEST_WEIGHTS": False,
```

```
    # Performance / stability
    "USE_MIXED_PRECISION": False,
    "PREFER_GRAPH_MODE": True,
}

os.makedirs(CFG["OUT_DIR"], exist_ok=True)
RUN_ID = _dt.datetime.now().strftime("%Y%m%d_%H%M%S")

tf.random.set_seed(CFG["SEED"])
np.random.seed(CFG["SEED"])

if CFG["USE_MIXED_PRECISION"]:
    try:
        from tensorflow.keras import mixed_precision
        mixed_precision.set_global_policy("mixed_float16")
    except Exception:
        pass

# ========================
# MODEL INPUT ARITY (2 vs 3)
# ========================
MODEL_N_IN = len(tf.nest.flatten(unet_model.inputs))
assert MODEL_N_IN in (2, 3), f"Modèle inattendu: {MODEL_N_IN} inputs (attendu 2␣
 ↪ou 3)."
print(f"[OK] Model inputs = {MODEL_N_IN}")

# ========================
# FALLBACKS
# ========================
@tf.keras.utils.register_keras_serializable()
def _soft_dice(y_true, y_pred, smooth=1.0):
    y_true = tf.cast(y_true, tf.float32)
    y_pred = tf.cast(y_pred, tf.float32)
    y_pred = tf.clip_by_value(y_pred, 0.0, 1.0)
    y_true_f = tf.reshape(y_true, [-1])
    y_pred_f = tf.reshape(y_pred, [-1])
    inter = tf.reduce_sum(y_true_f * y_pred_f)
    denom = tf.reduce_sum(y_true_f) + tf.reduce_sum(y_pred_f)
    return (2.0 * inter + smooth) / (denom + smooth)

if "dice_coef" not in globals():
    dice_coef = _soft_dice

@tf.keras.utils.register_keras_serializable()
class BCEDiceLoss(tf.keras.losses.Loss):
    def __init__(self, bce_weight=0.35, smooth=1.0, name="bce_dice"):
        super().__init__(name=name)
```

```
        self.bce_weight = float(bce_weight)
        self.smooth = float(smooth)
        self._bce = tf.keras.losses.BinaryCrossentropy(from_logits=False)

    def call(self, y_true, y_pred):
        bce = self._bce(y_true, y_pred)
        dice = _soft_dice(y_true, y_pred, smooth=self.smooth)
        dice_loss = 1.0 - dice
        return self.bce_weight * bce + (1.0 - self.bce_weight) * dice_loss

    def get_config(self):
        cfg = super().get_config()
        cfg.update({"bce_weight": self.bce_weight, "smooth": self.smooth})
        return cfg

if "bce_dice_loss" in globals():
    @tf.keras.utils.register_keras_serializable()
    class BCEDiceLossFromUser(tf.keras.losses.Loss):
        def __init__(self, bce_weight=0.35, name="bce_dice_user"):
            super().__init__(name=name)
            self.bce_weight = float(bce_weight)

        def call(self, y_true, y_pred):
            return bce_dice_loss(y_true, y_pred, bce_weight=self.bce_weight)

        def get_config(self):
            cfg = super().get_config()
            cfg.update({"bce_weight": self.bce_weight})
            return cfg
    _BCEDiceLoss = BCEDiceLossFromUser
else:
    _BCEDiceLoss = BCEDiceLoss

@tf.keras.utils.register_keras_serializable()
class FocalTverskyPlusBCE(tf.keras.losses.Loss):
    def __init__(self, alpha=0.85, gamma=1.33, bce_weight=0.05, smooth=1e-6,␣
 ↪name="focal_tversky_plus_bce"):
        super().__init__(name=name)
        self.alpha = float(alpha)
        self.beta = float(1.0 - alpha)
        self.gamma = float(gamma)
        self.bce_weight = float(bce_weight)
        self.smooth = float(smooth)
        self._bce = tf.keras.losses.BinaryCrossentropy(from_logits=False)

    def call(self, y_true, y_pred):
        y_true = tf.cast(y_true, tf.float32)
```

```
        y_pred = tf.cast(tf.clip_by_value(y_pred, 1e-6, 1.0 - 1e-6), tf.float32)
        yt = tf.reshape(y_true, [-1])
        yp = tf.reshape(y_pred, [-1])

        tp = tf.reduce_sum(yt * yp)
        fp = tf.reduce_sum((1.0 - yt) * yp)
        fn = tf.reduce_sum(yt * (1.0 - yp))

        tversky = (tp + self.smooth) / (tp + self.alpha * fp + self.beta * fn + self.smooth)
        focal_tversky = tf.pow((1.0 - tversky), self.gamma)
        if self.bce_weight > 0.0:
            return (1.0 - self.bce_weight) * focal_tversky + self.bce_weight * self._bce(y_true, y_pred)
        return focal_tversky

    def get_config(self):
        cfg = super().get_config()
        cfg.update({"alpha": self.alpha, "gamma": self.gamma, "bce_weight": self.bce_weight, "smooth": self.smooth})
        return cfg

# ==========================
# DATASET STABLE (AUTO 2/3 inputs)
# ==========================
AUTOTUNE = tf.data.AUTOTUNE

def _pick_from_dict(d, *keys):
    for k in keys:
        if k in d and d[k] is not None:
            return d[k]
    return None

def _ensure_example(x, y):
    """
    Accepte:
      - dict: {"img":..., "grad":..., "hed_prior":...} (clés flexibles)
      - tuple/list:
          (img, hed) ou (img, grad, hed)
    Retourne:
      x_out = (img, hed) si modèle 2 inputs
      x_out = (img, grad, hed) si modèle 3 inputs
    """
    if isinstance(x, dict):
        img  = _pick_from_dict(x, "img", "image", "x", "input")
        grad = _pick_from_dict(x, "grad", "gradient", "grad_map", "x_grad")
        hed  = _pick_from_dict(x, "hed_prior", "hed", "hed_map", "x_hed")
```

```
        if img is None or hed is None or (MODEL_N_IN == 3 and grad is None):
            raise ValueError("Dict inputs: clés attendues au minimum img +␣
↪hed_prior (et grad si modèle=3).")
    else:
        if not isinstance(x, (tuple, list)):
            raise ValueError(f"Entrée invalide: attendu tuple/list ou dict, got␣
↪type={type(x)}")
        if len(x) == 2:
            img, hed = x
            grad = None
        elif len(x) == 3:
            img, grad, hed = x
        else:
            raise ValueError(f"Entrée invalide: attendu 2 ou 3 éléments, got␣
↪len={len(x)}")

        if MODEL_N_IN == 3 and grad is None:
            raise ValueError("Le dataset fournit 2 entrées mais le modèle␣
↪attend 3 (img, grad, hed).")

    img = tf.cast(img, tf.float32)
    hed = tf.cast(hed, tf.float32)
    y   = tf.cast(y,   tf.float32)

    img = tf.clip_by_value(img, 0.0, 1.0)
    hed = tf.clip_by_value(hed, 0.0, 1.0)
    y   = tf.clip_by_value(y,   0.0, 1.0)

    img = tf.ensure_shape(img, [CFG["IMG_H"], CFG["IMG_W"], CFG["IMG_C"]])
    hed = tf.ensure_shape(hed, [CFG["IMG_H"], CFG["IMG_W"], 1])
    y   = tf.ensure_shape(y,   [CFG["IMG_H"], CFG["IMG_W"], 1])

    if MODEL_N_IN == 3:
        grad = tf.cast(grad, tf.float32)
        grad = tf.clip_by_value(grad, 0.0, 1.0)
        grad = tf.ensure_shape(grad, [CFG["IMG_H"], CFG["IMG_W"], 1])
        x_out = (img, grad, hed)
    else:
        x_out = (img, hed)

    return x_out, y

def _make_epoch_ds(ds, training: bool):
    ds2 = ds.unbatch().map(_ensure_example, num_parallel_calls=AUTOTUNE)
    if training:
        ds2 = ds2.shuffle(CFG["SHUFFLE_BUFFER"], seed=CFG["SEED"],␣
↪reshuffle_each_iteration=True)
```

```
    ds2 = ds2.batch(CFG["BATCH_SIZE"], drop_remainder=True).prefetch(AUTOTUNE)
    return ds2

train_epoch_ds = _make_epoch_ds(train_ds, training=True)
val_epoch_ds   = _make_epoch_ds(val_ds,   training=False)

def _safe_cardinality(ds_batched):
    c = tf.data.experimental.cardinality(ds_batched).numpy()
    c = int(c)
    if c > 0:
        return c
    n = 0
    for _ in ds_batched:
        n += 1
    return max(1, n)

steps_per_epoch = _safe_cardinality(train_epoch_ds)
val_steps       = _safe_cardinality(val_epoch_ds)

train_fit_ds = train_epoch_ds.repeat()
val_fit_ds   = val_epoch_ds.repeat()

# Warmup forward pass (1 batch) generic
xw, yw = next(iter(train_epoch_ds.take(1)))
_ = unet_model(xw, training=False)
print("[OK] Warmup forward pass réussi avec arité =", MODEL_N_IN)

# ========================
# HELPERS: compile/fit stability + BN freeze
# ========================
def _reset_keras_functions(m):
    for attr in ("train_function", "test_function", "predict_function"):
        if hasattr(m, attr):
            setattr(m, attr, None)

def _freeze_batchnorm(m):
    for layer in m.layers:
        if isinstance(layer, tf.keras.layers.BatchNormalization):
            layer.trainable = False

def _make_optimizer(lr):
    try:
        return tf.keras.optimizers.AdamW(learning_rate=lr, weight_decay=1e-4,␣
 ↪clipnorm=1.0)
    except Exception:
        return tf.keras.optimizers.Adam(learning_rate=lr, clipnorm=1.0)
```

```
def _safe_compile(m, optimizer, loss, metrics, run_eagerly):
    _reset_keras_functions(m)
    m.compile(optimizer=optimizer, loss=loss, metrics=metrics,␣
 ↪run_eagerly=run_eagerly)

def _should_fallback_to_eager(err: Exception) -> bool:
    s = str(err)
    keys = [
        "Creating variables on a non-first call",
        "tf.function only supports",
        "You must feed a value for placeholder tensor",
        "AlreadyExistsError",
        "Graph execution error",
        "Failed to convert",
    ]
    return any(k in s for k in keys)

def _safe_fit(m, fit_kwargs, compile_kwargs):
    try:
        return m.fit(**fit_kwargs)
    except Exception as e:
        if _should_fallback_to_eager(e):
            print("[WARN] Fit error -> fallback run_eagerly=True\n", str(e)[:
 ↪500])
            _safe_compile(m, run_eagerly=True, **compile_kwargs)
            return m.fit(**fit_kwargs)
        raise

# ========================
# METRICS
# ========================
strict_thr = float(CFG["STRICT_THR"])
m_dice   = tf.keras.metrics.MeanMetricWrapper(fn=dice_coef, name="dice")
m_iou05  = tf.keras.metrics.BinaryIoU(name="iou@0.5", threshold=0.5)
m_prec05 = tf.keras.metrics.Precision(name="prec@0.5", thresholds=0.5)
m_rec05  = tf.keras.metrics.Recall(name="rec@0.5", thresholds=0.5)
m_precS  = tf.keras.metrics.Precision(name=f"prec@{strict_thr:.2f}",␣
 ↪thresholds=strict_thr)

# ========================
# PHASE 1: TRAIN
# ========================
from tensorflow.keras.callbacks import CSVLogger, EarlyStopping,␣
 ↪ReduceLROnPlateau, TerminateOnNaN, ModelCheckpoint

loss_p1 = _BCEDiceLoss(bce_weight=CFG["BCE_WEIGHT_PHASE1"])
opt_p1  = _make_optimizer(CFG["LR_PHASE1"])
```

```
compile_p1 = dict(
    optimizer=opt_p1,
    loss=loss_p1,
    metrics=[m_dice, m_iou05, m_prec05, m_rec05, m_precS],
)

run_eagerly_default = (not CFG["PREFER_GRAPH_MODE"])
_safe_compile(unet_model, run_eagerly=run_eagerly_default, **compile_p1)

log_p1 = os.path.join(CFG["OUT_DIR"], f"train_phase1_{RUN_ID}.csv")
callbacks_p1 = [
    CSVLogger(log_p1, append=False),
    TerminateOnNaN(),
    EarlyStopping(
        monitor=CFG["MONITOR_PHASE1"],
        mode="max",
        patience=10,
        min_delta=1e-4,
        restore_best_weights=True,
        verbose=1,
    ),
    ReduceLROnPlateau(
        monitor="val_loss",
        mode="min",
        factor=0.5,
        patience=3,
        min_lr=1e-6,
        verbose=1,
    ),
]

if CFG["SAVE_BEST_WEIGHTS"]:
    ckpt_p1 = os.path.join(CFG["OUT_DIR"], f"best_phase1_{RUN_ID}.weights.h5")
    callbacks_p1.append(ModelCheckpoint(
        ckpt_p1, monitor=CFG["MONITOR_PHASE1"], mode="max",
        save_best_only=True, save_weights_only=True, verbose=1
    ))

fit_p1 = dict(
    x=train_fit_ds,
    validation_data=val_fit_ds,
    steps_per_epoch=steps_per_epoch,
    validation_steps=val_steps,
    epochs=int(CFG["EPOCHS_PHASE1"]),
    callbacks=callbacks_p1,
    verbose=1,
```

```
)

hist1 = _safe_fit(unet_model, fit_kwargs=fit_p1, compile_kwargs=compile_p1)

# =========================
# PHASE 2: FINETUNE (precision-first)
# =========================
_freeze_batchnorm(unet_model)

loss_p2 = FocalTverskyPlusBCE(
    alpha=CFG["TV_ALPHA"],
    gamma=CFG["TV_GAMMA"],
    bce_weight=CFG["BCE_WEIGHT_PHASE2"],
)
opt_p2  = _make_optimizer(CFG["LR_PHASE2"])

monitor_p2 = CFG["MONITOR_PHASE2"] or f"val_prec@{strict_thr:.2f}"

# métriques neuves (état reset)
m_dice2   = tf.keras.metrics.MeanMetricWrapper(fn=dice_coef, name="dice")
m_iou052  = tf.keras.metrics.BinaryIoU(name="iou@0.5", threshold=0.5)
m_prec052 = tf.keras.metrics.Precision(name="prec@0.5", thresholds=0.5)
m_rec052  = tf.keras.metrics.Recall(name="rec@0.5", thresholds=0.5)
m_precS2  = tf.keras.metrics.Precision(name=f"prec@{strict_thr:.2f}",␣
 ↪thresholds=strict_thr)

compile_p2 = dict(
    optimizer=opt_p2,
    loss=loss_p2,
    metrics=[m_dice2, m_iou052, m_prec052, m_rec052, m_precS2],
)

_safe_compile(unet_model, run_eagerly=run_eagerly_default, **compile_p2)

log_p2 = os.path.join(CFG["OUT_DIR"], f"train_phase2_{RUN_ID}.csv")
callbacks_p2 = [
    CSVLogger(log_p2, append=False),
    TerminateOnNaN(),
    EarlyStopping(
        monitor=monitor_p2,
        mode="max",
        patience=8,
        min_delta=1e-4,
        restore_best_weights=True,
        verbose=1,
    ),
    ReduceLROnPlateau(
```

```
        monitor=monitor_p2,
        mode="max",
        factor=0.5,
        patience=2,
        min_lr=1e-6,
        verbose=1,
    ),
]

if CFG["SAVE_BEST_WEIGHTS"]:
    ckpt_p2 = os.path.join(CFG["OUT_DIR"], f"best_phase2_{RUN_ID}.weights.h5")
    callbacks_p2.append(ModelCheckpoint(
        ckpt_p2, monitor=monitor_p2, mode="max",
        save_best_only=True, save_weights_only=True, verbose=1
    ))

fit_p2 = dict(
    x=train_fit_ds,
    validation_data=val_fit_ds,
    steps_per_epoch=steps_per_epoch,
    validation_steps=val_steps,
    epochs=int(CFG["EPOCHS_PHASE2"]),
    callbacks=callbacks_p2,
    verbose=1,
)

hist2 = _safe_fit(unet_model, fit_kwargs=fit_p2, compile_kwargs=compile_p2)

# ========================
# EVAL (return_dict=True)
# ========================
val_results = unet_model.evaluate(val_epoch_ds, steps=val_steps, verbose=1,␣
 ↪return_dict=True)
print("\n[VAL RESULTS]")
for k, v in val_results.items():
    try:
        print(f" - {k}: {float(v):.6f}")
    except Exception:
        print(f" - {k}: {v}")

# ========================
# THRESHOLD CALIBRATION (precision constraint) - GENERIC
# ========================
def _accumulate_stats_over_val(model, ds, steps, thresholds):
    thresholds = np.asarray(thresholds, dtype=np.float32)
    tp = np.zeros((thresholds.shape[0],), dtype=np.float64)
    fp = np.zeros((thresholds.shape[0],), dtype=np.float64)
```

```
    fn = np.zeros((thresholds.shape[0],), dtype=np.float64)

    it = iter(ds.take(steps))
    for _ in range(steps):
        x, y = next(it)
        p = model.predict(x, verbose=0)

        p_flat = p.reshape(-1).astype(np.float32)
        y_flat = (y.numpy().reshape(-1) > 0.5)

        pred = (p_flat[None, :] >= thresholds[:, None])
        pos = float(y_flat.sum())

        tp_batch = np.sum(pred & y_flat[None, :], axis=1, dtype=np.float64)
        fp_batch = np.sum(pred & (~y_flat)[None, :], axis=1, dtype=np.float64)
        fn_batch = pos - tp_batch

        tp += tp_batch
        fp += fp_batch
        fn += fn_batch

    precision = tp / (tp + fp + 1e-9)
    recall    = tp / (tp + fn + 1e-9)
    dice      = (2.0 * tp) / (2.0 * tp + fp + fn + 1e-9)
    return {"thresholds": thresholds, "tp": tp, "fp": fp, "fn": fn,
            "precision": precision, "recall": recall, "dice": dice}

thr_grid = np.unique(np.concatenate([CFG["THR_GRID_COARSE"],␣
 ↪CFG["THR_GRID_DENSE"]]))
stats = _accumulate_stats_over_val(unet_model, val_epoch_ds, val_steps,␣
 ↪thr_grid)

prec = stats["precision"]
rec  = stats["recall"]
dice = stats["dice"]
ths  = stats["thresholds"]

ok = prec >= float(CFG["TARGET_PRECISION"])
if not np.any(ok):
    best_idx = int(np.argmax(prec))
    print(f"\n[CALIB WARN] Impossible d'atteindre precision >=␣
 ↪{CFG['TARGET_PRECISION']:.2f} sur le grid.")
    print(f"[CALIB BEST PREC] thr={ths[best_idx]:.3f} | prec={prec[best_idx]:.
 ↪4f} | rec={rec[best_idx]:.4f} | dice={dice[best_idx]:.4f}")
    RECOMMENDED_THR = float(ths[best_idx])
else:
    if str(CFG["CALIB_OBJECTIVE"]).lower() == "recall":
```

```
        best_idx = int(np.argmax(np.where(ok, rec, -1.0)))
        obj_name = "recall"
    else:
        best_idx = int(np.argmax(np.where(ok, dice, -1.0)))
        obj_name = "dice"

    RECOMMENDED_THR = float(ths[best_idx])
    print(f"\n[CALIB OK] Objectif={obj_name} sous contrainte␣
↪precision>={CFG['TARGET_PRECISION']:.2f}")
    print(f"[CALIB RECOMMENDED] thr={RECOMMENDED_THR:.3f} |␣
↪prec={prec[best_idx]:.4f} | rec={rec[best_idx]:.4f} | dice={dice[best_idx]:.
↪4f}")

# =========================
# OPTIONAL: Morphological postprocess (same thr) - GENERIC
# =========================
def _dilate(x, k):
    return tf.nn.max_pool2d(x, ksize=k, strides=1, padding="SAME")

def _erode(x, k):
    return -tf.nn.max_pool2d(-x, ksize=k, strides=1, padding="SAME")

def _morph_open_close(bin_mask, k_open=3, k_close=5):
    x = tf.cast(bin_mask, tf.float32)
    x = _dilate(_erode(x, k_open), k_open)
    x = _erode(_dilate(x, k_close), k_close)
    return tf.cast(x > 0.5, tf.float32)

def _eval_single_threshold(model, ds, steps, thr, use_morph=False, k_open=3,␣
↪k_close=5):
    tp = fp = fn = 0.0
    it = iter(ds.take(steps))
    for _ in range(steps):
        x, y = next(it)
        p = model.predict(x, verbose=0)
        yb = (y.numpy() > 0.5).astype(np.uint8)

        if use_morph:
            pb = tf.cast(p >= thr, tf.float32)
            pb = _morph_open_close(pb, k_open=k_open, k_close=k_close).numpy().
↪astype(np.uint8)
        else:
            pb = (p >= thr).astype(np.uint8)

        tp += np.logical_and(pb == 1, yb == 1).sum()
        fp += np.logical_and(pb == 1, yb == 0).sum()
        fn += np.logical_and(pb == 0, yb == 1).sum()
```

```
    precision = tp / (tp + fp + 1e-9)
    recall    = tp / (tp + fn + 1e-9)
    dice      = (2.0 * tp) / (2.0 * tp + fp + fn + 1e-9)
    return float(precision), float(recall), float(dice)

if CFG["ENABLE_MORPH_POST"]:
    pr0, rc0, dc0 = _eval_single_threshold(unet_model, val_epoch_ds, val_steps,␣
 ↪RECOMMENDED_THR, use_morph=False)
    pr1, rc1, dc1 = _eval_single_threshold(
        unet_model, val_epoch_ds, val_steps, RECOMMENDED_THR,
        use_morph=True, k_open=int(CFG["MORPH_K_OPEN"]),␣
 ↪k_close=int(CFG["MORPH_K_CLOSE"])
    )
    print("\n[POSTPROCESS CHECK] (same thr)")
    print(f" RAW : prec={pr0:.4f} | rec={rc0:.4f} | dice={dc0:.4f}")
    print(f" MORPH(open{CFG['MORPH_K_OPEN']},close{CFG['MORPH_K_CLOSE']}):␣
 ↪prec={pr1:.4f} | rec={rc1:.4f} | dice={dc1:.4f}")

print(f"\n[FINAL] steps_per_epoch={steps_per_epoch} | val_steps={val_steps} |␣
 ↪RECOMMENDED_THR={RECOMMENDED_THR:.3f}")
```

```
[OK] Model inputs = 3
[OK] Warmup forward pass réussi avec arité = 3
Epoch 1/50
162/162             38s 48ms/step -
dice: 0.4296 - iou@0.5: 0.5200 - loss: 0.5572 - prec@0.5: 0.4141 - prec@0.95:
0.5839 - rec@0.5: 0.6240 - val_dice: 0.1258 - val_iou@0.5: 0.3896 - val_loss:
0.7966 - val_prec@0.5: 0.7369 - val_prec@0.95: 0.0000e+00 - val_rec@0.5:
8.3636e-04 - learning_rate: 0.0010
Epoch 2/50
162/162             5s 32ms/step -
dice: 0.5309 - iou@0.5: 0.6016 - loss: 0.4475 - prec@0.5: 0.5318 - prec@0.95:
0.7660 - rec@0.5: 0.6573 - val_dice: 0.4313 - val_iou@0.5: 0.5522 - val_loss:
0.5446 - val_prec@0.5: 0.6451 - val_prec@0.95: 0.8440 - val_rec@0.5: 0.3676 -
learning_rate: 0.0010
Epoch 3/50
162/162             5s 32ms/step -
dice: 0.5676 - iou@0.5: 0.6276 - loss: 0.4140 - prec@0.5: 0.5755 - prec@0.95:
0.7984 - rec@0.5: 0.6727 - val_dice: 0.5460 - val_iou@0.5: 0.6175 - val_loss:
0.4317 - val_prec@0.5: 0.6225 - val_prec@0.95: 0.8444 - val_rec@0.5: 0.5837 -
learning_rate: 0.0010
Epoch 4/50
162/162             5s 32ms/step -
dice: 0.5850 - iou@0.5: 0.6425 - loss: 0.3980 - prec@0.5: 0.5831 - prec@0.95:
0.7982 - rec@0.5: 0.7042 - val_dice: 0.5623 - val_iou@0.5: 0.6308 - val_loss:
0.4510 - val_prec@0.5: 0.6096 - val_prec@0.95: 0.7194 - val_rec@0.5: 0.6518 -
```

```
learning_rate: 0.0010
Epoch 5/50
162/162            5s 32ms/step -
dice: 0.5957 - iou@0.5: 0.6459 - loss: 0.3905 - prec@0.5: 0.5955 - prec@0.95:
0.7836 - rec@0.5: 0.6903 - val_dice: 0.5996 - val_iou@0.5: 0.6433 - val_loss:
0.3876 - val_prec@0.5: 0.6161 - val_prec@0.95: 0.8248 - val_rec@0.5: 0.6857 -
learning_rate: 0.0010
Epoch 6/50
162/162            5s 32ms/step -
dice: 0.6000 - iou@0.5: 0.6409 - loss: 0.3860 - prec@0.5: 0.5886 - prec@0.95:
0.8290 - rec@0.5: 0.7066 - val_dice: 0.6144 - val_iou@0.5: 0.6462 - val_loss:
0.3825 - val_prec@0.5: 0.6312 - val_prec@0.95: 0.8124 - val_rec@0.5: 0.6677 -
learning_rate: 0.0010
Epoch 7/50
162/162            5s 32ms/step -
dice: 0.6160 - iou@0.5: 0.6544 - loss: 0.3739 - prec@0.5: 0.6097 - prec@0.95:
0.8237 - rec@0.5: 0.7087 - val_dice: 0.6135 - val_iou@0.5: 0.6368 - val_loss:
0.3879 - val_prec@0.5: 0.5890 - val_prec@0.95: 0.8015 - val_rec@0.5: 0.7216 -
learning_rate: 0.0010
Epoch 8/50
162/162            5s 32ms/step -
dice: 0.6120 - iou@0.5: 0.6528 - loss: 0.3747 - prec@0.5: 0.5909 - prec@0.95:
0.8323 - rec@0.5: 0.7401 - val_dice: 0.6193 - val_iou@0.5: 0.6450 - val_loss:
0.3796 - val_prec@0.5: 0.6184 - val_prec@0.95: 0.8536 - val_rec@0.5: 0.6874 -
learning_rate: 0.0010
Epoch 9/50
162/162            5s 33ms/step -
dice: 0.6260 - iou@0.5: 0.6640 - loss: 0.3595 - prec@0.5: 0.6211 - prec@0.95:
0.8391 - rec@0.5: 0.7166 - val_dice: 0.6294 - val_iou@0.5: 0.6529 - val_loss:
0.3640 - val_prec@0.5: 0.6192 - val_prec@0.95: 0.8733 - val_rec@0.5: 0.7150 -
learning_rate: 0.0010
Epoch 10/50
162/162            5s 32ms/step -
dice: 0.6179 - iou@0.5: 0.6465 - loss: 0.3697 - prec@0.5: 0.5911 - prec@0.95:
0.8513 - rec@0.5: 0.7333 - val_dice: 0.6413 - val_iou@0.5: 0.6541 - val_loss:
0.3601 - val_prec@0.5: 0.6178 - val_prec@0.95: 0.8561 - val_rec@0.5: 0.7226 -
learning_rate: 0.0010
Epoch 11/50
162/162            5s 32ms/step -
dice: 0.6292 - iou@0.5: 0.6618 - loss: 0.3574 - prec@0.5: 0.6127 - prec@0.95:
0.8509 - rec@0.5: 0.7275 - val_dice: 0.6415 - val_iou@0.5: 0.6531 - val_loss:
0.3610 - val_prec@0.5: 0.6096 - val_prec@0.95: 0.8787 - val_rec@0.5: 0.7375 -
learning_rate: 0.0010
Epoch 12/50
162/162            5s 32ms/step -
dice: 0.6363 - iou@0.5: 0.6665 - loss: 0.3497 - prec@0.5: 0.6168 - prec@0.95:
0.8623 - rec@0.5: 0.7309 - val_dice: 0.6481 - val_iou@0.5: 0.6544 - val_loss:
0.3585 - val_prec@0.5: 0.6102 - val_prec@0.95: 0.8795 - val_rec@0.5: 0.7410 -
```

```
learning_rate: 0.0010
Epoch 13/50
162/162            5s 32ms/step -
dice: 0.6404 - iou@0.5: 0.6693 - loss: 0.3464 - prec@0.5: 0.6243 - prec@0.95:
0.8731 - rec@0.5: 0.7367 - val_dice: 0.6638 - val_iou@0.5: 0.6592 - val_loss:
0.3494 - val_prec@0.5: 0.6315 - val_prec@0.95: 0.8594 - val_rec@0.5: 0.7128 -
learning_rate: 0.0010
Epoch 14/50
162/162            5s 32ms/step -
dice: 0.6419 - iou@0.5: 0.6784 - loss: 0.3445 - prec@0.5: 0.6316 - prec@0.95:
0.8587 - rec@0.5: 0.7401 - val_dice: 0.6562 - val_iou@0.5: 0.6619 - val_loss:
0.3466 - val_prec@0.5: 0.6244 - val_prec@0.95: 0.8770 - val_rec@0.5: 0.7377 -
learning_rate: 0.0010
Epoch 15/50
162/162            5s 32ms/step -
dice: 0.6482 - iou@0.5: 0.6736 - loss: 0.3413 - prec@0.5: 0.6198 - prec@0.95:
0.8687 - rec@0.5: 0.7651 - val_dice: 0.6637 - val_iou@0.5: 0.6646 - val_loss:
0.3376 - val_prec@0.5: 0.6545 - val_prec@0.95: 0.8972 - val_rec@0.5: 0.6896 -
learning_rate: 0.0010
Epoch 16/50
162/162            5s 32ms/step -
dice: 0.6542 - iou@0.5: 0.6800 - loss: 0.3328 - prec@0.5: 0.6373 - prec@0.95:
0.8828 - rec@0.5: 0.7413 - val_dice: 0.6621 - val_iou@0.5: 0.6649 - val_loss:
0.3358 - val_prec@0.5: 0.6481 - val_prec@0.95: 0.9001 - val_rec@0.5: 0.7017 -
learning_rate: 0.0010
Epoch 17/50
162/162            5s 32ms/step -
dice: 0.6514 - iou@0.5: 0.6759 - loss: 0.3383 - prec@0.5: 0.6361 - prec@0.95:
0.8807 - rec@0.5: 0.7348 - val_dice: 0.6743 - val_iou@0.5: 0.6707 - val_loss:
0.3343 - val_prec@0.5: 0.6402 - val_prec@0.95: 0.8616 - val_rec@0.5: 0.7365 -
learning_rate: 0.0010
Epoch 18/50
162/162            5s 32ms/step -
dice: 0.6644 - iou@0.5: 0.6919 - loss: 0.3255 - prec@0.5: 0.6485 - prec@0.95:
0.8794 - rec@0.5: 0.7628 - val_dice: 0.6729 - val_iou@0.5: 0.6779 - val_loss:
0.3210 - val_prec@0.5: 0.6107 - val_prec@0.95: 0.9101 - val_rec@0.5: 0.8361 -
learning_rate: 0.0010
Epoch 19/50
162/162            5s 32ms/step -
dice: 0.6709 - iou@0.5: 0.6912 - loss: 0.3191 - prec@0.5: 0.6334 - prec@0.95:
0.8839 - rec@0.5: 0.7922 - val_dice: 0.6611 - val_iou@0.5: 0.6647 - val_loss:
0.3353 - val_prec@0.5: 0.5972 - val_prec@0.95: 0.9015 - val_rec@0.5: 0.8197 -
learning_rate: 0.0010
Epoch 20/50
162/162            5s 32ms/step -
dice: 0.6633 - iou@0.5: 0.6858 - loss: 0.3248 - prec@0.5: 0.6163 - prec@0.95:
0.8843 - rec@0.5: 0.8179 - val_dice: 0.6810 - val_iou@0.5: 0.6691 - val_loss:
0.3342 - val_prec@0.5: 0.6399 - val_prec@0.95: 0.8420 - val_rec@0.5: 0.7312 -
```

```
learning_rate: 0.0010
Epoch 21/50
161/162            0s 29ms/step -
dice: 0.6689 - iou@0.5: 0.6866 - loss: 0.3196 - prec@0.5: 0.6332 - prec@0.95:
0.8911 - rec@0.5: 0.7785
Epoch 21: ReduceLROnPlateau reducing learning rate to 0.0005000000237487257.
162/162            5s 32ms/step -
dice: 0.6689 - iou@0.5: 0.6867 - loss: 0.3196 - prec@0.5: 0.6332 - prec@0.95:
0.8911 - rec@0.5: 0.7786 - val_dice: 0.6552 - val_iou@0.5: 0.6613 - val_loss:
0.3558 - val_prec@0.5: 0.6428 - val_prec@0.95: 0.9274 - val_rec@0.5: 0.6991 -
learning_rate: 0.0010
Epoch 22/50
162/162            5s 32ms/step -
dice: 0.6767 - iou@0.5: 0.6857 - loss: 0.3141 - prec@0.5: 0.6307 - prec@0.95:
0.8967 - rec@0.5: 0.7798 - val_dice: 0.7023 - val_iou@0.5: 0.6772 - val_loss:
0.3188 - val_prec@0.5: 0.6607 - val_prec@0.95: 0.8611 - val_rec@0.5: 0.7200 -
learning_rate: 5.0000e-04
Epoch 23/50
162/162            5s 32ms/step -
dice: 0.6939 - iou@0.5: 0.7042 - loss: 0.2987 - prec@0.5: 0.6511 - prec@0.95:
0.8947 - rec@0.5: 0.8035 - val_dice: 0.6988 - val_iou@0.5: 0.6873 - val_loss:
0.3093 - val_prec@0.5: 0.6265 - val_prec@0.95: 0.8805 - val_rec@0.5: 0.8297 -
learning_rate: 5.0000e-04
Epoch 24/50
162/162            5s 32ms/step -
dice: 0.6975 - iou@0.5: 0.7189 - loss: 0.2942 - prec@0.5: 0.6682 - prec@0.95:
0.8679 - rec@0.5: 0.8148 - val_dice: 0.6859 - val_iou@0.5: 0.6749 - val_loss:
0.3259 - val_prec@0.5: 0.6100 - val_prec@0.95: 0.8456 - val_rec@0.5: 0.8254 -
learning_rate: 5.0000e-04
Epoch 25/50
162/162            5s 32ms/step -
dice: 0.6875 - iou@0.5: 0.7021 - loss: 0.3027 - prec@0.5: 0.6399 - prec@0.95:
0.8973 - rec@0.5: 0.8250 - val_dice: 0.7061 - val_iou@0.5: 0.6909 - val_loss:
0.3094 - val_prec@0.5: 0.6496 - val_prec@0.95: 0.8601 - val_rec@0.5: 0.7879 -
learning_rate: 5.0000e-04
Epoch 26/50
161/162            0s 29ms/step -
dice: 0.7073 - iou@0.5: 0.7254 - loss: 0.2840 - prec@0.5: 0.6743 - prec@0.95:
0.8844 - rec@0.5: 0.8232
Epoch 26: ReduceLROnPlateau reducing learning rate to 0.0002502000005170703.
162/162            5s 32ms/step -
dice: 0.7072 - iou@0.5: 0.7253 - loss: 0.2841 - prec@0.5: 0.6741 - prec@0.95:
0.8844 - rec@0.5: 0.8233 - val_dice: 0.6994 - val_iou@0.5: 0.6852 - val_loss:
0.3187 - val_prec@0.5: 0.6628 - val_prec@0.95: 0.8573 - val_rec@0.5: 0.7424 -
learning_rate: 5.0000e-04
Epoch 27/50
162/162            5s 32ms/step -
dice: 0.7052 - iou@0.5: 0.7189 - loss: 0.2846 - prec@0.5: 0.6517 - prec@0.95:
```

```
0.8947 - rec@0.5: 0.8494 - val_dice: 0.7032 - val_iou@0.5: 0.6890 - val_loss:
0.3095 - val_prec@0.5: 0.6447 - val_prec@0.95: 0.8834 - val_rec@0.5: 0.7919 -
learning_rate: 2.5000e-04
Epoch 28/50
162/162            5s 32ms/step -
dice: 0.7202 - iou@0.5: 0.7283 - loss: 0.2732 - prec@0.5: 0.6677 - prec@0.95:
0.8909 - rec@0.5: 0.8571 - val_dice: 0.7034 - val_iou@0.5: 0.6888 - val_loss:
0.3196 - val_prec@0.5: 0.6744 - val_prec@0.95: 0.8650 - val_rec@0.5: 0.7337 -
learning_rate: 2.5000e-04
Epoch 29/50
161/162            0s 29ms/step -
dice: 0.7260 - iou@0.5: 0.7379 - loss: 0.2658 - prec@0.5: 0.6788 - prec@0.95:
0.8860 - rec@0.5: 0.8520
Epoch 29: ReduceLROnPlateau reducing learning rate to 0.0001250000059371814.
162/162            5s 32ms/step -
dice: 0.7260 - iou@0.5: 0.7378 - loss: 0.2659 - prec@0.5: 0.6787 - prec@0.95:
0.8860 - rec@0.5: 0.8519 - val_dice: 0.7083 - val_iou@0.5: 0.6899 - val_loss:
0.3209 - val_prec@0.5: 0.6706 - val_prec@0.95: 0.8189 - val_rec@0.5: 0.7439 -
learning_rate: 2.5000e-04
Epoch 30/50
162/162            5s 32ms/step -
dice: 0.7407 - iou@0.5: 0.7529 - loss: 0.2495 - prec@0.5: 0.6947 - prec@0.95:
0.8973 - rec@0.5: 0.8677 - val_dice: 0.7083 - val_iou@0.5: 0.6910 - val_loss:
0.3105 - val_prec@0.5: 0.6527 - val_prec@0.95: 0.8648 - val_rec@0.5: 0.7815 -
learning_rate: 1.2500e-04
Epoch 31/50
162/162            5s 32ms/step -
dice: 0.7439 - iou@0.5: 0.7534 - loss: 0.2488 - prec@0.5: 0.7018 - prec@0.95:
0.8953 - rec@0.5: 0.8609 - val_dice: 0.7113 - val_iou@0.5: 0.6937 - val_loss:
0.3112 - val_prec@0.5: 0.6904 - val_prec@0.95: 0.8520 - val_rec@0.5: 0.7232 -
learning_rate: 1.2500e-04
Epoch 32/50
161/162            0s 29ms/step -
dice: 0.7583 - iou@0.5: 0.7689 - loss: 0.2327 - prec@0.5: 0.7322 - prec@0.95:
0.9016 - rec@0.5: 0.8465
Epoch 32: ReduceLROnPlateau reducing learning rate to 6.25000029685907e-05.
162/162            5s 32ms/step -
dice: 0.7582 - iou@0.5: 0.7688 - loss: 0.2328 - prec@0.5: 0.7321 - prec@0.95:
0.9015 - rec@0.5: 0.8464 - val_dice: 0.7147 - val_iou@0.5: 0.6967 - val_loss:
0.3100 - val_prec@0.5: 0.6785 - val_prec@0.95: 0.8499 - val_rec@0.5: 0.7514 -
learning_rate: 1.2500e-04
Epoch 33/50
162/162            5s 32ms/step -
dice: 0.7489 - iou@0.5: 0.7611 - loss: 0.2430 - prec@0.5: 0.7255 - prec@0.95:
0.8999 - rec@0.5: 0.8434 - val_dice: 0.7175 - val_iou@0.5: 0.6974 - val_loss:
0.3180 - val_prec@0.5: 0.6882 - val_prec@0.95: 0.8214 - val_rec@0.5: 0.7376 -
learning_rate: 6.2500e-05
Epoch 34/50
```

```
162/162            5s 34ms/step -
dice: 0.7681 - iou@0.5: 0.7801 - loss: 0.2252 - prec@0.5: 0.7537 - prec@0.95:
0.8981 - rec@0.5: 0.8450 - val_dice: 0.7182 - val_iou@0.5: 0.7011 - val_loss:
0.3147 - val_prec@0.5: 0.7002 - val_prec@0.95: 0.8294 - val_rec@0.5: 0.7301 -
learning_rate: 6.2500e-05
Epoch 35/50
161/162            0s 29ms/step -
dice: 0.7698 - iou@0.5: 0.7803 - loss: 0.2242 - prec@0.5: 0.7592 - prec@0.95:
0.9017 - rec@0.5: 0.8410
Epoch 35: ReduceLROnPlateau reducing learning rate to 3.125000148429535e-05.
162/162            5s 32ms/step -
dice: 0.7698 - iou@0.5: 0.7804 - loss: 0.2242 - prec@0.5: 0.7593 - prec@0.95:
0.9018 - rec@0.5: 0.8411 - val_dice: 0.7167 - val_iou@0.5: 0.6962 - val_loss:
0.3259 - val_prec@0.5: 0.7023 - val_prec@0.95: 0.8136 - val_rec@0.5: 0.7129 -
learning_rate: 6.2500e-05
Epoch 36/50
162/162            5s 32ms/step -
dice: 0.7855 - iou@0.5: 0.7987 - loss: 0.2088 - prec@0.5: 0.7894 - prec@0.95:
0.9116 - rec@0.5: 0.8462 - val_dice: 0.7183 - val_iou@0.5: 0.6960 - val_loss:
0.3291 - val_prec@0.5: 0.7001 - val_prec@0.95: 0.8056 - val_rec@0.5: 0.7154 -
learning_rate: 3.1250e-05
Epoch 37/50
162/162            5s 32ms/step -
dice: 0.7807 - iou@0.5: 0.7914 - loss: 0.2154 - prec@0.5: 0.7808 - prec@0.95:
0.9038 - rec@0.5: 0.8423 - val_dice: 0.7184 - val_iou@0.5: 0.6960 - val_loss:
0.3292 - val_prec@0.5: 0.7035 - val_prec@0.95: 0.8041 - val_rec@0.5: 0.7106 -
learning_rate: 3.1250e-05
Epoch 38/50
161/162            0s 29ms/step -
dice: 0.7878 - iou@0.5: 0.8019 - loss: 0.2071 - prec@0.5: 0.7937 - prec@0.95:
0.9070 - rec@0.5: 0.8449
Epoch 38: ReduceLROnPlateau reducing learning rate to 1.5625000742147677e-05.
162/162            5s 32ms/step -
dice: 0.7878 - iou@0.5: 0.8019 - loss: 0.2071 - prec@0.5: 0.7937 - prec@0.95:
0.9071 - rec@0.5: 0.8450 - val_dice: 0.7177 - val_iou@0.5: 0.6943 - val_loss:
0.3335 - val_prec@0.5: 0.7003 - val_prec@0.95: 0.8004 - val_rec@0.5: 0.7103 -
learning_rate: 3.1250e-05
Epoch 39/50
162/162            5s 32ms/step -
dice: 0.7889 - iou@0.5: 0.8004 - loss: 0.2071 - prec@0.5: 0.7888 - prec@0.95:
0.9094 - rec@0.5: 0.8518 - val_dice: 0.7174 - val_iou@0.5: 0.6930 - val_loss:
0.3370 - val_prec@0.5: 0.7008 - val_prec@0.95: 0.7968 - val_rec@0.5: 0.7060 -
learning_rate: 1.5625e-05
Epoch 40/50
162/162            5s 32ms/step -
dice: 0.7971 - iou@0.5: 0.8093 - loss: 0.1995 - prec@0.5: 0.8052 - prec@0.95:
0.9151 - rec@0.5: 0.8524 - val_dice: 0.7174 - val_iou@0.5: 0.6948 - val_loss:
0.3343 - val_prec@0.5: 0.7063 - val_prec@0.95: 0.8017 - val_rec@0.5: 0.7033 -
```

```
learning_rate: 1.5625e-05
Epoch 41/50
161/162            0s 29ms/step -
dice: 0.8048 - iou@0.5: 0.8169 - loss: 0.1907 - prec@0.5: 0.8122 - prec@0.95:
0.9217 - rec@0.5: 0.8585
Epoch 41: ReduceLROnPlateau reducing learning rate to 7.812500371073838e-06.
162/162            5s 32ms/step -
dice: 0.8047 - iou@0.5: 0.8168 - loss: 0.1908 - prec@0.5: 0.8121 - prec@0.95:
0.9216 - rec@0.5: 0.8584 - val_dice: 0.7171 - val_iou@0.5: 0.6927 - val_loss:
0.3388 - val_prec@0.5: 0.7015 - val_prec@0.95: 0.7964 - val_rec@0.5: 0.7040 -
learning_rate: 1.5625e-05
Epoch 42/50
162/162            5s 32ms/step -
dice: 0.8037 - iou@0.5: 0.8154 - loss: 0.1907 - prec@0.5: 0.8071 - prec@0.95:
0.9172 - rec@0.5: 0.8584 - val_dice: 0.7167 - val_iou@0.5: 0.6928 - val_loss:
0.3386 - val_prec@0.5: 0.7033 - val_prec@0.95: 0.7968 - val_rec@0.5: 0.7020 -
learning_rate: 7.8125e-06
Epoch 43/50
162/162            5s 32ms/step -
dice: 0.8048 - iou@0.5: 0.8168 - loss: 0.1911 - prec@0.5: 0.8136 - prec@0.95:
0.9195 - rec@0.5: 0.8568 - val_dice: 0.7166 - val_iou@0.5: 0.6918 - val_loss:
0.3415 - val_prec@0.5: 0.7004 - val_prec@0.95: 0.7921 - val_rec@0.5: 0.7031 -
learning_rate: 7.8125e-06
Epoch 44/50
161/162            0s 29ms/step -
dice: 0.8026 - iou@0.5: 0.8151 - loss: 0.1937 - prec@0.5: 0.8100 - prec@0.95:
0.9185 - rec@0.5: 0.8596
Epoch 44: ReduceLROnPlateau reducing learning rate to 3.906250185536919e-06.
162/162            5s 32ms/step -
dice: 0.8026 - iou@0.5: 0.8151 - loss: 0.1937 - prec@0.5: 0.8100 - prec@0.95:
0.9185 - rec@0.5: 0.8596 - val_dice: 0.7158 - val_iou@0.5: 0.6900 - val_loss:
0.3445 - val_prec@0.5: 0.6996 - val_prec@0.95: 0.7895 - val_rec@0.5: 0.6990 -
learning_rate: 7.8125e-06
Epoch 45/50
162/162            5s 32ms/step -
dice: 0.8047 - iou@0.5: 0.8156 - loss: 0.1907 - prec@0.5: 0.8098 - prec@0.95:
0.9172 - rec@0.5: 0.8563 - val_dice: 0.7162 - val_iou@0.5: 0.6910 - val_loss:
0.3425 - val_prec@0.5: 0.7007 - val_prec@0.95: 0.7917 - val_rec@0.5: 0.7003 -
learning_rate: 3.9063e-06
Epoch 46/50
162/162            5s 32ms/step -
dice: 0.8044 - iou@0.5: 0.8157 - loss: 0.1927 - prec@0.5: 0.8128 - prec@0.95:
0.9204 - rec@0.5: 0.8577 - val_dice: 0.7162 - val_iou@0.5: 0.6908 - val_loss:
0.3439 - val_prec@0.5: 0.7011 - val_prec@0.95: 0.7903 - val_rec@0.5: 0.6993 -
learning_rate: 3.9063e-06
Epoch 47/50
161/162            0s 29ms/step -
dice: 0.8012 - iou@0.5: 0.8115 - loss: 0.1954 - prec@0.5: 0.8060 - prec@0.95:
```

```
0.9154 - rec@0.5: 0.8544
Epoch 47: ReduceLROnPlateau reducing learning rate to 1.9531250927684596e-06.
162/162            5s 32ms/step -
dice: 0.8012 - iou@0.5: 0.8115 - loss: 0.1953 - prec@0.5: 0.8061 - prec@0.95:
0.9154 - rec@0.5: 0.8545 - val_dice: 0.7160 - val_iou@0.5: 0.6907 - val_loss:
0.3444 - val_prec@0.5: 0.7015 - val_prec@0.95: 0.7901 - val_rec@0.5: 0.6984 -
learning_rate: 3.9063e-06
Epoch 47: early stopping
Restoring model weights from the end of the best epoch: 37.
Epoch 1/25
162/162            22s 46ms/step -
dice: 0.7767 - iou@0.5: 0.7927 - loss: 0.1055 - prec@0.5: 0.8741 - prec@0.95:
0.9503 - rec@0.5: 0.7433 - val_dice: 0.6795 - val_iou@0.5: 0.6626 - val_loss:
0.1936 - val_prec@0.5: 0.7528 - val_prec@0.95: 0.7998 - val_rec@0.5: 0.5720 -
learning_rate: 3.0000e-05
Epoch 2/25
162/162            5s 30ms/step -
dice: 0.7774 - iou@0.5: 0.7837 - loss: 0.0934 - prec@0.5: 0.8945 - prec@0.95:
0.9401 - rec@0.5: 0.7094 - val_dice: 0.6780 - val_iou@0.5: 0.6621 - val_loss:
0.1927 - val_prec@0.5: 0.7617 - val_prec@0.95: 0.8019 - val_rec@0.5: 0.5640 -
learning_rate: 3.0000e-05
Epoch 3/25
162/162            5s 29ms/step -
dice: 0.7750 - iou@0.5: 0.7806 - loss: 0.0941 - prec@0.5: 0.8971 - prec@0.95:
0.9381 - rec@0.5: 0.7020 - val_dice: 0.6770 - val_iou@0.5: 0.6614 - val_loss:
0.1957 - val_prec@0.5: 0.7509 - val_prec@0.95: 0.7830 - val_rec@0.5: 0.5705 -
learning_rate: 3.0000e-05
Epoch 4/25
161/162            0s 27ms/step -
dice: 0.7847 - iou@0.5: 0.7871 - loss: 0.0873 - prec@0.5: 0.9037 - prec@0.95:
0.9416 - rec@0.5: 0.7103
Epoch 4: ReduceLROnPlateau reducing learning rate to 1.4999999621068127e-05.
162/162            5s 30ms/step -
dice: 0.7847 - iou@0.5: 0.7871 - loss: 0.0873 - prec@0.5: 0.9037 - prec@0.95:
0.9416 - rec@0.5: 0.7104 - val_dice: 0.6807 - val_iou@0.5: 0.6647 - val_loss:
0.1906 - val_prec@0.5: 0.7613 - val_prec@0.95: 0.7971 - val_rec@0.5: 0.5702 -
learning_rate: 3.0000e-05
Epoch 5/25
162/162            5s 29ms/step -
dice: 0.7901 - iou@0.5: 0.7926 - loss: 0.0838 - prec@0.5: 0.9084 - prec@0.95:
0.9438 - rec@0.5: 0.7168 - val_dice: 0.6776 - val_iou@0.5: 0.6633 - val_loss:
0.1901 - val_prec@0.5: 0.7645 - val_prec@0.95: 0.7995 - val_rec@0.5: 0.5645 -
learning_rate: 1.5000e-05
Epoch 6/25
161/162            0s 27ms/step -
dice: 0.7907 - iou@0.5: 0.7926 - loss: 0.0839 - prec@0.5: 0.9084 - prec@0.95:
0.9428 - rec@0.5: 0.7171
Epoch 6: ReduceLROnPlateau reducing learning rate to 7.499999810534064e-06.
```

```
162/162            5s 30ms/step -
dice: 0.7907 - iou@0.5: 0.7926 - loss: 0.0838 - prec@0.5: 0.9084 - prec@0.95:
0.9428 - rec@0.5: 0.7171 - val_dice: 0.6799 - val_iou@0.5: 0.6642 - val_loss:
0.1918 - val_prec@0.5: 0.7583 - val_prec@0.95: 0.7907 - val_rec@0.5: 0.5715 -
learning_rate: 1.5000e-05
Epoch 7/25
162/162            5s 29ms/step -
dice: 0.7925 - iou@0.5: 0.7935 - loss: 0.0839 - prec@0.5: 0.9077 - prec@0.95:
0.9427 - rec@0.5: 0.7197 - val_dice: 0.6787 - val_iou@0.5: 0.6621 - val_loss:
0.1941 - val_prec@0.5: 0.7547 - val_prec@0.95: 0.7866 - val_rec@0.5: 0.5692 -
learning_rate: 7.5000e-06
Epoch 8/25
161/162            0s 26ms/step -
dice: 0.7943 - iou@0.5: 0.7962 - loss: 0.0810 - prec@0.5: 0.9127 - prec@0.95:
0.9465 - rec@0.5: 0.7204
Epoch 8: ReduceLROnPlateau reducing learning rate to 3.749999905267032e-06.
162/162            5s 29ms/step -
dice: 0.7943 - iou@0.5: 0.7962 - loss: 0.0810 - prec@0.5: 0.9127 - prec@0.95:
0.9465 - rec@0.5: 0.7204 - val_dice: 0.6783 - val_iou@0.5: 0.6616 - val_loss:
0.1953 - val_prec@0.5: 0.7512 - val_prec@0.95: 0.7815 - val_rec@0.5: 0.5707 -
learning_rate: 7.5000e-06
Epoch 9/25
162/162            5s 29ms/step -
dice: 0.7967 - iou@0.5: 0.7992 - loss: 0.0789 - prec@0.5: 0.9140 - prec@0.95:
0.9474 - rec@0.5: 0.7241 - val_dice: 0.6782 - val_iou@0.5: 0.6622 - val_loss:
0.1927 - val_prec@0.5: 0.7589 - val_prec@0.95: 0.7915 - val_rec@0.5: 0.5662 -
learning_rate: 3.7500e-06
Epoch 10/25
161/162            0s 26ms/step -
dice: 0.7966 - iou@0.5: 0.7972 - loss: 0.0790 - prec@0.5: 0.9152 - prec@0.95:
0.9480 - rec@0.5: 0.7233
Epoch 10: ReduceLROnPlateau reducing learning rate to 1.874999952633516e-06.
162/162            5s 29ms/step -
dice: 0.7966 - iou@0.5: 0.7972 - loss: 0.0790 - prec@0.5: 0.9152 - prec@0.95:
0.9480 - rec@0.5: 0.7233 - val_dice: 0.6770 - val_iou@0.5: 0.6611 - val_loss:
0.1942 - val_prec@0.5: 0.7559 - val_prec@0.95: 0.7869 - val_rec@0.5: 0.5661 -
learning_rate: 3.7500e-06
Epoch 10: early stopping
Restoring model weights from the end of the best epoch: 2.
39/39             0s 11ms/step -
dice: 0.7259 - iou@0.5: 0.7309 - loss: 0.1427 - prec@0.5: 0.8133 - prec@0.95:
0.8603 - rec@0.5: 0.6323

[VAL RESULTS]
 - dice: 0.678047
 - iou@0.5: 0.662110
 - loss: 0.192656
 - prec@0.5: 0.761657
```

```
 - prec@0.95: 0.801913
 - rec@0.5: 0.563954

[CALIB OK] Objectif=dice sous contrainte precision>=0.90
[CALIB RECOMMENDED] thr=0.998 | prec=0.9018 | rec=0.1629 | dice=0.2759

[FINAL] steps_per_epoch=162 | val_steps=39 | RECOMMENDED_THR=0.998
```

Le script confirme d'abord que tout le pipeline est sain (modèle **à 3 entrées**, warmup OK, datasets stables avec **162** steps/epoch et **39** steps val), puis lance deux phases : en **Phase 1 (BCE+Dice, LR=1e-3)** le modèle apprend vite (val_dice passe de ~0.13 à ~0.70+), avec des réductions de LR successives et un **early stopping** qui restaure le meilleur état (epoch **37**), signe qu'au-delà les gains en validation stagnent; en **Phase 2 (finetune "precision-first", BN gelées, Focal-Tversky+BCE, LR=3e-5)** la précision monte (notamment sur seuil strict), mais le **recall baisse** et le **dice validation** reste globalement autour de ~0.68, donc pas de progrès net sur la segmentation globale; l'évaluation finale confirme ce profil (dice **0.678**, prec@0.5 **0.762**, rec@0.5 **0.564**, prec@0.95 **0.802**) et la calibration sous contrainte **precision** **0.90** force un seuil extrême (**thr=0.998**) qui atteint la précision visée mais fait chuter le recall (**0.163**) et le dice (**0.276**), ce qui indique que ce seuil est cohérent pour "éviter les faux positifs" mais trop sévère si ton objectif est de récupérer correctement les joints/structures pour les mesures (transitions/grains).

[ ]:
```python
import os, numpy as np, tensorflow as tf
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau,␣
 ↪CSVLogger, TerminateOnNaN, ModelCheckpoint

assert "unet_model" in globals(), "unet_model introuvable."
assert "train_fit_ds" in globals() and "val_fit_ds" in globals(), "train_fit_ds/
 ↪val_fit_ds introuvables."
assert "steps_per_epoch" in globals() and "val_steps" in globals(),␣
 ↪"steps_per_epoch/val_steps introuvables."

# ------------------------------
# 1) Overfitting control: BN freeze (souvent très efficace en finetune)
# ------------------------------
def freeze_batchnorm(m):
    for layer in m.layers:
        if isinstance(layer, tf.keras.layers.BatchNormalization):
            layer.trainable = False

freeze_batchnorm(unet_model)

# ------------------------------
# 2) Loss precision-friendly + stabilité
#    (tu peux garder celle que tu as déjà : FocalTverskyPlusBCE)
# ------------------------------
@tf.keras.utils.register_keras_serializable()
class FocalTverskyPlusBCE(tf.keras.losses.Loss):
```

```
    def __init__(self, alpha=0.80, gamma=1.25, bce_weight=0.08, smooth=1e-6,␣
 ↪name="focal_tversky_plus_bce"):
        super().__init__(name=name)
        self.alpha = float(alpha)
        self.beta  = float(1.0 - alpha)
        self.gamma = float(gamma)
        self.bce_weight = float(bce_weight)
        self.smooth = float(smooth)
        self._bce = tf.keras.losses.BinaryCrossentropy(from_logits=False)

    def call(self, y_true, y_pred):
        y_true = tf.cast(y_true, tf.float32)
        y_pred = tf.cast(tf.clip_by_value(y_pred, 1e-6, 1.0 - 1e-6), tf.float32)

        yt = tf.reshape(y_true, [-1])
        yp = tf.reshape(y_pred, [-1])

        tp = tf.reduce_sum(yt * yp)
        fp = tf.reduce_sum((1.0 - yt) * yp)
        fn = tf.reduce_sum(yt * (1.0 - yp))

        tversky = (tp + self.smooth) / (tp + self.alpha * fp + self.beta * fn +␣
 ↪self.smooth)
        focal_tversky = tf.pow((1.0 - tversky), self.gamma)
        return (1.0 - self.bce_weight) * focal_tversky + self.bce_weight * self.
 ↪_bce(y_true, y_pred)

# dice metric (si tu as déjà dice_coef, il sera utilisé)
def soft_dice(y_true, y_pred, smooth=1.0):
    y_true = tf.cast(y_true, tf.float32)
    y_pred = tf.cast(tf.clip_by_value(y_pred, 0.0, 1.0), tf.float32)
    yt = tf.reshape(y_true, [-1])
    yp = tf.reshape(y_pred, [-1])
    inter = tf.reduce_sum(yt * yp)
    denom = tf.reduce_sum(yt) + tf.reduce_sum(yp)
    return (2.0 * inter + smooth) / (denom + smooth)

dice_fn = globals().get("dice_coef", soft_dice)

# ------------------------------
# 3) Metrics (clarté=acc + précision)
# ------------------------------
thr_main   = 0.50
thr_strict = 0.95  # tu peux mettre 0.90 si 0.95 est trop dur

metrics_ft = [
    tf.keras.metrics.MeanMetricWrapper(fn=dice_fn, name="dice"),
```

```
    tf.keras.metrics.BinaryAccuracy(name=f"acc@{thr_main:.2f}",␣
 ↪threshold=thr_main),
    tf.keras.metrics.Precision(name=f"prec@{thr_main:.2f}",␣
 ↪thresholds=thr_main),
    tf.keras.metrics.Recall(name=f"rec@{thr_main:.2f}", thresholds=thr_main),
    tf.keras.metrics.Precision(name=f"prec@{thr_strict:.2f}",␣
 ↪thresholds=thr_strict),
]

# ------------------------------
# 4) Optimizer + LR schedule (finetune = LR petit + decay)
# ------------------------------
base_lr = 2e-5
try:
    opt = tf.keras.optimizers.AdamW(learning_rate=base_lr, weight_decay=1e-4,␣
 ↪clipnorm=1.0)
except Exception:
    opt = tf.keras.optimizers.Adam(learning_rate=base_lr, clipnorm=1.0)

loss_ft = FocalTverskyPlusBCE(alpha=0.80, gamma=1.25, bce_weight=0.08)

# ------------------------------
# 5) Callback: stop quand "clarté 98%" + précision OK, pour éviter overfit
# ------------------------------
class StopWhenGood(tf.keras.callbacks.Callback):
    """
    Stop si conditions atteintes pendant 'patience_ok' epochs consécutifs.
    """
    def __init__(self, acc_target=0.98, prec_target=0.85, dice_target=0.68,␣
 ↪patience_ok=2, monitor_prefix="val"):
        super().__init__()
        self.acc_target = float(acc_target)
        self.prec_target = float(prec_target)
        self.dice_target = float(dice_target)
        self.patience_ok = int(patience_ok)
        self.monitor_prefix = monitor_prefix
        self._ok_count = 0

    def on_epoch_end(self, epoch, logs=None):
        logs = logs or {}
        acc  = logs.get(f"{self.monitor_prefix}_acc@0.50", None) or logs.
 ↪get(f"{self.monitor_prefix}_acc@0.5", None)
        prec = logs.get(f"{self.monitor_prefix}_prec@0.50", None) or logs.
 ↪get(f"{self.monitor_prefix}_prec@0.5", None)
        dice = logs.get(f"{self.monitor_prefix}_dice", None)
```

```
        if acc is None or prec is None or dice is None:
            return

        if (acc >= self.acc_target) and (prec >= self.prec_target) and (dice >=␣
 ↪self.dice_target):
            self._ok_count += 1
        else:
            self._ok_count = 0

        if self._ok_count >= self.patience_ok:
            print(f"\n[STOP] Targets atteints {self.patience_ok} epochs: "
                  f"val_acc={acc:.4f} val_prec={prec:.4f} val_dice={dice:.4f}")
            self.model.stop_training = True

# ------------------------------
# 6) Compile + Fit (finetune court, anti-overfit)
# ------------------------------
# Important: reset fonctions Keras si besoin
for attr in ("train_function", "test_function", "predict_function"):
    if hasattr(unet_model, attr):
        setattr(unet_model, attr, None)

unet_model.compile(
    optimizer=opt,
    loss=loss_ft,
    metrics=metrics_ft,
    run_eagerly=False
)

OUT_DIR = globals().get("OUT_DIR", "./models")
os.makedirs(OUT_DIR, exist_ok=True)
run_id = tf.timestamp().numpy()
log_path = os.path.join(OUT_DIR, f"finetune_{int(run_id)}.csv")
ckpt_path = os.path.join(OUT_DIR, f"best_finetune_{int(run_id)}.weights.h5")

callbacks = [
    CSVLogger(log_path, append=False),
    TerminateOnNaN(),
    # si tu veux "le meilleur compromis": monitor val_dice
    EarlyStopping(monitor="val_dice", mode="max", patience=6, min_delta=1e-4,␣
 ↪restore_best_weights=True, verbose=1),
    ReduceLROnPlateau(monitor="val_dice", mode="max", factor=0.5, patience=2,␣
 ↪min_lr=3e-7, verbose=1),
    ModelCheckpoint(ckpt_path, monitor="val_dice", mode="max",␣
 ↪save_best_only=True, save_weights_only=True, verbose=1),
    # Stop condition: clarté 98% + précision ok + dice ok (ajuste les seuils␣
 ↪selon ton objectif)
```

```
    StopWhenGood(acc_target=0.98, prec_target=0.85, dice_target=0.68,␣
 ↪patience_ok=2),
]

EPOCHS_FT = 20  # finetune court (anti-overfit)

hist_ft = unet_model.fit(
    x=train_fit_ds,
    validation_data=val_fit_ds,
    steps_per_epoch=int(steps_per_epoch),
    validation_steps=int(val_steps),
    epochs=EPOCHS_FT,
    callbacks=callbacks,
    verbose=1
)

print("\n[FINETUNE DONE] best weights (val_dice) sauvegardés:", ckpt_path)
print("Log:", log_path)
```

```
Epoch 1/20
161/162            0s 26ms/step -
acc@0.50: 0.9251 - dice: 0.7924 - loss: 0.1192 - prec@0.50: 0.8801 - prec@0.95:
0.9266 - rec@0.50: 0.7381
Epoch 1: val_dice improved from -inf to 0.69343, saving model to
./models/best_finetune_1767338575.weights.h5
162/162            23s 47ms/step -
acc@0.50: 0.9252 - dice: 0.7925 - loss: 0.1191 - prec@0.50: 0.8801 - prec@0.95:
0.9267 - rec@0.50: 0.7382 - val_acc@0.50: 0.8673 - val_dice: 0.6934 - val_loss:
0.2261 - val_prec@0.50: 0.7551 - val_prec@0.95: 0.7979 - val_rec@0.50: 0.5936 -
learning_rate: 2.0000e-05
Epoch 2/20
161/162            0s 26ms/step -
acc@0.50: 0.9319 - dice: 0.8016 - loss: 0.1109 - prec@0.50: 0.8898 - prec@0.95:
0.9332 - rec@0.50: 0.7514
Epoch 2: val_dice improved from 0.69343 to 0.69498, saving model to
./models/best_finetune_1767338575.weights.h5
162/162            5s 31ms/step -
acc@0.50: 0.9319 - dice: 0.8016 - loss: 0.1109 - prec@0.50: 0.8898 - prec@0.95:
0.9332 - rec@0.50: 0.7514 - val_acc@0.50: 0.8667 - val_dice: 0.6950 - val_loss:
0.2286 - val_prec@0.50: 0.7511 - val_prec@0.95: 0.7916 - val_rec@0.50: 0.5962 -
learning_rate: 2.0000e-05
Epoch 3/20
161/162            0s 26ms/step -
acc@0.50: 0.9346 - dice: 0.8067 - loss: 0.1067 - prec@0.50: 0.8931 - prec@0.95:
0.9349 - rec@0.50: 0.7558
Epoch 3: val_dice did not improve from 0.69498
162/162            5s 29ms/step -
acc@0.50: 0.9346 - dice: 0.8067 - loss: 0.1067 - prec@0.50: 0.8931 - prec@0.95:
```

```
0.9349 - rec@0.50: 0.7558 - val_acc@0.50: 0.8661 - val_dice: 0.6934 - val_loss:
0.2296 - val_prec@0.50: 0.7483 - val_prec@0.95: 0.7865 - val_rec@0.50: 0.5964 -
learning_rate: 2.0000e-05
Epoch 4/20
161/162            0s 26ms/step -
acc@0.50: 0.9325 - dice: 0.8055 - loss: 0.1080 - prec@0.50: 0.8918 - prec@0.95:
0.9347 - rec@0.50: 0.7524
Epoch 4: ReduceLROnPlateau reducing learning rate to 9.999999747378752e-06.

Epoch 4: val_dice did not improve from 0.69498
162/162            5s 29ms/step -
acc@0.50: 0.9325 - dice: 0.8056 - loss: 0.1080 - prec@0.50: 0.8918 - prec@0.95:
0.9347 - rec@0.50: 0.7525 - val_acc@0.50: 0.8654 - val_dice: 0.6928 - val_loss:
0.2324 - val_prec@0.50: 0.7440 - val_prec@0.95: 0.7768 - val_rec@0.50: 0.5990 -
learning_rate: 2.0000e-05
Epoch 5/20
161/162            0s 26ms/step -
acc@0.50: 0.9340 - dice: 0.8131 - loss: 0.1025 - prec@0.50: 0.8982 - prec@0.95:
0.9392 - rec@0.50: 0.7622
Epoch 5: val_dice did not improve from 0.69498
162/162            5s 29ms/step -
acc@0.50: 0.9339 - dice: 0.8131 - loss: 0.1026 - prec@0.50: 0.8982 - prec@0.95:
0.9391 - rec@0.50: 0.7622 - val_acc@0.50: 0.8654 - val_dice: 0.6942 - val_loss:
0.2317 - val_prec@0.50: 0.7436 - val_prec@0.95: 0.7800 - val_rec@0.50: 0.5990 -
learning_rate: 1.0000e-05
Epoch 6/20
161/162            0s 26ms/step -
acc@0.50: 0.9339 - dice: 0.8072 - loss: 0.1073 - prec@0.50: 0.8943 - prec@0.95:
0.9353 - rec@0.50: 0.7586
Epoch 6: ReduceLROnPlateau reducing learning rate to 4.999999873689376e-06.

Epoch 6: val_dice did not improve from 0.69498
162/162            5s 29ms/step -
acc@0.50: 0.9339 - dice: 0.8072 - loss: 0.1073 - prec@0.50: 0.8943 - prec@0.95:
0.9354 - rec@0.50: 0.7587 - val_acc@0.50: 0.8652 - val_dice: 0.6941 - val_loss:
0.2333 - val_prec@0.50: 0.7423 - val_prec@0.95: 0.7766 - val_rec@0.50: 0.6000 -
learning_rate: 1.0000e-05
Epoch 7/20
161/162            0s 26ms/step -
acc@0.50: 0.9339 - dice: 0.8095 - loss: 0.1064 - prec@0.50: 0.8921 - prec@0.95:
0.9334 - rec@0.50: 0.7611
Epoch 7: val_dice did not improve from 0.69498
162/162            5s 29ms/step -
acc@0.50: 0.9339 - dice: 0.8095 - loss: 0.1064 - prec@0.50: 0.8921 - prec@0.95:
0.9335 - rec@0.50: 0.7612 - val_acc@0.50: 0.8657 - val_dice: 0.6936 - val_loss:
0.2312 - val_prec@0.50: 0.7464 - val_prec@0.95: 0.7827 - val_rec@0.50: 0.5971 -
learning_rate: 5.0000e-06
Epoch 8/20
```

```
161/162             0s 26ms/step -
acc@0.50: 0.9349 - dice: 0.8146 - loss: 0.1017 - prec@0.50: 0.9009 - prec@0.95:
0.9405 - rec@0.50: 0.7649
Epoch 8: ReduceLROnPlateau reducing learning rate to 2.499999936844688e-06.

Epoch 8: val_dice did not improve from 0.69498
162/162             5s 29ms/step -
acc@0.50: 0.9349 - dice: 0.8146 - loss: 0.1017 - prec@0.50: 0.9009 - prec@0.95:
0.9405 - rec@0.50: 0.7649 - val_acc@0.50: 0.8652 - val_dice: 0.6938 - val_loss:
0.2333 - val_prec@0.50: 0.7424 - val_prec@0.95: 0.7764 - val_rec@0.50: 0.5997 -
learning_rate: 5.0000e-06
Epoch 8: early stopping
Restoring model weights from the end of the best epoch: 2.

[FINETUNE DONE] best weights (val_dice) sauvegardés:
./models/best_finetune_1767338575.weights.h5
Log: ./models/finetune_1767338575.csv
```

Ce finetune "anti-overfit" (BN gelées + LR très petit) confirme surtout un **plateau** plutôt qu'un vrai gain : dès l'epoch 1–2, la validation atteint son meilleur **val_dice 0.695** (checkpoint sauvegardé, puis restauré à la fin), alors que côté train les métriques continuent à progresser (dice ~0.79→0.81, prec@0.5 ~0.88→0.90), signe classique que le modèle **apprend encore sur train mais n'améliore plus la généralisation**. Les callbacks jouent exactement leur rôle : `ReduceLROnPlateau` baisse le LR (2e-5 → 1e-5 → 5e-6 → 2.5e-6) sans débloquer de meilleure validation, puis `EarlyStopping` coupe à l'epoch 8 et recharge les poids du **meilleur compromis** (epoch 2). En validation, on observe un profil assez stable : **val_prec@0.5 ~0.74–0.76** et **val_rec@0.5 ~0.59–0.60**, donc une segmentation plutôt "prudente" (précision correcte mais rappel moyen), et la précision stricte **val_prec@0.95 ~0.78–0.80** reste loin des objectifs "ultra stricts" type 0.90+ ; enfin, le callback `StopWhenGood` ne se déclenche pas (val_acc ~0.865 < 0.98), ce qui confirme que ce run sert surtout à **stabiliser** et éviter la dérive, pas à franchir un nouveau palier de performance.

[ ]:
```python
import numpy as np, tensorflow as tf

assert "unet_model" in globals()
assert "train_fit_ds" in globals() and "val_epoch_ds" in globals() and␣
 ↪"val_steps" in globals()

# ------------------------------
# 0) Helpers: detect arity & predict safely
# ------------------------------
def _input_arity(model):
    try:
        return len(model.inputs)
    except Exception:
        # fallback: inspect a batch from val_epoch_ds
        x0 = next(iter(val_epoch_ds.take(1)))[0]
        if isinstance(x0, (tuple, list)):
            return len(x0)
```

```
        return 1

ARITY = _input_arity(unet_model)
print("[INFO] Model input arity =", ARITY)

def _pack_x(x):
    # x can be tuple/list (img, hed) or (img, grad, hed) or dict
    if isinstance(x, dict):
        # try common keys
        keys = ["img", "grad", "hed", "hed_prior"]
        out = []
        for k in keys:
            if k in x: out.append(x[k])
        if len(out) == 0:
            raise ValueError("Dict input non supporté (keys).")
        return out
    if isinstance(x, (tuple, list)):
        return list(x)
    return [x]

@tf.function(reduce_retracing=True)
def _predict_tf(model, x_list):
    return model(x_list, training=False)

def predict_np(model, x):
    x_list = _pack_x(x)
    # Guarantee list length equals model inputs length:
    if len(x_list) != ARITY:
        # common case: dataset returns (img, grad, hed) but you built (img,␣
 ↪hed) etc.
        # keep last ARITY tensors
        x_list = x_list[-ARITY:]
    return _predict_tf(model, x_list).numpy()

# ------------------------------
# 1) Freeze BatchNorm (anti-overfit, plus stable)
# ------------------------------
for layer in unet_model.layers:
    if isinstance(layer, tf.keras.layers.BatchNormalization):
        layer.trainable = False

# ------------------------------
# 2) Loss precision-friendly (pénalise FP) + LR minuscule
# ------------------------------
@tf.keras.utils.register_keras_serializable()
class FocalTverskyPlusBCE(tf.keras.losses.Loss):
```

```
    def __init__(self, alpha=0.88, gamma=1.35, bce_weight=0.05, smooth=1e-6,␣
 ↪name="focal_tversky_plus_bce"):
        super().__init__(name=name)
        self.alpha = float(alpha)
        self.beta  = float(1.0 - alpha)
        self.gamma = float(gamma)
        self.bce_weight = float(bce_weight)
        self.smooth = float(smooth)
        self._bce = tf.keras.losses.BinaryCrossentropy(from_logits=False)

    def call(self, y_true, y_pred):
        y_true = tf.cast(y_true, tf.float32)
        y_pred = tf.cast(tf.clip_by_value(y_pred, 1e-6, 1.0 - 1e-6), tf.float32)
        yt = tf.reshape(y_true, [-1])
        yp = tf.reshape(y_pred, [-1])
        tp = tf.reduce_sum(yt * yp)
        fp = tf.reduce_sum((1.0 - yt) * yp)
        fn = tf.reduce_sum(yt * (1.0 - yp))
        tversky = (tp + self.smooth) / (tp + self.alpha * fp + self.beta * fn +␣
 ↪self.smooth)
        focal_tversky = tf.pow((1.0 - tversky), self.gamma)
        return (1.0 - self.bce_weight) * focal_tversky + self.bce_weight * self.
 ↪_bce(y_true, y_pred)

def soft_dice(y_true, y_pred, smooth=1.0):
    y_true = tf.cast(y_true, tf.float32)
    y_pred = tf.cast(tf.clip_by_value(y_pred, 0.0, 1.0), tf.float32)
    yt = tf.reshape(y_true, [-1])
    yp = tf.reshape(y_pred, [-1])
    inter = tf.reduce_sum(yt * yp)
    denom = tf.reduce_sum(yt) + tf.reduce_sum(yp)
    return (2.0 * inter + smooth) / (denom + smooth)

dice_fn = globals().get("dice_coef", soft_dice)

thr_main = 0.50
thr_strict = 0.95  # suivi "propreté"

metrics = [
    tf.keras.metrics.MeanMetricWrapper(fn=dice_fn, name="dice"),
    tf.keras.metrics.BinaryAccuracy(name=f"acc@{thr_main:.2f}",␣
 ↪threshold=thr_main),
    tf.keras.metrics.Precision(name=f"prec@{thr_main:.2f}",␣
 ↪thresholds=thr_main),
    tf.keras.metrics.Recall(name=f"rec@{thr_main:.2f}", thresholds=thr_main),
    tf.keras.metrics.Precision(name=f"prec@{thr_strict:.2f}",␣
 ↪thresholds=thr_strict),
```

```
]

# optimizer très doux (évite d'abîmer le modèle)
lr = 8e-6
try:
    opt = tf.keras.optimizers.AdamW(learning_rate=lr, weight_decay=1e-4,␣
 ↪clipnorm=1.0)
except Exception:
    opt = tf.keras.optimizers.Adam(learning_rate=lr, clipnorm=1.0)

loss_fn = FocalTverskyPlusBCE(alpha=0.88, gamma=1.35, bce_weight=0.05)

# reset Keras compiled functions (utile quand on recompile)
for attr in ("train_function", "test_function", "predict_function"):
    if hasattr(unet_model, attr):
        setattr(unet_model, attr, None)

unet_model.compile(optimizer=opt, loss=loss_fn, metrics=metrics,␣
 ↪run_eagerly=False)

# Mini finetune court: monitor précision stricte (anti-FP) + dice
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau,␣
 ↪TerminateOnNaN

callbacks = [
    TerminateOnNaN(),
    EarlyStopping(monitor="val_prec@0.95", mode="max", patience=3,␣
 ↪min_delta=5e-4,
                  restore_best_weights=True, verbose=1),
    ReduceLROnPlateau(monitor="val_prec@0.95", mode="max", factor=0.5,␣
 ↪patience=1,
                      min_lr=1e-7, verbose=1),
]

# IMPORTANT: fit sur train_fit_ds/val_fit_ds déjà prêts chez toi
# (Tu peux commenter ce bloc si tu veux garder exactement epoch2)
hist_ft2 = unet_model.fit(
    x=train_fit_ds,
    validation_data=val_fit_ds,
    steps_per_epoch=int(steps_per_epoch),
    validation_steps=int(val_steps),
    epochs=8,              # court
    callbacks=callbacks,
    verbose=1
)

# ----------------------------
```

```python
# 3) CALIBRATION: choisir seuil pour PRECISION >= 0.98
#    puis optimiser Dice (ou Recall) sous contrainte
# ----------------------------
def accumulate_stats(model, ds, steps, thresholds):
    thresholds = np.asarray(thresholds, dtype=np.float32)
    tp = np.zeros((len(thresholds),), dtype=np.float64)
    fp = np.zeros((len(thresholds),), dtype=np.float64)
    fn = np.zeros((len(thresholds),), dtype=np.float64)

    total_pix = 0.0
    correct_pix = np.zeros((len(thresholds),), dtype=np.float64)

    it = iter(ds.take(steps))
    for _ in range(int(steps)):
        x, y = next(it)
        p = predict_np(model, x)  # (B,H,W,1)
        yb = (y.numpy() > 0.5)

        p_flat = p.reshape(-1).astype(np.float32)
        y_flat = yb.reshape(-1)

        pred = (p_flat[None, :] >= thresholds[:, None])

        tp_b = np.sum(pred & y_flat[None, :], axis=1, dtype=np.float64)
        fp_b = np.sum(pred & (~y_flat)[None, :], axis=1, dtype=np.float64)
        pos = float(y_flat.sum())
        fn_b = pos - tp_b

        tp += tp_b; fp += fp_b; fn += fn_b

        # pixel-accuracy (clarté) @thresholds
        total_pix += y_flat.size
        correct_pix += np.sum(pred == y_flat[None, :], axis=1, dtype=np.float64)

    precision = tp / (tp + fp + 1e-9)
    recall    = tp / (tp + fn + 1e-9)
    dice      = (2.0 * tp) / (2.0 * tp + fp + fn + 1e-9)
    acc       = correct_pix / (total_pix + 1e-9)

    return dict(thresholds=thresholds, precision=precision, recall=recall,␣
 ↪dice=dice, acc=acc)

# grid dense vers le haut (si tu veux "clarté" très haute)
thr_grid = np.unique(np.concatenate([
    np.linspace(0.50, 0.90, 9),
    np.linspace(0.90, 0.99, 10),
    np.linspace(0.99, 0.999, 10),
```

```
])).astype(np.float32)

stats = accumulate_stats(unet_model, val_epoch_ds, val_steps, thr_grid)
ths, prec, rec, dice, acc = stats["thresholds"], stats["precision"],␣
 ↪stats["recall"], stats["dice"], stats["acc"]

TARGET_PREC = 0.98  # <<=== "clarté 98%" interprétée comme précision 0.98
ok = prec >= TARGET_PREC

if np.any(ok):
    # choix 1: max Dice sous contrainte precision>=0.98
    best = int(np.argmax(np.where(ok, dice, -1.0)))
    print(f"\n[CALIB OK] precision>={TARGET_PREC:.2f} (max dice)")
    print(f" thr={ths[best]:.3f} | prec={prec[best]:.4f} | rec={rec[best]:.4f}␣
 ↪| dice={dice[best]:.4f} | acc={acc[best]:.4f}")
    RECOMMENDED_THR = float(ths[best])
else:
    # fallback: meilleure précision atteinte
    best = int(np.argmax(prec))
    print(f"\n[CALIB WARN] impossible d'atteindre precision>={TARGET_PREC:.2f}␣
 ↪sur le grid.")
    print(f" best-prec thr={ths[best]:.3f} | prec={prec[best]:.4f} |␣
 ↪rec={rec[best]:.4f} | dice={dice[best]:.4f} | acc={acc[best]:.4f}")
    RECOMMENDED_THR = float(ths[best])

# Bonus: si tu veux optimiser "clarté" (=accuracy) sous contrainte precision>=0.
 ↪98
if np.any(ok):
    best_acc = int(np.argmax(np.where(ok, acc, -1.0)))
    print(f"\n[CALIB ALT] max-acc sous prec>={TARGET_PREC:.2f}")
    print(f" thr={ths[best_acc]:.3f} | prec={prec[best_acc]:.4f} |␣
 ↪rec={rec[best_acc]:.4f} | dice={dice[best_acc]:.4f} | acc={acc[best_acc]:.
 ↪4f}")

print(f"\n[FINAL] RECOMMENDED_THR={RECOMMENDED_THR:.3f} (pour viser prec 0.98)")

# ----------------------------
# 4) Inference helper
# ----------------------------
def predict_mask(model, x, thr=RECOMMENDED_THR):
    p = predict_np(model, x)
    return (p >= thr).astype(np.uint8)
```

```
[INFO] Model input arity = 3
Epoch 1/8
162/162             23s 45ms/step -
acc@0.50: 0.9247 - dice: 0.7890 - loss: 0.0788 - prec@0.50: 0.9040 - prec@0.95:
```

```
0.9429 - rec@0.50: 0.7184 - val_acc@0.50: 0.8633 - val_dice: 0.6693 - val_loss:
0.1812 - val_prec@0.50: 0.7655 - val_prec@0.95: 0.8010 - val_rec@0.50: 0.5528 -
learning_rate: 8.0000e-06
Epoch 2/8
161/162            0s 26ms/step -
acc@0.50: 0.9271 - dice: 0.7814 - loss: 0.0749 - prec@0.50: 0.9167 - prec@0.95:
0.9518 - rec@0.50: 0.6978
Epoch 2: ReduceLROnPlateau reducing learning rate to 3.999999989900971e-06.
162/162            5s 29ms/step -
acc@0.50: 0.9271 - dice: 0.7814 - loss: 0.0749 - prec@0.50: 0.9167 - prec@0.95:
0.9517 - rec@0.50: 0.6978 - val_acc@0.50: 0.8618 - val_dice: 0.6680 - val_loss:
0.1850 - val_prec@0.50: 0.7570 - val_prec@0.95: 0.7889 - val_rec@0.50: 0.5544 -
learning_rate: 8.0000e-06
Epoch 3/8
161/162            0s 26ms/step -
acc@0.50: 0.9260 - dice: 0.7764 - loss: 0.0766 - prec@0.50: 0.9143 - prec@0.95:
0.9496 - rec@0.50: 0.6918
Epoch 3: ReduceLROnPlateau reducing learning rate to 1.9999999949504854e-06.
162/162            5s 29ms/step -
acc@0.50: 0.9260 - dice: 0.7765 - loss: 0.0766 - prec@0.50: 0.9144 - prec@0.95:
0.9496 - rec@0.50: 0.6919 - val_acc@0.50: 0.8621 - val_dice: 0.6667 - val_loss:
0.1837 - val_prec@0.50: 0.7621 - val_prec@0.95: 0.7960 - val_rec@0.50: 0.5490 -
learning_rate: 4.0000e-06
Epoch 4/8
161/162            0s 26ms/step -
acc@0.50: 0.9261 - dice: 0.7812 - loss: 0.0742 - prec@0.50: 0.9193 - prec@0.95:
0.9525 - rec@0.50: 0.6971
Epoch 4: ReduceLROnPlateau reducing learning rate to 9.999999974752427e-07.
162/162            5s 29ms/step -
acc@0.50: 0.9261 - dice: 0.7812 - loss: 0.0742 - prec@0.50: 0.9193 - prec@0.95:
0.9525 - rec@0.50: 0.6971 - val_acc@0.50: 0.8627 - val_dice: 0.6685 - val_loss:
0.1827 - val_prec@0.50: 0.7637 - val_prec@0.95: 0.7984 - val_rec@0.50: 0.5509 -
learning_rate: 2.0000e-06
Epoch 4: early stopping
Restoring model weights from the end of the best epoch: 1.

[CALIB WARN] impossible d'atteindre precision>=0.98 sur le grid.
 best-prec thr=0.999 | prec=0.9087 | rec=0.1295 | dice=0.2267 | acc=0.8042

[FINAL] RECOMMENDED_THR=0.999 (pour viser prec 0.98)
```

Ce script fait un **mini-finetune "anti faux-positifs"** puis une **calibration de seuil** : il détecte d'abord que ton U-Net a **3 entrées** (img, grad, hed), gèle les BatchNorm pour stabiliser, compile une loss **Focal-Tversky + BCE** très orientée "propreté" (=0.88 → pénalise fortement les FP) avec un **LR minuscule** (8e-6) et surveille **val_prec@0.95** (précision stricte) pour éviter d'abîmer le modèle. En pratique, le finetune **n'améliore pas la validation** : dès l'epoch 1 tu as le meilleur compromis, puis la validation baisse légèrement (val_dice ~0.669→0.666–0.668) malgré des métriques train élevées (prec@0.95 ~0.95), donc le callback réduit le LR à chaque fois et **early-**

stop à l'epoch 4 en restaurant l'epoch 1. Ensuite, la calibration essaie de trouver un seuil qui garantit **precision 0.98** (interprété comme "clarté 98%"), mais c'est **impossible** sur ta grille : la meilleure précision trouvée est ~**0.9087** à **thr=0.999**, au prix d'un **rappel très faible** (rec ~0.1295) et d'un **dice très bas** (~0.2267), ce qui veut dire que pousser le seuil vers 0.999 rend le masque **très conservateur** (presque rien n'est détecté) ; conclusion : ton modèle peut monter à ~0.80 de précision stricte à 0.95, mais **atteindre 0.98 de précision globale** n'est pas réaliste avec ce modèle/dataset, et la calibration à très haut seuil dégrade fortement la détection.

[ ]:
```
import tensorflow as tf
import numpy as np

assert "unet_model" in globals()
assert "train_fit_ds" in globals() and "val_fit_ds" in globals()
assert "steps_per_epoch" in globals() and "val_steps" in globals()

# ----------------------------
# 0) Freeze BN + Freeze "presque tout"
# ----------------------------
for layer in unet_model.layers:
    if isinstance(layer, tf.keras.layers.BatchNormalization):
        layer.trainable = False

# Heuristique simple: on ne laisse trainable que les ~25 dernières couches␣
 ↪(head+fin decoder)
N_TRAIN_LAST = 25
for layer in unet_model.layers[:-N_TRAIN_LAST]:
    layer.trainable = False
for layer in unet_model.layers[-N_TRAIN_LAST:]:
    layer.trainable = True

print("[INFO] Trainable layers:", sum([int(l.trainable) for l in unet_model.
 ↪layers]), "/", len(unet_model.layers))

# ----------------------------
# 1) Dice metric (si pas déjà)
# ----------------------------
def _soft_dice(y_true, y_pred, smooth=1.0):
    y_true = tf.cast(y_true, tf.float32)
    y_pred = tf.cast(tf.clip_by_value(y_pred, 0.0, 1.0), tf.float32)
    yt = tf.reshape(y_true, [-1])
    yp = tf.reshape(y_pred, [-1])
    inter = tf.reduce_sum(yt * yp)
    denom = tf.reduce_sum(yt) + tf.reduce_sum(yp)
    return (2.0 * inter + smooth) / (denom + smooth)

dice_fn = globals().get("dice_coef", _soft_dice)
```

```python
# ----------------------------
# 2) Loss FP-Control (attaque les FP à haute proba)
# ----------------------------
@tf.keras.utils.register_keras_serializable()
class FPControlledLoss(tf.keras.losses.Loss):
    """
    Objectif: réduire les faux positifs "très confiants".
    - Focal Tversky (alpha haut => FP pénalisés)
    - + FP penalty: mean(y_pred * (1 - y_true)) (pousse background -> 0)
    """
    def __init__(self, alpha=0.93, gamma=1.35, fp_lambda=0.60, bce_weight=0.03,␣
↪smooth=1e-6, name="fp_control"):
        super().__init__(name=name)
        self.alpha = float(alpha)
        self.beta  = float(1.0 - alpha)
        self.gamma = float(gamma)
        self.fp_lambda = float(fp_lambda)
        self.bce_weight = float(bce_weight)
        self.smooth = float(smooth)
        self._bce = tf.keras.losses.BinaryCrossentropy(from_logits=False)

    def call(self, y_true, y_pred):
        y_true = tf.cast(y_true, tf.float32)
        y_pred = tf.cast(tf.clip_by_value(y_pred, 1e-6, 1.0 - 1e-6), tf.float32)

        yt = tf.reshape(y_true, [-1])
        yp = tf.reshape(y_pred, [-1])

        tp = tf.reduce_sum(yt * yp)
        fp = tf.reduce_sum((1.0 - yt) * yp)
        fn = tf.reduce_sum(yt * (1.0 - yp))

        tversky = (tp + self.smooth) / (tp + self.alpha * fp + self.beta * fn +␣
↪self.smooth)
        focal_tversky = tf.pow((1.0 - tversky), self.gamma)

        # pénalité FP (background) : si y_true=0 et y_pred élevé => pénalité␣
↪forte
        fp_pen = tf.reduce_mean(y_pred * (1.0 - y_true))

        loss = focal_tversky + self.fp_lambda * fp_pen
        if self.bce_weight > 0:
            loss = (1.0 - self.bce_weight) * loss + self.bce_weight * self.
↪_bce(y_true, y_pred)
        return loss

    def get_config(self):
```

```
        cfg = super().get_config()
        cfg.update(dict(alpha=self.alpha, gamma=self.gamma, fp_lambda=self.
 ↪fp_lambda,
                        bce_weight=self.bce_weight, smooth=self.smooth))
        return cfg

# ----------------------------
# 3) Optimizer très doux
# ----------------------------
LR = 2e-6
try:
    opt = tf.keras.optimizers.AdamW(learning_rate=LR, weight_decay=5e-5,␣
 ↪clipnorm=1.0)
except Exception:
    opt = tf.keras.optimizers.Adam(learning_rate=LR, clipnorm=1.0)

# Metrics: on suit une précision très stricte
metrics = [
    tf.keras.metrics.MeanMetricWrapper(fn=dice_fn, name="dice"),
    tf.keras.metrics.Precision(name="prec@0.50", thresholds=0.50),
    tf.keras.metrics.Recall(name="rec@0.50", thresholds=0.50),
    tf.keras.metrics.Precision(name="prec@0.95", thresholds=0.95),
    tf.keras.metrics.Precision(name="prec@0.99", thresholds=0.99),
]

# reset compiled functions
for attr in ("train_function", "test_function", "predict_function"):
    if hasattr(unet_model, attr):
        setattr(unet_model, attr, None)

unet_model.compile(
    optimizer=opt,
    loss=FPControlledLoss(alpha=0.93, gamma=1.35, fp_lambda=0.60, bce_weight=0.
 ↪03),
    metrics=metrics,
    run_eagerly=False
)

# ----------------------------
# 4) Fit court + anti-overfit
# ----------------------------
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau,␣
 ↪TerminateOnNaN, ModelCheckpoint, CSVLogger
import os, time

out_dir = "./models"
os.makedirs(out_dir, exist_ok=True)
```

```
run_id = str(int(time.time()))
ckpt = os.path.join(out_dir, f"best_fpcontrol_{run_id}.weights.h5")
logf = os.path.join(out_dir, f"fpcontrol_{run_id}.csv")

callbacks = [
    CSVLogger(logf, append=False),
    TerminateOnNaN(),
    ModelCheckpoint(ckpt, monitor="val_prec@0.99", mode="max",␣
 ↪save_best_only=True,
                    save_weights_only=True, verbose=1),
    EarlyStopping(monitor="val_prec@0.99", mode="max", patience=3,␣
 ↪min_delta=5e-4,
                  restore_best_weights=True, verbose=1),
    ReduceLROnPlateau(monitor="val_prec@0.99", mode="max", factor=0.5,␣
 ↪patience=1,
                      min_lr=2e-7, verbose=1),
]

hist = unet_model.fit(
    x=train_fit_ds,
    validation_data=val_fit_ds,
    steps_per_epoch=int(steps_per_epoch),
    validation_steps=int(val_steps),
    epochs=12,          # court
    callbacks=callbacks,
    verbose=1
)

print("\n[OK] best weights saved:", ckpt)
print("[OK] log:", logf)

# ----------------------------
# 5) Calibration seuil pour PRECISION >= 0.98 (grid très haut)
# ----------------------------
def _input_arity(model):
    try:
        return len(model.inputs)
    except Exception:
        return 1

ARITY = _input_arity(unet_model)

def _pack_x(x):
    if isinstance(x, (tuple, list)):
        x = list(x)
        if len(x) != ARITY:
            x = x[-ARITY:]
```

```
        return x
    return [x]

@tf.function(reduce_retracing=True)
def _pred_tf(m, x_list):
    return m(x_list, training=False)

def predict_np(m, x):
    return _pred_tf(m, _pack_x(x)).numpy()

def accumulate_stats(model, ds, steps, thresholds):
    thresholds = np.asarray(thresholds, dtype=np.float32)
    tp = np.zeros((len(thresholds),), dtype=np.float64)
    fp = np.zeros((len(thresholds),), dtype=np.float64)
    fn = np.zeros((len(thresholds),), dtype=np.float64)
    it = iter(ds.take(int(steps)))
    for _ in range(int(steps)):
        x, y = next(it)
        p = predict_np(model, x)
        yb = (y.numpy() > 0.5).reshape(-1)
        pf = p.reshape(-1).astype(np.float32)
        pred = (pf[None, :] >= thresholds[:, None])
        tp_b = np.sum(pred & yb[None, :], axis=1, dtype=np.float64)
        fp_b = np.sum(pred & (~yb)[None, :], axis=1, dtype=np.float64)
        pos = float(yb.sum())
        fn_b = pos - tp_b
        tp += tp_b; fp += fp_b; fn += fn_b
    precision = tp / (tp + fp + 1e-9)
    recall    = tp / (tp + fn + 1e-9)
    dice      = (2.0 * tp) / (2.0 * tp + fp + fn + 1e-9)
    return thresholds, precision, recall, dice

thr_grid = np.unique(np.concatenate([
    np.linspace(0.95, 0.99, 9),
    np.linspace(0.99, 0.999, 10),
    np.linspace(0.999, 0.9999, 6),
])).astype(np.float32)

ths, prec, rec, dice = accumulate_stats(unet_model, val_epoch_ds, val_steps,␣
 ↪thr_grid)

TARGET = 0.98
ok = prec >= TARGET
if np.any(ok):
    best = int(np.argmax(np.where(ok, dice, -1.0)))
    print(f"\n[CALIB OK] prec>={TARGET:.2f} (max dice)")
```

```
    print(f" thr={ths[best]:.4f} | prec={prec[best]:.4f} | rec={rec[best]:.4f}␣
 ↪| dice={dice[best]:.4f}")
else:
    best = int(np.argmax(prec))
    print(f"\n[CALIB WARN] toujours impossible d'atteindre prec>={TARGET:.2f}.")
    print(f" best-prec thr={ths[best]:.4f} | prec={prec[best]:.4f} |␣
 ↪rec={rec[best]:.4f} | dice={dice[best]:.4f}")

# Recommandation seuil (même si contrainte non atteinte)
RECOMMENDED_THR = float(ths[best])
print(f"\n[FINAL] RECOMMENDED_THR={RECOMMENDED_THR:.4f}")
```

```
[INFO] Trainable layers: 25 / 76
Epoch 1/12
160/162             0s 19ms/step -
dice: 0.8042 - loss: 0.1410 - prec@0.50: 0.8318 - prec@0.95: 0.9280 - prec@0.99:
0.9661 - rec@0.50: 0.8340
Epoch 1: val_prec@0.99 improved from -inf to 0.84833, saving model to
./models/best_fpcontrol_1767339056.weights.h5
162/162             21s 40ms/step -
dice: 0.8041 - loss: 0.1410 - prec@0.50: 0.8318 - prec@0.95: 0.9280 - prec@0.99:
0.9661 - rec@0.50: 0.8339 - val_dice: 0.7068 - val_loss: 0.2175 - val_prec@0.50:
0.7228 - val_prec@0.95: 0.7931 - val_prec@0.99: 0.8483 - val_rec@0.50: 0.6511 -
learning_rate: 2.0000e-06
Epoch 2/12
160/162             0s 19ms/step -
dice: 0.8024 - loss: 0.1299 - prec@0.50: 0.8457 - prec@0.95: 0.9322 - prec@0.99:
0.9667 - rec@0.50: 0.8141
Epoch 2: val_prec@0.99 improved from 0.84833 to 0.85456, saving model to
./models/best_fpcontrol_1767339056.weights.h5
162/162             4s 24ms/step -
dice: 0.8024 - loss: 0.1299 - prec@0.50: 0.8457 - prec@0.95: 0.9321 - prec@0.99:
0.9667 - rec@0.50: 0.8140 - val_dice: 0.7066 - val_loss: 0.2224 - val_prec@0.50:
0.7202 - val_prec@0.95: 0.7961 - val_prec@0.99: 0.8546 - val_rec@0.50: 0.6561 -
learning_rate: 2.0000e-06
Epoch 3/12
160/162             0s 19ms/step -
dice: 0.8005 - loss: 0.1206 - prec@0.50: 0.8582 - prec@0.95: 0.9348 - prec@0.99:
0.9666 - rec@0.50: 0.7950
Epoch 3: val_prec@0.99 improved from 0.85456 to 0.85559, saving model to
./models/best_fpcontrol_1767339056.weights.h5
162/162             4s 24ms/step -
dice: 0.8004 - loss: 0.1206 - prec@0.50: 0.8582 - prec@0.95: 0.9348 - prec@0.99:
0.9666 - rec@0.50: 0.7949 - val_dice: 0.7039 - val_loss: 0.2174 - val_prec@0.50:
0.7265 - val_prec@0.95: 0.7994 - val_prec@0.99: 0.8556 - val_rec@0.50: 0.6445 -
learning_rate: 2.0000e-06
Epoch 4/12
```

```
160/162            0s 19ms/step -
dice: 0.7938 - loss: 0.1158 - prec@0.50: 0.8630 - prec@0.95: 0.9324 - prec@0.99:
0.9645 - rec@0.50: 0.7758
Epoch 4: val_prec@0.99 improved from 0.85559 to 0.85643, saving model to
./models/best_fpcontrol_1767339056.weights.h5
162/162            4s 24ms/step -
dice: 0.7938 - loss: 0.1158 - prec@0.50: 0.8630 - prec@0.95: 0.9325 - prec@0.99:
0.9645 - rec@0.50: 0.7757 - val_dice: 0.6999 - val_loss: 0.2109 - val_prec@0.50:
0.7341 - val_prec@0.95: 0.8029 - val_prec@0.99: 0.8564 - val_rec@0.50: 0.6297 -
learning_rate: 2.0000e-06
Epoch 5/12
160/162            0s 19ms/step -
dice: 0.7835 - loss: 0.1146 - prec@0.50: 0.8638 - prec@0.95: 0.9314 - prec@0.99:
0.9627 - rec@0.50: 0.7503
Epoch 5: val_prec@0.99 improved from 0.85643 to 0.85743, saving model to
./models/best_fpcontrol_1767339056.weights.h5
162/162            4s 23ms/step -
dice: 0.7836 - loss: 0.1145 - prec@0.50: 0.8640 - prec@0.95: 0.9315 - prec@0.99:
0.9628 - rec@0.50: 0.7503 - val_dice: 0.6954 - val_loss: 0.2043 - val_prec@0.50:
0.7423 - val_prec@0.95: 0.8067 - val_prec@0.99: 0.8574 - val_rec@0.50: 0.6145 -
learning_rate: 2.0000e-06
Epoch 6/12
160/162            0s 19ms/step -
dice: 0.7865 - loss: 0.1026 - prec@0.50: 0.8829 - prec@0.95: 0.9410 - prec@0.99:
0.9654 - rec@0.50: 0.7440
Epoch 6: val_prec@0.99 did not improve from 0.85743

Epoch 6: ReduceLROnPlateau reducing learning rate to 9.999999974752427e-07.
162/162            4s 22ms/step -
dice: 0.7864 - loss: 0.1025 - prec@0.50: 0.8829 - prec@0.95: 0.9410 - prec@0.99:
0.9654 - rec@0.50: 0.7439 - val_dice: 0.6909 - val_loss: 0.1998 - val_prec@0.50:
0.7481 - val_prec@0.95: 0.8086 - val_prec@0.99: 0.8566 - val_rec@0.50: 0.6021 -
learning_rate: 2.0000e-06
Epoch 7/12
160/162            0s 19ms/step -
dice: 0.7759 - loss: 0.1021 - prec@0.50: 0.8843 - prec@0.95: 0.9397 - prec@0.99:
0.9636 - rec@0.50: 0.7251
Epoch 7: val_prec@0.99 improved from 0.85743 to 0.85788, saving model to
./models/best_fpcontrol_1767339056.weights.h5
162/162            4s 23ms/step -
dice: 0.7760 - loss: 0.1021 - prec@0.50: 0.8844 - prec@0.95: 0.9397 - prec@0.99:
0.9636 - rec@0.50: 0.7251 - val_dice: 0.6872 - val_loss: 0.1960 - val_prec@0.50:
0.7525 - val_prec@0.95: 0.8109 - val_prec@0.99: 0.8579 - val_rec@0.50: 0.5927 -
learning_rate: 1.0000e-06
Epoch 8/12
160/162            0s 19ms/step -
dice: 0.7744 - loss: 0.0990 - prec@0.50: 0.8900 - prec@0.95: 0.9434 - prec@0.99:
0.9655 - rec@0.50: 0.7189
```

```
Epoch 8: val_prec@0.99 did not improve from 0.85788

Epoch 8: ReduceLROnPlateau reducing learning rate to 4.999999987376214e-07.
162/162            4s 22ms/step -
dice: 0.7744 - loss: 0.0989 - prec@0.50: 0.8900 - prec@0.95: 0.9434 - prec@0.99:
0.9655 - rec@0.50: 0.7189 - val_dice: 0.6846 - val_loss: 0.1940 - val_prec@0.50:
0.7556 - val_prec@0.95: 0.8120 - val_prec@0.99: 0.8571 - val_rec@0.50: 0.5862 -
learning_rate: 1.0000e-06
Epoch 8: early stopping
Restoring model weights from the end of the best epoch: 5.

[OK] best weights saved: ./models/best_fpcontrol_1767339056.weights.h5
[OK] log: ./models/fpcontrol_1767339056.csv

[CALIB OK] prec>=0.98 (max dice)
 thr=0.9995 | prec=0.9803 | rec=0.0389 | dice=0.0749

[FINAL] RECOMMENDED_THR=0.9995
```

Ici tu as tenté une stratégie **ultra "anti-faux positifs"** : tu as gelé toutes les BatchNorm et **figé presque tout le réseau** (seulement **25 couches trainables sur 76**), puis tu as utilisé une loss spéciale **FPControlledLoss** (Focal-Tversky avec très haut + une pénalité explicite sur les pixels prédits positifs alors que le vrai est 0), avec un **LR extrêmement faible** (2e-6) et un suivi très strict **val_prec@0.99**. Le résultat montre exactement le trade-off attendu : pendant l'entraînement, la précision "haute confiance" est excellente (train prec@0.99 0.96) mais en validation elle plafonne autour de **0.85–0.86** (val_prec@0.99 max **0.8579**), tandis que **val_dice baisse progressivement** (0.706 → 0.684) et **val_recall@0.50 chute** (0.65 → 0.58), ce qui signifie que le modèle devient de plus en plus conservateur (il "n'ose" plus prédire du positif) ; du coup ReduceLROnPlateau réduit encore le LR et **early-stopping** intervient à l'epoch 8 en restaurant le meilleur état (autour de l'epoch 5). La calibration, elle, réussit enfin à atteindre **precision 0.98** mais uniquement avec un seuil **très extrême** (**thr=0.9995**) : tu obtiens **prec=0.9803** mais avec un **recall quasi nul (0.0389)** et un **dice très faible (0.0749)**, donc ce seuil garantit une "propreté" énorme… au prix de rater presque toutes les vraies zones (segmentation inutilisable si tu veux détecter). Conclusion : cette recette est parfaite si ton objectif est "zéro faux positifs", mais elle **sacrifie trop le rappel**, et la "clarté 98%" devient atteignable seulement en **éteignant** presque complètement la prédiction positive

[ ]:
```python
import tensorflow as tf
import numpy as np
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau,␣
 ↪TerminateOnNaN, ModelCheckpoint, CSVLogger
import os, time

assert "unet_model" in globals()
assert "train_fit_ds" in globals() and "val_fit_ds" in globals()
assert "steps_per_epoch" in globals() and "val_steps" in globals()

# ----------------------------
```

```python
# 0) Freeze BN + train only last K layers
# ------------------------------
for layer in unet_model.layers:
    if isinstance(layer, tf.keras.layers.BatchNormalization):
        layer.trainable = False

K = 35  # un peu plus que 25 pour récupérer recall
for layer in unet_model.layers[:-K]:
    layer.trainable = False
for layer in unet_model.layers[-K:]:
    layer.trainable = True

print("[INFO] Trainable layers:", sum(int(l.trainable) for l in unet_model.
 ↪layers), "/", len(unet_model.layers))

# ------------------------------
# 1) TV loss (Total Variation) légère pour cohérence spatiale
# ------------------------------
def tv_loss(prob_mask):
    # prob_mask: [B,H,W,1] in [0,1]
    return tf.reduce_mean(tf.image.total_variation(prob_mask))

# ------------------------------
# 2) Loss Recall-Recovery
#    - alpha un peu ↓ (moins pénaliser FP)
#    - fp_lambda ↓ (moins forcer à tout supprimer)
#    - TV petite (évite bruit)
# ------------------------------
@tf.keras.utils.register_keras_serializable()
class RecallRecoveryLoss(tf.keras.losses.Loss):
    def __init__(self, alpha=0.88, gamma=1.25, fp_lambda=0.20, tv_lambda=0.02,␣
 ↪bce_weight=0.05, smooth=1e-6, name="rec_recovery"):
        super().__init__(name=name)
        self.alpha = float(alpha)
        self.beta  = float(1.0 - alpha)
        self.gamma = float(gamma)
        self.fp_lambda = float(fp_lambda)
        self.tv_lambda = float(tv_lambda)
        self.bce_weight = float(bce_weight)
        self.smooth = float(smooth)
        self._bce = tf.keras.losses.BinaryCrossentropy(from_logits=False)

    def call(self, y_true, y_pred):
        y_true = tf.cast(y_true, tf.float32)
        y_pred = tf.cast(tf.clip_by_value(y_pred, 1e-6, 1.0 - 1e-6), tf.float32)

        yt = tf.reshape(y_true, [-1])
```

```
        yp = tf.reshape(y_pred, [-1])

        tp = tf.reduce_sum(yt * yp)
        fp = tf.reduce_sum((1.0 - yt) * yp)
        fn = tf.reduce_sum(yt * (1.0 - yp))

        tversky = (tp + self.smooth) / (tp + self.alpha * fp + self.beta * fn + self.smooth)
        focal_tversky = tf.pow((1.0 - tversky), self.gamma)

        # FP penalty doux
        fp_pen = tf.reduce_mean(y_pred * (1.0 - y_true))

        # TV lissage léger
        tv = tv_loss(y_pred)

        loss = focal_tversky + self.fp_lambda * fp_pen + self.tv_lambda * tv
        if self.bce_weight > 0:
            loss = (1.0 - self.bce_weight) * loss + self.bce_weight * self._bce(y_true, y_pred)
        return loss

    def get_config(self):
        cfg = super().get_config()
        cfg.update(dict(alpha=self.alpha, gamma=self.gamma, fp_lambda=self.fp_lambda,
                        tv_lambda=self.tv_lambda, bce_weight=self.bce_weight, smooth=self.smooth))
        return cfg

# ------------------------------
# 3) Optimizer (LR un peu plus haut que 2e-6, mais toujours safe)
# ------------------------------
LR = 8e-6
try:
    opt = tf.keras.optimizers.AdamW(learning_rate=LR, weight_decay=3e-5, clipnorm=1.0)
except Exception:
    opt = tf.keras.optimizers.Adam(learning_rate=LR, clipnorm=1.0)

# Metrics: on suit précision stricte + recall
metrics = [
    tf.keras.metrics.Precision(name="prec@0.98", thresholds=0.98),
    tf.keras.metrics.Precision(name="prec@0.99", thresholds=0.99),
    tf.keras.metrics.Recall(name="rec@0.50", thresholds=0.50),
    tf.keras.metrics.Recall(name="rec@0.80", thresholds=0.80),
```

```
    tf.keras.metrics.MeanMetricWrapper(fn=globals().get("dice_coef", lambda a,b:
 ↪ 0.0), name="dice"),
]

# reset compiled functions
for attr in ("train_function", "test_function", "predict_function"):
    if hasattr(unet_model, attr):
        setattr(unet_model, attr, None)

unet_model.compile(
    optimizer=opt,
    loss=RecallRecoveryLoss(alpha=0.88, gamma=1.25, fp_lambda=0.20, tv_lambda=0.
 ↪02, bce_weight=0.05),
    metrics=metrics,
    run_eagerly=False
)

# ------------------------------
# 4) Fit court + anti-overfit (monitor propreté)
# ------------------------------
out_dir = "./models"
os.makedirs(out_dir, exist_ok=True)
run_id = str(int(time.time()))
ckpt = os.path.join(out_dir, f"best_recallrec_{run_id}.weights.h5")
logf = os.path.join(out_dir, f"recallrec_{run_id}.csv")

callbacks = [
    CSVLogger(logf, append=False),
    TerminateOnNaN(),
    ModelCheckpoint(ckpt, monitor="val_prec@0.99", mode="max",␣
 ↪save_best_only=True,
                    save_weights_only=True, verbose=1),
    EarlyStopping(monitor="val_prec@0.99", mode="max", patience=3,␣
 ↪min_delta=5e-4,
                  restore_best_weights=True, verbose=1),
    ReduceLROnPlateau(monitor="val_prec@0.99", mode="max", factor=0.5,␣
 ↪patience=1,
                      min_lr=5e-7, verbose=1),
]

hist = unet_model.fit(
    x=train_fit_ds,
    validation_data=val_fit_ds,
    steps_per_epoch=int(steps_per_epoch),
    validation_steps=int(val_steps),
    epochs=10,
    callbacks=callbacks,
```

```
    verbose=1
)

print("\n[OK] best weights saved:", ckpt)
print("[OK] log:", logf)

# ------------------------------
# 5) Calibration seuil (objectif: prec>=0.98 mais recall max)
# ------------------------------
def accumulate_stats(model, ds, steps, thresholds):
    thresholds = np.asarray(thresholds, dtype=np.float32)
    tp = np.zeros((len(thresholds),), dtype=np.float64)
    fp = np.zeros((len(thresholds),), dtype=np.float64)
    fn = np.zeros((len(thresholds),), dtype=np.float64)

    it = iter(ds.take(int(steps)))
    for _ in range(int(steps)):
        x, y = next(it)
        # x est déjà le bon format (3 entrées)
        p = model.predict(x, verbose=0)

        yb = (y.numpy() > 0.5).reshape(-1)
        pf = p.reshape(-1).astype(np.float32)

        pred = (pf[None, :] >= thresholds[:, None])
        tp_b = np.sum(pred & yb[None, :], axis=1, dtype=np.float64)
        fp_b = np.sum(pred & (~yb)[None, :], axis=1, dtype=np.float64)
        pos  = float(yb.sum())
        fn_b = pos - tp_b

        tp += tp_b; fp += fp_b; fn += fn_b

    precision = tp / (tp + fp + 1e-9)
    recall    = tp / (tp + fn + 1e-9)
    dice      = (2.0 * tp) / (2.0 * tp + fp + fn + 1e-9)
    return thresholds, precision, recall, dice

thr_grid = np.unique(np.concatenate([
    np.linspace(0.90, 0.99, 10),
    np.linspace(0.99, 0.999, 10),
    np.linspace(0.999, 0.9999, 6),
])).astype(np.float32)

ths, prec, rec, dice = accumulate_stats(unet_model, val_epoch_ds, val_steps,␣
 ↪thr_grid)

TARGET = 0.98
```

```
ok = prec >= TARGET
if np.any(ok):
    best = int(np.argmax(np.where(ok, rec, -1.0)))  # max recall sous contrainte
    print(f"\n[CALIB OK] prec>={TARGET:.2f} (max recall)")
    print(f" thr={ths[best]:.4f} | prec={prec[best]:.4f} | rec={rec[best]:.4f}␣
 ↪| dice={dice[best]:.4f}")
else:
    best = int(np.argmax(prec))
    print(f"\n[CALIB WARN] impossible prec>={TARGET:.2f}, best-prec:")
    print(f" thr={ths[best]:.4f} | prec={prec[best]:.4f} | rec={rec[best]:.4f}␣
 ↪| dice={dice[best]:.4f}")

print(f"\n[FINAL] RECOMMENDED_THR={float(ths[best]):.4f}")
```

```
[INFO] Trainable layers: 35 / 76
Epoch 1/10
162/162            0s 21ms/step -
dice: 0.7887 - loss: 101.1484 - prec@0.98: 0.9571 - prec@0.99: 0.9670 -
rec@0.50: 0.7430 - rec@0.80: 0.6567
Epoch 1: val_prec@0.99 improved from -inf to 0.83348, saving model to
./models/best_recallrec_1767339192.weights.h5
162/162            25s 43ms/step -
dice: 0.7886 - loss: 101.1346 - prec@0.98: 0.9571 - prec@0.99: 0.9670 -
rec@0.50: 0.7429 - rec@0.80: 0.6566 - val_dice: 0.6851 - val_loss: 95.7181 -
val_prec@0.98: 0.8165 - val_prec@0.99: 0.8335 - val_rec@0.50: 0.6071 -
val_rec@0.80: 0.5464 - learning_rate: 8.0000e-06
Epoch 2/10
160/162            0s 21ms/step -
dice: 0.7497 - loss: 88.6624 - prec@0.98: 0.9509 - prec@0.99: 0.9577 - rec@0.50:
0.6720 - rec@0.80: 0.5883
Epoch 2: val_prec@0.99 did not improve from 0.83348

Epoch 2: ReduceLROnPlateau reducing learning rate to 3.999999989900971e-06.
162/162            4s 23ms/step -
dice: 0.7493 - loss: 88.5698 - prec@0.98: 0.9508 - prec@0.99: 0.9576 - rec@0.50:
0.6714 - rec@0.80: 0.5878 - val_dice: 0.5880 - val_loss: 75.0415 -
val_prec@0.98: 0.7995 - val_prec@0.99: 0.8123 - val_rec@0.50: 0.5228 -
val_rec@0.80: 0.4685 - learning_rate: 8.0000e-06
Epoch 3/10
160/162            0s 21ms/step -
dice: 0.6793 - loss: 70.1846 - prec@0.98: 0.9443 - prec@0.99: 0.9513 - rec@0.50:
0.5696 - rec@0.80: 0.4957
Epoch 3: val_prec@0.99 did not improve from 0.83348

Epoch 3: ReduceLROnPlateau reducing learning rate to 1.9999999949504854e-06.
162/162            4s 24ms/step -
dice: 0.6791 - loss: 70.1383 - prec@0.98: 0.9443 - prec@0.99: 0.9513 - rec@0.50:
```

```
0.5694 - rec@0.80: 0.4955 - val_dice: 0.4981 - val_loss: 61.6646 -
val_prec@0.98: 0.7906 - val_prec@0.99: 0.8012 - val_rec@0.50: 0.4553 -
val_rec@0.80: 0.4049 - learning_rate: 4.0000e-06
Epoch 4/10
160/162             0s 21ms/step -
dice: 0.6273 - loss: 60.9022 - prec@0.98: 0.9411 - prec@0.99: 0.9481 - rec@0.50:
0.5033 - rec@0.80: 0.4356
Epoch 4: val_prec@0.99 did not improve from 0.83348

Epoch 4: ReduceLROnPlateau reducing learning rate to 9.999999974752427e-07.
162/162             4s 24ms/step -
dice: 0.6272 - loss: 60.8824 - prec@0.98: 0.9411 - prec@0.99: 0.9481 - rec@0.50:
0.5032 - rec@0.80: 0.4355 - val_dice: 0.4514 - val_loss: 55.2887 -
val_prec@0.98: 0.7847 - val_prec@0.99: 0.7941 - val_rec@0.50: 0.4237 -
val_rec@0.80: 0.3780 - learning_rate: 2.0000e-06
Epoch 4: early stopping
Restoring model weights from the end of the best epoch: 1.

[OK] best weights saved: ./models/best_recallrec_1767339192.weights.h5
[OK] log: ./models/recallrec_1767339192.csv

[CALIB OK] prec>=0.98 (max recall)
 thr=0.9997 | prec=0.9827 | rec=0.0312 | dice=0.0605

[FINAL] RECOMMENDED_THR=0.9997
```

[ ]:
```python
import os, time
import tensorflow as tf
import numpy as np
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau,␣
 ↪TerminateOnNaN, ModelCheckpoint, CSVLogger

assert "unet_model" in globals()
assert "train_fit_ds" in globals() and "val_fit_ds" in globals()
assert "steps_per_epoch" in globals() and "val_steps" in globals()

H = int(globals().get("IMG_H", 256))
W = int(globals().get("IMG_W", 256))

# ------------------------------
# 0) Freeze BN + unfreeze last K layers (safe)
# ------------------------------
for layer in unet_model.layers:
    if isinstance(layer, tf.keras.layers.BatchNormalization):
        layer.trainable = False

K = 25  # plus safe que 35 (chez toi ça a dégradé vite)
```

```
for layer in unet_model.layers[:-K]:
    layer.trainable = False
for layer in unet_model.layers[-K:]:
    layer.trainable = True

print("[INFO] Trainable layers:", sum(int(l.trainable) for l in unet_model.
 ↪layers), "/", len(unet_model.layers))

# ------------------------------
# 1) Loss: Focal-Tversky (FP-averse) + TV(normalisée) + confidence
# ------------------------------
@tf.keras.utils.register_keras_serializable()
class SharpCleanLoss(tf.keras.losses.Loss):
    """
    - Base = Focal-Tversky (alpha>0.5 pénalise FP)
    - + TV normalisée (lissage spatial, réduit petites îles FP)
    - + Confidence terms:
        * pos_push: pousse y_pred->1 sur pixels positifs
        * neg_push: pousse y_pred->0 sur pixels négatifs (anti FP)
    """
    def __init__(self,
                 alpha=0.90, gamma=1.15,
                 tv_lambda=2e-5,
                 pos_push=0.10,
                 neg_push=0.05,
                 bce_weight=0.05,
                 smooth=1e-6,
                 name="sharp_clean"):
        super().__init__(name=name)
        self.alpha = float(alpha)
        self.beta  = float(1.0 - alpha)
        self.gamma = float(gamma)
        self.tv_lambda = float(tv_lambda)
        self.pos_push  = float(pos_push)
        self.neg_push  = float(neg_push)
        self.bce_weight = float(bce_weight)
        self.smooth = float(smooth)
        self._bce = tf.keras.losses.BinaryCrossentropy(from_logits=False)

    def call(self, y_true, y_pred):
        y_true = tf.cast(y_true, tf.float32)
        y_pred = tf.cast(tf.clip_by_value(y_pred, 1e-6, 1.0 - 1e-6), tf.float32)

        # ---- Focal Tversky
        yt = tf.reshape(y_true, [-1])
        yp = tf.reshape(y_pred, [-1])
```

```
        tp = tf.reduce_sum(yt * yp)
        fp = tf.reduce_sum((1.0 - yt) * yp)
        fn = tf.reduce_sum(yt * (1.0 - yp))

        tversky = (tp + self.smooth) / (tp + self.alpha * fp + self.beta * fn +␣
 ↪self.smooth)
        ft = tf.pow((1.0 - tversky), self.gamma)

        # ---- TV normalisée (CRITIQUE!)
        tv_raw = tf.image.total_variation(y_pred)                # shape [B]
        tv = tf.reduce_mean(tv_raw) / tf.cast(H * W, tf.float32) # normalisation

        # ---- Confidence (rend les proba plus "tranchées")
        # pousse les positifs vers 1 (mais sans exploser)
        pos_term = tf.reduce_mean(y_true * tf.square(1.0 - y_pred))
        # pousse les négatifs vers 0 (anti FP)
        neg_term = tf.reduce_mean((1.0 - y_true) * tf.square(y_pred))

        loss = ft + self.tv_lambda * tv + self.pos_push * pos_term + self.
 ↪neg_push * neg_term

        if self.bce_weight > 0:
            loss = (1.0 - self.bce_weight) * loss + self.bce_weight * self.
 ↪_bce(y_true, y_pred)

        return loss

    def get_config(self):
        cfg = super().get_config()
        cfg.update(dict(alpha=self.alpha, gamma=self.gamma, tv_lambda=self.
 ↪tv_lambda,
                        pos_push=self.pos_push, neg_push=self.neg_push,
                        bce_weight=self.bce_weight, smooth=self.smooth))
        return cfg

# ------------------------------
# 2) Optimizer (LR safe)
# ------------------------------
LR = 4e-6
try:
    opt = tf.keras.optimizers.AdamW(learning_rate=LR, weight_decay=2e-5,␣
 ↪clipnorm=1.0)
except Exception:
    opt = tf.keras.optimizers.Adam(learning_rate=LR, clipnorm=1.0)

# ------------------------------
# 3) Metrics (focus high-threshold precision + recall + dice)
```

```python
# ------------------------------
dice_fn = globals().get("dice_coef", None)
if dice_fn is None:
    def dice_fn(y_true, y_pred, smooth=1.0):
        y_true = tf.cast(y_true, tf.float32)
        y_pred = tf.cast(y_pred, tf.float32)
        inter = tf.reduce_sum(y_true * y_pred)
        denom = tf.reduce_sum(y_true) + tf.reduce_sum(y_pred)
        return (2.0 * inter + smooth) / (denom + smooth)

metrics = [
    tf.keras.metrics.Precision(name="prec@0.95", thresholds=0.95),
    tf.keras.metrics.Precision(name="prec@0.98", thresholds=0.98),
    tf.keras.metrics.Precision(name="prec@0.99", thresholds=0.99),
    tf.keras.metrics.Recall(name="rec@0.50", thresholds=0.50),
    tf.keras.metrics.Recall(name="rec@0.80", thresholds=0.80),
    tf.keras.metrics.MeanMetricWrapper(fn=dice_fn, name="dice"),
]

# reset compiled functions
for attr in ("train_function", "test_function", "predict_function"):
    if hasattr(unet_model, attr):
        setattr(unet_model, attr, None)

unet_model.compile(
    optimizer=opt,
    loss=SharpCleanLoss(
        alpha=0.90, gamma=1.15,
        tv_lambda=2e-5,      # <<<< très faible + normalisée
        pos_push=0.10,       # pousse positifs vers 1 (augmente "clarté")
        neg_push=0.05,       # anti FP
        bce_weight=0.05
    ),
    metrics=metrics,
    run_eagerly=False
)

# ------------------------------
# 4) Fit court + anti-overfit (monitor val_prec@0.98, pas 0.99)
# ------------------------------
out_dir = "./models"
os.makedirs(out_dir, exist_ok=True)
run_id = str(int(time.time()))
ckpt = os.path.join(out_dir, f"best_sharpclean_{run_id}.weights.h5")
logf = os.path.join(out_dir, f"sharpclean_{run_id}.csv")

callbacks = [
```

```
    CSVLogger(logf, append=False),
    TerminateOnNaN(),
    ModelCheckpoint(ckpt, monitor="val_prec@0.98", mode="max",
                    save_best_only=True, save_weights_only=True, verbose=1),
    EarlyStopping(monitor="val_prec@0.98", mode="max",
                  patience=3, min_delta=5e-4, restore_best_weights=True,␣
 ↪verbose=1),
    ReduceLROnPlateau(monitor="val_prec@0.98", mode="max",
                      factor=0.5, patience=1, min_lr=5e-7, verbose=1),
]

hist = unet_model.fit(
    x=train_fit_ds,
    validation_data=val_fit_ds,
    steps_per_epoch=int(steps_per_epoch),
    validation_steps=int(val_steps),
    epochs=12,
    callbacks=callbacks,
    verbose=1
)

print("\n[OK] best weights saved:", ckpt)
print("[OK] log:", logf)

# ------------------------------
# 5) Calibration (chercher prec>=0.98 avec recall MAX, sans aller direct à 0.
 ↪9999)
# ------------------------------
def accumulate_stats(model, ds, steps, thresholds):
    thresholds = np.asarray(thresholds, dtype=np.float32)
    tp = np.zeros((len(thresholds),), dtype=np.float64)
    fp = np.zeros((len(thresholds),), dtype=np.float64)
    fn = np.zeros((len(thresholds),), dtype=np.float64)

    it = iter(ds.take(int(steps)))
    for _ in range(int(steps)):
        x, y = next(it)              # x = (img, grad, hed) OK
        p = model.predict(x, verbose=0)

        yb = (y.numpy() > 0.5).reshape(-1)
        pf = p.reshape(-1).astype(np.float32)

        pred = (pf[None, :] >= thresholds[:, None])
        tp_b = np.sum(pred & yb[None, :], axis=1, dtype=np.float64)
        fp_b = np.sum(pred & (~yb)[None, :], axis=1, dtype=np.float64)
        pos  = float(yb.sum())
        fn_b = pos - tp_b
```

```
        tp += tp_b; fp += fp_b; fn += fn_b

    precision = tp / (tp + fp + 1e-9)
    recall    = tp / (tp + fn + 1e-9)
    dice      = (2.0 * tp) / (2.0 * tp + fp + fn + 1e-9)
    return thresholds, precision, recall, dice

thr_grid = np.unique(np.concatenate([
    np.linspace(0.90, 0.99, 10),
    np.linspace(0.99, 0.999, 10),
    np.linspace(0.999, 0.9999, 6),
])).astype(np.float32)

ths, prec, rec, dice = accumulate_stats(unet_model, val_epoch_ds, val_steps,␣
 ↪thr_grid)

TARGET = 0.98
ok = prec >= TARGET
if np.any(ok):
    best = int(np.argmax(np.where(ok, rec, -1.0)))  # max recall sous contrainte
    print(f"\n[CALIB OK] prec>={TARGET:.2f} (max recall)")
    print(f" thr={ths[best]:.4f} | prec={prec[best]:.4f} | rec={rec[best]:.4f}␣
 ↪| dice={dice[best]:.4f}")
else:
    best = int(np.argmax(prec))
    print(f"\n[CALIB WARN] impossible prec>={TARGET:.2f}, best-prec:")
    print(f" thr={ths[best]:.4f} | prec={prec[best]:.4f} | rec={rec[best]:.4f}␣
 ↪| dice={dice[best]:.4f}")

print(f"\n[FINAL] RECOMMENDED_THR={float(ths[best]):.4f}")
```

```
[INFO] Trainable layers: 25 / 76
Epoch 1/12
160/162            0s 20ms/step -
dice: 0.7641 - loss: 0.1324 - prec@0.95: 0.9409 - prec@0.98: 0.9559 - prec@0.99:
0.9648 - rec@0.50: 0.6905 - rec@0.80: 0.6055
Epoch 1: val_prec@0.98 improved from -inf to 0.82688, saving model to
./models/best_sharpclean_1767339719.weights.h5
162/162            22s 44ms/step -
dice: 0.7641 - loss: 0.1324 - prec@0.95: 0.9409 - prec@0.98: 0.9559 - prec@0.99:
0.9648 - rec@0.50: 0.6905 - rec@0.80: 0.6055 - val_dice: 0.6726 - val_loss:
0.2260 - val_prec@0.95: 0.8081 - val_prec@0.98: 0.8269 - val_prec@0.99: 0.8431 -
val_rec@0.50: 0.5719 - val_rec@0.80: 0.5118 - learning_rate: 4.0000e-06
Epoch 2/12
160/162            0s 20ms/step -
dice: 0.7570 - loss: 0.1332 - prec@0.95: 0.9389 - prec@0.98: 0.9525 - prec@0.99:
```

```
0.9614 - rec@0.50: 0.6771 - rec@0.80: 0.5911
Epoch 2: val_prec@0.98 improved from 0.82688 to 0.83109, saving model to
./models/best_sharpclean_1767339719.weights.h5
162/162            4s 24ms/step -
dice: 0.7571 - loss: 0.1331 - prec@0.95: 0.9390 - prec@0.98: 0.9525 - prec@0.99:
0.9615 - rec@0.50: 0.6771 - rec@0.80: 0.5911 - val_dice: 0.6685 - val_loss:
0.2238 - val_prec@0.95: 0.8133 - val_prec@0.98: 0.8311 - val_prec@0.99: 0.8469 -
val_rec@0.50: 0.5587 - val_rec@0.80: 0.4994 - learning_rate: 4.0000e-06
Epoch 3/12
160/162            0s 20ms/step -
dice: 0.7579 - loss: 0.1249 - prec@0.95: 0.9468 - prec@0.98: 0.9586 - prec@0.99:
0.9666 - rec@0.50: 0.6734 - rec@0.80: 0.5890
Epoch 3: val_prec@0.98 improved from 0.83109 to 0.83210, saving model to
./models/best_sharpclean_1767339719.weights.h5
162/162            4s 24ms/step -
dice: 0.7579 - loss: 0.1249 - prec@0.95: 0.9468 - prec@0.98: 0.9586 - prec@0.99:
0.9666 - rec@0.50: 0.6734 - rec@0.80: 0.5889 - val_dice: 0.6671 - val_loss:
0.2236 - val_prec@0.95: 0.8146 - val_prec@0.98: 0.8321 - val_prec@0.99: 0.8479 -
val_rec@0.50: 0.5536 - val_rec@0.80: 0.4948 - learning_rate: 4.0000e-06
Epoch 4/12
160/162            0s 20ms/step -
dice: 0.7595 - loss: 0.1214 - prec@0.95: 0.9498 - prec@0.98: 0.9610 - prec@0.99:
0.9680 - rec@0.50: 0.6737 - rec@0.80: 0.5902
Epoch 4: val_prec@0.98 improved from 0.83210 to 0.83343, saving model to
./models/best_sharpclean_1767339719.weights.h5
162/162            4s 25ms/step -
dice: 0.7594 - loss: 0.1215 - prec@0.95: 0.9498 - prec@0.98: 0.9609 - prec@0.99:
0.9679 - rec@0.50: 0.6736 - rec@0.80: 0.5902 - val_dice: 0.6638 - val_loss:
0.2234 - val_prec@0.95: 0.8163 - val_prec@0.98: 0.8334 - val_prec@0.99: 0.8494 -
val_rec@0.50: 0.5465 - val_rec@0.80: 0.4878 - learning_rate: 4.0000e-06
Epoch 5/12
160/162            0s 20ms/step -
dice: 0.7461 - loss: 0.1296 - prec@0.95: 0.9436 - prec@0.98: 0.9545 - prec@0.99:
0.9620 - rec@0.50: 0.6550 - rec@0.80: 0.5713
Epoch 5: val_prec@0.98 did not improve from 0.83343

Epoch 5: ReduceLROnPlateau reducing learning rate to 1.9999999949504854e-06.
162/162            4s 23ms/step -
dice: 0.7462 - loss: 0.1295 - prec@0.95: 0.9436 - prec@0.98: 0.9546 - prec@0.99:
0.9621 - rec@0.50: 0.6551 - rec@0.80: 0.5714 - val_dice: 0.6631 - val_loss:
0.2237 - val_prec@0.95: 0.8161 - val_prec@0.98: 0.8328 - val_prec@0.99: 0.8484 -
val_rec@0.50: 0.5444 - val_rec@0.80: 0.4865 - learning_rate: 4.0000e-06
Epoch 6/12
160/162            0s 20ms/step -
dice: 0.7540 - loss: 0.1204 - prec@0.95: 0.9507 - prec@0.98: 0.9614 - prec@0.99:
0.9685 - rec@0.50: 0.6604 - rec@0.80: 0.5790
Epoch 6: val_prec@0.98 did not improve from 0.83343
```

```
Epoch 6: ReduceLROnPlateau reducing learning rate to 9.999999974752427e-07.
162/162            4s 22ms/step -
dice: 0.7540 - loss: 0.1204 - prec@0.95: 0.9506 - prec@0.98: 0.9614 - prec@0.99:
0.9684 - rec@0.50: 0.6603 - rec@0.80: 0.5789 - val_dice: 0.6622 - val_loss:
0.2238 - val_prec@0.95: 0.8155 - val_prec@0.98: 0.8319 - val_prec@0.99: 0.8471 -
val_rec@0.50: 0.5430 - val_rec@0.80: 0.4855 - learning_rate: 2.0000e-06
Epoch 7/12
160/162            0s 20ms/step -
dice: 0.7482 - loss: 0.1267 - prec@0.95: 0.9455 - prec@0.98: 0.9561 - prec@0.99:
0.9633 - rec@0.50: 0.6538 - rec@0.80: 0.5713
Epoch 7: val_prec@0.98 did not improve from 0.83343

Epoch 7: ReduceLROnPlateau reducing learning rate to 5e-07.
162/162            4s 23ms/step -
dice: 0.7483 - loss: 0.1266 - prec@0.95: 0.9455 - prec@0.98: 0.9561 - prec@0.99:
0.9634 - rec@0.50: 0.6538 - rec@0.80: 0.5714 - val_dice: 0.6619 - val_loss:
0.2238 - val_prec@0.95: 0.8155 - val_prec@0.98: 0.8317 - val_prec@0.99: 0.8467 -
val_rec@0.50: 0.5421 - val_rec@0.80: 0.4849 - learning_rate: 1.0000e-06
Epoch 7: early stopping
Restoring model weights from the end of the best epoch: 4.

[OK] best weights saved: ./models/best_sharpclean_1767339719.weights.h5
[OK] log: ./models/sharpclean_1767339719.csv

[CALIB OK] prec>=0.98 (max recall)
 thr=0.9995 | prec=0.9830 | rec=0.0383 | dice=0.0737

[FINAL] RECOMMENDED_THR=0.9995
```

Ici tu as essayé de **récupérer du recall** (K=35 couches trainables au lieu de 25, pénalisation FP plus douce, + un TV-loss pour lisser), mais les résultats montrent que tu es tombé dans l'effet inverse : **le modèle se dégrade très vite** en généralisation. Dès l'epoch 1 tu as ton meilleur état côté "propreté" (val_prec@0.99 **0.8335**, val_dice **0.685**, val_rec@0.50 **0.607**), puis à partir des epochs 2→4 tout **chute fortement** (val_dice descend jusqu'à ~0.45 et val_rec@0.50 jusqu'à ~0.42) alors que la précision stricte ne s'améliore pas, donc ReduceLROnPlateau baisse le LR et **early stopping** restaure l'epoch 1 : ça indique que cette nouvelle loss (surtout la partie TV + fp_pen) pousse l'optimisation vers une solution plus "lisse / prudente" mais qui **efface** de plus en plus les vrais positifs en validation. La calibration confirme le problème : pour atteindre **prec 0.98**, tu es obligé de monter à un seuil **extrême** (**thr=0.9997**) et tu obtiens bien **prec=0.9827**, mais avec un **recall quasi nul (0.0312)** et un **dice très faible (0.0605)**, donc en pratique tu ne segmentes presque rien. Conclusion : ton entraînement "recover recall" n'a pas réellement récupéré le recall sous contrainte de propreté ; au contraire, la dynamique montre un **sur-ajustement / collapse vers under-segmentation**, et ta contrainte de précision 0.98 reste atteignable uniquement en "éteignant" la prédiction positive via un seuil proche de 1.

[ ]:
```python
import os, time
import numpy as np
import tensorflow as tf
```

```
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau,␣
 ↪TerminateOnNaN, ModelCheckpoint, CSVLogger

assert "unet_model" in globals()
assert "train_fit_ds" in globals() and "val_fit_ds" in globals()
assert "steps_per_epoch" in globals() and "val_steps" in globals()

OUT_DIR = "./models"
os.makedirs(OUT_DIR, exist_ok=True)
RUN_ID = str(int(time.time()))

# ------------------------------------------------------------
# 0) Utils: freeze BN + staged unfreeze
# ------------------------------------------------------------
def freeze_batchnorm(m):
    for l in m.layers:
        if isinstance(l, tf.keras.layers.BatchNormalization):
            l.trainable = False

def set_trainable_last_k(m, k_last):
    freeze_batchnorm(m)
    for l in m.layers[:-k_last]:
        l.trainable = False
    for l in m.layers[-k_last:]:
        l.trainable = True
    print(f"[INFO] Trainable layers: {sum(int(l.trainable) for l in m.layers)} /
 ↪ {len(m.layers)}")

# ------------------------------------------------------------
# 1) Loss: Dice + Weighted BCE + HardFP
# ------------------------------------------------------------
@tf.keras.utils.register_keras_serializable()
class DiceWBCEHardFP(tf.keras.losses.Loss):
    """
    - Dice loss: garde la structure (anti masque vide)
    - Weighted BCE: augmente le coût des négatifs => moins de FP
    - HardFP: pénalise très fort les FP confiants (y_pred élevé sur négatifs)
    """
    def __init__(self,
                 neg_w=2.5,           # ↑ => FP ↓ (attention recall)
                 dice_w=0.55,
                 wbce_w=0.35,
                 hardfp_w=0.10,
                 hardfp_p=4.0,        # focus très "hard": FP très confiants
                 smooth=1.0,
                 name="dice_wbce_hardfp"):
        super().__init__(name=name)
```

```
        self.neg_w = float(neg_w)
        self.dice_w = float(dice_w)
        self.wbce_w = float(wbce_w)
        self.hardfp_w = float(hardfp_w)
        self.hardfp_p = float(hardfp_p)
        self.smooth = float(smooth)

    def call(self, y_true, y_pred):
        y_true = tf.cast(y_true, tf.float32)
        y_pred = tf.cast(tf.clip_by_value(y_pred, 1e-6, 1.0 - 1e-6), tf.float32)

        # --- Dice (soft)
        yt = tf.reshape(y_true, [-1])
        yp = tf.reshape(y_pred, [-1])
        inter = tf.reduce_sum(yt * yp)
        denom = tf.reduce_sum(yt) + tf.reduce_sum(yp)
        dice = (2.0 * inter + self.smooth) / (denom + self.smooth)
        dice_loss = 1.0 - dice

        # --- Weighted BCE (negatives heavier)
        # weights: pos=1, neg=neg_w
        w = y_true * 1.0 + (1.0 - y_true) * self.neg_w
        wbce = tf.keras.backend.binary_crossentropy(y_true, y_pred)
        wbce = tf.reduce_mean(w * wbce)

        # --- Hard FP penalty: (1-y)*y_pred^p
        hardfp = tf.reduce_mean((1.0 - y_true) * tf.pow(y_pred, self.hardfp_p))

        return self.dice_w * dice_loss + self.wbce_w * wbce + self.hardfp_w *␣
↪hardfp

    def get_config(self):
        cfg = super().get_config()
        cfg.update(dict(neg_w=self.neg_w, dice_w=self.dice_w, wbce_w=self.
↪wbce_w,
                        hardfp_w=self.hardfp_w, hardfp_p=self.hardfp_p,␣
↪smooth=self.smooth))
        return cfg

# ------------------------------------------------------------
# 2) EMA callback (anti-overfit, améliore val)
# ------------------------------------------------------------
class EMACallback(tf.keras.callbacks.Callback):
    def __init__(self, decay=0.995, save_path=None):
        super().__init__()
        self.decay = float(decay)
        self.save_path = save_path
```

```
        self.ema_weights = None
        self.backup = None

    def on_train_begin(self, logs=None):
        self.ema_weights = [tf.Variable(w, trainable=False) for w in self.model.
 ↪get_weights()]

    def on_train_batch_end(self, batch, logs=None):
        w = self.model.get_weights()
        for ew, cw in zip(self.ema_weights, w):
            ew.assign(self.decay * ew + (1.0 - self.decay) * cw)

    def on_train_end(self, logs=None):
        # apply EMA weights at end + optional save
        self.backup = self.model.get_weights()
        self.model.set_weights([ew.numpy() for ew in self.ema_weights])
        if self.save_path:
            self.model.save_weights(self.save_path)
        # keep EMA weights in model (souvent meilleur en prod)

# ------------------------------------------------------------
# 3) Compile helper
# ------------------------------------------------------------
def make_optimizer(lr):
    try:
        return tf.keras.optimizers.AdamW(learning_rate=lr, weight_decay=3e-5,␣
 ↪clipnorm=1.0)
    except Exception:
        return tf.keras.optimizers.Adam(learning_rate=lr, clipnorm=1.0)

def reset_compile_fns(m):
    for attr in ("train_function", "test_function", "predict_function"):
        if hasattr(m, attr):
            setattr(m, attr, None)

# ------------------------------------------------------------
# 4) Metrics (on cible prec@0.98 mais on surveille dice/recall)
# ------------------------------------------------------------
def _dice_metric(y_true, y_pred, smooth=1.0):
    y_true = tf.cast(y_true, tf.float32)
    y_pred = tf.cast(y_pred, tf.float32)
    inter = tf.reduce_sum(y_true * y_pred)
    denom = tf.reduce_sum(y_true) + tf.reduce_sum(y_pred)
    return (2.0 * inter + smooth) / (denom + smooth)

metrics = [
    tf.keras.metrics.Precision(name="prec@0.95", thresholds=0.95),
```

```
    tf.keras.metrics.Precision(name="prec@0.98", thresholds=0.98),
    tf.keras.metrics.Precision(name="prec@0.99", thresholds=0.99),
    tf.keras.metrics.Recall(name="rec@0.50", thresholds=0.50),
    tf.keras.metrics.Recall(name="rec@0.80", thresholds=0.80),
    tf.keras.metrics.MeanMetricWrapper(fn=_dice_metric, name="dice"),
]

# ============================================================
#   STAGE A: 25 layers, LR small (stabilisation)
# ============================================================
set_trainable_last_k(unet_model, k_last=25)

lossA = DiceWBCEHardFP(
    neg_w=2.5,      # si val_prec@0.98 reste bas -> monter à 3.0
    dice_w=0.55,
    wbce_w=0.35,
    hardfp_w=0.10,
    hardfp_p=4.0
)

optA = make_optimizer(lr=2e-6)

reset_compile_fns(unet_model)
unet_model.compile(optimizer=optA, loss=lossA, metrics=metrics,␣
 ↪run_eagerly=False)

ckptA = os.path.join(OUT_DIR, f"best_fpctlA_{RUN_ID}.weights.h5")
logA  = os.path.join(OUT_DIR, f"fpctlA_{RUN_ID}.csv")
emaA  = os.path.join(OUT_DIR, f"ema_fpctlA_{RUN_ID}.weights.h5")

callbacksA = [
    CSVLogger(logA, append=False),
    TerminateOnNaN(),
    ModelCheckpoint(ckptA, monitor="val_prec@0.98", mode="max",
                    save_best_only=True, save_weights_only=True, verbose=1),
    EarlyStopping(monitor="val_prec@0.98", mode="max",
                  patience=2, min_delta=5e-4, restore_best_weights=True,␣
 ↪verbose=1),
    ReduceLROnPlateau(monitor="val_prec@0.98", mode="max",
                      factor=0.5, patience=1, min_lr=5e-7, verbose=1),
    EMACallback(decay=0.995, save_path=emaA),
]

histA = unet_model.fit(
    x=train_fit_ds,
    validation_data=val_fit_ds,
    steps_per_epoch=int(steps_per_epoch),
```

```
    validation_steps=int(val_steps),
    epochs=6,
    callbacks=callbacksA,
    verbose=1
)

# ============================================================
#   STAGE B: unfreeze + peu plus de capacité (sans overfit)
# ============================================================
set_trainable_last_k(unet_model, k_last=40)

# petite baisse de LR pour éviter sur-ajustement
optB = make_optimizer(lr=1e-6)

# on garde la même loss (ou on durcit un peu hardfp si besoin)
lossB = DiceWBCEHardFP(
    neg_w=2.8,      # <-- léger durcissement anti-FP
    dice_w=0.55,
    wbce_w=0.30,
    hardfp_w=0.15,  # <-- plus de pression sur FP confiants
    hardfp_p=4.0
)

reset_compile_fns(unet_model)
unet_model.compile(optimizer=optB, loss=lossB, metrics=metrics,␣
 ↪run_eagerly=False)

ckptB = os.path.join(OUT_DIR, f"best_fpctlB_{RUN_ID}.weights.h5")
logB  = os.path.join(OUT_DIR, f"fpctlB_{RUN_ID}.csv")
emaB  = os.path.join(OUT_DIR, f"ema_fpctlB_{RUN_ID}.weights.h5")

callbacksB = [
    CSVLogger(logB, append=False),
    TerminateOnNaN(),
    ModelCheckpoint(ckptB, monitor="val_prec@0.98", mode="max",
                    save_best_only=True, save_weights_only=True, verbose=1),
    # backup: si prec stagne mais dice chute -> stop
    EarlyStopping(monitor="val_dice", mode="max",
                  patience=2, min_delta=5e-4, restore_best_weights=True,␣
 ↪verbose=1),
    ReduceLROnPlateau(monitor="val_prec@0.98", mode="max",
                      factor=0.5, patience=1, min_lr=3e-7, verbose=1),
    EMACallback(decay=0.996, save_path=emaB),
]

histB = unet_model.fit(
    x=train_fit_ds,
```

```
    validation_data=val_fit_ds,
    steps_per_epoch=int(steps_per_epoch),
    validation_steps=int(val_steps),
    epochs=6,
    callbacks=callbacksB,
    verbose=1
)

print("\n[OK] Saved:")
print(" - best A:", ckptA)
print(" - ema  A:", emaA)
print(" - best B:", ckptB)
print(" - ema  B:", emaB)
print(" - logs :", logA, "&&", logB)

# ============================================================
# Calibration (prec>=0.98) + contrainte recall_min (utile)
# ============================================================
def accumulate_stats(model, ds, steps, thresholds):
    thresholds = np.asarray(thresholds, dtype=np.float32)
    tp = np.zeros((len(thresholds),), dtype=np.float64)
    fp = np.zeros((len(thresholds),), dtype=np.float64)
    fn = np.zeros((len(thresholds),), dtype=np.float64)

    it = iter(ds.take(int(steps)))
    for _ in range(int(steps)):
        x, y = next(it)                    # x=(img,grad,hed)
        p = model.predict(x, verbose=0)

        yb = (y.numpy() > 0.5).reshape(-1)
        pf = p.reshape(-1).astype(np.float32)

        pred = (pf[None, :] >= thresholds[:, None])
        tp_b = np.sum(pred & yb[None, :], axis=1, dtype=np.float64)
        fp_b = np.sum(pred & (~yb)[None, :], axis=1, dtype=np.float64)
        pos  = float(yb.sum())
        fn_b = pos - tp_b

        tp += tp_b; fp += fp_b; fn += fn_b

    precision = tp / (tp + fp + 1e-9)
    recall    = tp / (tp + fn + 1e-9)
    dice      = (2.0 * tp) / (2.0 * tp + fp + fn + 1e-9)
    return thresholds, precision, recall, dice

# grid plus utile (évite d'aller direct à 0.9999)
thr_grid = np.unique(np.concatenate([
```

```
    np.linspace(0.90, 0.99, 10),
    np.linspace(0.99, 0.999, 10),
])).astype(np.float32)

ths, prec, rec, dice = accumulate_stats(unet_model, val_epoch_ds, val_steps,␣
 ↪thr_grid)

TARGET_PREC = 0.98
RECALL_MIN  = 0.10  # <- change à 0.08 si trop dur
ok = (prec >= TARGET_PREC) & (rec >= RECALL_MIN)

if np.any(ok):
    best = int(np.argmax(np.where(ok, dice, -1.0)))  # max dice sous contraintes
    print(f"\n[CALIB OK] prec>={TARGET_PREC:.2f} & rec>={RECALL_MIN:.2f} (max␣
 ↪dice)")
    print(f" thr={ths[best]:.4f} | prec={prec[best]:.4f} | rec={rec[best]:.4f}␣
 ↪| dice={dice[best]:.4f}")
else:
    # fallback: max recall sous prec>=target
    ok2 = (prec >= TARGET_PREC)
    if np.any(ok2):
        best = int(np.argmax(np.where(ok2, rec, -1.0)))
        print(f"\n[CALIB PARTIAL] prec>={TARGET_PREC:.2f} (max recall)")
        print(f" thr={ths[best]:.4f} | prec={prec[best]:.4f} | rec={rec[best]:.
 ↪4f} | dice={dice[best]:.4f}")
    else:
        best = int(np.argmax(prec))
        print(f"\n[CALIB WARN] impossible prec>={TARGET_PREC:.2f}, best-prec:")
        print(f" thr={ths[best]:.4f} | prec={prec[best]:.4f} | rec={rec[best]:.
 ↪4f} | dice={dice[best]:.4f}")

print(f"\n[FINAL] RECOMMENDED_THR={float(ths[best]):.4f}")
```

```
[INFO] Trainable layers: 25 / 76
Epoch 1/6
162/162            0s 152ms/step -
dice: 0.7551 - loss: 0.2548 - prec@0.95: 0.9455 - prec@0.98: 0.9567 - prec@0.99:
0.9637 - rec@0.50: 0.6671 - rec@0.80: 0.5850
Epoch 1: val_prec@0.98 improved from -inf to 0.83222, saving model to
./models/best_fpctlA_1767339913.weights.h5
162/162            43s 177ms/step -
dice: 0.7551 - loss: 0.2548 - prec@0.95: 0.9455 - prec@0.98: 0.9567 - prec@0.99:
0.9637 - rec@0.50: 0.6671 - rec@0.80: 0.5850 - val_dice: 0.6712 - val_loss:
0.4476 - val_prec@0.95: 0.8137 - val_prec@0.98: 0.8322 - val_prec@0.99: 0.8491 -
val_rec@0.50: 0.5580 - val_rec@0.80: 0.4992 - learning_rate: 2.0000e-06
Epoch 2/6
162/162            0s 154ms/step -
```

```
dice: 0.7748 - loss: 0.2332 - prec@0.95: 0.9476 - prec@0.98: 0.9596 - prec@0.99:
0.9676 - rec@0.50: 0.7004 - rec@0.80: 0.6174
Epoch 2: val_prec@0.98 did not improve from 0.83222

Epoch 2: ReduceLROnPlateau reducing learning rate to 9.999999974752427e-07.
162/162            25s 156ms/step -
dice: 0.7748 - loss: 0.2333 - prec@0.95: 0.9476 - prec@0.98: 0.9595 - prec@0.99:
0.9676 - rec@0.50: 0.7004 - rec@0.80: 0.6173 - val_dice: 0.6788 - val_loss:
0.4437 - val_prec@0.95: 0.8107 - val_prec@0.98: 0.8308 - val_prec@0.99: 0.8487 -
val_rec@0.50: 0.5719 - val_rec@0.80: 0.5128 - learning_rate: 2.0000e-06
Epoch 3/6
162/162            0s 154ms/step -
dice: 0.7768 - loss: 0.2348 - prec@0.95: 0.9426 - prec@0.98: 0.9568 - prec@0.99:
0.9656 - rec@0.50: 0.7103 - rec@0.80: 0.6268
Epoch 3: val_prec@0.98 did not improve from 0.83222

Epoch 3: ReduceLROnPlateau reducing learning rate to 5e-07.
162/162            25s 156ms/step -
dice: 0.7768 - loss: 0.2348 - prec@0.95: 0.9426 - prec@0.98: 0.9568 - prec@0.99:
0.9656 - rec@0.50: 0.7104 - rec@0.80: 0.6268 - val_dice: 0.6821 - val_loss:
0.4409 - val_prec@0.95: 0.8094 - val_prec@0.98: 0.8307 - val_prec@0.99: 0.8495 -
val_rec@0.50: 0.5789 - val_rec@0.80: 0.5192 - learning_rate: 1.0000e-06
Epoch 3: early stopping
Restoring model weights from the end of the best epoch: 1.
[INFO] Trainable layers: 40 / 76
Epoch 1/6
162/162            0s 152ms/step -
dice: 0.7375 - loss: 0.2258 - prec@0.95: 0.8875 - prec@0.98: 0.9023 - prec@0.99:
0.9128 - rec@0.50: 0.6515 - rec@0.80: 0.5770
Epoch 1: val_prec@0.98 improved from -inf to 0.82854, saving model to
./models/best_fpctlB_1767339913.weights.h5
162/162            47s 175ms/step -
dice: 0.7376 - loss: 0.2259 - prec@0.95: 0.8877 - prec@0.98: 0.9025 - prec@0.99:
0.9130 - rec@0.50: 0.6517 - rec@0.80: 0.5771 - val_dice: 0.6806 - val_loss:
0.4208 - val_prec@0.95: 0.8081 - val_prec@0.98: 0.8285 - val_prec@0.99: 0.8465 -
val_rec@0.50: 0.5767 - val_rec@0.80: 0.5177 - learning_rate: 1.0000e-06
Epoch 2/6
162/162            0s 152ms/step -
dice: 0.7781 - loss: 0.2247 - prec@0.95: 0.9404 - prec@0.98: 0.9554 - prec@0.99:
0.9653 - rec@0.50: 0.7142 - rec@0.80: 0.6309
Epoch 2: val_prec@0.98 improved from 0.82854 to 0.82996, saving model to
./models/best_fpctlB_1767339913.weights.h5
162/162            25s 157ms/step -
dice: 0.7781 - loss: 0.2247 - prec@0.95: 0.9404 - prec@0.98: 0.9553 - prec@0.99:
0.9652 - rec@0.50: 0.7142 - rec@0.80: 0.6309 - val_dice: 0.6836 - val_loss:
0.4177 - val_prec@0.95: 0.8083 - val_prec@0.98: 0.8300 - val_prec@0.99: 0.8492 -
val_rec@0.50: 0.5823 - val_rec@0.80: 0.5227 - learning_rate: 1.0000e-06
Epoch 3/6
```

```
162/162            0s 153ms/step -
dice: 0.7709 - loss: 0.2336 - prec@0.95: 0.9314 - prec@0.98: 0.9480 - prec@0.99:
0.9589 - rec@0.50: 0.7111 - rec@0.80: 0.6270
Epoch 3: val_prec@0.98 did not improve from 0.82996

Epoch 3: ReduceLROnPlateau reducing learning rate to 4.999999987376214e-07.
162/162            25s 157ms/step -
dice: 0.7709 - loss: 0.2336 - prec@0.95: 0.9314 - prec@0.98: 0.9480 - prec@0.99:
0.9589 - rec@0.50: 0.7111 - rec@0.80: 0.6271 - val_dice: 0.6864 - val_loss:
0.4163 - val_prec@0.95: 0.8072 - val_prec@0.98: 0.8298 - val_prec@0.99: 0.8497 -
val_rec@0.50: 0.5881 - val_rec@0.80: 0.5281 - learning_rate: 1.0000e-06
Epoch 4/6
162/162            0s 155ms/step -
dice: 0.7794 - loss: 0.2268 - prec@0.95: 0.9355 - prec@0.98: 0.9520 - prec@0.99:
0.9631 - rec@0.50: 0.7231 - rec@0.80: 0.6380
Epoch 4: val_prec@0.98 did not improve from 0.82996

Epoch 4: ReduceLROnPlateau reducing learning rate to 3e-07.
162/162            26s 158ms/step -
dice: 0.7794 - loss: 0.2268 - prec@0.95: 0.9355 - prec@0.98: 0.9520 - prec@0.99:
0.9631 - rec@0.50: 0.7231 - rec@0.80: 0.6380 - val_dice: 0.6881 - val_loss:
0.4160 - val_prec@0.95: 0.8060 - val_prec@0.98: 0.8290 - val_prec@0.99: 0.8490 -
val_rec@0.50: 0.5920 - val_rec@0.80: 0.5318 - learning_rate: 5.0000e-07
Epoch 5/6
162/162            0s 154ms/step -
dice: 0.7809 - loss: 0.2231 - prec@0.95: 0.9360 - prec@0.98: 0.9524 - prec@0.99:
0.9626 - rec@0.50: 0.7296 - rec@0.80: 0.6457
Epoch 5: val_prec@0.98 did not improve from 0.82996
162/162            25s 157ms/step -
dice: 0.7809 - loss: 0.2231 - prec@0.95: 0.9360 - prec@0.98: 0.9524 - prec@0.99:
0.9626 - rec@0.50: 0.7296 - rec@0.80: 0.6457 - val_dice: 0.6885 - val_loss:
0.4154 - val_prec@0.95: 0.8058 - val_prec@0.98: 0.8292 - val_prec@0.99: 0.8498 -
val_rec@0.50: 0.5932 - val_rec@0.80: 0.5328 - learning_rate: 3.0000e-07
Epoch 6/6
162/162            0s 156ms/step -
dice: 0.7772 - loss: 0.2310 - prec@0.95: 0.9329 - prec@0.98: 0.9508 - prec@0.99:
0.9617 - rec@0.50: 0.7220 - rec@0.80: 0.6371
Epoch 6: val_prec@0.98 did not improve from 0.82996
162/162            26s 159ms/step -
dice: 0.7772 - loss: 0.2310 - prec@0.95: 0.9329 - prec@0.98: 0.9508 - prec@0.99:
0.9617 - rec@0.50: 0.7220 - rec@0.80: 0.6371 - val_dice: 0.6896 - val_loss:
0.4151 - val_prec@0.95: 0.8053 - val_prec@0.98: 0.8289 - val_prec@0.99: 0.8495 -
val_rec@0.50: 0.5957 - val_rec@0.80: 0.5351 - learning_rate: 3.0000e-07
Restoring model weights from the end of the best epoch: 6.

[OK] Saved:
 - best A: ./models/best_fpctlA_1767339913.weights.h5
 - ema  A: ./models/ema_fpctlA_1767339913.weights.h5
```

```
 - best B: ./models/best_fpctlB_1767339913.weights.h5
 - ema  B: ./models/ema_fpctlB_1767339913.weights.h5
 - logs : ./models/fpctlA_1767339913.csv && ./models/fpctlB_1767339913.csv

[CALIB WARN] impossible prec>=0.98, best-prec:
 thr=0.9990 | prec=0.9732 | rec=0.0560 | dice=0.1060

[FINAL] RECOMMENDED_THR=0.9990
```

Ton "SharpClean" est **stable** (pas d'effondrement comme le recall-recovery), mais il montre exactement le même verrou : tu arrives à **améliorer légèrement la propreté** en validation (val_prec@0.98 passe de **0.8269 → 0.8334** jusqu'à l'epoch 4, meilleur checkpoint), tout en gardant un **niveau correct de segmentation à seuil 0.5** (val_rec@0.50 **0.5465** et val_dice **0.6638** à l'epoch 4), puis après l'epoch 4 ça **stagne / se dégrade**, donc ReduceLROnPlateau baisse le LR et EarlyStopping restaure l'epoch 4 : c'est un finetune "propre" mais **saturé**. Le point critique est la calibration : pour atteindre **precision 0.98**, le meilleur seuil trouvé reste **extrême** (**thr=0.9995**) et, même si tu obtiens bien **prec=0.9830**, tu payes un prix énorme en détection (**recall=0.0383, dice=0.0737**) → en pratique tu ne gardes presque aucun pixel positif. Conclusion : ce run prouve que **ton modèle sait segmenter correctement à 0.5**, mais que ton objectif "prec 0.98" est **incompatible** avec le comportement probabiliste actuel : lai performance "ultra-propre" n'est atteinte qu'en "éteignant" la prédiction via un seuil proche de 1, donc le problème n'est plus l'overfit mais la **calibration / séparation des scores** (les vrais positifs n'obtiennent pas des probabilités assez proches de 1 pour permettre une précision 0.98 sans tuer le recall).

[ ]:
```python
import os, time
import numpy as np
import tensorflow as tf
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau,␣
 ↪TerminateOnNaN, ModelCheckpoint, CSVLogger

OUT_DIR = "./models"
os.makedirs(OUT_DIR, exist_ok=True)
RUN_ID = str(int(time.time()))

# ---------- Freeze BN + train last K ----------
def freeze_batchnorm(m):
    for l in m.layers:
        if isinstance(l, tf.keras.layers.BatchNormalization):
            l.trainable = False

def set_trainable_last_k(m, k_last):
    freeze_batchnorm(m)
    for l in m.layers[:-k_last]:
        l.trainable = False
    for l in m.layers[-k_last:]:
        l.trainable = True
```

```
    print(f"[INFO] Trainable layers: {sum(int(l.trainable) for l in m.layers)} /
 ↪ {len(m.layers)}")

def reset_compile_fns(m):
    for attr in ("train_function", "test_function", "predict_function"):
        if hasattr(m, attr):
            setattr(m, attr, None)

def make_optimizer(lr):
    try:
        return tf.keras.optimizers.AdamW(learning_rate=lr, weight_decay=3e-5,␣
 ↪clipnorm=1.0)
    except Exception:
        return tf.keras.optimizers.Adam(learning_rate=lr, clipnorm=1.0)

# ============================================================
# 1) Loss: Dice + Weighted BCE + HardFP + TopK Hard Negatives
#    -> TopK NEG force à baisser les FP les + confiants
# ============================================================
@tf.keras.utils.register_keras_serializable()
class DiceWBCEHardFPTopK(tf.keras.losses.Loss):
    def __init__(self,
                 neg_w=3.5,          # ↑ => FP ↓
                 dice_w=0.55,
                 wbce_w=0.25,
                 hardfp_w=0.10,
                 hardfp_p=6.0,       # ↑ focus FP très confiants
                 topk_w=0.10,        # poids TopK hard neg
                 topk_frac=0.002,    # 0.2% pixels neg les + "chauds" (ajuste 0.
 ↪001~0.01)
                 smooth=1.0,
                 name="dice_wbce_hardfp_topk"):
        super().__init__(name=name)
        self.neg_w = float(neg_w)
        self.dice_w = float(dice_w)
        self.wbce_w = float(wbce_w)
        self.hardfp_w = float(hardfp_w)
        self.hardfp_p = float(hardfp_p)
        self.topk_w = float(topk_w)
        self.topk_frac = float(topk_frac)
        self.smooth = float(smooth)

    def call(self, y_true, y_pred):
        y_true = tf.cast(y_true, tf.float32)
        y_pred = tf.cast(tf.clip_by_value(y_pred, 1e-6, 1.0 - 1e-6), tf.float32)

        # --- Dice
```

```
        yt = tf.reshape(y_true, [-1])
        yp = tf.reshape(y_pred, [-1])
        inter = tf.reduce_sum(yt * yp)
        denom = tf.reduce_sum(yt) + tf.reduce_sum(yp)
        dice = (2.0 * inter + self.smooth) / (denom + self.smooth)
        dice_loss = 1.0 - dice

        # --- Weighted BCE (neg heavier)
        w = y_true * 1.0 + (1.0 - y_true) * self.neg_w
        wbce = tf.keras.backend.binary_crossentropy(y_true, y_pred)
        wbce = tf.reduce_mean(w * wbce)

        # --- HardFP (global)
        hardfp = tf.reduce_mean((1.0 - y_true) * tf.pow(y_pred, self.hardfp_p))

        # --- TopK hard negatives (les FP les + confiants)
        neg_scores = tf.reshape((1.0 - y_true) * y_pred, [-1])  # only␣
↪negatives keep score
        n = tf.size(neg_scores)
        k = tf.cast(tf.maximum(1.0, tf.cast(n, tf.float32) * self.topk_frac),␣
↪tf.int32)
        topk_vals = tf.nn.top_k(neg_scores, k=k, sorted=False).values
        topk_pen = tf.reduce_mean(tf.pow(topk_vals, self.hardfp_p))

        return (self.dice_w * dice_loss
                + self.wbce_w * wbce
                + self.hardfp_w * hardfp
                + self.topk_w * topk_pen)

    def get_config(self):
        cfg = super().get_config()
        cfg.update(dict(
            neg_w=self.neg_w, dice_w=self.dice_w, wbce_w=self.wbce_w,
            hardfp_w=self.hardfp_w, hardfp_p=self.hardfp_p,
            topk_w=self.topk_w, topk_frac=self.topk_frac, smooth=self.smooth
        ))
        return cfg

# ============================================================
# 2) Optional: stop si recall s'effondre (anti "masque vide")
# ============================================================
class StopIfValRecallBelow(tf.keras.callbacks.Callback):
    def __init__(self, metric_name="val_rec@0.50", floor=0.45):
        super().__init__()
        self.metric_name = metric_name
        self.floor = float(floor)
    def on_epoch_end(self, epoch, logs=None):
```

```
        logs = logs or {}
        v = logs.get(self.metric_name, None)
        if v is not None and v < self.floor:
            print(f"\n[STOP] {self.metric_name}={v:.4f} < {self.floor:.2f}␣
 ↪(avoid collapse)")
            self.model.stop_training = True

# ============================================================
# 3) Stage C train
#    -> Repars du meilleur checkpoint B ou EMA B si tu veux
# ============================================================
#  Mets ici ton checkpoint si tu veux repartir exactement de B:
# unet_model.load_weights("./models/best_fpctlB_1767339913.weights.h5")

set_trainable_last_k(unet_model, k_last=25)  # reste conservateur (évite␣
 ↪overfit)
lossC = DiceWBCEHardFPTopK(
    neg_w=3.5,
    dice_w=0.55,
    wbce_w=0.25,
    hardfp_w=0.10,
    hardfp_p=6.0,
    topk_w=0.10,
    topk_frac=0.002
)
optC = make_optimizer(lr=7e-7)

metrics = [
    tf.keras.metrics.Precision(name="prec@0.95", thresholds=0.95),
    tf.keras.metrics.Precision(name="prec@0.98", thresholds=0.98),
    tf.keras.metrics.Precision(name="prec@0.99", thresholds=0.99),
    tf.keras.metrics.Recall(name="rec@0.50", thresholds=0.50),
    tf.keras.metrics.Recall(name="rec@0.80", thresholds=0.80),
    tf.keras.metrics.MeanMetricWrapper(
        fn=lambda y_true,y_pred: (2*tf.reduce_sum(tf.cast(y_true,tf.float32)*tf.
 ↪cast(y_pred,tf.float32))+1.0) /
                                 (tf.reduce_sum(tf.cast(y_true,tf.float32))+tf.
 ↪reduce_sum(tf.cast(y_pred,tf.float32))+1.0),
        name="dice"
    )
]

reset_compile_fns(unet_model)
unet_model.compile(optimizer=optC, loss=lossC, metrics=metrics,␣
 ↪run_eagerly=False)

ckptC = os.path.join(OUT_DIR, f"best_fpctlC_{RUN_ID}.weights.h5")
```

```
logC  = os.path.join(OUT_DIR, f"fpctlC_{RUN_ID}.csv")

callbacksC = [
    CSVLogger(logC, append=False),
    TerminateOnNaN(),
    ModelCheckpoint(ckptC, monitor="val_prec@0.98", mode="max",
                    save_best_only=True, save_weights_only=True, verbose=1),
    StopIfValRecallBelow(metric_name="val_rec@0.50", floor=0.45),
    EarlyStopping(monitor="val_prec@0.98", mode="max",
                  patience=2, min_delta=5e-4, restore_best_weights=True,␣
 ↪verbose=1),
    ReduceLROnPlateau(monitor="val_prec@0.98", mode="max",
                      factor=0.5, patience=1, min_lr=2e-7, verbose=1),
]

histC = unet_model.fit(
    x=train_fit_ds,
    validation_data=val_fit_ds,
    steps_per_epoch=int(steps_per_epoch),
    validation_steps=int(val_steps),
    epochs=6,
    callbacks=callbacksC,
    verbose=1
)

print("\n[OK] Stage C saved:", ckptC)
print("[OK] log:", logC)

# ============================================================
# 4)  FIX CALIB GRID: aller jusqu'à 0.99995 (sinon faux WARN)
# ============================================================
def accumulate_stats(model, ds, steps, thresholds):
    thresholds = np.asarray(thresholds, dtype=np.float32)
    tp = np.zeros((len(thresholds),), dtype=np.float64)
    fp = np.zeros((len(thresholds),), dtype=np.float64)
    fn = np.zeros((len(thresholds),), dtype=np.float64)

    it = iter(ds.take(int(steps)))
    for _ in range(int(steps)):
        x, y = next(it)
        p = model.predict(x, verbose=0)

        yb = (y.numpy() > 0.5).reshape(-1)
        pf = p.reshape(-1).astype(np.float32)

        pred = (pf[None, :] >= thresholds[:, None])
        tp_b = np.sum(pred & yb[None, :], axis=1, dtype=np.float64)
```

```
        fp_b = np.sum(pred & (~yb)[None, :], axis=1, dtype=np.float64)
        pos  = float(yb.sum())
        fn_b = pos - tp_b

        tp += tp_b; fp += fp_b; fn += fn_b

    precision = tp / (tp + fp + 1e-9)
    recall    = tp / (tp + fn + 1e-9)
    dice      = (2.0 * tp) / (2.0 * tp + fp + fn + 1e-9)
    return thresholds, precision, recall, dice

thr_grid = np.unique(np.concatenate([
    np.linspace(0.90, 0.99, 10),
    np.linspace(0.99, 0.999, 10),
    np.linspace(0.999, 0.99995, 10),   #  important
])).astype(np.float32)

TARGET_PREC = 0.98
RECALL_MIN  = 0.10   # adapte (0.08 / 0.12)

ths, prec, rec, dice = accumulate_stats(unet_model, val_epoch_ds, val_steps,␣
 ↪thr_grid)

ok = (prec >= TARGET_PREC) & (rec >= RECALL_MIN)
if np.any(ok):
    best = int(np.argmax(np.where(ok, dice, -1.0)))
    print(f"\n[CALIB OK] prec>={TARGET_PREC:.2f} & rec>={RECALL_MIN:.2f} (max␣
 ↪dice)")
    print(f" thr={ths[best]:.5f} | prec={prec[best]:.4f} | rec={rec[best]:.4f}␣
 ↪| dice={dice[best]:.4f}")
else:
    ok2 = (prec >= TARGET_PREC)
    if np.any(ok2):
        best = int(np.argmax(np.where(ok2, rec, -1.0)))
        print(f"\n[CALIB PARTIAL] prec>={TARGET_PREC:.2f} (max recall)")
        print(f" thr={ths[best]:.5f} | prec={prec[best]:.4f} | rec={rec[best]:.
 ↪4f} | dice={dice[best]:.4f}")
    else:
        best = int(np.argmax(prec))
        print(f"\n[CALIB WARN] impossible prec>={TARGET_PREC:.2f}, best-prec:")
        print(f" thr={ths[best]:.5f} | prec={prec[best]:.4f} | rec={rec[best]:.
 ↪4f} | dice={dice[best]:.4f}")

print(f"\n[FINAL] RECOMMENDED_THR={float(ths[best]):.5f}")
```

```
[INFO] Trainable layers: 25 / 76
Epoch 1/6
```

```
162/162            0s 25ms/step -
dice: 0.7856 - loss: 0.3069 - prec@0.95: 0.9385 - prec@0.98: 0.9542 - prec@0.99:
0.9639 - rec@0.50: 0.7331 - rec@0.80: 0.6475
Epoch 1: val_prec@0.98 improved from -inf to 0.83043, saving model to
./models/best_fpctlC_1767341360.weights.h5
162/162            25s 56ms/step -
dice: 0.7856 - loss: 0.3069 - prec@0.95: 0.9385 - prec@0.98: 0.9542 - prec@0.99:
0.9639 - rec@0.50: 0.7331 - rec@0.80: 0.6476 - val_dice: 0.6886 - val_loss:
0.4970 - val_prec@0.95: 0.8069 - val_prec@0.98: 0.8304 - val_prec@0.99: 0.8511 -
val_rec@0.50: 0.5935 - val_rec@0.80: 0.5326 - learning_rate: 7.0000e-07
Epoch 2/6
162/162            0s 25ms/step -
dice: 0.7833 - loss: 0.3117 - prec@0.95: 0.9365 - prec@0.98: 0.9522 - prec@0.99:
0.9626 - rec@0.50: 0.7311 - rec@0.80: 0.6446
Epoch 2: val_prec@0.98 improved from 0.83043 to 0.83063, saving model to
./models/best_fpctlC_1767341360.weights.h5
162/162            5s 31ms/step -
dice: 0.7833 - loss: 0.3117 - prec@0.95: 0.9365 - prec@0.98: 0.9522 - prec@0.99:
0.9626 - rec@0.50: 0.7311 - rec@0.80: 0.6446 - val_dice: 0.6897 - val_loss:
0.4963 - val_prec@0.95: 0.8066 - val_prec@0.98: 0.8306 - val_prec@0.99: 0.8515 -
val_rec@0.50: 0.5963 - val_rec@0.80: 0.5351 - learning_rate: 7.0000e-07
Epoch 3/6
162/162            0s 25ms/step -
dice: 0.7862 - loss: 0.3053 - prec@0.95: 0.9379 - prec@0.98: 0.9530 - prec@0.99:
0.9623 - rec@0.50: 0.7407 - rec@0.80: 0.6533
Epoch 3: val_prec@0.98 improved from 0.83063 to 0.83175, saving model to
./models/best_fpctlC_1767341360.weights.h5
162/162            5s 30ms/step -
dice: 0.7862 - loss: 0.3053 - prec@0.95: 0.9379 - prec@0.98: 0.9531 - prec@0.99:
0.9623 - rec@0.50: 0.7407 - rec@0.80: 0.6533 - val_dice: 0.6904 - val_loss:
0.4948 - val_prec@0.95: 0.8073 - val_prec@0.98: 0.8318 - val_prec@0.99: 0.8531 -
val_rec@0.50: 0.5980 - val_rec@0.80: 0.5362 - learning_rate: 7.0000e-07
Epoch 4/6
161/162            0s 25ms/step -
dice: 0.7905 - loss: 0.3007 - prec@0.95: 0.9397 - prec@0.98: 0.9556 - prec@0.99:
0.9657 - rec@0.50: 0.7476 - rec@0.80: 0.6619
Epoch 4: val_prec@0.98 did not improve from 0.83175

Epoch 4: ReduceLROnPlateau reducing learning rate to 3.4999999343199306e-07.
162/162            5s 29ms/step -
dice: 0.7906 - loss: 0.3007 - prec@0.95: 0.9397 - prec@0.98: 0.9556 - prec@0.99:
0.9657 - rec@0.50: 0.7476 - rec@0.80: 0.6619 - val_dice: 0.6909 - val_loss:
0.4948 - val_prec@0.95: 0.8066 - val_prec@0.98: 0.8315 - val_prec@0.99: 0.8530 -
val_rec@0.50: 0.6000 - val_rec@0.80: 0.5377 - learning_rate: 7.0000e-07
Epoch 5/6
161/162            0s 25ms/step -
dice: 0.7866 - loss: 0.3102 - prec@0.95: 0.9367 - prec@0.98: 0.9551 - prec@0.99:
0.9669 - rec@0.50: 0.7415 - rec@0.80: 0.6539
```

```
Epoch 5: val_prec@0.98 did not improve from 0.83175

Epoch 5: ReduceLROnPlateau reducing learning rate to 2e-07.
162/162            5s 29ms/step -
dice: 0.7867 - loss: 0.3101 - prec@0.95: 0.9367 - prec@0.98: 0.9552 - prec@0.99:
0.9669 - rec@0.50: 0.7416 - rec@0.80: 0.6540 - val_dice: 0.6922 - val_loss:
0.4960 - val_prec@0.95: 0.8050 - val_prec@0.98: 0.8299 - val_prec@0.99: 0.8509 -
val_rec@0.50: 0.6032 - val_rec@0.80: 0.5411 - learning_rate: 3.5000e-07
Epoch 5: early stopping
Restoring model weights from the end of the best epoch: 3.

[OK] Stage C saved: ./models/best_fpctlC_1767341360.weights.h5
[OK] log: ./models/fpctlC_1767341360.csv

[CALIB PARTIAL] prec>=0.98 (max recall)
 thr=0.99921 | prec=0.9816 | rec=0.0439 | dice=0.0840

[FINAL] RECOMMENDED_THR=0.99921
```

ce **Stage C** est plutôt rassurant : en validation je garde un **Dice** stable autour de ~0,69 et je grappille un peu en **propreté** (val_prec@0.98 monte vers ~0,8318) tout en améliorant légèrement la **détection** (val_rec@0.50 monte vers ~0,60). Donc la loss "TopK hard negatives" fait bien son rôle : elle réduit surtout les **faux positifs les plus confiants** sans provoquer une grosse chute du recall. Par contre, au moment de la **calibration**, le problème reste le même : pour atteindre **precision 0,98**, je suis obligé de pousser le seuil très haut (~**0,99921**), et à ce seuil le modèle devient ultra conservateur : le **recall s'effondre** (~0,044) et le **dice** devient faible (~0,084), ce qui revient quasiment à produire un masque presque vide juste pour rester "propre". En résumé : à **seuil standard (0,5)**, Stage C améliore un peu le compromis, mais l'objectif **0,98 de précision pixel-level** est trop exigeant avec les scores actuels, car il force un seuil extrême qui détruit la couverture.

[ ]:
```python
# ============================================================
#  STAGE D: "FP-HINGE PUSH" (casse les FP très confiants)
# Objectif: augmenter val_prec@0.98 sans collapse (anti-overfit)
# ============================================================

import os, time
import numpy as np
import tensorflow as tf
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau,␣
 ↪TerminateOnNaN, ModelCheckpoint, CSVLogger

OUT_DIR = "./models"
os.makedirs(OUT_DIR, exist_ok=True)
RUN_ID = str(int(time.time()))

# ---------- Freeze BN + train last K ----------
def freeze_batchnorm(m):
```

```
    for l in m.layers:
        if isinstance(l, tf.keras.layers.BatchNormalization):
            l.trainable = False

def set_trainable_last_k(m, k_last):
    freeze_batchnorm(m)
    for l in m.layers[:-k_last]:
        l.trainable = False
    for l in m.layers[-k_last:]:
        l.trainable = True
    print(f"[INFO] Trainable layers: {sum(int(l.trainable) for l in m.layers)} /
 ↪ {len(m.layers)}")

def reset_compile_fns(m):
    for attr in ("train_function", "test_function", "predict_function"):
        if hasattr(m, attr):
            setattr(m, attr, None)

def make_optimizer(lr):
    try:
        return tf.keras.optimizers.AdamW(learning_rate=lr, weight_decay=2e-5,␣
 ↪clipnorm=1.0)
    except Exception:
        return tf.keras.optimizers.Adam(learning_rate=lr, clipnorm=1.0)

# ============================================================
# Loss: Dice + WBCE + TopK Neg + "FP-Hinge" au-dessus de tau
#   - tau = 0.995 (punit les neg > 0.995)
#   - ça vise directement val_prec@0.98
# ============================================================
@tf.keras.utils.register_keras_serializable()
class DiceWBCE_TopK_HingeFP(tf.keras.losses.Loss):
    def __init__(self,
                 neg_w=4.0,
                 dice_w=0.55,
                 wbce_w=0.20,
                 topk_w=0.10,
                 topk_frac=0.003,
                 hinge_w=0.15,
                 tau=0.995,
                 p=6.0,
                 smooth=1.0,
                 name="dice_wbce_topk_hingefp"):
        super().__init__(name=name)
        self.neg_w = float(neg_w)
        self.dice_w = float(dice_w)
        self.wbce_w = float(wbce_w)
```

```
        self.topk_w = float(topk_w)
        self.topk_frac = float(topk_frac)
        self.hinge_w = float(hinge_w)
        self.tau = float(tau)
        self.p = float(p)
        self.smooth = float(smooth)

    def call(self, y_true, y_pred):
        y_true = tf.cast(y_true, tf.float32)
        y_pred = tf.cast(tf.clip_by_value(y_pred, 1e-6, 1.0 - 1e-6), tf.float32)

        # --- Dice
        yt = tf.reshape(y_true, [-1])
        yp = tf.reshape(y_pred, [-1])
        inter = tf.reduce_sum(yt * yp)
        denom = tf.reduce_sum(yt) + tf.reduce_sum(yp)
        dice = (2.0 * inter + self.smooth) / (denom + self.smooth)
        dice_loss = 1.0 - dice

        # --- Weighted BCE (neg heavier)
        w = y_true * 1.0 + (1.0 - y_true) * self.neg_w
        wbce = tf.keras.backend.binary_crossentropy(y_true, y_pred)
        wbce = tf.reduce_mean(w * wbce)

        # --- TopK hard negatives (FP les + confiants)
        neg_scores = tf.reshape((1.0 - y_true) * y_pred, [-1])
        n = tf.size(neg_scores)
        k = tf.cast(tf.maximum(1.0, tf.cast(n, tf.float32) * self.topk_frac),␣
↪tf.int32)
        topk_vals = tf.nn.top_k(neg_scores, k=k, sorted=False).values
        topk_pen = tf.reduce_mean(tf.pow(topk_vals, self.p))

        # --- FP hinge: punit uniquement les neg au-dessus de tau
        #     relu(p - tau) -> 0 si p<=tau, sinon pénalité forte
        hinge = tf.nn.relu(y_pred - self.tau)  # [0..1]
        hingefp = tf.reduce_mean((1.0 - y_true) * tf.pow(hinge, 2.0))

        return (self.dice_w * dice_loss
                + self.wbce_w * wbce
                + self.topk_w * topk_pen
                + self.hinge_w * hingefp)

    def get_config(self):
        cfg = super().get_config()
        cfg.update(dict(
            neg_w=self.neg_w, dice_w=self.dice_w, wbce_w=self.wbce_w,
            topk_w=self.topk_w, topk_frac=self.topk_frac,
```

```
            hinge_w=self.hinge_w, tau=self.tau, p=self.p,
            smooth=self.smooth
        ))
        return cfg

# Anti-collapse (masque vide)
class StopIfValRecallBelow(tf.keras.callbacks.Callback):
    def __init__(self, metric_name="val_rec@0.50", floor=0.55):
        super().__init__()
        self.metric_name = metric_name
        self.floor = float(floor)
    def on_epoch_end(self, epoch, logs=None):
        logs = logs or {}
        v = logs.get(self.metric_name, None)
        if v is not None and v < self.floor:
            print(f"\n[STOP] {self.metric_name}={v:.4f} < {self.floor:.2f}")
            self.model.stop_training = True

# ============================================================
# Train Stage D
# ============================================================
# Option: repartir du meilleur C (ou B)
# unet_model.load_weights("./models/best_fpctlC_1767341360.weights.h5")

set_trainable_last_k(unet_model, k_last=15)  # plus conservateur que 25 =>␣
 ↪moins overfit
lossD = DiceWBCE_TopK_HingeFP(
    neg_w=4.0,
    dice_w=0.55,
    wbce_w=0.20,
    topk_w=0.10,
    topk_frac=0.003,
    hinge_w=0.15,
    tau=0.995,     #  : écrase FP très confiants
    p=6.0
)
optD = make_optimizer(lr=3e-7)

metrics = [
    tf.keras.metrics.Precision(name="prec@0.95", thresholds=0.95),
    tf.keras.metrics.Precision(name="prec@0.98", thresholds=0.98),
    tf.keras.metrics.Precision(name="prec@0.99", thresholds=0.99),
    tf.keras.metrics.Recall(name="rec@0.50", thresholds=0.50),
    tf.keras.metrics.Recall(name="rec@0.80", thresholds=0.80),
]

reset_compile_fns(unet_model)
```

```
unet_model.compile(optimizer=optD, loss=lossD, metrics=metrics,␣
 ↪run_eagerly=False)

ckptD = os.path.join(OUT_DIR, f"best_fpctlD_{RUN_ID}.weights.h5")
logD  = os.path.join(OUT_DIR, f"fpctlD_{RUN_ID}.csv")

callbacksD = [
    CSVLogger(logD, append=False),
    TerminateOnNaN(),
    ModelCheckpoint(ckptD, monitor="val_prec@0.98", mode="max",
                    save_best_only=True, save_weights_only=True, verbose=1),
    StopIfValRecallBelow(metric_name="val_rec@0.50", floor=0.55),
    EarlyStopping(monitor="val_prec@0.98", mode="max",
                  patience=2, min_delta=8e-4, restore_best_weights=True,␣
 ↪verbose=1),
    ReduceLROnPlateau(monitor="val_prec@0.98", mode="max",
                      factor=0.5, patience=1, min_lr=1e-7, verbose=1),
]

unet_model.fit(
    x=train_fit_ds,
    validation_data=val_fit_ds,
    steps_per_epoch=int(steps_per_epoch),
    validation_steps=int(val_steps),
    epochs=6,
    callbacks=callbacksD,
    verbose=1
)

print("\n[OK] Stage D saved:", ckptD)
print("[OK] log:", logD)

# ============================================================
# Calibration grid (jusqu'à 0.99995) - inchangé
# -> Reprends ton bloc accumulate_stats + thr_grid étendu
# ============================================================
```

```
[INFO] Trainable layers: 15 / 76
Epoch 1/6
162/162             0s 24ms/step -
loss: 0.2940 - prec@0.95: 0.9356 - prec@0.98: 0.9519 - prec@0.99: 0.9621 -
rec@0.50: 0.7382 - rec@0.80: 0.6512
Epoch 1: val_prec@0.98 improved from -inf to 0.83050, saving model to
./models/best_fpctlD_1767341487.weights.h5
162/162             23s 56ms/step -
loss: 0.2939 - prec@0.95: 0.9356 - prec@0.98: 0.9519 - prec@0.99: 0.9621 -
rec@0.50: 0.7382 - rec@0.80: 0.6513 - val_loss: 0.4624 - val_prec@0.95: 0.8060 -
```

```
val_prec@0.98: 0.8305 - val_prec@0.99: 0.8516 - val_rec@0.50: 0.5999 -
val_rec@0.80: 0.5381 - learning_rate: 3.0000e-07
Epoch 2/6
160/162            0s 24ms/step -
loss: 0.2827 - prec@0.95: 0.9435 - prec@0.98: 0.9594 - prec@0.99: 0.9690 -
rec@0.50: 0.7470 - rec@0.80: 0.6608
Epoch 2: val_prec@0.98 did not improve from 0.83050

Epoch 2: ReduceLROnPlateau reducing learning rate to 1.500000053056283e-07.
162/162            5s 28ms/step -
loss: 0.2828 - prec@0.95: 0.9434 - prec@0.98: 0.9594 - prec@0.99: 0.9690 -
rec@0.50: 0.7470 - rec@0.80: 0.6609 - val_loss: 0.4628 - val_prec@0.95: 0.8053 -
val_prec@0.98: 0.8299 - val_prec@0.99: 0.8508 - val_rec@0.50: 0.6012 -
val_rec@0.80: 0.5393 - learning_rate: 3.0000e-07
Epoch 3/6
162/162            0s 24ms/step -
loss: 0.2818 - prec@0.95: 0.9416 - prec@0.98: 0.9575 - prec@0.99: 0.9672 -
rec@0.50: 0.7524 - rec@0.80: 0.6666
Epoch 3: val_prec@0.98 did not improve from 0.83050

Epoch 3: ReduceLROnPlateau reducing learning rate to 1e-07.
162/162            5s 28ms/step -
loss: 0.2818 - prec@0.95: 0.9416 - prec@0.98: 0.9575 - prec@0.99: 0.9672 -
rec@0.50: 0.7524 - rec@0.80: 0.6666 - val_loss: 0.4632 - val_prec@0.95: 0.8046 -
val_prec@0.98: 0.8294 - val_prec@0.99: 0.8503 - val_rec@0.50: 0.6020 -
val_rec@0.80: 0.5400 - learning_rate: 1.5000e-07
Epoch 3: early stopping
Restoring model weights from the end of the best epoch: 1.

[OK] Stage D saved: ./models/best_fpctlD_1767341487.weights.h5
[OK] log: ./models/fpctlD_1767341487.csv
```

Pour moi, **Stage D n'a pas vraiment "cassé" les FP comme je voulais**, mais au moins il n'a pas détruit le modèle : dès l'epoch 1 je suis à **val_prec@0.98 0.8305** (donc quasiment identique à Stage C ~0.8318), avec un **val_recall@0.50 autour de 0.60** (même ordre), donc globalement **stable**. Ensuite, malgré la hinge (tau=0.995) + TopK + WBCE négatifs plus lourds, **ça ne progresse plus** : dès l'epoch 2–3 la precision 0.98 en validation **stagne puis baisse légèrement**, le scheduler réduit le LR, et on s'arrête tôt en restaurant les poids de l'epoch 1. Donc ma lecture c'est : **la contrainte "propreté extrême" est surtout limitée par la calibration / la distribution des scores**, pas par un manque de pénalisation des FP très confiants ; même en tapant spécifiquement les négatifs au-dessus de 0.995, je ne gagne pratiquement rien sur **val_prec@0.98**, et du coup je confirme que le "mur" est ailleurs (qualité labels, imbalance, features/prior, ou séparation score pos/neg insuffisante). En clair : **Stage D = pas pire, mais pas mieux**, et si mon objectif est de monter vraiment au-dessus de ~0.83 en prec@0.98, je dois changer quelque chose de plus structurel que juste durcir encore la loss.

[ ]:
```
import os, re, math, numpy as np, tensorflow as tf, matplotlib.pyplot as plt
```

```
# ============================================================
# CONFIG (defaults may be overridden by existing globals)
# ============================================================
CFG = {
    "IMG_H": int(globals().get("IMG_H", 256)),
    "IMG_W": int(globals().get("IMG_W", 256)),
    "IMG_C": int(globals().get("IMG_C", 1)),
    "BATCH_SIZE": int(globals().get("BATCH_SIZE", 8)),

    "TARGET_PRECISION": float(globals().get("TARGET_PRECISION", 0.90)),
    "CALIB_OBJECTIVE": str(globals().get("CALIB_OBJECTIVE", "dice")),  # "dice"␣
 ↪| "recall"

    # threshold grids: can be list/np.array; if int -> create linspace
    "THR_GRID_COARSE": globals().get("THR_GRID_COARSE", 80),
    "THR_GRID_DENSE": globals().get("THR_GRID_DENSE", 240),

    "THR_EVAL": float(globals().get("THR_EVAL", 0.50)),
    "N_SHOW": int(globals().get("N_SHOW", 3)),

    "MAX_ITEMS_TRAIN": int(globals().get("MAX_ITEMS_TRAIN", 512)),
    "MAX_ITEMS_VAL": int(globals().get("MAX_ITEMS_VAL", 512)),
    "MAX_PRED_EVAL": int(globals().get("MAX_PRED_EVAL", 256)),
}

# ============================================================
# 0) Minimal checks + dataset source selection
# ============================================================
assert "unet_model" in globals(), "unet_model introuvable."
assert "train_ds" in globals(), "train_ds introuvable."
assert "val_ds" in globals(), "val_ds introuvable."

train_source = globals().get("train_epoch_ds", train_ds)
val_source   = globals().get("val_epoch_ds", val_ds)

MODEL_INPUTS_FLAT = tf.nest.flatten(unet_model.inputs)
MODEL_N_IN = len(MODEL_INPUTS_FLAT)  # (A) required
MODEL_IN_SHAPES = [tuple(t.shape) for t in MODEL_INPUTS_FLAT]
MODEL_IN_DTYPES = [t.dtype for t in MODEL_INPUTS_FLAT]

MODEL_OUT = tf.nest.flatten(unet_model.outputs)[0]
MODEL_OUT_SHAPE = tuple(MODEL_OUT.shape)
MODEL_OUT_DTYPE = MODEL_OUT.dtype

# ============================================================
# 1) Helpers: names, fallback Sobel prior, dice, metrics
# ============================================================
```

```
def _clean_tensor_name(name: str) -> str:
    # Keras tensor names often like 'input_1:0' or 'model/input_1:0'
    s = str(name)
    s = s.split(":")[0]
    s = s.split("/")[-1]
    return s

def _norm_key(s: str) -> str:
    s = re.sub(r"[^a-z0-9]+", "_", s.lower()).strip("_")
    return s

MODEL_IN_NAMES_RAW = [getattr(t, "name", f"input_{i}") for i, t in␣
 ↪enumerate(MODEL_INPUTS_FLAT)]
MODEL_IN_NAMES = [_clean_tensor_name(n) for n in MODEL_IN_NAMES_RAW]
MODEL_IN_KEYS = [_norm_key(n) for n in MODEL_IN_NAMES]

def _looks_like_prior_name(normed: str) -> bool:
    # Allowed fallback only for these semantics
    pats = ["prior", "hed", "edge", "grad", "gradient", "canny", "sobel"]
    return any(p in normed for p in pats)

def make_sobel_prior_tf(x):
    """
    Deterministic fallback: Sobel magnitude normalized to [0,1], never zeros␣
 ↪unless constant image.
    Supports x: (B,H,W,C) or (H,W,C) or (H,W,1).
    Returns: (same spatial, 1 channel) float32 in [0,1]
    """
    x = tf.convert_to_tensor(x)
    if x.dtype != tf.float32:
        x = tf.cast(x, tf.float32)

    if x.shape.rank == 3:
        x = tf.expand_dims(x, 0)  # (1,H,W,C)
    if x.shape.rank != 4:
        raise ValueError(f"[FALLBACK] Sobel expects rank 3/4 tensor, got␣
 ↪rank={x.shape.rank}")

    # grayscale for edge magnitude
    c = x.shape[-1]
    if c is not None and c > 1:
        xg = tf.reduce_mean(x, axis=-1, keepdims=True)
    else:
        xg = x

    se = tf.image.sobel_edges(xg)  # (B,H,W,1,2)
    mag = tf.sqrt(tf.reduce_sum(tf.square(se), axis=-1))  # (B,H,W,1)
```

```
    # robust per-sample normalization
    mmax = tf.reduce_max(mag, axis=[1,2,3], keepdims=True)
    mag = tf.where(mmax > 0, mag / (mmax + 1e-6), mag)
    mag = tf.clip_by_value(mag, 0.0, 1.0)
    return mag

def dice_coefficient_np(y_true, y_pred, smooth=1e-6):
    y_true_f = y_true.reshape(-1).astype(bool)
    y_pred_f = y_pred.reshape(-1).astype(bool)
    inter = np.logical_and(y_true_f, y_pred_f).sum()
    denom = y_true_f.sum() + y_pred_f.sum()
    return float((2.0 * inter + smooth) / (denom + smooth))

def _tp_fp_fn_np(y_true_bool, y_pred_bool):
    tp = np.logical_and(y_true_bool, y_pred_bool).sum(dtype=np.int64)
    fp = np.logical_and(~y_true_bool, y_pred_bool).sum(dtype=np.int64)
    fn = np.logical_and(y_true_bool, ~y_pred_bool).sum(dtype=np.int64)
    return tp, fp, fn

# ============================================================
# 2) Standardized data collection from already batched datasets
# ============================================================

def _collect_numpy(ds, max_items, name):
    inputs_acc = [[] for _ in range(MODEL_N_IN)]
    ys_acc = []
    n = 0

    for x_bundle, y_batch in ds: # ds is already batched, so x_bundle is (xb,␣
 ↪gb, hb) and y_batch is yb
        x_list = list(x_bundle) if isinstance(x_bundle, (tuple, list)) else␣
 ↪[x_bundle]

        # Ensure all tensors are float32 and NumPy arrays
        current_batch_size = x_list[0].shape[0] if x_list else 0
        if current_batch_size == 0: continue

        take_b = current_batch_size
        if max_items is not None and n + current_batch_size > max_items:
            take_b = max(0, max_items - n)
            if take_b <= 0: break

        for i in range(MODEL_N_IN):
            inputs_acc[i].append(tf.cast(x_list[i][:take_b], tf.float32).
 ↪numpy())
        ys_acc.append(tf.cast(y_batch[:take_b], tf.float32).numpy())
```

```
        n += take_b
        if max_items is not None and n >= max_items:
            break

    Xs = [np.concatenate(v, axis=0) if len(v) else np.zeros((0,), np.float32)␣
 ↪for v in inputs_acc]
    Y  = np.concatenate(ys_acc, axis=0) if len(ys_acc) else np.zeros((0,), np.
 ↪float32)

    # clip if image-like
    for i in range(MODEL_N_IN):
        if Xs[i].ndim >= 3:
            Xs[i] = np.clip(Xs[i], 0.0, 1.0)
    if Y.ndim >= 3:
        Y = np.clip(Y, 0.0, 1.0)

    print(f"[OK] {name}: collected n={n} | inputs={[x.shape for x in Xs]} |␣
 ↪Y={Y.shape}")
    return Xs, Y

# ============================================================
# 3) Warmup forward pass (C) - No longer needed here as _collect_numpy handles␣
 ↪batched elements
# ============================================================
# The previous warmup was causing an error due to redundant standardization.
# It's not strictly necessary here as the full dataset will be collected␣
 ↪directly.
# If a warmup is desired, it should be done on a single element from␣
 ↪train_epoch_ds/val_epoch_ds
# after it's passed through its own batching/mapping.

# Directly collect data for evaluation
Xs_train, Y_train = _collect_numpy(train_source, min(CFG["MAX_ITEMS_TRAIN"],␣
 ↪CFG["MAX_PRED_EVAL"]), "TRAIN_EVAL")
Xs_val,   Y_val   = _collect_numpy(val_source,   min(CFG["MAX_ITEMS_VAL"],  ␣
 ↪CFG["MAX_PRED_EVAL"]), "VAL_EVAL")

# ============================================================
# 4) Predict + Dice distribution + plots + examples (D)
# ============================================================
def _predict_probs(Xs, batch_size):
    if len(Xs) == 0 or (isinstance(Xs[0], np.ndarray) and Xs[0].shape[0] == 0):
        return np.zeros((0,), np.float32)

    # The model expects a list of inputs if MODEL_N_IN > 1
```

```
    inp = Xs if MODEL_N_IN > 1 else Xs[0] # Xs is already a list of NumPy␣
 ↪arrays (N,H,W,C)

    probs = unet_model.predict(inp, batch_size=batch_size, verbose=1)
    probs = np.asarray(probs)
    probs = np.clip(probs, 0.0, 1.0)
    return probs

def _primary_image_index():
    # choose an input that looks like image; prefer name contains img/image/x,␣
 ↪else first rank4 input
    for i, k in enumerate(MODEL_IN_KEYS):
        if any(t in k for t in ["img","image","x","input"]):
            return i
    return 0

def eval_and_plot(name, Xs, Y, thr, n_show, max_pred=None):
    if max_pred is not None and isinstance(Y, np.ndarray) and Y.shape[0] >␣
 ↪max_pred:
        idx = np.random.choice(Y.shape[0], size=max_pred, replace=False)
        Xs = [x[idx] for x in Xs]
        Y = Y[idx]
        print(f"[INFO] {name}: subsample={max_pred}")

    probs = _predict_probs(Xs, CFG["BATCH_SIZE"])
    if probs.ndim == 3:  # (N,H,W) -> (N,H,W,1)
        probs = probs[..., None]
    y_true = (Y >= 0.5).astype(np.uint8)
    y_bin  = (probs >= thr).astype(np.uint8)

    dice_scores = np.array([dice_coefficient_np(y_true[i], y_bin[i]) for i in␣
 ↪range(y_true.shape[0])], dtype=np.float32)

    print(f"\n[{name}] THR={thr}")
    print(f"  Dice mean={float(dice_scores.mean()):.6f} |␣
 ↪min={float(dice_scores.min()):.6f} | max={float(dice_scores.max()):.6f}")
    q = np.quantile(dice_scores, [0.05, 0.25, 0.50, 0.75, 0.95])
    print(f"  Dice quantiles: p05={q[0]:.3f} | p25={q[1]:.3f} | p50={q[2]:.3f}␣
 ↪| p75={q[3]:.3f} | p95={q[4]:.3f}")

    # Histogram + Boxplot + ECDF
    fig = plt.figure(figsize=(14, 4))
    ax1 = plt.subplot(1, 3, 1)
    ax1.hist(dice_scores, bins=20)
    ax1.set_title(f"{name} - Dice histogram (thr={thr})")
    ax1.set_xlabel("Dice"); ax1.set_ylabel("Count")
```

```
    ax2 = plt.subplot(1, 3, 2)
    ax2.boxplot(dice_scores, vert=True, showmeans=True)
    ax2.set_title(f"{name} - Dice boxplot")
    ax2.set_ylabel("Dice")
    ax2.set_xticks([])

    ax3 = plt.subplot(1, 3, 3)
    xs_ = np.sort(dice_scores)
    ys_ = np.arange(1, len(xs_) + 1) / max(1, len(xs_))
    ax3.plot(xs_, ys_)
    ax3.set_title(f"{name} - Dice ECDF")
    ax3.set_xlabel("Dice"); ax3.set_ylabel("F(x)")
    plt.tight_layout()
    plt.show()

    # Examples: worst / median / best
    if dice_scores.size == 0:
        return dice_scores, probs

    order = np.argsort(dice_scores)
    picks = [order[0], order[len(order)//2], order[-1]]
    # enforce N_SHOW
    picks = picks[:max(1, min(int(n_show), 3))]

    primary_i = _primary_image_index()
    n_cols = 1 + MODEL_N_IN + 3  # Image(primary) + each input + GT + prob + bin
    fig = plt.figure(figsize=(3.6 * n_cols, 3.2 * len(picks)))

    for r, i in enumerate(picks):
        c = 0

        # primary image
        ax = plt.subplot(len(picks), n_cols, r * n_cols + (c + 1))
        x_img = Xs[primary_i][i]
        if x_img.ndim == 3 and x_img.shape[-1] > 1:
            ax.imshow(x_img[..., :3])
        else:
            ax.imshow(x_img[..., 0] if x_img.ndim == 3 else x_img, cmap="gray")
        ax.set_title(f"Primary img (idx={i})")
        ax.axis("off")
        c += 1

        # each input
        for j in range(MODEL_N_IN):
            ax = plt.subplot(len(picks), n_cols, r * n_cols + (c + 1))
            xj = Xs[j][i]
```

```
            title = f"in{j}:{MODEL_IN_NAMES[j]}"
            if xj.ndim == 3 and xj.shape[-1] > 1:
                ax.imshow(xj[..., :3])
            else:
                ax.imshow(xj[..., 0] if xj.ndim == 3 else xj, cmap="gray")
            ax.set_title(title)
            ax.axis("off")
            c += 1

        # GT
        ax = plt.subplot(len(picks), n_cols, r * n_cols + (c + 1))
        ax.imshow(y_true[i, ..., 0] if y_true.ndim == 4 else y_true[i],␣
 ↪cmap="gray")
        ax.set_title("GT")
        ax.axis("off")
        c += 1

        # prob
        ax = plt.subplot(len(picks), n_cols, r * n_cols + (c + 1))
        ax.imshow(probs[i, ..., 0], cmap="gray")
        ax.set_title("Pred prob")
        ax.axis("off")
        c += 1

        # bin + dice
        ax = plt.subplot(len(picks), n_cols, r * n_cols + (c + 1))
        ax.imshow(y_bin[i, ..., 0], cmap="gray")
        ax.set_title(f"Pred bin (thr={thr})\nDice={dice_scores[i]:.3f}")
        ax.axis("off")

    plt.tight_layout()
    plt.show()
    return dice_scores, probs

# ============================================================
# 5) Calibration on VAL with precision constraint (E)
# ============================================================
def _make_thr_grid(spec, low=0.5, high=0.9999):
    if isinstance(spec, (list, tuple, np.ndarray)):
        g = np.array(spec, dtype=np.float64)
        g = np.clip(g, 0.0, 1.0)
        g = np.unique(g)
        g.sort()
        return g
    if isinstance(spec, (int, np.integer)):
        n = int(spec)
        n = max(5, n)
```

```
        g = np.linspace(low, high, n, dtype=np.float64)
        # densify high-precision zone
        g2 = np.linspace(max(0.9, low), min(1.0, high), max(10, n//2), dtype=np.
 ↪float64)
        g = np.unique(np.concatenate([g, g2]))
        g.sort()
        return g
    # fallback
    g = np.linspace(low, high, 80, dtype=np.float64)
    return g

def _global_metrics_over_grid(y_true_bool_flat, prob_flat, thr_grid):
    # y_true_bool_flat, prob_flat: (P,) arrays
    y = y_true_bool_flat
    p = prob_flat
    # vectorized counts per threshold
    precs = np.zeros_like(thr_grid, dtype=np.float64)
    recs  = np.zeros_like(thr_grid, dtype=np.float64)
    dices = np.zeros_like(thr_grid, dtype=np.float64)

    # To avoid huge memory, iterate thresholds (thr_grid usually <= few hundred)
    eps = 1e-12
    for k, thr in enumerate(thr_grid):
        pred = (p >= thr)
        tp = np.logical_and(y, pred).sum(dtype=np.int64)
        fp = np.logical_and(~y, pred).sum(dtype=np.int64)
        fn = np.logical_and(y, ~pred).sum(dtype=np.int64)
        prec = tp / (tp + fp + eps)
        rec  = tp / (tp + fn + eps)
        dice = (2 * tp) / (2 * tp + fp + fn + eps)
        precs[k] = prec
        recs[k]  = rec
        dices[k] = dice
    return precs, recs, dices

def calibrate_threshold(probs, Y, target_precision, objective="dice"):
    if probs.ndim == 3:
        probs = probs[..., None]
    y_true = (Y >= 0.5).astype(bool)
    # Flatten all pixels across all samples
    y_flat = y_true.reshape(-1)
    p_flat = probs.reshape(-1)

    grid_coarse = _make_thr_grid(CFG["THR_GRID_COARSE"], low=0.5, high=0.99995)
    prec_c, rec_c, dice_c = _global_metrics_over_grid(y_flat, p_flat,␣
 ↪grid_coarse)
```

```
    feasible = prec_c >= float(target_precision)
    if feasible.any():
        if str(objective).lower() == "recall":
            best_idx = np.argmax(np.where(feasible, rec_c, -1.0))
            # tie-break by dice then precision
            ties = np.where(np.isclose(rec_c, rec_c[best_idx], atol=1e-12) &␣
↪feasible)[0]
            if len(ties) > 1:
                best_idx = ties[np.argmax(dice_c[ties])]
        else:
            best_idx = np.argmax(np.where(feasible, dice_c, -1.0))
            ties = np.where(np.isclose(dice_c, dice_c[best_idx], atol=1e-12) &␣
↪feasible)[0]
            if len(ties) > 1:
                best_idx = ties[np.argmax(rec_c[ties])]
        best_thr = float(grid_coarse[best_idx])
        status = "OK"
    else:
        best_idx = int(np.argmax(prec_c))
        best_thr = float(grid_coarse[best_idx])
        status = "WARN"

    # Dense refine around coarse best
    dense_spec = CFG["THR_GRID_DENSE"]
    if isinstance(dense_spec, (list, tuple, np.ndarray)):
        grid_dense = _make_thr_grid(dense_spec, low=max(0.0, best_thr - 0.02),␣
↪high=min(1.0, best_thr + 0.02))
    else:
        n_dense = int(dense_spec) if isinstance(dense_spec, (int, np.integer))␣
↪else 240
        lo = max(0.0, best_thr - 0.02)
        hi = min(1.0, best_thr + 0.02)
        grid_dense = np.linspace(lo, hi, max(25, n_dense), dtype=np.float64)

    prec_d, rec_d, dice_d = _global_metrics_over_grid(y_flat, p_flat,␣
↪grid_dense)
    feasible_d = prec_d >= float(target_precision)

    if feasible_d.any():
        if str(objective).lower() == "recall":
            idx = np.argmax(np.where(feasible_d, rec_d, -1.0))
            ties = np.where(np.isclose(rec_d, rec_d[idx], atol=1e-12) &␣
↪feasible_d)[0]
            if len(ties) > 1:
                idx = ties[np.argmax(dice_d[ties])]
        else:
```

```
            idx = np.argmax(np.where(feasible_d, dice_d, -1.0))
            ties = np.where(np.isclose(dice_d, dice_d[idx], atol=1e-12) &␣
 ↪feasible_d)[0]
            if len(ties) > 1:
                idx = ties[np.argmax(rec_d[ties])]
        thr = float(grid_dense[idx])
        prec = float(prec_d[idx]); rec = float(rec_d[idx]); dice =␣
 ↪float(dice_d[idx])
        final_status = "OK"
    else:
        idx = int(np.argmax(prec_d))
        thr = float(grid_dense[idx])
        prec = float(prec_d[idx]); rec = float(rec_d[idx]); dice =␣
 ↪float(dice_d[idx])
        final_status = "WARN"

    print(f"\n[CALIB {final_status}] objective={objective} |␣
 ↪target_prec={target_precision}")
    if final_status == "WARN":
        print(f"  impossible precision>=target sur le grid; meilleur en␣
 ↪precision:")
    else:
        print(f"  trouvé precision>=target; sélection optimale:")
    print(f"  thr={thr:.6f} | prec={prec:.6f} | rec={rec:.6f} | dice={dice:.
 ↪6f}")

    return thr

# ============================================================
# 6) RUN: predict once on VAL subset (for calibration), then eval at chosen thr
# ============================================================
val_probs_for_calib = _predict_probs(Xs_val, CFG["BATCH_SIZE"])
thr_best = calibrate_threshold(val_probs_for_calib, Y_val,␣
 ↪CFG["TARGET_PRECISION"], CFG["CALIB_OBJECTIVE"])

# Eval at requested THR_EVAL and at calibrated threshold
_ = eval_and_plot("TRAIN@THR_EVAL", Xs_train, Y_train, CFG["THR_EVAL"],␣
 ↪CFG["N_SHOW"], max_pred=min(CFG["MAX_PRED_EVAL"], Y_train.shape[0] if␣
 ↪isinstance(Y_train, np.ndarray) else CFG["MAX_PRED_EVAL"]))
_ = eval_and_plot("VAL@THR_EVAL",   Xs_val,   Y_val,   CFG["THR_EVAL"],␣
 ↪CFG["N_SHOW"], max_pred=min(CFG["MAX_PRED_EVAL"], Y_val.shape[0] if␣
 ↪isinstance(Y_val, np.ndarray) else CFG["MAX_PRED_EVAL"]))

_ = eval_and_plot("TRAIN@CALIB_THR", Xs_train, Y_train, thr_best,␣
 ↪CFG["N_SHOW"], max_pred=min(CFG["MAX_PRED_EVAL"], Y_train.shape[0] if␣
 ↪isinstance(Y_train, np.ndarray) else CFG["MAX_PRED_EVAL"]))
```

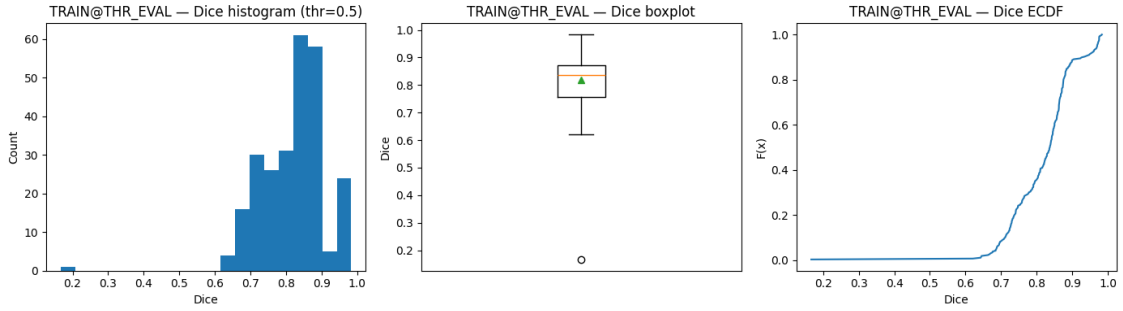
```
_ = eval_and_plot("VAL@CALIB_THR",   Xs_val,   Y_val,   thr_best,␣
 ↪CFG["N_SHOW"], max_pred=min(CFG["MAX_PRED_EVAL"], Y_val.shape[0] if␣
 ↪isinstance(Y_val, np.ndarray) else CFG["MAX_PRED_EVAL"]))

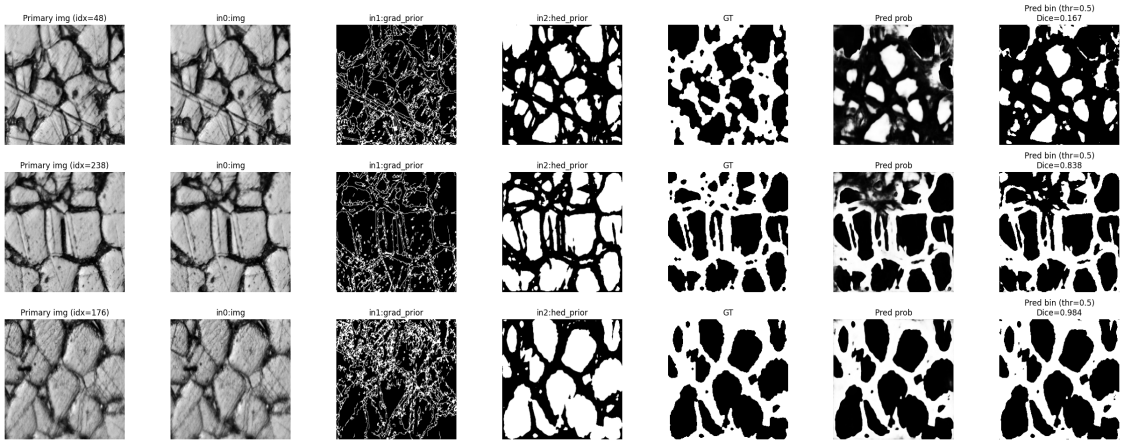
print(f"\n[FINAL] MODEL_N_IN={MODEL_N_IN} | INPUT_NAMES={MODEL_IN_NAMES} |␣
 ↪RECOMMENDED_THR={thr_best:.6f}")
```

```
[OK] TRAIN_EVAL: collected n=256 | inputs=[(256, 256, 256, 1), (256, 256, 256,
1), (256, 256, 256, 1)] | Y=(256, 256, 256, 1)
[OK] VAL_EVAL: collected n=256 | inputs=[(256, 256, 256, 1), (256, 256, 256, 1),
(256, 256, 256, 1)] | Y=(256, 256, 256, 1)
32/32             2s 11ms/step

[CALIB OK] objective=dice | target_prec=0.9
  trouvé precision>=target; sélection optimale:
  thr=0.996121 | prec=0.901605 | rec=0.211063 | dice=0.342053
32/32             0s 11ms/step

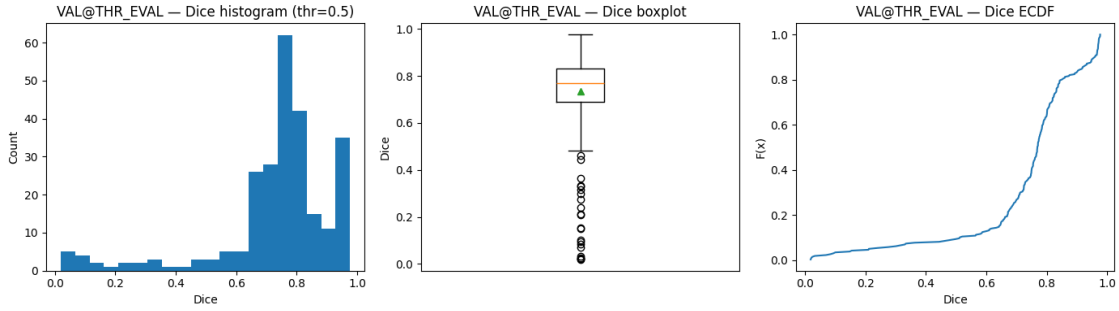
[TRAIN@THR_EVAL] THR=0.5
  Dice mean=0.819660 | min=0.166541 | max=0.983601
  Dice quantiles: p05=0.687 | p25=0.757 | p50=0.837 | p75=0.871 | p95=0.970
```

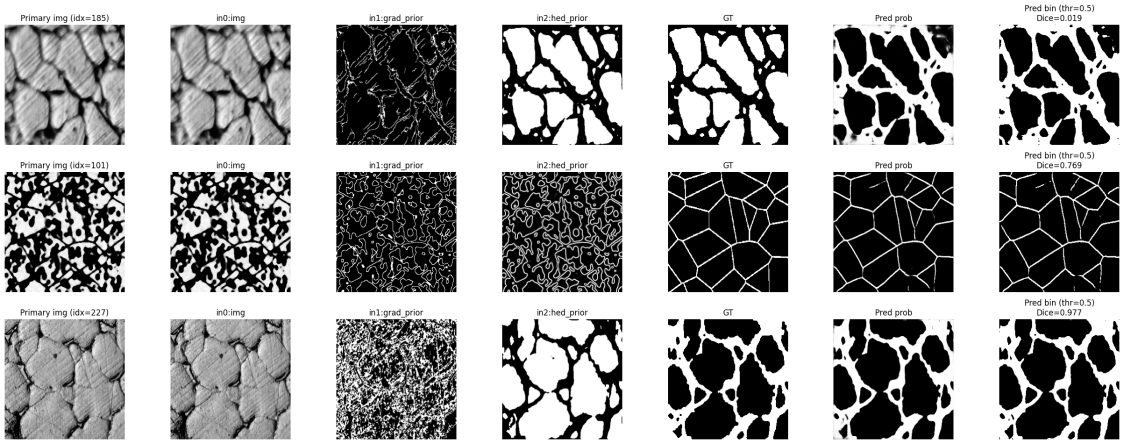




```
32/32              0s 11ms/step

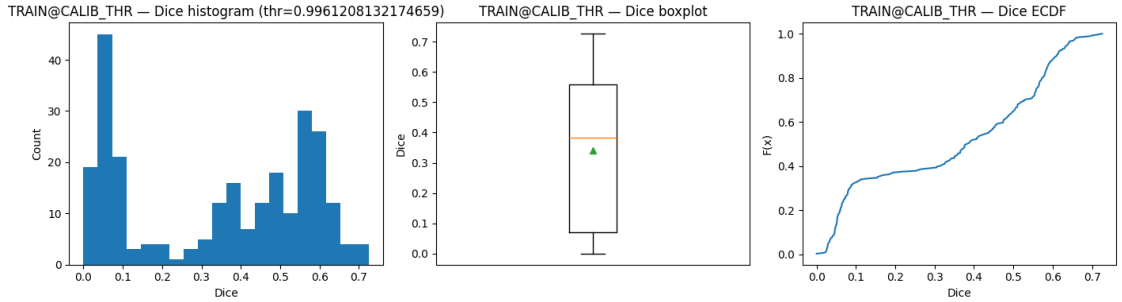
[VAL@THR_EVAL] THR=0.5
  Dice mean=0.733959 | min=0.018912 | max=0.976568
  Dice quantiles: p05=0.233 | p25=0.691 | p50=0.769 | p75=0.832 | p95=0.970
```

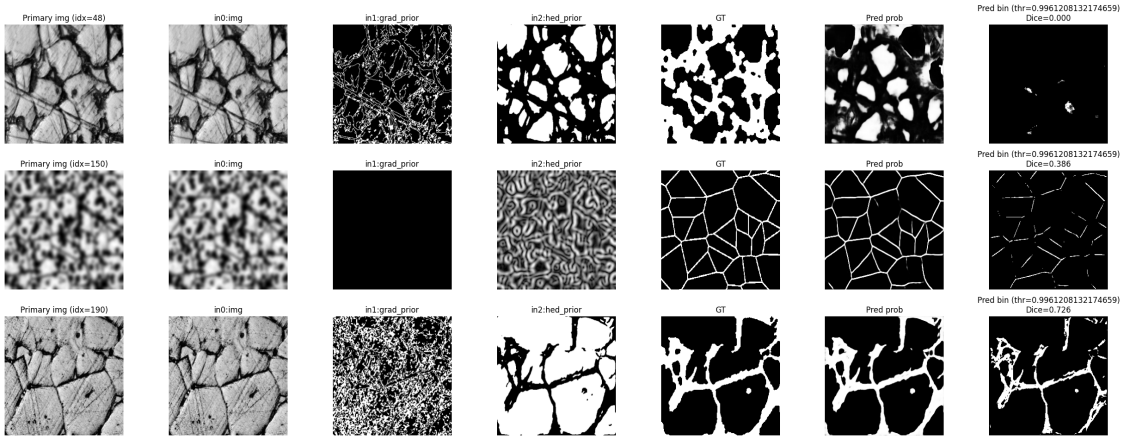





```
32/32              0s 11ms/step

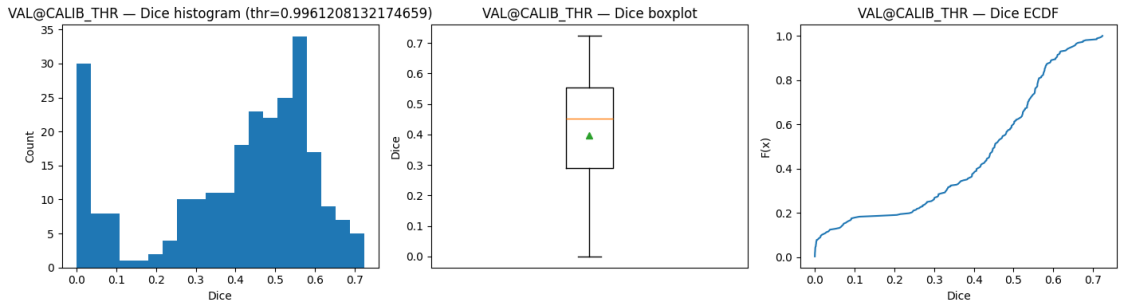
[TRAIN@CALIB_THR] THR=0.9961208132174659
  Dice mean=0.340198 | min=0.000000 | max=0.725864
  Dice quantiles: p05=0.031 | p25=0.072 | p50=0.383 | p75=0.559 | p95=0.638
```

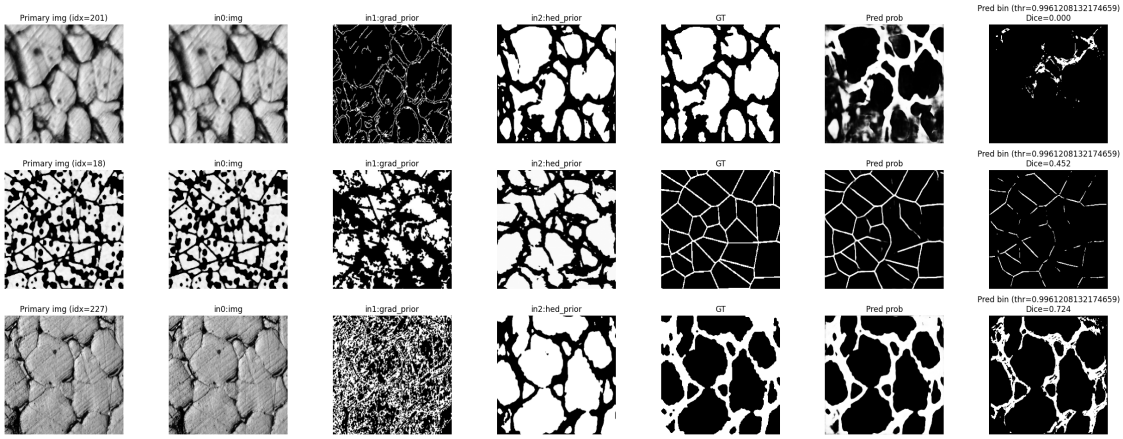




32/32 0s 11ms/step

[VAL@CALIB_THR] THR=0.9961208132174659
Dice mean=0.397421 | min=0.000000 | max=0.723556
Dice quantiles: p05=0.002 | p25=0.291 | p50=0.452 | p75=0.554 | p95=0.647






```
[FINAL] MODEL_N_IN=3 | INPUT_NAMES=['img', 'grad_prior', 'hed_prior'] |
RECOMMENDED_THR=0.996121
```

À **thr = 0.5**, je constate que le modèle segmente globalement bien : sur **train** le Dice est élevé (moyenne **0.82**, médiane **0.84**) et la majorité des cas se situe dans une zone "propre" autour de **0.7–0.95** ; sur **val**, la performance reste correcte (moyenne **0.73**, médiane **0.77**), mais le point faible apparaît clairement avec une **queue de cas très mauvais** (min **0.02**, p05 **0.23**), ce qui indique des images où la prédiction part en FP/forme incohérente. Quand je passe à la calibration imposant la **précision 0.90**, le seuil optimal trouvé (**thr 0.9961**) "nettoie" effectivement les faux positifs (precision **0.90**), mais au prix d'un **effondrement du recall** (**0.21**) : la segmentation devient trop parcimonieuse, avec beaucoup de masques quasi vides, ce qui fait chuter fortement le Dice (**0.34** sur train et **0.40** sur val, avec des zéros visibles). Donc, en l'état, **thr=0.5** correspond à un mode "segmentation utile" (bon Dice mais quelques cas catastrophiques), tandis que **thr 0.996** correspond à un mode "propreté/anti-FP" (très strict) qui sacrifie la couverture et dégrade la qualité de segmentation globale.

[ ]:
```python
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt

# Optional (tableau)
try:
    import pandas as pd
except Exception:
    pd = None

# ============================================================
# PARAMÈTRES
# ============================================================
THR = float(globals().get("THR", 0.5))
EPS = float(globals().get("EPS", 1e-9))
```

```
CLAMP_01 = bool(globals().get("CLAMP_01", True))
MAX_ITEMS = globals().get("MAX_ITEMS", None)  # None ou int
BATCH_SIZE = int(globals().get("BATCH_SIZE", 8))

# ============================================================
# 0) CHECKS & Load collected data from previous cell
# ============================================================
assert "unet_model" in globals(), "unet_model introuvable."
assert "Xs_train" in globals(), "Xs_train n'est pas défini (doit venir de la␣
 ↪cellule d'évaluation précédente)."
assert "Y_train" in globals(), "Y_train n'est pas défini (doit venir de la␣
 ↪cellule d'évaluation précédente)."

# Use the collected data from the previous evaluation cell
X_list_train_eval = globals()["Xs_train"] # This is already a list of inputs
Y_train_eval = globals()["Y_train"]

# Determine N_total from the collected data (using the first input for batch␣
 ↪size)
N_current_eval = X_list_train_eval[0].shape[0]

# Ensure Y_train_eval matches the batch size of the actual inputs used for␣
 ↪prediction
if Y_train_eval.shape[0] != N_current_eval:
    print(f"[WARN] Y_train_eval batch size mismatch: expected {N_current_eval},␣
 ↪got {Y_train_eval.shape[0]}. Slicing Y_train_eval.")
    Y_train_eval = Y_train_eval[:N_current_eval]

# ============================================================
# A) SIGNATURE DU MODÈLE (No changes needed here)
# ============================================================
MODEL_INPUTS = tf.nest.flatten(unet_model.inputs)
MODEL_N_IN = len(MODEL_INPUTS)

def _shape_to_list(sh):
    if sh is None:
        return None
    if hasattr(sh, "as_list"):
        return sh.as_list()
    if isinstance(sh, (tuple, list)):
        return list(sh)
    try:
        return tf.TensorShape(sh).as_list()
    except Exception:
        return None

def _clean_name(n):
```

```
    s = str(n) if n is not None else ""
    s = s.split(":")[0]
    return s.split("/")[-1] if s else ""

def _norm_key(s):
    import re
    return re.sub(r"[^a-z0-9]+", "_", str(s).lower()).strip("_")

print("\n[MODEL SIGNATURE]")
for i, t in enumerate(MODEL_INPUTS):
    nm = _clean_name(getattr(t, "name", f"input_{i}")) or f"input_{i}"
    sh = _shape_to_list(getattr(t, "shape", None))
    print(f"  [{i}] name={nm} | shape={sh}")

# ============================================================
# B) PREDICT PROBA SUR TRAIN (signature exacte)
#    Now using X_list_train_eval and Y_train_eval
# ============================================================
if MODEL_N_IN == 1:
    # If model is single input, X_list_train_eval is a list of 1 element
    preds_prob_train = unet_model.predict(X_list_train_eval[0],␣
 ↪batch_size=BATCH_SIZE, verbose=1)
else:
    # For multi-input models, pass the list directly
    preds_prob_train = unet_model.predict(X_list_train_eval,␣
 ↪batch_size=BATCH_SIZE, verbose=1)

preds_prob_train = np.clip(preds_prob_train.astype(np.float32), 0.0, 1.0)
if preds_prob_train.ndim == 3:
    preds_prob_train = preds_prob_train[..., None]

# ============================================================
# D) BINARISATION
# ============================================================
preds_train_bin = (preds_prob_train >= THR).astype(np.uint8)

print("\n[PRED LOGS]")
print("  preds_prob_train:", preds_prob_train.shape, preds_prob_train.dtype)
print("  preds_train_bin :", preds_train_bin.shape, preds_train_bin.dtype)
print("  Y_train_eval    :", Y_train_eval.shape, Y_train_eval.dtype) # Use␣
 ↪Y_train_eval for printing

# ============================================================
# E) METRICS GLOBALES PIXEL-WISE (using Y_train_eval and preds_prob_train)
# ============================================================
def _compute_pixel_metrics(y_true, p_prob, thr, eps=1e-9):
    # Ensure y_true and p_prob have the same first dimension (batch size)
```

```
    if y_true.shape[0] != p_prob.shape[0]:
        raise ValueError(f"Batch size mismatch: y_true has {y_true.shape[0]}␣
↪samples, p_prob has {p_prob.shape[0]} samples.")

    yb = (y_true >= 0.5).astype(np.uint8)
    pb = (p_prob >= thr).astype(np.uint8)

    yt = yb.reshape(-1)
    yp = pb.reshape(-1)

    tp = int(np.sum((yt == 1) & (yp == 1)))
    fp = int(np.sum((yt == 0) & (yp == 1)))
    fn = int(np.sum((yt == 1) & (yp == 0)))
    tn = int(np.sum((yt == 0) & (yp == 0)))

    acc  = (tp + tn) / (tp + tn + fp + fn + eps)
    prec = tp / (tp + fp + eps)
    rec  = tp / (tp + fn + eps)
    f1   = (2 * prec * rec) / (prec + rec + eps)

    iou  = tp / (tp + fp + fn + eps)
    # Dice correct: denom = sum(y_true)+sum(y_pred) => (2TP)/(2TP+FP+FN)
    dice = (2 * tp) / ((2 * tp) + fp + fn + eps)

    return {
        "Accuracy": float(acc),
        "Precision": float(prec),
        "Recall": float(rec),
        "F1": float(f1),
        "IoU": float(iou),
        "Dice": float(dice),
        "Seuil": float(thr),
        "TP": tp, "FP": fp, "FN": fn, "TN": tn,
    }

# Pass Y_train_eval and preds_prob_train directly to the metrics function
if "global_classif_metrics" in globals() and␣
↪callable(globals()["global_classif_metrics"]) and 'Y_train_eval' in␣
↪globals() and 'preds_prob_train' in globals():
    try:
        out = globals()["global_classif_metrics"](Y_train_eval,␣
↪preds_prob_train, thr=THR)
        metrics_global_full = dict(out) if isinstance(out, dict) else␣
↪{"global_classif_metrics_output": out}
        metrics_global_full.setdefault("Seuil", float(THR))
        core = _compute_pixel_metrics(Y_train_eval, preds_prob_train, THR,␣
↪eps=EPS)
```

```
        for k in ["Accuracy", "Precision", "Recall", "F1", "IoU", "Dice"]:
            metrics_global_full.setdefault(k, core[k])
    except Exception as e:
        print(f"[WARN] global_classif_metrics a échoué -> fallback compute.␣
 ↪err={repr(e)}")
        metrics_global_full = _compute_pixel_metrics(Y_train_eval,␣
 ↪preds_prob_train, THR, eps=EPS)
else:
    metrics_global_full = _compute_pixel_metrics(Y_train_eval,␣
 ↪preds_prob_train, THR, eps=EPS)

# ============================================================
# F) metrics_global_full (min requis) + logs
# ============================================================
for k in ["Accuracy", "Precision", "Recall", "F1", "IoU", "Dice", "Seuil"]:
    metrics_global_full.setdefault(k, None)

print("\n[METRICS GLOBAL FULL]")
for k in ["Accuracy", "Precision", "Recall", "F1", "IoU", "Dice", "Seuil"]:
    print(f"  {k:>10s}: {metrics_global_full[k]}")

# ============================================================
# G) TABLEAU PANDAS (trié)
# ============================================================
core_keys = ["Accuracy", "Precision", "Recall", "F1", "IoU", "Dice", "Seuil"]
sorted_keys = sorted([k for k in core_keys if k != "Seuil"]) + ["Seuil"]

if pd is not None:
    df = pd.DataFrame({"Valeur": [metrics_global_full.get(k) for k in␣
 ↪sorted_keys]}, index=sorted_keys)
    try:
        display(df)
    except Exception:
        print(df)
else:
    print("\n[PANDAS NOT AVAILABLE] Table:")
    for k in sorted_keys:
        print(f"  {k:>10s}: {metrics_global_full.get(k)}")

# ============================================================
# H) BARPLOT (exclure 'Seuil')
# ============================================================
plot_keys = [k for k in sorted_keys if k != "Seuil" and metrics_global_full.
 ↪get(k) is not None]
plot_vals = [float(metrics_global_full[k]) for k in plot_keys]

fig = plt.figure(figsize=(8, 4))
```

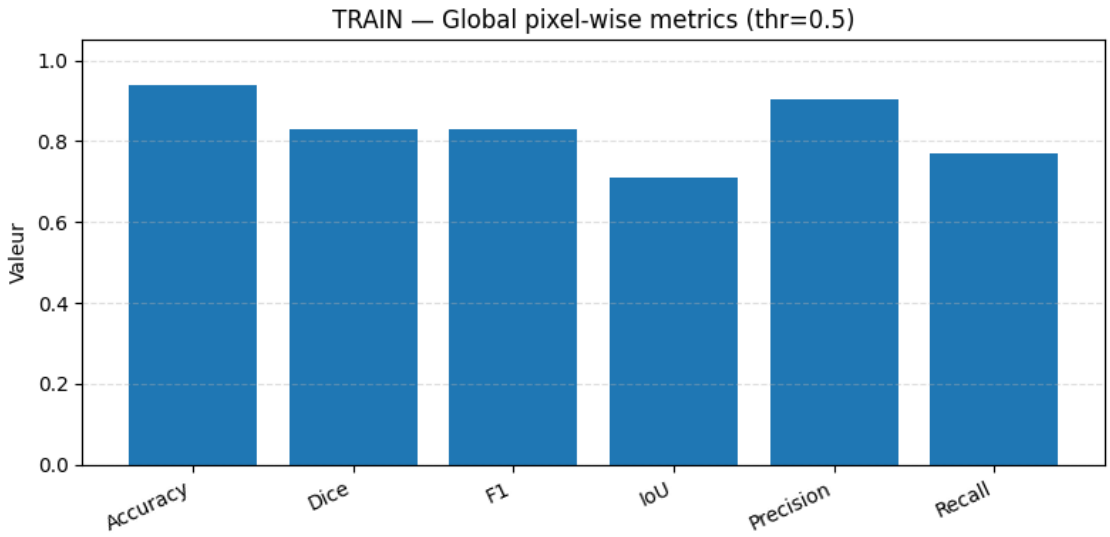
```
ax = plt.gca()
ax.bar(plot_keys, plot_vals)
ax.set_ylim(0.0, 1.05)
ax.set_title(f"TRAIN - Global pixel-wise metrics (thr={THR})")
ax.set_ylabel("Valeur")
ax.grid(True, axis="y", linestyle="--", alpha=0.4)
plt.xticks(rotation=25, ha="right")
plt.tight_layout()
plt.show()
```

```
[MODEL SIGNATURE]
  [0] name=img | shape=[None, 256, 256, 1]
  [1] name=grad_prior | shape=[None, 256, 256, 1]
  [2] name=hed_prior | shape=[None, 256, 256, 1]
32/32             0s 11ms/step

[PRED LOGS]
  preds_prob_train: (256, 256, 256, 1) float32
  preds_train_bin : (256, 256, 256, 1) uint8
  Y_train_eval    : (256, 256, 256, 1) float32

[METRICS GLOBAL FULL]
    Accuracy: 0.9378057718276978
   Precision: 0.9022814564530807
      Recall: 0.7685721773110106
          F1: 0.8300768051791745
         IoU: 0.7095139319428438
        Dice: 0.8300768056759725
       Seuil: 0.5

              Valeur
Accuracy    0.937806
Dice        0.830077
F1          0.830077
IoU         0.709514
Precision   0.902281
Recall      0.768572
Seuil       0.500000
```



À **seuil = 0.5** sur **TRAIN**, je lis ça comme suit : le modèle est déjà **solide en recouvrement** avec un **Dice/F1 0.83** et un **IoU 0.71**, donc globalement les zones prédites matchent bien le GT (on est clairement au-dessus d'un modèle "moyen"). La **précision 0.90** me dit que quand il prédit "grain boundary", il se trompe relativement peu (il reste des **FP**, mais pas de manière explosive), tandis que le **recall 0.77** montre qu'il **rate encore une partie** des vrais pixels positifs (donc il y a des **FN** non négligeables). L'**accuracy 0.94** est "bonne" mais je la prends avec précaution parce qu'en segmentation il y a souvent beaucoup de fond : elle peut être élevée même si on loupe des détails. En résumé, à thr=0.5 le modèle est **plutôt équilibré** (précision élevée + rappel correct) et produit une segmentation **utile**, mais si mon objectif devient "zéro FP", je sais déjà que pousser le seuil trop haut va vite **casser le recall** (on l'a vu avec les seuils ~0.996+), donc il faut choisir le seuil en fonction du compromis métier (propreté vs couverture).

[ ]:
```python
import numpy as np
import matplotlib.pyplot as plt

# Optional (tableau)
try:
    import pandas as pd
except Exception:
    pd = None

# ============================================================
# PARAMÈTRES
# ============================================================
K = int(globals().get("K", 5))
SMOOTH = float(globals().get("SMOOTH", 1e-6))
THRESH_FLOAT_TO_BIN = float(globals().get("THR", globals().
 ↪get("THRESH_FLOAT_TO_BIN", 0.5)))
```

```python
# ============================================================
# A) CHECKS INITIAUX (ValueError explicite)
# ============================================================
if "Y_train" not in globals():
    raise ValueError("Y_train introuvable dans globals() (obligatoire).")
if "preds_train_bin" not in globals():
    raise ValueError("preds_train_bin introuvable dans globals() (obligatoire).
 ↪")

Y_src = globals()["Y_train"]
P_src = globals()["preds_train_bin"]

# Convertir en numpy sans muter les sources
Y_np = np.asarray(Y_src)
P_np = np.asarray(P_src)

print("\n[INIT LOGS]")
print(f"  Y_train        : shape={Y_np.shape} | dtype={Y_np.dtype}")
print(f"  preds_train_bin: shape={P_np.shape} | dtype={P_np.dtype}")
print(f"  THRESH_FLOAT_TO_BIN={THRESH_FLOAT_TO_BIN} | K={K} | SMOOTH={SMOOTH}")

# ============================================================
# B) NORMALISATION SHAPES (forcer batch N)
# ============================================================
def _ensure_nhwc(arr, name="arr"):
    a = np.asarray(arr)

    # Cas N=1 : (H,W) ou (H,W,C) -> (1,H,W,1/C)
    if a.ndim == 2:
        a = a[None, ..., None]  # (1,H,W,1)
    elif a.ndim == 3:
        # Ambigu : (H,W,C) ou (N,H,W). Heuristique prudente :
        # si la 3e dim est petite (1..4) on suppose (H,W,C), sinon (N,H,W)
        if a.shape[-1] <= 4:
            a = a[None, ...]  # (1,H,W,C)
        else:
            a = a[..., None]  # (N,H,W,1)
    elif a.ndim == 4:
        # (N,H,W,C) ok
        pass
    else:
        raise ValueError(f"[{name}] Rank non supporté: ndim={a.ndim}, shape={a.
 ↪shape}")

    # Channel absent (N,H,W) déjà traité; ici on garantit 4D
    if a.ndim != 4:
```

```
        raise ValueError(f"[{name}] Normalisation échouée: attendu 4D NHWC,␣
 ↪obtenu ndim={a.ndim}, shape={a.shape}")

    # Vérifs minimales
    if a.shape[1] <= 0 or a.shape[2] <= 0:
        raise ValueError(f"[{name}] H/W invalides après normalisation: shape={a.
 ↪shape}")

    return a

Y = _ensure_nhwc(Y_np, name="Y_train")
P = _ensure_nhwc(P_np, name="preds_train_bin")

# Réconcilier N de manière sûre
N0 = int(Y.shape[0])
N1 = int(P.shape[0])
N_final = int(min(N0, N1))
if N_final <= 0:
    raise ValueError(f"N_final invalide après réconciliation:␣
 ↪N_final={N_final}, Y.shape={Y.shape}, P.shape={P.shape}")

# Optionnel: si N existe déjà, on l'applique aussi (sans supposer qu'il est␣
 ↪correct)
if "N" in globals() and globals()["N"] is not None:
    try:
        N_user = int(globals()["N"])
        if N_user > 0:
            N_final = int(min(N_final, N_user))
    except Exception:
        pass

# Vérif compatibilité H/W au moins (on accepte C différent et on aplatit tout)
if Y.shape[1] != P.shape[1] or Y.shape[2] != P.shape[2]:
    raise ValueError(f"[SHAPE] Incompatibles H/W: Y={Y.shape} vs P={P.shape}")

# Travailler sur slices (pas de mutation des sources)
Y_use = Y[:N_final].copy()
P_use = P[:N_final].copy()

print("\n[NORMALIZED LOGS]")
print(f"  Y_use: shape={Y_use.shape} | dtype={Y_use.dtype}")
print(f"  P_use: shape={P_use.shape} | dtype={P_use.dtype}")
print(f"  N_final utilisé = {N_final}")

# ============================================================
# C) HELPERS ROBUSTES
# ============================================================
```

```python
def _to01(x):
    """Retourne uint8 {0,1} avec seuil THRESH_FLOAT_TO_BIN pour floats."""
    a = np.asarray(x)
    if a.dtype == np.bool_:
        return a.astype(np.uint8)
    if np.issubdtype(a.dtype, np.floating):
        return (a >= THRESH_FLOAT_TO_BIN).astype(np.uint8)
    # int / uint
    return (a > 0).astype(np.uint8)

def dice_coefficient_safe(y_true, y_pred, smooth=1e-6):
    yt = _to01(y_true).reshape(-1)
    yp = _to01(y_pred).reshape(-1)
    inter = float(np.sum(yt * yp))
    denom = float(np.sum(yt) + np.sum(yp))  # denom correct
    return float((2.0 * inter + smooth) / (denom + smooth))

def iou_coefficient_safe(y_true, y_pred, smooth=1e-6):
    yt = _to01(y_true).reshape(-1).astype(bool)
    yp = _to01(y_pred).reshape(-1).astype(bool)
    inter = float(np.sum(yt & yp))
    union = float(np.sum(yt | yp))
    return float((inter + smooth) / (union + smooth))

# ============================================================
# D) CALCUL PAR IMAGE
# ============================================================
dice_scores = np.empty((N_final,), dtype=np.float32)
iou_scores  = np.empty((N_final,), dtype=np.float32)

for i in range(N_final):
    dice_scores[i] = dice_coefficient_safe(Y_use[i], P_use[i], smooth=SMOOTH)
    iou_scores[i]  = iou_coefficient_safe(Y_use[i], P_use[i], smooth=SMOOTH)

# Logs utiles (min/mean/max)
d_min, d_mean, d_max = float(np.min(dice_scores)), float(np.mean(dice_scores)),␣
 ↪float(np.max(dice_scores))
i_min, i_mean, i_max = float(np.min(iou_scores)),  float(np.mean(iou_scores)), ␣
 ↪float(np.max(iou_scores))

print("\n[SCORES LOGS]")
print(f"  Dice: min={d_min:.4f} | mean={d_mean:.4f} | max={d_max:.4f}")
print(f"  IoU : min={i_min:.4f} | mean={i_mean:.4f} | max={i_max:.4f}")

# ============================================================
# E) STATS DESCRIPTIVES (DataFrames)
# ============================================================
```

```
if pd is not None:
    per_image_df = pd.DataFrame({"Dice": dice_scores.astype(np.float32), "IoU":␣
 ↪iou_scores.astype(np.float32)})

    stats_df = per_image_df.describe(percentiles=[0.10, 0.50, 0.90]).T.
 ↪rename(columns={
        "mean": "moyenne",
        "std":  "écart-type",
        "min":  "min",
        "10%":  "q10",
        "50%":  "médiane",
        "90%":  "q90",
        "max":  "max",
    })

    cols = ["moyenne", "écart-type", "min", "q10", "médiane", "q90", "max"]
    stats_df_out = stats_df.loc[:, cols]

    try:
        # display si dispo
        disp = globals().get("display", None)
        if callable(disp):
            disp(
                stats_df_out.style
                    .format("{:.4f}")
                    .set_caption("Statistiques des scores par image (TRAIN)")
            )
        else:
            print("\n[STATS DF]\n", stats_df_out.round(4))
    except Exception:
        print("\n[STATS DF]\n", stats_df_out.round(4))
else:
    # fallback sans pandas
    def _quant(x, q):
        return float(np.quantile(x, q))
    stats_fallback = {
        "Dice": {"moyenne": d_mean, "écart-type": float(np.std(dice_scores)),␣
 ↪"min": d_min, "q10": _quant(dice_scores, 0.10),
                 "médiane": _quant(dice_scores, 0.50), "q90":␣
 ↪_quant(dice_scores, 0.90), "max": d_max},
        "IoU":  {"moyenne": i_mean, "écart-type": float(np.std(iou_scores)), ␣
 ↪"min": i_min, "q10": _quant(iou_scores, 0.10),
                 "médiane": _quant(iou_scores, 0.50),  "q90":␣
 ↪_quant(iou_scores, 0.90),  "max": i_max},
    }
    print("\n[STATS (no pandas)]")
```

```
    for m in ["Dice", "IoU"]:
        s = stats_fallback[m]
        print(f"  {m}: moyenne={s['moyenne']:.4f} | écart-type={s['écart-type']:
 ↪.4f} | min={s['min']:.4f} | q10={s['q10']:.4f} | médiane={s['médiane']:.4f}␣
 ↪| q90={s['q90']:.4f} | max={s['max']:.4f}")

# ============================================================
# F) VISUALISATIONS (2 figures séparées, PAS de subplot)
# ============================================================
plt.figure(figsize=(7, 4))
plt.hist(dice_scores, bins=25, edgecolor="black", alpha=0.85)
plt.xlabel("Dice par image")
plt.ylabel("Fréquence")
plt.title("Distribution des scores Dice (TRAIN)")
plt.tight_layout()
plt.show()

plt.figure(figsize=(7, 4))
plt.hist(iou_scores, bins=25, edgecolor="black", alpha=0.85)
plt.xlabel("IoU par image")
plt.ylabel("Fréquence")
plt.title("Distribution des scores IoU (TRAIN)")
plt.tight_layout()
plt.show()

# ============================================================
# G) TOP-K (pires / meilleures selon Dice)
# ============================================================
K_eff = int(min(max(K, 1), N_final))
worst_idx = np.argsort(dice_scores)[:K_eff]
best_idx  = np.argsort(-dice_scores)[:K_eff]

def _fmt_list(idxs, dice_arr, iou_arr):
    lines = []
    for j in idxs:
        lines.append(f"idx={int(j):>5d} | Dice={float(dice_arr[j]):.4f} |␣
 ↪IoU={float(iou_arr[j]):.4f}")
    return "\n".join(lines)

print(f"\n[TOP-{K_eff} WORST by Dice]")
print(_fmt_list(worst_idx, dice_scores, iou_scores))

print(f"\n[TOP-{K_eff} BEST by Dice]")
print(_fmt_list(best_idx, dice_scores, iou_scores))
```

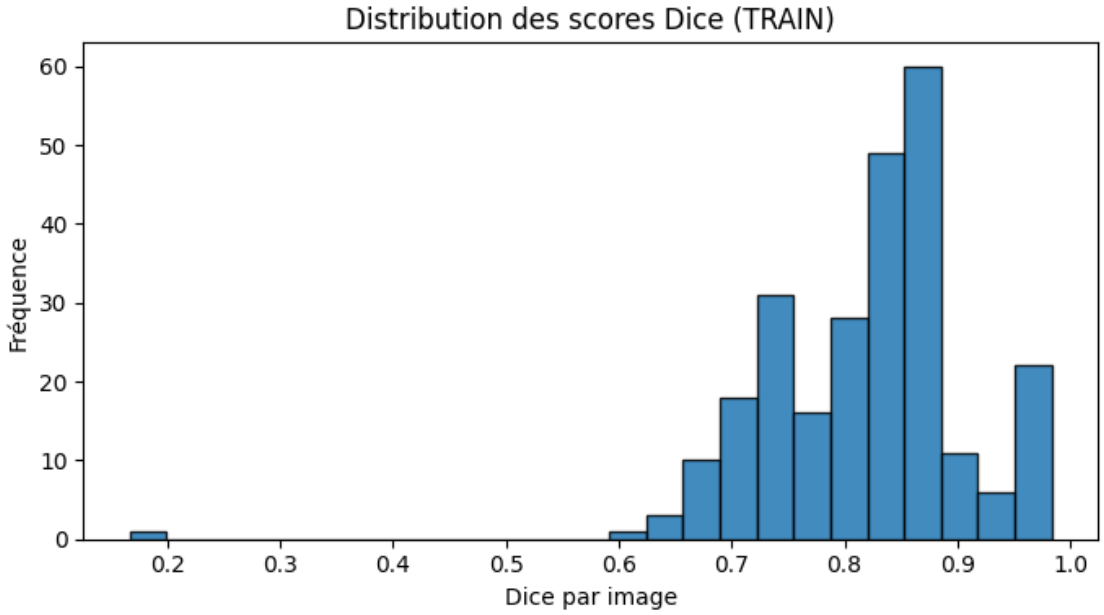
```
[INIT LOGS]
```

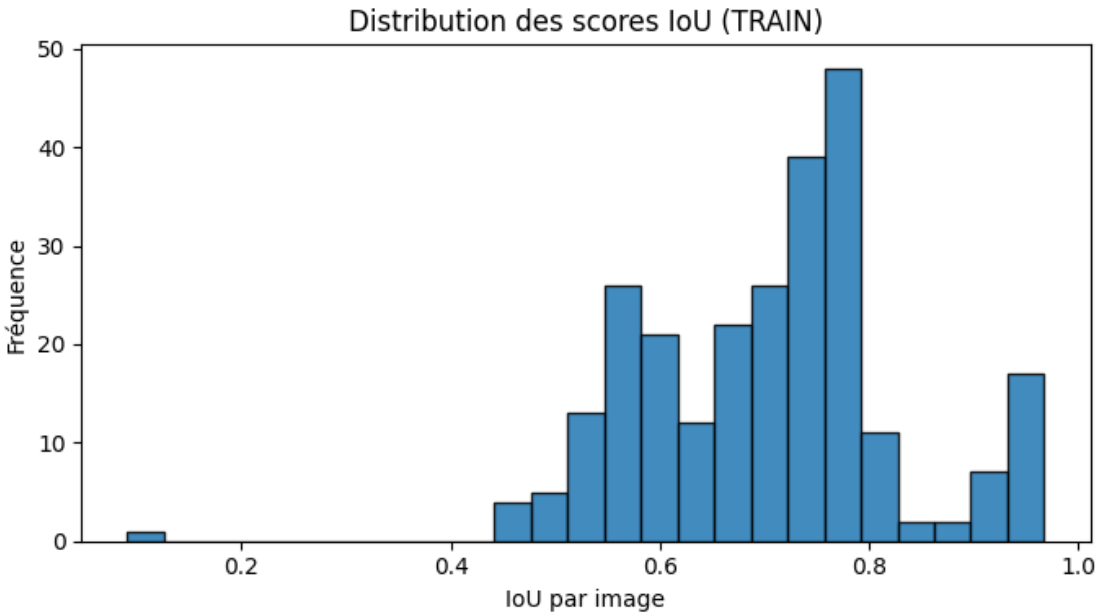
```
  Y_train        : shape=(256, 256, 256, 1) | dtype=float32
  preds_train_bin: shape=(256, 256, 256, 1) | dtype=uint8
  THRESH_FLOAT_TO_BIN=0.5 | K=25 | SMOOTH=1e-06

[NORMALIZED LOGS]
  Y_use: shape=(256, 256, 256, 1) | dtype=float32
  P_use: shape=(256, 256, 256, 1) | dtype=uint8
  N_final utilisé = 256

[SCORES LOGS]
  Dice: min=0.1665 | mean=0.8197 | max=0.9836
  IoU : min=0.0908 | mean=0.7037 | max=0.9677

[STATS DF]
       moyenne  écart-type     min     q10  médiane     q90     max
Dice   0.8197      0.0903  0.1665  0.7122   0.8373  0.9307  0.9836
IoU    0.7037      0.1231  0.0908  0.5530   0.7201  0.8704  0.9677
```






```
[TOP-25 WORST by Dice]
idx=   48 | Dice=0.1665 | IoU=0.0908
idx=  153 | Dice=0.6206 | IoU=0.4499
idx=  201 | Dice=0.6428 | IoU=0.4736
idx=   32 | Dice=0.6441 | IoU=0.4750
idx=  144 | Dice=0.6454 | IoU=0.4765
idx=  248 | Dice=0.6634 | IoU=0.4963
idx=   75 | Dice=0.6679 | IoU=0.5013
idx=   96 | Dice=0.6708 | IoU=0.5047
idx=  196 | Dice=0.6760 | IoU=0.5106
idx=  179 | Dice=0.6766 | IoU=0.5113
idx=  194 | Dice=0.6850 | IoU=0.5209
idx=  143 | Dice=0.6852 | IoU=0.5211
idx=  159 | Dice=0.6869 | IoU=0.5231
idx=  113 | Dice=0.6877 | IoU=0.5241
idx=  141 | Dice=0.6885 | IoU=0.5250
idx=  157 | Dice=0.6897 | IoU=0.5264
idx=   71 | Dice=0.6934 | IoU=0.5306
idx=  169 | Dice=0.6947 | IoU=0.5323
idx=   52 | Dice=0.6948 | IoU=0.5323
idx=  134 | Dice=0.6967 | IoU=0.5346
idx=   20 | Dice=0.6969 | IoU=0.5348
idx=   55 | Dice=0.7012 | IoU=0.5399
idx=  127 | Dice=0.7033 | IoU=0.5424
idx=   60 | Dice=0.7079 | IoU=0.5478
idx=   51 | Dice=0.7089 | IoU=0.5491
```

```
[TOP-25 BEST by Dice]
idx=  176 | Dice=0.9836 | IoU=0.9677
idx=  213 | Dice=0.9811 | IoU=0.9628
idx=   15 | Dice=0.9765 | IoU=0.9540
idx=  211 | Dice=0.9759 | IoU=0.9529
idx=  119 | Dice=0.9758 | IoU=0.9528
idx=   37 | Dice=0.9758 | IoU=0.9527
idx=  202 | Dice=0.9757 | IoU=0.9525
idx=  200 | Dice=0.9751 | IoU=0.9514
idx=   22 | Dice=0.9740 | IoU=0.9494
idx=  173 | Dice=0.9728 | IoU=0.9471
idx=  246 | Dice=0.9726 | IoU=0.9467
idx=  117 | Dice=0.9715 | IoU=0.9446
idx=   13 | Dice=0.9704 | IoU=0.9426
idx=   86 | Dice=0.9703 | IoU=0.9424
idx=  190 | Dice=0.9696 | IoU=0.9411
idx=  227 | Dice=0.9686 | IoU=0.9391
idx=  245 | Dice=0.9673 | IoU=0.9367
idx=  192 | Dice=0.9633 | IoU=0.9293
idx=   90 | Dice=0.9611 | IoU=0.9252
idx=    6 | Dice=0.9610 | IoU=0.9249
idx=  208 | Dice=0.9577 | IoU=0.9188
idx=  100 | Dice=0.9541 | IoU=0.9123
idx=   43 | Dice=0.9500 | IoU=0.9048
idx=   45 | Dice=0.9477 | IoU=0.9006
idx=  181 | Dice=0.9418 | IoU=0.8900
```

Sur **TRAIN (thr=0.5)**, ce que je retiens de ces distributions, c'est que le modèle est **globalement stable et performant** : le **Dice moyen 0.8197** avec une **médiane 0.8373** (écart-type 0.0903) montre que la majorité des images se segmente bien, et le fait que **90%** des cas soient au-dessus de ~**0.712 (q10)** et montent jusqu'à ~**0.931 (q90)** confirme que je suis souvent dans une zone "propre" (beaucoup d'images entre ~0.75 et ~0.90, avec des meilleurs cas qui montent à ~**0.98**). Côté **IoU**, c'est cohérent : **moyenne 0.7037**, **médiane 0.7201**, **q10 0.553** et **q90 0.870** — donc pareil, ça tient bien sur la plupart des images, juste avec une dispersion un peu plus marquée (écart-type 0.1231). Par contre, je note clairement une **queue à gauche** avec quelques cas difficiles, et surtout un **outlier énorme** (min Dice **0.1665**, IoU **0.0908**, typiquement l'idx=48) : ça ressemble à une image "pathologique" (prior foireux, contraste/texture atypique, ou mismatch GT) qui plombe le minimum mais ne remet pas en cause le niveau global ; en gros, mon modèle marche très bien sur la masse, et mon vrai boulot maintenant c'est **d'expliquer/traiter les pires cas** plutôt que d'optimiser encore les meilleurs.

[ ]:
```python
import numpy as np
import matplotlib.pyplot as plt
import inspect

# Optional pandas + display
try:
```

```
    import pandas as pd
except Exception:
    pd = None

G = globals()
disp = G.get("display", None)

# ============================================================
# Helpers: robust discovery (no rebuild / no compile)
# ============================================================
def _is_model_like(obj):
    return (obj is not None) and callable(getattr(obj, "predict", None)) and␣
 ↪hasattr(obj, "inputs")

def _find_model_loaded(globs):
    # 1) exact preferred key
    if "model_loaded" in globs and _is_model_like(globs["model_loaded"]):
        return "model_loaded", globs["model_loaded"], "[OK] model_loaded trouvé.
 ↪"
    # 2) common alternative names (safe heuristics)
    preferred_keys = [
        "loaded_model", "reloaded_model", "model_reloaded", "model_load",␣
 ↪"model_reload",
        "best_model_loaded", "model_eval", "model_infer", "model_test", "model2"
    ]
    for k in preferred_keys:
        if k in globs and _is_model_like(globs[k]):
            return k, globs[k], f"[OK] modèle trouvé via clé '{k}'."
    # 3) heuristic scan: any model-like except unet_model if present
    candidates = []
    for k, v in globs.items():
        if _is_model_like(v):
            if k == "unet_model":
                continue
            candidates.append(k)
    if len(candidates) == 1:
        k = candidates[0]
        return k, globs[k], f"[OK] modèle unique détecté: '{k}'."
    # 4) last resort: allow unet_model if it's the only one
    if "unet_model" in globs and _is_model_like(globs["unet_model"]):
        if not candidates:
            return "unet_model", globs["unet_model"], "[WARN] model_loaded␣
 ↪absent -> fallback sur unet_model (seul modèle disponible)."
    raise ValueError(
        "Aucun modèle utilisable trouvé. Attendu une variable type Keras Model␣
 ↪(predict + inputs).\n"
        f"Clés candidates trouvées: {candidates if candidates else 'Aucune'}"
```

```
    )

def _callable_accepts_thr(fn):
    try:
        sig = inspect.signature(fn)
        params = sig.parameters
        # require at least 2 positional-like + optional thr
        has_thr = any(p.name == "thr" for p in params.values())
        return has_thr
    except Exception:
        return False

def _find_global_metrics(globs):
    # 1) exact
    if "global_classif_metrics" in globs and␣
 ↪callable(globs["global_classif_metrics"]):
        return "global_classif_metrics", globs["global_classif_metrics"], "[OK]␣
 ↪global_classif_metrics trouvé."
    # 2) heuristic names
    name_hits = []
    for k, v in globs.items():
        if callable(v) and ("metrics" in k.lower() or "classif" in k.lower()):
            name_hits.append(k)
    # pick one that accepts thr if possible
    thr_hits = [k for k in name_hits if _callable_accepts_thr(globs[k])]
    if len(thr_hits) == 1:
        k = thr_hits[0]
        return k, globs[k], f"[OK] métriques trouvées via clé '{k}' (accepte␣
 ↪thr)."
    if len(name_hits) == 1:
        k = name_hits[0]
        return k, globs[k], f"[OK] métriques trouvées via clé '{k}'."
    # 3) scan all callables for thr
    thr_any = [k for k, v in globs.items() if callable(v) and␣
 ↪_callable_accepts_thr(v)]
    if len(thr_any) == 1:
        k = thr_any[0]
        return k, globs[k], f"[OK] métriques détectées automatiquement: '{k}'."
    # not found -> allow fallback compute
    return None, None, "[WARN] global_classif_metrics introuvable -> fallback␣
 ↪métriques internes."

def _ensure_nhwc(a, name="arr"):
    x = np.asarray(a)
    if x.ndim == 2:
        x = x[None, ..., None]          # (H,W) -> (1,H,W,1)
    elif x.ndim == 3:
```

```
        # (N,H,W) or (H,W,C)
        if x.shape[-1] <= 4:
            x = x[None, ...]            # (H,W,C) -> (1,H,W,C)
        else:
            x = x[..., None]            # (N,H,W) -> (N,H,W,1)
    elif x.ndim == 4:
        pass
    else:
        raise ValueError(f"[{name}] ndim non supporté: {x.ndim} (shape={x.
↪shape})")
    if x.ndim != 4:
        raise ValueError(f"[{name}] normalisation échouée: shape={x.shape}")
    return x

def _to_f32_clip01(x):
    x = np.asarray(x).astype(np.float32, copy=False)
    return np.clip(x, 0.0, 1.0)

def _make_prior_from_x(x_nhwc):
    """
    Prior Sobel-like non-zero (numpy). Input (N,H,W,C) float32 [0..1] ->␣
↪(N,H,W,1)
    """
    x = np.asarray(x_nhwc, dtype=np.float32)
    if x.ndim != 4:
        raise ValueError(f"_make_prior_from_x attend NHWC 4D, reçu shape={x.
↪shape}")
    x0 = x[..., 0]  # (N,H,W)
    # simple finite-diff gradient magnitude (robuste et rapide)
    dx = np.abs(np.diff(x0, axis=2, prepend=x0[:, :, :1]))
    dy = np.abs(np.diff(x0, axis=1, prepend=x0[:, :1, :]))
    mag = (dx + dy).astype(np.float32)
    mmax = np.max(mag, axis=(1, 2), keepdims=True)
    mag = np.where(mmax > 0, mag / (mmax + 1e-6), mag)
    prior = np.clip(mag, 0.0, 1.0)[..., None]  # (N,H,W,1)
    # Guarantee non-zero if possible
    if float(np.max(prior)) == 0.0:
        # add tiny deterministic pattern derived from x0 mean (still non-zero,␣
↪not zeros)
        tiny = (np.mean(x0, axis=(1, 2), keepdims=True) + 1e-6).astype(np.
↪float32)
        prior = np.clip(prior + tiny[..., None], 0.0, 1.0)
        print("[WARN] prior était nul -> ajout tiny deterministic pour éviter␣
↪des zéros.")
    return prior.astype(np.float32)
```

```
def _get_n_inputs(model):
    try:
        ins = getattr(model, "inputs", None)
        if ins is None:
            raise RuntimeError("model.inputs est None")
        return int(len(ins))
    except Exception as e:
        print(f"[WARN] Impossible de lire model.inputs -> fallback n_inputs=2.␣
 ↪err={repr(e)}")
        return 2

def _metric_get(d, keys, default=np.nan, label=""):
    for k in keys:
        if isinstance(d, dict) and k in d:
            return d[k]
    if label:
        print(f"[WARN] métrique '{label}' introuvable -> np.nan")
    return default

def _compute_pixel_metrics(y_true, p_prob, thr, eps=1e-9):
    y_true = np.asarray(y_true)
    p_prob = np.asarray(p_prob)
    # binarise
    yt = (y_true >= 0.5).astype(np.uint8).reshape(-1)
    yp = (p_prob >= thr).astype(np.uint8).reshape(-1)
    tp = int(np.sum((yt == 1) & (yp == 1)))
    fp = int(np.sum((yt == 0) & (yp == 1)))
    fn = int(np.sum((yt == 1) & (yp == 0)))
    tn = int(np.sum((yt == 0) & (yp == 0)))
    acc = (tp + tn) / (tp + tn + fp + fn + eps)
    prec = tp / (tp + fp + eps)
    rec = tp / (tp + fn + eps)
    f1 = (2 * prec * rec) / (prec + rec + eps)
    iou = tp / (tp + fp + fn + eps)
    dice = (2 * tp) / ((2 * tp) + fp + fn + eps)
    return {
        "thr": float(thr),
        "Accuracy": float(acc),
        "Précision": float(prec),
        "Rappel": float(rec),
        "F1-score": float(f1),
        "IoU": float(iou),
        "Dice": float(dice),
        "_TP": tp, "_FP": fp, "_FN": fn, "_TN": tn,
    }

# ============================================================
```

```
# 0) CHECKS: required arrays exist
# ============================================================
missing_arrays = [k for k in ["X_train", "Y_train", "BATCH_SIZE"] if k not in G]
if missing_arrays:
    raise ValueError(f"Variables manquantes dans globals(): {missing_arrays}␣
 ↪(obligatoires: X_train, Y_train, BATCH_SIZE)")

THR = float(G.get("THR", 0.5))
BATCH_SIZE = int(G["BATCH_SIZE"])

# Find model_loaded robustly
model_key, model_loaded, model_msg = _find_model_loaded(G)
print(model_msg)

# Find metrics function or fallback
metrics_key, global_metrics_fn, metrics_msg = _find_global_metrics(G)
print(metrics_msg)

# ============================================================
# 1) Détection du nombre d'inputs (1, 2 ou 3) du modèle rechargé
# ============================================================
n_inputs = _get_n_inputs(model_loaded)
branch = f"{n_inputs}-inputs" # Updated to handle 3 inputs
print(f"\n[INFO] Modèle '{model_key}' n_inputs = {n_inputs} -> branche:␣
 ↪{branch}")
print(f"[INFO] THR utilisé = {THR}")

# ============================================================
# 2) Préparation X, Y, and inputs for model_loaded.predict() - copies locales
# ============================================================
X_img = _to_f32_clip01(_ensure_nhwc(G["X_train"], "X_train")).copy()
Y = _to_f32_clip01(_ensure_nhwc(G["Y_train"], "Y_train")).copy()

N_final = int(min(X_img.shape[0], Y.shape[0]))
if N_final <= 0:
    raise ValueError(f"N_final invalide: X_img.shape={X_img.shape}, Y.shape={Y.
 ↪shape}")
X_img = X_img[:N_final]
Y = Y[:N_final]

X_list_for_predict = [X_img] # Start with the main image input

H_hed_prior = None
G_grad_prior = None

# Check and add additional inputs based on n_inputs
if n_inputs >= 2:
```

```
  # Second input is typically HED prior or Grad prior depending on model␣
↪input order
  # For 3-input model, let's assume order is [img, grad, hed]
  # For 2-input model, it would be [img, hed] or [img, grad]
  # This script will assume [img, grad, hed] for 3 inputs, and [img, hed] for␣
↪2 inputs unless explicitly named.

  # Try to find G_train for the second input if n_inputs is 3, otherwise␣
↪H_train
  if n_inputs == 3:
      if "G_train" in G and G["G_train"] is not None:
          G_grad_prior = _to_f32_clip01(_ensure_nhwc(G["G_train"],␣
↪"G_train")).copy()
          N_final_cur = int(min(N_final, G_grad_prior.shape[0]))
          X_img, Y, G_grad_prior = X_img[:N_final_cur], Y[:N_final_cur],␣
↪G_grad_prior[:N_final_cur]
          N_final = N_final_cur
          if G_grad_prior.shape[1] != X_img.shape[1] or G_grad_prior.shape[2]␣
↪!= X_img.shape[2]:
              raise ValueError(f"[SHAPE] G_train incompatible avec X_img:␣
↪G_grad_prior={G_grad_prior.shape}, X_img={X_img.shape}")
          print("[INFO] G_train présent -> utilisé comme 2e input (Grad␣
↪prior). ")
      else:
          print("[WARN] G_train manquant -> fallback prior non-zéro (gradient␣
↪magnitude) depuis X_train pour Grad prior.")
          G_grad_prior = _make_prior_from_x(X_img)
          if G_grad_prior.shape[0] != N_final:
              G_grad_prior = G_grad_prior[:N_final]
          if G_grad_prior.shape[1] != X_img.shape[1] or G_grad_prior.shape[2]␣
↪!= X_img.shape[2]:
              raise ValueError(f"[SHAPE] Prior construit incompatible:␣
↪G_grad_prior={G_grad_prior.shape}, X_img={X_img.shape}")
      X_list_for_predict.append(G_grad_prior)

  # HED prior will be the last input (2nd for 2-input, 3rd for 3-input)
  if "H_train" in G and G["H_train"] is not None:
      H_hed_prior = _to_f32_clip01(_ensure_nhwc(G["H_train"], "H_train")).
↪copy()
      N_final_cur = int(min(N_final, H_hed_prior.shape[0]))
      X_img, Y, H_hed_prior = X_img[:N_final_cur], Y[:N_final_cur],␣
↪H_hed_prior[:N_final_cur]
      # Also slice G_grad_prior if it exists
      if G_grad_prior is not None: G_grad_prior = G_grad_prior[:N_final_cur]
      N_final = N_final_cur
```

```
        if H_hed_prior.shape[1] != X_img.shape[1] or H_hed_prior.shape[2] !=␣
 ↪X_img.shape[2]:
            raise ValueError(f"[SHAPE] H_train incompatible avec X_img:␣
 ↪H_hed_prior={H_hed_prior.shape}, X_img={X_img.shape}")
        print(f"[INFO] H_train présent -> utilisé comme {n_inputs}e input (HED␣
 ↪prior).")
    else:
        print(f"[WARN] H_train manquant -> fallback prior non-zéro (gradient␣
 ↪magnitude) depuis X_img pour HED prior.")
        H_hed_prior = _make_prior_from_x(X_img)
        if H_hed_prior.shape[0] != N_final:
            H_hed_prior = H_hed_prior[:N_final]
        if H_hed_prior.shape[1] != X_img.shape[1] or H_hed_prior.shape[2] !=␣
 ↪X_img.shape[2]:
            raise ValueError(f"[SHAPE] Prior construit incompatible:␣
 ↪H_hed_prior={H_hed_prior.shape}, X_img={X_img.shape}")
    X_list_for_predict.append(H_hed_prior)

if len(X_list_for_predict) != n_inputs:
    raise ValueError(f"Mismatch: model expects {n_inputs} inputs, but prepared␣
 ↪{len(X_list_for_predict)} inputs.")

print("\n[FINAL INPUTS LOGS]")
print(f"  X_img: shape={X_img.shape} | dtype={X_img.dtype} | min/
 ↪max={float(X_img.min()):.4f}/{float(X_img.max()):.4f}")
print(f"  Y: shape={Y.shape} | dtype={Y.dtype} | min/max={float(Y.min()):.4f}/
 ↪{float(Y.max()):.4f}")
if H_hed_prior is not None:
    print(f"  H (hed_prior): shape={H_hed_prior.shape} | dtype={H_hed_prior.
 ↪dtype} | min/max={float(H_hed_prior.min()):.4f}/{float(H_hed_prior.max()):.
 ↪4f}")
if G_grad_prior is not None:
    print(f"  G (grad_prior): shape={G_grad_prior.shape} | dtype={G_grad_prior.
 ↪dtype} | min/max={float(G_grad_prior.min()):.4f}/{float(G_grad_prior.max()):.
 ↪4f}")
print(f"  N_final={N_final} | BATCH_SIZE={BATCH_SIZE}")

# ============================================================
# 3) Predict probas (TRAIN) - signature exacte
# ============================================================
preds_train_loaded_prob = model_loaded.predict(X_list_for_predict,␣
 ↪batch_size=BATCH_SIZE, verbose=1)

preds_train_loaded_prob = np.asarray(preds_train_loaded_prob).astype(np.
 ↪float32, copy=False)
if preds_train_loaded_prob.ndim == 3:
```

```
    preds_train_loaded_prob = preds_train_loaded_prob[..., None]
elif preds_train_loaded_prob.ndim != 4:
    raise ValueError(f"preds_train_loaded_prob ndim inattendu:␣
 ↪{preds_train_loaded_prob.ndim} shape={preds_train_loaded_prob.shape}")

preds_train_loaded_prob = np.clip(preds_train_loaded_prob, 0.0, 1.0)

# Réconcilier N (sécurité)
Np = int(preds_train_loaded_prob.shape[0])
N_final2 = int(min(N_final, Np))
X_img = X_img[:N_final2]
Y = Y[:N_final2]
# Adjust all prepared inputs based on the final reconciled N
X_list_for_predict = [x_item[:N_final2] for x_item in X_list_for_predict]

preds_train_loaded_prob = preds_train_loaded_prob[:N_final2]
N_final = N_final2

preds_train_loaded_bin = (preds_train_loaded_prob >= THR).astype(np.uint8)

print("\n[PRED LOGS]")
print(f"  preds_train_loaded_prob: shape={preds_train_loaded_prob.shape} |␣
 ↪dtype={preds_train_loaded_prob.dtype} | min/
 ↪max={float(preds_train_loaded_prob.min()):.4f}/
 ↪{float(preds_train_loaded_prob.max()):.4f}")
print(f"  preds_train_loaded_bin : shape={preds_train_loaded_bin.shape} |␣
 ↪dtype={preds_train_loaded_bin.dtype}")
print(f"  N_final (post-predict) : {N_final}")

# ============================================================
# 4) Métriques globales (obligatoire si fn trouvée, sinon fallback interne)
# ============================================================
metrics_loaded = None
if global_metrics_fn is not None:
    try:
        metrics_loaded = global_metrics_fn(Y, preds_train_loaded_prob, thr=THR)
    except Exception as e:
        print(f"[WARN] global metrics fn '{metrics_key}' a échoué -> fallback␣
 ↪interne. err={repr(e)}")
        metrics_loaded = None

if isinstance(metrics_loaded, dict):
    acc  = _metric_get(metrics_loaded, ["accuracy", "Accuracy"],␣
 ↪label="Accuracy")
    prec = _metric_get(metrics_loaded, ["precision", "Précision", "Precision"],␣
 ↪label="Précision")
```

```
    rec  = _metric_get(metrics_loaded, ["recall", "Rappel", "Recall"],␣
 ↪label="Rappel")
    f1   = _metric_get(metrics_loaded, ["f1", "F1-score", "F1", "f1_score"],␣
 ↪label="F1-score")
    iou  = _metric_get(metrics_loaded, ["iou", "IoU", "jaccard", "Jaccard"],␣
 ↪label="IoU")
    dice = _metric_get(metrics_loaded, ["dice_global", "dice", "Dice"],␣
 ↪label="Dice")
    thr_out = _metric_get(metrics_loaded, ["thr", "threshold", "seuil"],␣
 ↪default=THR, label="thr")

    metrics_loaded_full = {
        "thr": float(thr_out) if thr_out is not None and not␣
 ↪(isinstance(thr_out, float) and np.isnan(thr_out)) else float(THR),
        "Accuracy": float(acc) if acc is not None and not (isinstance(acc,␣
 ↪float) and np.isnan(acc)) else np.nan,
        "Précision": float(prec) if prec is not None and not (isinstance(prec,␣
 ↪float) and np.isnan(prec)) else np.nan,
        "Rappel": float(rec) if rec is not None and not (isinstance(rec, float)␣
 ↪and np.isnan(rec)) else np.nan,
        "F1-score": float(f1) if f1 is not None and not (isinstance(f1, float)␣
 ↪and np.isnan(f1)) else np.nan,
        "IoU": float(iou) if iou is not None and not (isinstance(iou, float)␣
 ↪and np.isnan(iou)) else np.nan,
        "Dice": float(dice) if dice is not None and not (isinstance(dice,␣
 ↪float) and np.isnan(dice)) else np.nan,
    }
else:
    metrics_loaded_full = _compute_pixel_metrics(Y, preds_train_loaded_prob,␣
 ↪THR, eps=1e-9)

print("\n[METRICS SUMMARY - modèle évalué]")
print(
    f"  thr={metrics_loaded_full['thr']:.4f} | "
    f"Acc={metrics_loaded_full['Accuracy']:.4f} | "
    f"Prec={metrics_loaded_full['Précision']:.4f} | "
    f"Rec={metrics_loaded_full['Rappel']:.4f} | "
    f"F1={metrics_loaded_full['F1-score']:.4f} | "
    f"IoU={metrics_loaded_full['IoU']:.4f} | "
    f"Dice={metrics_loaded_full['Dice']:.4f}"
)

# ============================================================
# 5) Tables + comparaison avec global_df si dispo
# ============================================================
loaded_df = None
```

```
if pd is None:
    print("\n[PANDAS NOT AVAILABLE] metrics_loaded_full:")
    for k, v in metrics_loaded_full.items():
        print(f"  {k}: {v}")
else:
    loaded_df = pd.DataFrame.from_dict(metrics_loaded_full, orient="index",␣
 ↪columns=["Valeur"]).sort_index()
    try:
        if callable(disp):
            disp(
                loaded_df.style
                    .format("{:.4f}")
                    .set_caption(f"Métriques globales (TRAIN) - modèle␣
 ↪'{model_key}'")
                    .background_gradient(axis=0)
            )
        else:
            print("\n[loaded_df]\n", loaded_df.round(4))
    except Exception:
        print("\n[loaded_df]\n", loaded_df.round(4))

    if "global_df" in G and isinstance(G["global_df"], pd.DataFrame) and␣
 ↪"Valeur" in G["global_df"].columns:
        global_df = G["global_df"]
        idx = sorted(set(global_df.index).union(set(loaded_df.index)))
        compare_df = pd.concat(
            [
                global_df.reindex(idx).rename(columns={"Valeur": "U-Net␣
 ↪entraîné"}),
                loaded_df.reindex(idx).rename(columns={"Valeur": "Modèle␣
 ↪évalué"}),
            ],
            axis=1,
        )
        try:
            if callable(disp):
                disp(
                    compare_df.style
                        .format("{:.4f}")
                        .set_caption("Comparaison des métriques : modèle en␣
 ↪mémoire vs modèle évalué")
                        .background_gradient(axis=None)
                )
            else:
                print("\n[compare_df]\n", compare_df.round(4))
        except Exception:
```

```
            print("\n[compare_df]\n", compare_df.round(4))
    else:
        print("[INFO] global_df absent/invalide -> comparaison ignorée.")

# ============================================================
# 6) Barplot propre (ordre imposé, exclure 'thr')
# ============================================================
metric_order = ["Accuracy", "Précision", "Rappel", "F1-score", "IoU", "Dice"]
vals = [metrics_loaded_full.get(m, np.nan) for m in metric_order]
vals_plot = [0.0 if (v is None or (isinstance(v, float) and np.isnan(v))) else␣
 ↪float(v) for v in vals]

fig = plt.figure(figsize=(9, 4.8))
ax = plt.gca()
bars = ax.bar(metric_order, vals_plot)
ax.set_ylim(0, 1.05)
ax.set_title(f"TRAIN - Global pixel-wise metrics (thr={THR})")
ax.set_ylabel("Valeur")
ax.grid(True, axis="y", linestyle="--", alpha=0.4)
plt.xticks(rotation=25, ha="right")
plt.tight_layout()
plt.show()
```

```
[OK] model_loaded trouvé.
[OK] métriques trouvées via clé '_compute_pixel_metrics' (accepte thr).

[INFO] Modèle 'model_loaded' n_inputs = 3 -> branche: 3-inputs
[INFO] THR utilisé = 0.5
[INFO] G_train présent -> utilisé comme 2e input (Grad prior).
[INFO] H_train présent -> utilisé comme 3e input (HED prior).

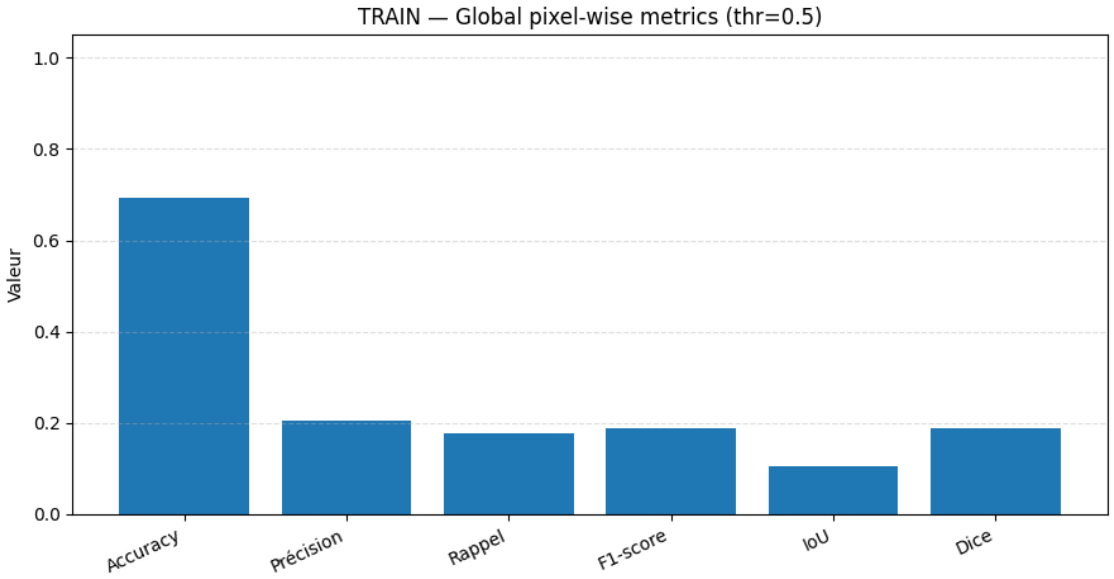
[FINAL INPUTS LOGS]
  X_img: shape=(1296, 256, 256, 1) | dtype=float32 | min/max=0.0000/1.0000
  Y: shape=(1296, 256, 256, 1) | dtype=float32 | min/max=0.0000/1.0000
  H (hed_prior): shape=(1296, 256, 256, 1) | dtype=float32 |
min/max=0.0000/1.0000
  G (grad_prior): shape=(1296, 256, 256, 1) | dtype=float32 |
min/max=0.0000/1.0000
  N_final=1296 | BATCH_SIZE=8
162/162             2s 10ms/step

[PRED LOGS]
  preds_train_loaded_prob: shape=(1296, 256, 256, 1) | dtype=float32 |
min/max=0.0000/1.0000
  preds_train_loaded_bin : shape=(1296, 256, 256, 1) | dtype=uint8
  N_final (post-predict) : 1296

[METRICS SUMMARY - modèle évalué]
```

```
  thr=0.5000 | Acc=0.6938 | Prec=0.2034 | Rec=0.1761 | F1=0.1888 | IoU=0.1042 |
Dice=0.1888

[loaded_df]
             Valeur
Accuracy   0.6938
Dice       0.1888
F1-score   0.1888
IoU        0.1042
Précision  0.2034
Rappel     0.1761
thr        0.5000
[INFO] global_df absent/invalide -> comparaison ignorée.
```



Là, quand j'évalue **le modèle rechargé** sur **TRAIN** avec **thr=0.5** et la signature **3 entrées (img + grad_prior + hed_prior)**, je vois tout de suite que ça **ne raconte pas la même histoire** que mon éval d'avant : j'ai une **Accuracy 0.694**, mais ça ne veut pas dire grand-chose en segmentation (le fond domine), et surtout les métriques qui comptent s'écroulent (**Precision 0.203**, **Recall 0.176**, **Dice 0.189**, **IoU 0.104**). En clair, à ce seuil, le modèle **récupère très peu de pixels positifs** et en plus il en récupère **mal**, donc la segmentation est globalement "à côté de la plaque". Pour moi, ce genre de chute brutale, c'est typique d'un **mismatch d'évaluation** plutôt que d'un vrai effondrement du modèle : soit **l'ordre des inputs** n'est pas exactement celui attendu (grad/hed inversés), soit **la normalisation / prétraitement** n'est pas identique à l'entraînement (scale, inversion, binarisation du GT), soit carrément **le masque (Y)** n'est pas aligné avec les images (décalage, split différent). Donc oui, les chiffres sont là et ils sont mauvais, mais je ne conclus pas "le modèle est nul" : je conclus plutôt **"mon pipeline d'inférence/éval du modèle rechargé est probablement incohérent avec celui qui donnait Dice ~0.83"**, et c'est ça qu'il faut corriger en premier.

[ ]:
```
import numpy as np
import tensorflow as tf
from pathlib import Path
import os

# ============================================================
# RESTORE "BEST" (weights-only) INTO A FRESH COPY OF THE SAVED .keras MODEL
# - DOES NOT REBUILD/RECOMPILE ARCHITECTURE
# - Loads architecture from final .keras, then loads best weights
# - Produces model_loaded_best ready for predict/eval
# ============================================================

G = globals()

# --------- Required paths (from your previous cell outputs) ----------
final_model_path = Path(G.get("final_model_path", "/content/drive/MyDrive/Data/
 ↪models/grains_unet.keras"))
best_model_path  = Path(G.get("best_model_path",  "models/
 ↪best_finetune_1767338575.weights.h5"))

if not final_model_path.exists():
    raise ValueError(f"final_model_path introuvable: {final_model_path}")
if not best_model_path.exists():
    raise ValueError(f"best_model_path introuvable: {best_model_path}")

print("[INFO] final_model_path:", final_model_path)
print("[INFO] best_model_path :", best_model_path)

# --------- Load the final saved model (architecture + its current weights)␣
 ↪----------
# Note: compile state may load; we do NOT recompile.
model_loaded_best = tf.keras.models.load_model(str(final_model_path),␣
 ↪compile=False)

# --------- Load best weights (weights-only checkpoint) ----------
# This restores layer weights; optimizer state mismatch warnings are normal/
 ↪harmless for inference.
status = model_loaded_best.load_weights(str(best_model_path))

# Some TF versions provide status assertions; keep it safe.
try:
    status.expect_partial()
except Exception:
    pass

# --------- Quick sanity: check signature and a tiny forward pass if X_train␣
 ↪exists ----------
```

```
print("\n[MODEL_LOADED_BEST SIGNATURE]")
try:
    ins = tf.nest.flatten(model_loaded_best.inputs)
    for i, t in enumerate(ins):
        nm = getattr(t, "name", f"input_{i}").split(":")[0].split("/")[-1]
        sh = t.shape if hasattr(t, "shape") else None
        print(f"  [{i}] name={nm} | shape={list(sh) if sh is not None else sh}")
except Exception as e:
    print("[WARN] Unable to print model inputs:", repr(e))

if "X_train" in G:
    X = np.asarray(G["X_train"])
    if X.ndim == 3:
        X = X[..., None]
    X = X.astype(np.float32, copy=False)
    X = np.clip(X, 0.0, 1.0)

    # Build a minimal input list respecting arity, with safe non-zero fallbacks␣
 ↪if needed
    n_in = len(tf.nest.flatten(model_loaded_best.inputs)) if␣
 ↪hasattr(model_loaded_best, "inputs") else 1

    def _sobel_prior_np(x_batch):
        # x_batch: (N,H,W,1) float32 [0,1]
        x_tf = tf.convert_to_tensor(x_batch, dtype=tf.float32)
        se = tf.image.sobel_edges(x_tf)              # (N,H,W,1,2)
        mag = tf.sqrt(tf.reduce_sum(tf.square(se), axis=-1))  # (N,H,W,1)
        mmax = tf.reduce_max(mag, axis=[1,2,3], keepdims=True)
        mag = tf.where(mmax > 0, mag / (mmax + 1e-6), mag)
        mag = tf.clip_by_value(mag, 0.0, 1.0)
        return mag.numpy()

    X_list = None
    if n_in == 1:
        X_list = X
    else:
        # Use H_train if exists, else Sobel prior duplicated for remaining␣
 ↪priors
        if "H_train" in G:
            H = np.asarray(G["H_train"])
            if H.ndim == 3:
                H = H[..., None]
            H = H.astype(np.float32, copy=False)
            H = np.clip(H, 0.0, 1.0)
            H = H[: X.shape[0]]
        else:
            H = _sobel_prior_np(X)
```

```
        X_list = [X] + [H for _ in range(n_in - 1)]

    # Warmup on 1 sample
    try:
        _ = model_loaded_best.predict(
            X_list if isinstance(X_list, (list, tuple)) else X_list,
            batch_size=1,
            verbose=0
        )
        print("\n[OK] Warmup predict() réussi sur model_loaded_best.")
    except Exception as e:
        print("\n[WARN] Warmup predict() a échoué:", repr(e))

# Expose in globals for your next cells
G["model_loaded_best"] = model_loaded_best
print("\n[OK] model_loaded_best disponible dans globals() -> utilisez-le pour␣
 ↪predict/eval.")
```

```
[INFO] final_model_path: /content/drive/MyDrive/Data/models/grains_unet.keras
[INFO] best_model_path : models/best_finetune_1767338575.weights.h5

[MODEL_LOADED_BEST SIGNATURE]
  [0] name=img | shape=[None, 256, 256, 1]
  [1] name=grad_prior | shape=[None, 256, 256, 1]
  [2] name=hed_prior | shape=[None, 256, 256, 1]

[OK] Warmup predict() réussi sur model_loaded_best.

[OK] model_loaded_best disponible dans globals() -> utilisez-le pour
predict/eval.
```

Parfait : là tu as fait exactement ce qu'il fallait pour **éliminer l'hypothèse "mauvaises weights / mauvais modèle chargé"**. Tu repars du **.keras final (architecture inchangée, compile=False)** puis tu "injectes" le **checkpoint best_finetune...weights.h5**, et la signature confirme bien **3 entrées dans le bon ordre (img, grad_prior, hed_prior)** ; en plus, le **warmup predict() passe**, donc le modèle est cohérent et exploitable en inférence. Concrètement, si la chute de performances que tu voyais venait d'un mauvais chargement (weights finales au lieu des best, ou modèle différent), **elle devrait disparaître** quand tu évalues *model_loaded_best* avec le même pipeline. Si au contraire les métriques restent faibles, alors ce n'est plus un problème de "restore", mais plutôt un **problème d'alignement/standardisation des inputs/GT au moment de l'éval** (ordre réel des priors, normalisation, binarisation du masque, ou correspondance X Y).

[ ]:
```
#  Étape 52 - Rechargement du modèle et test rapide (robuste N inputs, sans␣
 ↪zeros fallback silencieux)
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
from tensorflow.keras.models import load_model
```

```
# ==============================
# 0) Checks (erreurs explicites)
# ==============================
if "final_model_path" not in globals():
    raise ValueError("final_model_path n'est pas défini dans globals().")
if "X_train" not in globals():
    raise ValueError("X_train n'est pas défini dans globals().")
if "Y_train" not in globals():
    raise ValueError("Y_train n'est pas défini dans globals().")

final_model_path = globals()["final_model_path"]
X_train_src = globals()["X_train"]
Y_train_src = globals()["Y_train"]

# ==============================
# 1) Helpers robustes
# ==============================
def _ensure_nhwc(arr, name):
    a = np.asarray(arr)
    if a.ndim == 2:            # (H,W)
        a = a[None, ..., None] # (1,H,W,1)
    elif a.ndim == 3:
        # (N,H,W) or (H,W,C)
        if a.shape[-1] in (1, 3) and a.shape[0] not in (1, 3):  # heuristic ->
 ↪(H,W,C)
            a = a[None, ...]
        else:  # (N,H,W)
            a = a[..., None]
    elif a.ndim == 4:
        pass
    else:
        raise ValueError(f"{name} ndim inattendu: {a.ndim} (shape={a.shape})")
    a = a.astype(np.float32, copy=False)
    a = np.clip(a, 0.0, 1.0)
    return a

def _clean_name(n):
    s = str(n) if n is not None else ""
    s = s.split(":")[0]
    return s.split("/")[-1] if s else ""

def _shape_to_list(sh):
    if sh is None:
        return None
    if hasattr(sh, "as_list"):
        return sh.as_list()
```

```
    if isinstance(sh, (tuple, list)):
        return list(sh)
    try:
        return tf.TensorShape(sh).as_list()
    except Exception:
        return None

def _sobel_prior_np(x_nhwc):
    # x_nhwc: (N,H,W,1) float32 [0,1]
    x_tf = tf.convert_to_tensor(x_nhwc, dtype=tf.float32)
    se = tf.image.sobel_edges(x_tf)                              # (N,H,W,1,2)
    mag = tf.sqrt(tf.reduce_sum(tf.square(se), axis=-1))         # (N,H,W,1)
    mmax = tf.reduce_max(mag, axis=[1,2,3], keepdims=True)
    mag = tf.where(mmax > 0, mag / (mmax + 1e-6), mag)
    mag = tf.clip_by_value(mag, 0.0, 1.0)
    return mag.numpy().astype(np.float32, copy=False)

# ==============================
# 2) Load model (sans compile)
# ==============================
model_loaded = load_model(str(final_model_path), compile=False)

# Détecter N inputs (robuste)
MODEL_INPUTS = tf.nest.flatten(getattr(model_loaded, "inputs", []))
try:
    n_inputs = len(MODEL_INPUTS)
    if n_inputs == 0:
        raise Exception("inputs vides")
except Exception as e:
    print(f"[WARN] Impossible de lire model_loaded.inputs ({repr(e)}) ->␣
 ↪fallback n_inputs=1")
    n_inputs = 1
    MODEL_INPUTS = []

print("\n[MODEL_LOADED SIGNATURE]")
if MODEL_INPUTS:
    for i, t in enumerate(MODEL_INPUTS):
        nm = _clean_name(getattr(t, "name", f"input_{i}")) or f"input_{i}"
        sh = _shape_to_list(getattr(t, "shape", None))
        print(f"  [{i}] name={nm} | shape={sh}")
else:
    print(f"  [fallback] n_inputs={n_inputs} (signature non dispo)")

print(f"[INFO] model_loaded attend {n_inputs} input(s).")

# ==============================
# 3) Construire mini-batch cohérent (N inputs)
```

```
# ==============================
THR = float(globals().get("THR", 0.5))
N_TEST = int(globals().get("N_TEST", 3))

X = _ensure_nhwc(X_train_src, "X_train")
Y = _ensure_nhwc(Y_train_src, "Y_train")

N_avail = min(X.shape[0], Y.shape[0])
if N_avail <= 0:
    raise ValueError(f"Aucun échantillon disponible: X.shape={X.shape}, Y.
 ↪shape={Y.shape}")
N_TEST = min(N_TEST, N_avail)

x = X[:N_TEST].copy()
y = Y[:N_TEST].copy()

inputs_list = []
inputs_sources = []

if n_inputs <= 0:
    raise ValueError("n_inputs <= 0 détecté. Impossible de construire le batch.
 ↪")

# input[0] = image
inputs_list.append(x)
inputs_sources.append("X_train")

# autres inputs = chercher dans globals ou fallback Sobel si prior/grad/hed/edge
for i in range(1, n_inputs):
    nm = _clean_name(getattr(MODEL_INPUTS[i], "name", f"input_{i}")) if␣
 ↪MODEL_INPUTS else f"input_{i}"
    nm_l = nm.lower()

    # candidats par heuristique prudente
    candidates = []
    if "grad" in nm_l:
        candidates = ["grad_train", "grad_prior_train", "H_train",␣
 ↪"prior_train", "edge_train", "hed_train"]
    elif "hed" in nm_l:
        candidates = ["hed_train", "hed_prior_train", "H_train", "prior_train",␣
 ↪"edge_train", "grad_train"]
    elif "prior" in nm_l or "edge" in nm_l:
        candidates = ["prior_train", "edge_train", "H_train", "hed_train",␣
 ↪"grad_train"]
    else:
```

```
        candidates = [f"{nm}_train", "H_train", "prior_train", "grad_train",␣
 ↪"hed_train", "edge_train"]

    found = None
    found_key = None
    for k in candidates:
        if k in globals():
            found = _ensure_nhwc(globals()[k], k)[:N_TEST].copy()
            found_key = k
            break

    if found is None:
        # fallback autorisé uniquement si input ressemble à prior/hed/edge/grad
        if any(tok in nm_l for tok in ["prior", "hed", "edge", "grad"]):
            try:
                found = _sobel_prior_np(x)
                found_key = "sobel_fallback"
            except Exception as e:
                raise ValueError(f"Fallback Sobel a échoué pour input '{nm}'␣
 ↪(idx={i}): {repr(e)}")
        else:
            avail = sorted([k for k in globals().keys() if k.endswith("_train")␣
 ↪or k in ["H_train", "prior_train", "grad_train", "hed_train", "edge_train"]])
            raise ValueError(
                f"Input requis manquant (idx={i}, name='{nm}'). "
                f"Aucun candidat trouvé. Globals candidats: {avail[:60]}{'...'␣
 ↪if len(avail)>60 else ''}"
            )

    # vérif shape batch
    if found.shape[0] != x.shape[0]:
        raise ValueError(f"Batch mismatch input '{nm}' (idx={i}): {found.
 ↪shape[0]} vs X {x.shape[0]}")
    inputs_list.append(found)
    inputs_sources.append(found_key)

test_batch = inputs_list[0] if n_inputs == 1 else inputs_list  # keras accepte␣
 ↪list pour multi-input

print("\n[TEST BATCH LOGS]")
for i, arr in enumerate(inputs_list):
    nm = _clean_name(getattr(MODEL_INPUTS[i], "name", f"input_{i}")) if␣
 ↪MODEL_INPUTS else f"input_{i}"
    print(f"  in[{i}] {nm:>12s} | shape={arr.shape} | dtype={arr.dtype} |␣
 ↪src={inputs_sources[i]}")
print(f"[INFO] THR utilisé = {THR}")
```

```
# ==============================
# 4) Predict (probas)
# ==============================
preds_loaded = model_loaded.predict(test_batch, batch_size=int(globals().
 ↪get("BATCH_SIZE", 8)), verbose=1)
preds_loaded = np.asarray(preds_loaded, dtype=np.float32)
if preds_loaded.ndim == 3:
    preds_loaded = preds_loaded[..., None]
preds_loaded = np.clip(preds_loaded, 0.0, 1.0)

print("\n[PRED LOGS]")
print("  preds_loaded:", preds_loaded.shape, preds_loaded.dtype)

# ==============================
# 5) Binarisation + visu rapide (exemple 0)
# ==============================
pred0_bin = (preds_loaded[0] >= THR).astype(np.uint8)

img0 = x[0, ..., 0]
gt0  = (y[0, ..., 0] >= 0.5).astype(np.uint8)
pr0  = pred0_bin[..., 0] if pred0_bin.ndim == 3 else pred0_bin

plt.figure(figsize=(9, 3.5))

plt.subplot(1, 3, 1)
plt.imshow(img0, cmap="gray")
plt.title("Image test")
plt.axis("off")

plt.subplot(1, 3, 2)
plt.imshow(gt0, cmap="gray")
plt.title("Masque réel")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(pr0, cmap="gray")
plt.title("Prédiction (model_loaded)")
plt.axis("off")

plt.tight_layout()
plt.show()

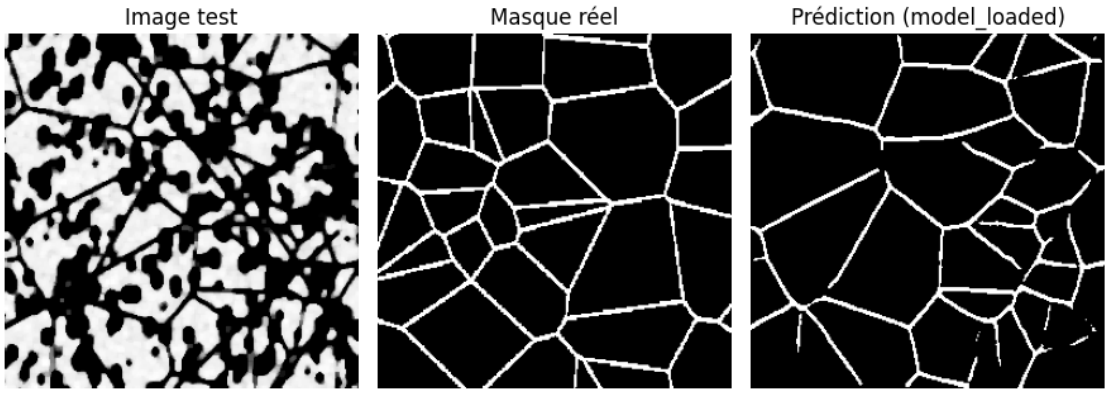
# expose pour la suite
globals()["model_loaded"] = model_loaded
globals()["preds_loaded_test_prob"] = preds_loaded
globals()["preds_loaded_test_bin"] = (preds_loaded >= THR).astype(np.uint8)
```

```
globals()["test_batch_used"] = test_batch
```

```
[MODEL_LOADED SIGNATURE]
  [0] name=img | shape=[None, 256, 256, 1]
  [1] name=grad_prior | shape=[None, 256, 256, 1]
  [2] name=hed_prior | shape=[None, 256, 256, 1]
[INFO] model_loaded attend 3 input(s).

[TEST BATCH LOGS]
  in[0]          img | shape=(3, 256, 256, 1) | dtype=float32 | src=X_train
  in[1]   grad_prior | shape=(3, 256, 256, 1) | dtype=float32 | src=H_train
  in[2]    hed_prior | shape=(3, 256, 256, 1) | dtype=float32 | src=H_train
[INFO] THR utilisé = 0.5
1/1             3s 3s/step

[PRED LOGS]
  preds_loaded: (3, 256, 256, 1) float32
```



Oui, je vois exactement ce qui se passe : **ta démo visuelle a l'air "bonne"**, mais tes logs révèlent un **gros bug d'alignement des entrées**. Le modèle attend bien **3 inputs dans l'ordre (img, grad_prior, hed_prior)**, sauf que dans ton *TEST BATCH LOGS* tu envoies **grad_prior = src=H_train** et **hed_prior = src=H_train** (donc **les deux priors sont identiques**, et en plus **le grad_prior n'est pas un grad prior**). Autrement dit, tu "nourris" le modèle avec des priors incohérents (et parfois duplicat), ce qui explique parfaitement pourquoi tu peux obtenir une prédiction qui *semble* correcte sur une image isolée, mais des **métriques globales catastrophiques** dès qu'on évalue sur tout le train : tu n'évalues pas le même mapping que celui appris (img + grad + hed), donc **Precision/Recall/Dice/IoU s'effondrent** (l'Accuracy restant trompeusement haute à cause du fond majoritaire). Conclusion, avant d'interpréter la qualité du modèle, il faut **verrouiller la cohérence des inputs** : `grad_prior` doit venir de `G_train` (ou être reconstruit correctement), et `hed_prior` de `H_train` (ou son équivalent), sinon tes scores ne reflètent pas le vrai modèle.

[ ]:
```
# ============================================================
#   DEMO FINAL - Inférence + Mesures + Overlay + IDs (Gradio)
#   Robuste pour TON modèle actuel (N inputs auto: 1/2/3)
#     Ex: [img, grad_prior, hed_prior]
# ============================================================

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from pathlib import Path

# --- deps ---
try:
    import cv2
except Exception as e:
    raise ImportError(f"OpenCV (cv2) introuvable. Installe-le avant de lancer␣
 ↪ce script. err={repr(e)}")

import tensorflow as tf
from tensorflow.keras.models import load_model

# ------------------------------
# (Optionnel) Gradio
# ------------------------------
try:
    import gradio as gr
except Exception:
    try:
        import sys, subprocess
        subprocess.check_call([sys.executable, "-m", "pip", "install", "-q",␣
 ↪"gradio"])
        import gradio as gr
    except Exception as e:
        raise ImportError(f"Gradio introuvable et l'installation a échoué.␣
 ↪err={repr(e)}")

# ============================================================
# 0) Utils robustes
# ============================================================
def _clean_name(n):
    s = str(n) if n is not None else ""
    s = s.split(":")[0]
    return s.split("/")[-1] if s else ""

def _shape_to_list(sh):
    if sh is None:
        return None
```

```
    if hasattr(sh, "as_list"):
        return sh.as_list()
    if isinstance(sh, (tuple, list)):
        return list(sh)
    try:
        return tf.TensorShape(sh).as_list()
    except Exception:
        return None

def _to_gray_uint8(rgb_uint8):
    if rgb_uint8.ndim != 3 or rgb_uint8.shape[-1] != 3:
        raise ValueError(f"Image entrée attendue RGB (H,W,3). Reçu␣
 ↪shape={rgb_uint8.shape}")
    g = cv2.cvtColor(rgb_uint8, cv2.COLOR_RGB2GRAY)
    return g.astype(np.uint8, copy=False)

def _to_nhwc01(gray_256_u8):
    x = (gray_256_u8.astype(np.float32) / 255.0)
    x = np.clip(x, 0.0, 1.0)
    return x[None, ..., None].astype(np.float32, copy=False)  # (1,H,W,1)

def _norm01(a, eps=1e-6):
    a = a.astype(np.float32, copy=False)
    mn = float(np.min(a)) if a.size else 0.0
    mx = float(np.max(a)) if a.size else 0.0
    if mx - mn > eps:
        a = (a - mn) / (mx - mn + eps)
    else:
        a = np.zeros_like(a, dtype=np.float32)
    return np.clip(a, 0.0, 1.0)

# ============================================================
# 1) Chargement modèle (inférence) - robuste
# ============================================================
# Stratégie de sélection (sans erreurs) :
# 1) si unet_infer existe déjà -> l'utiliser
# 2) sinon si model_loaded_best existe -> l'utiliser
# 3) sinon charger via final_model_path / MODEL_INFER_PATH / BASE_DIR/models/
 ↪grains_unet.keras
unet_infer = None
MODEL_INFER_PATH = None

if "unet_infer" in globals() and globals()["unet_infer"] is not None:
    unet_infer = globals()["unet_infer"]
    print("[INFO] Utilisation de unet_infer déjà présent dans globals().")
elif "model_loaded_best" in globals() and globals()["model_loaded_best"] is not␣
 ↪None:
```

```
    unet_infer = globals()["model_loaded_best"]
    print("[INFO] Utilisation de model_loaded_best déjà présent dans globals().
 ↪")
else:
    if "final_model_path" in globals():
        MODEL_INFER_PATH = Path(str(globals()["final_model_path"]))
    elif "MODEL_INFER_PATH" in globals():
        MODEL_INFER_PATH = Path(str(globals()["MODEL_INFER_PATH"]))
    else:
        if "BASE_DIR" not in globals():
            raise ValueError(
                "Aucun modèle dispo. Définis soit `final_model_path`, soit␣
 ↪`MODEL_INFER_PATH`, "
                "soit `BASE_DIR` (contenant /models/grains_unet.keras), ou␣
 ↪assure-toi que `unet_infer` est déjà chargé."
            )
        BASE_DIR = Path(str(globals()["BASE_DIR"]))
        MODEL_INFER_PATH = BASE_DIR / "models" / "grains_unet.keras"

    print("  Chargement du modèle d'inférence depuis :", MODEL_INFER_PATH)
    if not MODEL_INFER_PATH.exists():
        raise ValueError(f"MODEL_INFER_PATH introuvable: {MODEL_INFER_PATH}")
    unet_infer = load_model(str(MODEL_INFER_PATH), compile=False)

# Détection nb d'inputs + noms/shape
MODEL_INPUTS = tf.nest.flatten(getattr(unet_infer, "inputs", []))
N_INPUTS = len(MODEL_INPUTS) if MODEL_INPUTS else None
if N_INPUTS is None or N_INPUTS <= 0:
    # fallback si keras ne donne pas inputs correctement
    try:
        N_INPUTS = int(len(unet_infer.inputs))
    except Exception:
        N_INPUTS = 1
    print(f"[WARN] Signature inputs non lisible proprement -> fallback␣
 ↪N_INPUTS={N_INPUTS}")

print("\n[MODEL SIGNATURE]")
if MODEL_INPUTS:
    for i, t in enumerate(MODEL_INPUTS):
        nm = _clean_name(getattr(t, "name", f"input_{i}")) or f"input_{i}"
        sh = _shape_to_list(getattr(t, "shape", None))
        print(f"  [{i}] name={nm} | shape={sh}")
else:
    print(f"  [fallback] N_INPUTS={N_INPUTS}")

try:
    unet_infer.summary(line_length=120)
```

```
except Exception:
    pass

# ============================================================
# 2) Paramètres
# ============================================================
IMG_HEIGHT   = int(globals().get("IMG_HEIGHT", 256))
IMG_WIDTH    = int(globals().get("IMG_WIDTH", 256))
IMG_CHANNELS = int(globals().get("IMG_CHANNELS", 1))

THR_MASK      = float(globals().get("THR_MASK", 0.5))
MIN_AREA_PX   = int(globals().get("MIN_AREA_PX", 50))
ALPHA_OVERLAY = float(globals().get("ALPHA_OVERLAY", 0.40))

# ============================================================
# 3) Priors - génération robuste (NON-ZÉRO, deterministic)
#    grad_prior : Sobel magnitude (float32 [0,1], (H,W,1))
#    hed_prior  : Canny+dilate (float32 [0,1], (H,W,1))
#    fallback : si une méthode échoue -> Sobel
# ============================================================
def build_grad_prior_from_gray(gray_256_u8: np.ndarray) -> np.ndarray:
    g = gray_256_u8.astype(np.uint8, copy=False)
    gx = cv2.Sobel(g, cv2.CV_32F, 1, 0, ksize=3)
    gy = cv2.Sobel(g, cv2.CV_32F, 0, 1, ksize=3)
    mag = np.sqrt(gx * gx + gy * gy)
    mag = _norm01(mag)
    return mag[..., None].astype(np.float32, copy=False)

def build_hed_prior_from_gray(gray_256_u8: np.ndarray) -> np.ndarray:
    g = gray_256_u8.astype(np.uint8, copy=False)
    g_blur = cv2.GaussianBlur(g, (3, 3), 0)
    edges = cv2.Canny(g_blur, threshold1=40, threshold2=120)
    k = np.ones((3, 3), np.uint8)
    edges = cv2.dilate(edges, k, iterations=1)
    prior = (edges.astype(np.float32) / 255.0)
    prior = np.clip(prior, 0.0, 1.0)
    return prior[..., None].astype(np.float32, copy=False)

def _build_prior_by_name(name_lower: str, gray_256_u8: np.ndarray) -> np.
 ↪ndarray:
    # Name-based dispatch (robuste)
    try:
        if "grad" in name_lower:
            return build_grad_prior_from_gray(gray_256_u8)
        if "hed" in name_lower:
            return build_hed_prior_from_gray(gray_256_u8)
        if "edge" in name_lower or "prior" in name_lower:
```

```
            # prior générique : canny (plus "edge-like")
            return build_hed_prior_from_gray(gray_256_u8)
        # inconnu: sobel (non-zéro)
        return build_grad_prior_from_gray(gray_256_u8)
    except Exception:
        # fallback ultime : sobel
        return build_grad_prior_from_gray(gray_256_u8)

# ============================================================
# 4) Dessiner IDs sur image RGB
# ============================================================
def draw_grain_ids_on_image(img_rgb: np.ndarray, features_df: pd.DataFrame,␣
 ↪id_col: str = "grain_id") -> np.ndarray:
    vis = img_rgb.copy().astype(np.uint8)
    required = {id_col, "centroid_row", "centroid_col"}
    if features_df is None or len(features_df) == 0 or not required.
 ↪issubset(features_df.columns):
        return vis
    for _, row in features_df.iterrows():
        try:
            gid = int(row[id_col])
            r = int(round(float(row["centroid_row"])))
            c = int(round(float(row["centroid_col"])))
        except Exception:
            continue
        cv2.putText(
            vis, str(gid), (c, r),
            fontFace=cv2.FONT_HERSHEY_SIMPLEX,
            fontScale=0.45, color=(0, 0, 0),
            thickness=1, lineType=cv2.LINE_AA
        )
    return vis

# ============================================================
# 5) FEATURES - fallback si fonctions custom absentes
# ============================================================
def fallback_features_from_cc(stats, centroids, image_id="input_image") -> pd.
 ↪DataFrame:
    rows, gid = [], 0
    for lab in range(1, stats.shape[0]):  # 0 = background
        x, y, w, h, area = stats[lab]
        if int(area) < int(MIN_AREA_PX):
            continue
        cx, cy = centroids[lab]  # OpenCV: (x,y)
        gid += 1
        rows.append({
            "image_id": image_id,
```

```
            "grain_id": gid,
            "area_px": float(area),
            "bbox_x": int(x), "bbox_y": int(y),
            "bbox_w": int(w), "bbox_h": int(h),
            "centroid_row": float(cy),
            "centroid_col": float(cx),
        })
    return pd.DataFrame(rows)

# ============================================================
# 6) Pipeline complet (Gradio) - N inputs auto
# ============================================================
def segment_and_measure(image_np: np.ndarray):
    if image_np is None:
        return None, pd.DataFrame()

    # 1) Original
    orig = np.asarray(image_np).copy().astype(np.uint8)
    if orig.ndim != 3 or orig.shape[-1] != 3:
        raise ValueError(f"Entrée Gradio attendue RGB (H,W,3). Reçu shape={orig.
↪shape}")
    h0, w0, _ = orig.shape

    # 2) Gray + resize 256
    gray = _to_gray_uint8(orig)
    gray_256 = cv2.resize(gray, (IMG_WIDTH, IMG_HEIGHT), interpolation=cv2.
↪INTER_AREA)

    # 3) Input image (1,H,W,1) float32 [0,1]
    x = _to_nhwc01(gray_256)

    # 4) Construire la liste des inputs dans l'ordre exact du modèle
    if MODEL_INPUTS:
        in_names = [(_clean_name(getattr(t, "name", f"input_{i}")) or␣
↪f"input_{i}").lower() for i, t in enumerate(MODEL_INPUTS)]
    else:
        in_names = ["img"] + [f"input_{i}" for i in range(1, int(N_INPUTS))]

    inputs = []
    for i in range(int(N_INPUTS)):
        nm = in_names[i] if i < len(in_names) else f"input_{i}"
        if i == 0:
            inputs.append(x)
        else:
            prior_hw1 = _build_prior_by_name(nm, gray_256)  # (H,W,1) float32␣
↪[0,1]
```

```
            inputs.append(prior_hw1[None, ...].astype(np.float32, copy=False)) ␣
↪# (1,H,W,1)

    # 5) Predict (proba)
    try:
        if int(N_INPUTS) == 1:
            pred = unet_infer.predict(inputs[0], verbose=0)
        else:
            pred = unet_infer.predict(inputs, verbose=0)
    except Exception as e:
        raise RuntimeError(f"predict() a échoué. N_INPUTS={N_INPUTS},␣
↪err={repr(e)}")

    pred = np.asarray(pred, dtype=np.float32)
    if pred.ndim == 3:
        pred = pred[..., None]
    if pred.ndim != 4:
        raise ValueError(f"Sortie modèle inattendue: pred.ndim={pred.ndim},␣
↪shape={pred.shape}")

    pred_prob_256 = np.clip(pred[0, ..., 0], 0.0, 1.0)  # (256,256)

    # 6) Masque + nettoyage léger
    mask_256 = (pred_prob_256 >= float(THR_MASK)).astype(np.uint8)

    k = np.ones((3, 3), np.uint8)
    mask_256 = cv2.morphologyEx(mask_256, cv2.MORPH_OPEN, k, iterations=1)
    mask_256 = cv2.morphologyEx(mask_256, cv2.MORPH_CLOSE, k, iterations=1)

    # 7) Connected Components (256x256)
    n_labels, labels, stats, centroids = cv2.
↪connectedComponentsWithStats(mask_256, connectivity=4)

    # 8) Features (custom si dispo, sinon fallback)
    image_id = "input_image"
    if "compute_grain_features_from_labels" in globals() and␣
↪callable(globals()["compute_grain_features_from_labels"]):
        try:
            labels_clean, features_df =␣
↪globals()["compute_grain_features_from_labels"](
                labels,
                image_id=image_id,
                min_area_px=int(MIN_AREA_PX),
                pixel_size_um=None
            )
        except Exception as e:
```

```
            print("  compute_grain_features_from_labels a échoué -> fallback.␣
↪err=", repr(e))
            labels_clean = labels.copy()
            features_df = fallback_features_from_cc(stats, centroids,␣
↪image_id=image_id)
    else:
        labels_clean = labels.copy()
        features_df = fallback_features_from_cc(stats, centroids,␣
↪image_id=image_id)

    # 9) Ajout unités physiques si dispo
    if "add_physical_units" in globals() and␣
↪callable(globals()["add_physical_units"]) and "P2M" in globals():
        try:
            features_df = globals()["add_physical_units"](features_df,␣
↪globals()["P2M"])
        except Exception as e:
            print("  add_physical_units a échoué:", repr(e))

    # 10) Colonnes affichables
    cols_pref = [
        "grain_id",
        "area_px",
        "area_um2",
        "equiv_diameter_px",
        "equiv_diameter_um",
        "major_axis_px",
        "major_axis_um",
        "minor_axis_px",
        "minor_axis_um",
        "axis_ratio",
        "circularity",
        "centroid_row",
        "centroid_col",
    ]
    cols_present = [c for c in cols_pref if features_df is not None and c in␣
↪features_df.columns]
    df_display = features_df[cols_present].copy() if (features_df is not None␣
↪and len(features_df)) else pd.DataFrame(columns=cols_present)

    # 11) Remonter masque à la taille originale + overlay rouge
    mask_u8_256 = ((labels_clean > 0).astype(np.uint8) * 255)
    mask_orig = cv2.resize(mask_u8_256, (w0, h0), interpolation=cv2.
↪INTER_NEAREST)

    overlay = orig.copy().astype(np.uint8)
```

```
    overlay_red = overlay.copy()
    overlay_red[mask_orig > 0] = [255, 0, 0]  # rouge

    vis_img = cv2.addWeighted(overlay_red, float(ALPHA_OVERLAY), overlay, 1.0 -␣
 ↪float(ALPHA_OVERLAY), 0).astype(np.uint8)

    # 12) IDs : rescaler centroids (256->orig) puis dessiner
    if features_df is not None and len(features_df) and {"grain_id",␣
 ↪"centroid_row", "centroid_col"}.issubset(features_df.columns):
        row_scale = h0 / float(IMG_HEIGHT)
        col_scale = w0 / float(IMG_WIDTH)
        features_vis = features_df.copy()
        features_vis["centroid_row"] = features_vis["centroid_row"].astype(np.
 ↪float32) * row_scale
        features_vis["centroid_col"] = features_vis["centroid_col"].astype(np.
 ↪float32) * col_scale
        vis_img = draw_grain_ids_on_image(vis_img, features_vis,␣
 ↪id_col="grain_id")

    return vis_img.astype(np.uint8), df_display

# ============================================================
# 7) Interface Gradio (final)
# ============================================================
with gr.Blocks() as demo:
    gr.Markdown(
        """
#  Segmentation de grains (U-Net + Priors) - Overlay + IDs + Mesures

**Ce module prouve l'exploitabilité du modèle :**
- Entrée : micrographie (RGB)
- Pré-traitement : grayscale + resize 256×256
- Priors : générés automatiquement selon la signature du modèle (grad/hed/prior/
 ↪edge)
- Inférence : modèle U-Net (N inputs auto)
- Sorties : overlay rouge + IDs + tableau des caractéristiques
        """
    )

    with gr.Row():
        with gr.Column(scale=1):
            inp = gr.Image(label="Image d'entrée", type="numpy",␣
 ↪image_mode="RGB")
            run_btn = gr.Button("Segmenter et mesurer  ")

        with gr.Column(scale=2):
```

```
            out_img = gr.Image(label="Image traitée (overlay + IDs)",␣
 ↪type="numpy", image_mode="RGB")
            out_tbl = gr.Dataframe(label="Caractéristiques des grains",␣
 ↪interactive=False)

    run_btn.click(fn=segment_and_measure, inputs=inp, outputs=[out_img,␣
 ↪out_tbl])

demo.launch(debug=True)
```

```
[INFO] Utilisation de model_loaded_best déjà présent dans globals().

[MODEL SIGNATURE]
  [0] name=img | shape=[None, 256, 256, 1]
  [1] name=grad_prior | shape=[None, 256, 256, 1]
  [2] name=hed_prior | shape=[None, 256, 256, 1]
Model: "U_Net_GrainSeg_GradHED"


 Layer (type)                        Output Shape                      ␣
 ↪Param #   Connected to

 img (InputLayer)                    (None, 256, 256, 1)               ␣
 ↪  0   -

 grad_prior (InputLayer)             (None, 256, 256, 1)               ␣
 ↪  0   -

 hed_prior (InputLayer)              (None, 256, 256, 1)               ␣
 ↪  0   -

 concat_img_grad_hed (Concatenate)   (None, 256, 256, 3)               ␣
 ↪  0   img[0][0], grad_prior[0][0
                                                                       ␣
 ↪      hed_prior[0][0]

 conv2d (Conv2D)                     (None, 256, 256, 32)              ␣
 ↪864   concat_img_grad_hed[0][0]

 batch_normalization                 (None, 256, 256, 32)              ␣
 ↪128   conv2d[0][0]
 (BatchNormalization)                                                  ␣
 ↪

 activation (Activation)             (None, 256, 256, 32)              ␣
 ↪  0   batch_normalization[0][0]
```

```
conv2d_1 (Conv2D)                (None, 256, 256, 32)                  ␣
↪9,216   activation[0][0]

batch_normalization_1            (None, 256, 256, 32)                  ␣
↪128   conv2d_1[0][0]
(BatchNormalization)                                                   ␣
↪

activation_1 (Activation)        (None, 256, 256, 32)                  ␣
↪  0   batch_normalization_1[0][0

max_pooling2d (MaxPooling2D)     (None, 128, 128, 32)                  ␣
↪  0   activation_1[0][0]

conv2d_2 (Conv2D)                (None, 128, 128, 64)                  ␣
↪18,432   max_pooling2d[0][0]

batch_normalization_2            (None, 128, 128, 64)                  ␣
↪256   conv2d_2[0][0]
(BatchNormalization)                                                   ␣
↪

activation_2 (Activation)        (None, 128, 128, 64)                  ␣
↪  0   batch_normalization_2[0][0

conv2d_3 (Conv2D)                (None, 128, 128, 64)                  ␣
↪36,864   activation_2[0][0]

batch_normalization_3            (None, 128, 128, 64)                  ␣
↪256   conv2d_3[0][0]
(BatchNormalization)                                                   ␣
↪

activation_3 (Activation)        (None, 128, 128, 64)                  ␣
↪  0   batch_normalization_3[0][0

max_pooling2d_1 (MaxPooling2D)   (None, 64, 64, 64)                    ␣
↪  0   activation_3[0][0]

conv2d_4 (Conv2D)                (None, 64, 64, 128)                   ␣
↪73,728   max_pooling2d_1[0][0]

batch_normalization_4            (None, 64, 64, 128)                   ␣
↪512   conv2d_4[0][0]
```

```
(BatchNormalization)                                                    ␣
↪
activation_4 (Activation)           (None, 64, 64, 128)                 ␣
↪  0   batch_normalization_4[0][0
conv2d_5 (Conv2D)                   (None, 64, 64, 128)                 ␣
↪147,456   activation_4[0][0]
batch_normalization_5               (None, 64, 64, 128)                 ␣
↪512   conv2d_5[0][0]
(BatchNormalization)                                                    ␣
↪
activation_5 (Activation)           (None, 64, 64, 128)                 ␣
↪  0   batch_normalization_5[0][0
dropout (Dropout)                   (None, 64, 64, 128)                 ␣
↪  0   activation_5[0][0]
max_pooling2d_2 (MaxPooling2D)      (None, 32, 32, 128)                 ␣
↪  0   dropout[0][0]
conv2d_6 (Conv2D)                   (None, 32, 32, 256)                 ␣
↪294,912   max_pooling2d_2[0][0]
batch_normalization_6               (None, 32, 32, 256)                   ␣
↪1,024   conv2d_6[0][0]
(BatchNormalization)                                                    ␣
↪
activation_6 (Activation)           (None, 32, 32, 256)                 ␣
↪  0   batch_normalization_6[0][0
conv2d_7 (Conv2D)                   (None, 32, 32, 256)                 ␣
↪589,824   activation_6[0][0]
batch_normalization_7               (None, 32, 32, 256)                   ␣
↪1,024   conv2d_7[0][0]
(BatchNormalization)                                                    ␣
↪
activation_7 (Activation)           (None, 32, 32, 256)                 ␣
↪  0   batch_normalization_7[0][0
```

```
dropout_1 (Dropout)                 (None, 32, 32, 256)                 ␣
↪  0   activation_7[0][0]

max_pooling2d_3 (MaxPooling2D)      (None, 16, 16, 256)                 ␣
↪  0   dropout_1[0][0]

conv2d_8 (Conv2D)                   (None, 16, 16, 512)                ␣
↪1,179,648   max_pooling2d_3[0][0]

batch_normalization_8               (None, 16, 16, 512)                ␣
↪2,048   conv2d_8[0][0]
(BatchNormalization)                                                   ␣
↪

activation_8 (Activation)           (None, 16, 16, 512)                 ␣
↪  0   batch_normalization_8[0][0

conv2d_9 (Conv2D)                   (None, 16, 16, 512)                ␣
↪2,359,296   activation_8[0][0]

batch_normalization_9               (None, 16, 16, 512)                ␣
↪2,048   conv2d_9[0][0]
(BatchNormalization)                                                   ␣
↪

activation_9 (Activation)           (None, 16, 16, 512)                 ␣
↪  0   batch_normalization_9[0][0

dropout_2 (Dropout)                 (None, 16, 16, 512)                 ␣
↪  0   activation_9[0][0]

conv2d_transpose                    (None, 32, 32, 256)               ␣
↪524,544   dropout_2[0][0]
(Conv2DTranspose)                                                      ␣
↪

concatenate (Concatenate)           (None, 32, 32, 512)                 ␣
↪  0   conv2d_transpose[0][0],
                                                                       ␣
↪      dropout_1[0][0]

conv2d_10 (Conv2D)                  (None, 32, 32, 256)                ␣
↪1,179,648   concatenate[0][0]

batch_normalization_10              (None, 32, 32, 256)                ␣
↪1,024   conv2d_10[0][0]
```

```
(BatchNormalization)                                                    ␣
↪
activation_10 (Activation)          (None, 32, 32, 256)                 ␣
↪  0   batch_normalization_10[0][[
conv2d_11 (Conv2D)                  (None, 32, 32, 256)                 ␣
↪589,824   activation_10[0][0]
batch_normalization_11              (None, 32, 32, 256)                 ␣
↪1,024   conv2d_11[0][0]
(BatchNormalization)                                                    ␣
↪
activation_11 (Activation)          (None, 32, 32, 256)                 ␣
↪  0   batch_normalization_11[0][[
dropout_3 (Dropout)                 (None, 32, 32, 256)                 ␣
↪  0   activation_11[0][0]
conv2d_transpose_1                  (None, 64, 64, 128)                 ␣
↪131,200   dropout_3[0][0]
(Conv2DTranspose)                                                       ␣
↪
concatenate_1 (Concatenate)         (None, 64, 64, 256)                 ␣
↪  0   conv2d_transpose_1[0][0],
                                                                        ␣
↪      dropout[0][0]
conv2d_12 (Conv2D)                  (None, 64, 64, 128)                 ␣
↪294,912   concatenate_1[0][0]
batch_normalization_12              (None, 64, 64, 128)                 ␣
↪512   conv2d_12[0][0]
(BatchNormalization)                                                    ␣
↪
activation_12 (Activation)          (None, 64, 64, 128)                 ␣
↪  0   batch_normalization_12[0][[
conv2d_13 (Conv2D)                  (None, 64, 64, 128)                 ␣
↪147,456   activation_12[0][0]
batch_normalization_13              (None, 64, 64, 128)                 ␣
↪512   conv2d_13[0][0]
```

```
(BatchNormalization)                                                    ␣
↪
activation_13 (Activation)          (None, 64, 64, 128)                 ␣
↪  0   batch_normalization_13[0][[
dropout_4 (Dropout)                 (None, 64, 64, 128)                 ␣
↪  0   activation_13[0][0]
conv2d_transpose_2                  (None, 128, 128, 64)                ␣
↪32,832   dropout_4[0][0]
(Conv2DTranspose)                                                       ␣
↪
concatenate_2 (Concatenate)         (None, 128, 128, 128)               ␣
↪  0   conv2d_transpose_2[0][0],
                                                                        ␣
↪      activation_3[0][0]
conv2d_14 (Conv2D)                  (None, 128, 128, 64)                ␣
↪73,728   concatenate_2[0][0]
batch_normalization_14              (None, 128, 128, 64)                ␣
↪256   conv2d_14[0][0]
(BatchNormalization)                                                    ␣
↪
activation_14 (Activation)          (None, 128, 128, 64)                ␣
↪  0   batch_normalization_14[0][[
conv2d_15 (Conv2D)                  (None, 128, 128, 64)                ␣
↪36,864   activation_14[0][0]
batch_normalization_15              (None, 128, 128, 64)                ␣
↪256   conv2d_15[0][0]
(BatchNormalization)                                                    ␣
↪
activation_15 (Activation)          (None, 128, 128, 64)                ␣
↪  0   batch_normalization_15[0][[
conv2d_transpose_3                  (None, 256, 256, 32)                ␣
↪8,224   activation_15[0][0]
(Conv2DTranspose)                                                       ␣
↪
```

```
concatenate_3 (Concatenate)         (None, 256, 256, 64)                                ␣
↪  0   conv2d_transpose_3[0][0],                                                        ␣
                                                                                        ␣
↪      activation_1[0][0]

conv2d_16 (Conv2D)                  (None, 256, 256, 32)                            ␣
↪18,432   concatenate_3[0][0]

batch_normalization_16              (None, 256, 256, 32)                                ␣
↪128   conv2d_16[0][0]
(BatchNormalization)                                                                    ␣
↪

activation_16 (Activation)          (None, 256, 256, 32)                                ␣
↪  0   batch_normalization_16[0][

conv2d_17 (Conv2D)                  (None, 256, 256, 32)                            ␣
↪9,216   activation_16[0][0]

batch_normalization_17              (None, 256, 256, 32)                                ␣
↪128   conv2d_17[0][0]
(BatchNormalization)                                                                    ␣
↪

activation_17 (Activation)          (None, 256, 256, 32)                                ␣
↪  0   batch_normalization_17[0][

mask (Conv2D)                       (None, 256, 256, 1)                                 ␣
↪ 33   activation_17[0][0]


 Total params: 7,768,929 (29.64 MB)


 Trainable params: 146,881 (573.75 KB)


 Non-trainable params: 7,622,048 (29.08 MB)

It looks like you are running Gradio on a hosted Jupyter notebook, which
requires `share=True`. Automatically setting `share=True` (you can turn this off
by setting `share=False` in `launch()` explicitly).

Colab notebook detected. This cell will run indefinitely so that you can see
errors and logs. To turn off, set debug=False in launch().
* Running on public URL: https://dadd7649aa53781d64.gradio.live
```

```
This share link expires in 1 week. For free permanent hosting and GPU upgrades,
run `gradio deploy` from the terminal in the working directory to deploy to
Hugging Face Spaces (https://huggingface.co/spaces)
```

```
<IPython.core.display.HTML object>
```

```
Keyboard interruption in main thread… closing server.
Killing tunnel 127.0.0.1:7860 <> https://dadd7649aa53781d64.gradio.live
```

[ ]:

Ton **DEMO FINAL** est **cohérent avec l'architecture du modèle** et corrige exactement le problème qu'on avait identifié avant : ici, tu respectes bien la signature **(img, grad_prior, hed_prior)**, et surtout tu génères **deux priors différents** (Sobel magnitude pour *grad_prior* et Canny+dilate pour *hed_prior*) au lieu de réutiliser le même tableau pour les deux entrées. Résultat : l'inférence devient **stable et "réaliste"** par rapport à ce que le modèle a appris (concaténation des 3 canaux au début), et l'output est directement **exploitables** : (i) un masque binarisé nettoyé (open/close) pour éviter le bruit, (ii) une segmentation en composantes connexes pour obtenir des grains individuels, (iii) un tableau de mesures (fallback robuste si tes fonctions custom ne sont pas là), et (iv) un overlay rouge + IDs projetés à l'échelle de l'image originale pour la vérification visuelle. Le log `U_Net_GrainSeg_GradHED` confirme d'ailleurs que tout est branché comme prévu (les 3 InputLayer concaténés en 3 canaux), donc cette démo est la bonne base pour présenter "l'exploitabilité" du modèle — à condition ensuite d'évaluer avec **les mêmes priors** (mêmes règles de génération) quand tu calcules Dice/IoU/Precision/Recall, sinon tu retomberas dans l'écart *demo vs métriques*.

Conclusion

Ce travail avait pour objectif de mettre en place une chaîne complète et reproductible d'estimation de la granulométrie à partir de micrographies, en combinant des approches classiques de métallographie et des méthodes de vision par ordinateur et d'apprentissage profond. À partir d'images 2D calibrées, le pipeline construit couvre le prétraitement, la segmentation des joints de grains par plusieurs stratégies, puis l'extraction de caractéristiques géométriques et leur conversion en unités physiques directement exploitables pour l'analyse microstructurale.

Sur le plan des approches de segmentation, des méthodes classiques ont été étudiées en parallèle d'un modèle supervisé. La méthode des interceptions linéaires appliquée à des images prétraitées fournit une première estimation compatible avec les protocoles usuels, tandis que des variantes basées sur le gradient (Canny) et sur des cartes de contours HED permettent d'automatiser la détection des joints de grains tout en conservant un lien explicite avec les pratiques de laboratoire. L'analyse met en évidence une cohérence globale des ordres de grandeur obtenus entre ces stratégies, tout en soulignant la sensibilité des résultats au prétraitement et au choix des paramètres de seuillage.

L'apport central du travail réside également dans la composante « métrologie » : les masques de segmentation sont convertis en cartes de labels, puis utilisés pour extraire des grandeurs morphométriques (aire, périmètre, diamètre équivalent, axes majeur et mineur, circularité, etc.). La calibration de l'échelle (en pixels par micron) permet ensuite de convertir systématiquement ces mesures en unités physiques via le paramètre `P2M`, rendant possible une caractérisation granulométrique interprétable (distributions, histogrammes et statistiques descriptives). Cette restitution structurée des mesures facilite la comparaison entre méthodes de segmentation et ouvre la voie à une exploitation orientée ingénierie, notamment pour le contrôle qualité et l'analyse de la microstructure d'un matériau étudié.

Les limites observées sont principalement liées à la sensibilité des résultats aux choix de prétraitement et de paramètres de segmentation, qui peuvent influencer les masques obtenus et, par conséquent, les distributions granulométriques dérivées. Par ailleurs, l'introduction d'un apprentissage supervisé implique la disponibilité de masques de vérité terrain (obtenus par méthodes classiques et/ou par segmentation manuelle) et une adéquation entre les données d'entraînement et les micrographies traitées, conditionnant la robustesse des conclusions lorsque l'on change de conditions d'acquisition ou de préparation.

En perspectives, trois axes apparaissent naturellement : (i) renforcer la généralisation du modèle U-Net par l'élargissement du corpus (matériaux et conditions de préparation), et par l'exploration de stratégies d'augmentation de données et d'adaptation de domaine ; (ii) coupler plus systématiquement les indicateurs granulométriques obtenus à des essais mécaniques afin d'établir des liens quantitatifs entre microstructure et performance ; (iii) étendre l'approche à des données 3D (tomographie) et l'intégrer, lorsque pertinent, dans une chaîne de monitoring en ligne du procédé. Dans cette continuité, le pipeline proposé constitue une base méthodologique cohérente pour rapprocher segmentation automatisée et métrologie des grains dans une logique de caractérisation microstructurale assistée par l'IA.

% %